Documentation technique — HYDRA

Méthodes de dépouillement, propagation d'incertitudes, corrélation d'images et corrections de dispersion

Document généré à partir d'une lecture intégrale du code source

Septembre 2026

Table des matières

Avant-propos **9**
Objet du document . . . 9
Méthode de rédaction et niveau de détail des équations . . . 9
Pourquoi la Monte-Carlo, et pas une propagation analytique – sauf dans un coin de l'application . 10
Sur les points de vigilance signalés dans ce document . . . 10
Conventions . . . 11

1. Propagation d'incertitudes par Monte-Carlo et calculs de covariance **12**
1. Principe général : échantillonnage Monte-Carlo, pas de propagation analytique linéarisée . . . 12
1.1 Étape 1 — définition et tirage des paramètres d'entrée . . . 12
1.2 Étape 2 — rejeu du pipeline complet pour chaque tirage . . . 13
1.3 Étape 3 — agrégation : moyenne et écart-type empiriques point par point . . . 14
2. Distributions de tirage supportées . . . 15
2.1 Loi normale (`Distribution.NORMAL`) . . . 16
2.2 Loi uniforme (`Distribution.UNIFORM`) . . . 16
2.3 Cas dégénéré (valeur fixe) . . . 16
3. Covariance / corrélation Monte-Carlo entre points d'une même courbe . . . 17
3.1 Nécessité d'une grille commune, et interpolation . . . 17
3.2 Formule de la matrice de covariance . . . 17
3.3 Corrélation : pas de normalisation implémentée dans le code . . . 18
3.4 Origine des tirages bruts utilisés . . . 18
3.5 Bibliothèque de calibration (`covariance_library`) — passerelle vers la Calibration bayésienne . . . 19
4. Bande d'incertitude d'un capteur personnalisé (`custom_sensor.py`) . . . 20
4.1 Donnée calibrée : facteur multiplicatif, filtre passe-bas, correction thermique . . . 21
4.2 Bande d'incertitude (`uncertainty_band`) . . . 21
4.3 Recadrage temporel (`cut_from`) . . . 22
4.4 Correction de sortie thermique variable dans le temps (optionnelle) . . . 22
5. Paramétrage utilisateur du nombre de tirages (N) . . . 24
6. Synthèse des points de traçabilité (fichier :: fonction) . . . 24
7. Un autre paradigme d'incertitude : covariance asymptotique de moindres carrés . . . 25
8. Limitations et hypothèses à signaler explicitement . . . 26

2. Dépouillement Quasi-Statique (QS) et géométries d'éprouvette **28**
1. Vue d'ensemble de la chaîne de calcul QS . . . 28
Chaîne amont (avant `process_qs`) . . . 29

2. Prétraitement des signaux 29
2.1 Découpe du début de courbe (core/curve_cutting.py: cut_curve_start) 29
2.2 Correction de complaisance machine (core/machine_compliance.py: apply_compliance_correction) 30
2.3 Incertitudes de mesure Monte-Carlo (coeff_load, coeff_dd) 30
3. Les quatre cinématiques de dépouillement (type_check) 31
3.1 Compression et Tension (type_check == 1 ou 2) 31
3.2 Éprouvette entaillée — Shear-Compression/Tension (type_check == 3) 32
3.3 Cisaillement (type_check == 4) 32
3.4 Modèle de température adiabatique (Taylor-Quinney), commun aux 4 modes 33
4. Géométries d'éprouvette (core/models.py) 33
4.1 Cylindrique (Compression) 33
4.2 Carrée/Rectangulaire (Compression) 34
4.3 Axisymétrique (Tension) 34
4.4 Plate (Tension, "Flat") 34
4.5 Éprouvette entaillée (Shear-Compression/Tension, Notched) 34
4.6 Hat-shaped (Cisaillement) 34
4.7 Double cisaillement 35
4.8 Simple cisaillement 35
5. Coefficients k / sk pour éprouvette entaillée 35
5.1 Format de fichier (core/k_value_io.py) 35
5.2 Rôle cinématique de k_value (sans split, sk_check faux) 35
5.3 Mécanisme de "split fit" (sk_check vrai) 36
5.4 Cohérence avec la géométrie "entaillée" (§4.5) 37
6. Limitations, hypothèses et ambiguïtés explicitement relevées 37
6.1 Sur l'onglet QS, la géométrie détaillée (cylindrique vs carrée/rectangulaire, axisymétrique vs plate) n'est pas prise en compte 37
6.2 Sur l'onglet QS, le sous-type de cisaillement n'est pas sélectionnable 38
6.3 Angle d'entaille : unité « deg » déclarée, mais utilisée directement dans cos() 38
6.4 Champs géométriques déclarés mais non utilisés en QS 38
6.5 Absence de recalcul de section en grande déformation 38
6.6 Modèle de température : hypothèse adiabatique non justifiée dans le code 39
7. Récapitulatif fichier / fonction 39

3. Dépouillement SHPB (barres de Hopkinson classiques à deux barres) 40
Avertissement du code lui-même (à reproduire tel quel) 40
1. Principe physique et chaîne de calcul générale 40
1.1 Hypothèses classiques 40
1.2 Convention exacte utilisée dans le code — vérifiée ligne à ligne 41
1.3 Découpe temporelle des trois ondes (« Cutting waves ») 42
1.4 Vitesse de déformation, déformation nominale, déformation vraie (cas compression/traction) 42
1.5 Contrainte nominale et contrainte vraie (cas compression/traction) 43
1.6 Indicateurs de qualité d'équilibrage (« balance quality ») 43
1.7 Sécurités numériques (pas des corrections physiques) 44
2. Variantes de kinématique du même moteur de calcul 44
2.1 Compression / traction (type_check = 1 ou 2) 44
2.2 Cisaillement-compression/traction entaillé — « Notched » (type_check == 3, lignes 383-497) 44

2.3 Cisaillement pur — « Shear » (type_check == 4, lignes 609-723) 45
3. Choix de la force utilisée pour la contrainte (force_check) 45
4. Corrections physiques disponibles 46
4.1 Correction de gradient thermique de barre (« heat_check », éprouvette chauffée) 46
4.2 Correction de platens (plat_check) 47
4.3 Correction de poinçonnement / indentation (punch_check) 48
4.4 Correction d'inertie (inertia_check) 49
4.5 Coefficient de friction (friction) 50
5. Correction du « pied » / « toe » de courbe (core/toe_correction.py) 50
5.1 Détection automatique de la zone linéaire — detect_linear_region() (lignes 51-89) 50
5.2 Ajustement manuel — fit_linear_region() (lignes 40-48) 51
5.3 Recalage de l'origine — apply_toe_correction() (lignes 92-102) 51
5.4 Recalcul de la déformation vraie après recalage — infer_true_strain_relation() / true_strain_from_nominal() (lignes 105-148) 51
5.5 Cohérence avec les tirages Monte-Carlo bruts et les grandeurs de confinement (ui/pages/page_process.py) 52
6. Outil de vérification de l'alignement des ondes (core/wave_alignment.py) 53
6.1 Impédance mécanique et coefficient de transmission 53
6.2 Onde transmise théorique — predicted_transmitted_wave() (lignes 45-73) 54
6.3 Auto-alignement par corrélation croisée — best_alignment_shift() (lignes 88-129) . 54
7. Alignement temporel d'un capteur monté directement sur l'éprouvette (core/specimen_sensor_alignment.py) 55
8. Récapitulatif des limitations, hypothèses et points d'ambiguïté relevés dans le code 55

4. Dépouillement « Direct Impact » (DI) 57
1. Principe physique et différences avec le SHPB classique 57
2. Vue d'ensemble du pipeline (process_di()) 58
3. Découpe des signaux et vitesse incidente 58
3.1 Fenêtrage temporel 58
3.2 Signal dispersion-corrigé (disp_ela / disp_visco) 59
3.3 Correction « platen » 59
3.4 Correction de gradient thermique de barre (« heat ») 60
3.5 Attention — Interaction plat_check / heat_check sur E_mat_OB / C0_mat_OB 61
3.6 Diamètre/propriétés retenus pour F_output vs pour la correction de poinçonnement . . 61
3.7 Module et célérité viscoélastiques (_viscoelastic_modulus_and_speed, disp_visco) 62
4. Chaîne déformation / vitesse de déformation / force / contrainte 62
4.1 Compression / Traction (type_check 1 ou 2) 62
4.2 Cisaillement (type_check == 4 : hat-shaped / double / simple) 63
4.3 Éprouvette entaillée — Shear-Compression/Tension (type_check == 3) 64
5. Corrections physiques complémentaires 66
5.1 Poinçonnement / indentation (punch_check) 66
5.2 Friction (friction[d] != 0) 67
5.3 Inertie radiale/axiale (inertia_check) 67
5.4 Échauffement adiabatique (température) 67
6. Recalage du profil de vitesse incidente (« Cut »/« shift ») et facteur de calibration 68
6.1 Découpe/décalage (« Cut velocity profile at selected start » / « shift by (s) ») 68
6.2 Facteur de calibration multiplicatif de la vitesse incidente 68
6.3 Aide auxiliaire pour capteurs personnalisés montés sur l'éprouvette 69

7. Réutilisation de `toe_correction.py` et `specimen_sensor_alignment.py` 69
8. Table de correspondance équations <-> code 70
9. Limitations, avertissements et points à vérifier 71

5. Essai quasi-œdométrique (confiné) : contrainte radiale, pression hydrostatique et déviateur 73
1. Principe physique de l'essai 73
2. Modèle LINÉAIRE de cellule 74
2.1 Formule 74
2.2 Interprétation — C1, C2 et le mécanisme de transition 74
3. Modèle NON-LINÉAIRE de cellule 75
3.1 Vue d'ensemble 75
3.2 Étape 1 — Normalisation en déformation orthoradiale et ajustement polynomial 76
3.3 Étape 2 — Évaluation du polynôme sans terme constant 76
3.4 Étape 3 — Interpolation entre les deux courbes selon la hauteur courante de l'éprouvette 76
4. Constantes alpha00 et alpha0^(l/2) — calibration de la déformation radiale 78
4.1 Relation implicite 78
4.2 Résolution numérique 78
4.3 Convention de signe — point d'attention explicite du code 78
5. Grandeurs dérivées : contrainte radiale, pression, déviateur, déformations 79
5.1 Invariants de contrainte : pression hydrostatique et déviateur 79
5.2 Déformation radiale 79
5.3 Déformation volumétrique 79
5.4 Déformation équivalente (von Mises) 80
5.5 Résumé — chaîne de calcul complète 80
5.6 Contrainte axiale alternative par vélocimétrie laser 81
5.7 Enveloppe synthétique de repli (« imitation ») — `triaxial_synthetic_draws` 81
5.8 Calcul réel en boucle Monte-Carlo (SHPB / Impact Direct / Quasi-Statique) 82
6. Récapitulatif des correspondances fonctions <-> grandeurs 84
7. Limitations, hypothèses et avertissements relevés dans le code 85

6. Corrélation d'Images Numériques (DIC) 88
1. Vue d'ensemble du pipeline 89
2. Prétraitement (`core/dic_preprocessing.py`) 89
3. ROI et maillage de corrélation (`core/dic_correlation.py`) 90
3.1 Formes de ROI 90
3.2 Construction de la grille — `build_mesh(roi, subset_size, step)` 90
3.3 ROI non rectangulaires 91
3.4 Structure `MeshGrid` 91
4. Algorithme de suivi/corrélation (`core/dic_processing.py`) 91
4.1 Principe général 91
4.2 Corrélation croisée normalisée et fenêtre de recherche 91
4.3 Raffinement sous-pixel — `_subpixel_peak` 92
4.4 Modes de suivi « Fixed » vs « Moving » 93
4.5 Filtre de qualité de corrélation — `correlation_threshold` 93
4.6 Filtre de cohérence de voisinage — `_apply_neighbor_consistency_filter` 93
4.7 Boucle principale `run_dic` 94
4.8 Export et réimport des résultats (CSV) — `core/dic_result_io.py` 94
5. Champ de déformation local — `_grid_strain` (`core/dic_processing.py`, lignes 188–202) . 96

5.1 Déformation équivalente de von Mises — `core/dic_kinematics.von_mises_strain` . 97
6. Cinématique dérivée — `core/dic_kinematics.compute_kinematics` 97
6.1 Axe temporel 97
6.2 Vitesse et accélération 97
6.3 Force dynamique ROI — hypothèse de section constante 98
6.4 Lissage — `smooth_series` 98
7. Conversion en déformation vraie — `nominal_to_true_strain` (`core/dic_kinematics.py`, lignes 204–229) 98
8. Time scale / Time offset — synchronisation caméra <-> jauges 99
9. Modèle d'incertitude de mesure DIC (`core/dic_uncertainty.py`) 100
9.1 Positionnement du modèle — ce qu'il EST et n'est PAS 100
9.2 Mécanisme 1 — Plancher de précision de recalage sub-pixel (photométrique) 101
9.3 Mécanisme 2 — Dégradation liée à la qualité de corrélation 102
9.4 Mécanisme 3 — Distorsion du subset (moyennage intra-subset) 102
9.5 Combinaison des trois termes en quadrature 103
9.6 Propagation aux déformations 103
9.7 Propagation temporelle (taux, vitesse, accélération, force) 104
9.8 Nombre de points effectif et incertitude de moyenne ROI 105
9.9 Orchestration — `compute_uncertainty` 105
10. Rendu visuel (`core/dic_render.py`) — résumé bref 106
11. Récapitulatif des correspondances fichier → fonction 107
12. Synthèse des limitations et avertissements relevés dans le code 108

7. Correction de dispersion des ondes — régime élastique 110
1. Principe physique : pourquoi une barre finie disperse l'onde 111
2. Théorie de dispersion implémentée : équation de Pochhammer-Chree 111
2.1 Variables et notations exactes du code (`core/dispersion.py::_solve_pochhammer_branch`, lignes 162–197) 112
2.2 Équation caractéristique résolue 112
2.3 Vitesse de barre c0 et coefficient de Poisson : `_bar_elastic_wave_speed()` 113
3. Méthode numérique de résolution : root-search fréquence par fréquence 113
3.1 Algorithme (`_solve_pochhammer_branch`, lignes 162–197) 113
3.2 Parallélisation entre branches (`_solve_pochhammer_branches`, lignes 209–239) 114
4. Algorithme complet de correction du signal (FFT → correction par bin → IFFT) 115
4.1 Découpage temporel des fenêtres (avant FFT) 115
4.2 Passage au domaine fréquentiel (FFT) 115
4.3 Correction par composante fréquentielle : `_corrected()` (lignes 373–385) 115
4.4 Distance signée z et convention de signe des trois ondes 116
4.5 Cas « barre de sortie seule » (Impact Direct) : `elastic_correction_direct_impact()` 117
5. Fenêtrage fréquentiel et bornage de la bande passante de la correction 117
5.1 Absence de fenêtrage temporel (apodisation) avant FFT — toujours vrai 117
5.2 Limitation de la bande passante propre de la correction — `_bandwidth_rolloff_weight()` (lignes 92-153) 118
6. Bascule élastique / viscoélastique dans le module 120
7. Résultats exposés (`DispersionResult`, lignes 99–110) 120
8. Synthèse des correspondances code 121
9. Limitations et avertissements identifiés dans le code 121

8. Correction de dispersion viscoélastique, calibration bayésienne et amortissement 123
8.1 Pourquoi un modèle viscoélastique ? . . . 123
8.2 Les quatre modèles rhéologiques (`core/rheology_models.py`) . . . 125
8.2.1 Kelvin–Voigt (`_kelvin_voigt_modulus`, ligne 120) . . . 125
8.2.2 Standard Linear Solid / Zener (`_sls_modulus`, ligne 126) — seul modèle branché sur la page Dispersion Corrections . . . 125
8.2.3 Generalized Maxwell / Wiechert, 2 branches (`_generalized_maxwell_2_modulus`, ligne 146) . . . 126
8.2.4 Burgers (`_burgers_modulus`, ligne 157) . . . 127
8.2.5 Tableau récapitulatif . . . 128
8.3 Propagation du pulse à travers le modèle de barre viscoélastique (`core/visco_calibration.py`) 128
8.3.1 Module complexe, vitesse et modules de Lamé (`_complex_modulus_and_speed`, ligne 78) . . . 128
8.3.2 Recherche de racine de Pochhammer-Chree par bin de fréquence (`_predict_pulse`, ligne 90) . . . 128
8.3.3 Correction de phase, atténuation et amortissement, puis reconstruction temporelle . . . 129
8.3.4 Score de comparaison : Pearson vs R² (`SCORE_FUNCS`, ligne 223) . . . 130
8.4 Calibration bayésienne (`core/bayes_visco_calibration.py`) . . . 131
8.4.1 Pourquoi une recherche bayésienne plutôt qu'une grille . . . 131
8.4.2 Évaluation d'un point candidat . . . 131
8.4.3 Échantillonnage initial (Latin Hypercube Sampling) . . . 131
8.4.4 Boucle adaptative : surrogate Processus Gaussien + Expected Improvement . . . 132
8.4.5 Moyenne, écart-type et corrélation a posteriori des paramètres . . . 132
8.4.6 Pulse prédit retourné : réutilisation, pas de recalcul . . . 133
8.5 Le Processus Gaussien surrogate (`core/gaussian_process.py`) . . . 133
8.5.1 Noyau de covariance . . . 134
8.5.2 Ajustement des hyperparamètres (maximisation de la vraisemblance marginale) . . . 134
8.5.3 Prédiction (moyenne/variance a posteriori) . . . 134
8.6 Auto-alignement des pulses par corrélation sub-échantillon (`core/pulse_alignment.py`) . . . 135
8.6.1 Pourquoi . . . 135
8.6.2 Algorithme (`align_pulse2`, ligne 50) . . . 135
8.7 Outil standalone « Free-Decay Damping » (`core/decay.py`) . . . 136
8.7.1 Localisation précise du pic et des points à mi-puissance . . . 136
8.7.2 Facteur d'amortissement . . . 136
8.7.3 Utilisation en aval . . . 137
8.8 Synthèse des renvois fichier/fonction . . . 137
8.9 Récapitulatif des limitations et avertissements relevés dans le code . . . 138

9. Outils standalone (CPG, synchronisation vidéo, analyse par lot, calibration bayésienne) 140
1. Calculateur de coefficient de pont de jauges (CPG) . . . 140
1.1 Contexte et rôle . . . 140
1.2 Entrées physiques . . . 141
1.3 Terme thermique commun . . . 141
1.4 Formule du coefficient CPG par type de pont . . . 142
1.5 Propagation d'incertitude . . . 142
1.6 Interface / synchronisation . . . 143
1.7 Correction de sensibilité transverse (optionnelle) . . . 143

1.8 Correction de sortie thermique (optionnelle, informative) . . . 144
2. Synchronisation caméra / vidéo . . . 145
2.1 Contexte . . . 145
2.2 Lecture du fichier d'horloge . . . 145
2.3 Détection du train d'impulsions d'exposition . . . 146
2.4 Estimation du temps absolu de chaque frame . . . 146
2.5 Détection d'un front montant (signal de trigger à palier unique) . . . 147
2.6 Méthode 1 — Localisation du trigger par signal de trigger . . . 147
2.7 Méthode 2 — Localisation du trigger par numéro de frame connu + fréquence . . . 148
2.8 Résultat retourné . . . 148
2.9 Limitations notées dans le code . . . 149
3. Analyse par lot (Batch Analysis) . . . 149
3.1 Contexte . . . 149
3.2 Chargement des fichiers . . . 149
3.3 Correction isotherme (échauffement adiabatique) . . . 149
3.4 Rééchantillonnage sur une grille commune . . . 150
3.5 Courbe moyenne et bandes de variabilité . . . 151
3.6 Export . . . 152
3.7 Limitations et remarques notées dans le code . . . 153
4. Sous-échantillonnage (`core/downsampling.py`) . . . 153
5. Abstraction vidéo / séquence d'images (`core/frame_source.py`) . . . 153
6. Export vidéo annoté DIC (`core/dic_video_export.py`) . . . 154
7. Calibration bayésienne de lois de comportement — portage de MAMBA (`core/calib_*.py`, `ui/pages/page_bayes_calib.py`) . . . 154
7.1 Contexte : MAMBA et le choix de portage . . . 154
7.2 Analyseur et évaluateur d'équation (`core/calib_equation.py`) . . . 155
7.3 Trois traitements de prior par paramètre libre (`core/calib_priors.py`) . . . 156
7.4 De la bibliothèque de covariance au jeu de données d'un essai (`core/calib_library_adapter.py`) 156
7.5 Construction et échantillonnage du modèle PyMC (`core/calib_pymc_model.py`) . . . 157
7.6 Orchestration d'une exécution (`core/calib_run.py`) . . . 158
7.7 Diagnostics postérieurs et validation croisée réelle (`core/calib_diagnostics.py`) . . . 158
7.8 Fil d'exécution partagé (`core/qt_worker.py`) . . . 159
7.9 Interface — les onze cartes de la page Bayesian Calibration . . . 159
7.10 Limitations et avertissements notés dans le code . . . 161
7.11 Synthèse des renvois fichier/fonction . . . 161

10. Analyse fissure & vitesse d'onde (Crack & Wave Analysis) 163
1. Squelette de fissure tracé (`core/crack_skeleton.py`) . . . 164
2. Vitesse des ondes élastiques par suivi de front (`core/elastic_wave_speed.py`) . . . 165
2.1 Principe : temps de vol le long d'un front . . . 165
2.2 Détection automatique de branches multiples (crête + RANSAC) . . . 165
2.3 Cueillette manuelle par polygone ROI (`extract_ridge_in_polygon`) . . . 166
2.4 Absence d'incertitude formelle . . . 166
3. Détection et suivi de la pointe de fissure (`core/crack_tip_tracking.py`) . . . 167
3.1 Vue d'ensemble des méthodes disponibles . . . 167
3.2 Méthode retenue : saut d'ouverture (`crack_opening_jump_profile`, `track_tip_opening_jump`) 168
3.3 Incertitude de position (`opening_extent_position_uncertainty_px`) . . . 170

3.4 Vitesse locale et son incertitude . . . 171
3.5 Vitesse ajustée (régression, pondérée quand une incertitude est disponible) . . . 172
3.6 Export CSV . . . 172
4. Facteurs d'intensité de contrainte K_I/K_II : X-FEM-DIC (`core/xfem_dic.py`) . . . 173
4.1 Principe et justification : pourquoi les images brutes . . . 173
4.2 Modèle de déplacement et simplifications volontaires . . . 173
4.3 Maillage Q4 centré sur la pointe (uniforme ou gradué) . . . 174
4.4 Classification de l'enrichissement (`classify_mesh_enrichment`) . . . 174
4.5 Résolution : Gauss-Newton amorti, mise à l'échelle de Jacobi . . . 175
4.6 Détection automatique de la position de la pointe . . . 176
4.7 Incertitude formelle (covariance de Gauss-Newton asymptotique) . . . 177
4.8 Convention "K_I critique" et enveloppe d'incertitude sur le tracé . . . 178
4.9 Export CSV (`export_xfem_k_csv`) . . . 179
5. Méthodes additionnelles disponibles dans le code (`core/crack_k1c.py`, `core/crack_interaction_integral.py`) 17
5.1 Ajustement DIC standard en anneau (`fit_k_single_frame`, `compute_k_time_series`) 179
5.2 Extension optionnelle : terme de T-stress (série de Williams étendue) . . . 181
5.3 Intégrale d'interaction / J-integral (`core/crack_interaction_integral.py`) . . . 181
6. Limitations et remarques notées dans le code . . . 183

Annexe — Synthèse des points de vigilance relevés dans le code 186
Chapitre 1 — Incertitudes Monte-Carlo et covariance . . . 186
Chapitre 2 — Quasi-Statique (QS) et géométries d'éprouvette . . . 187
Chapitre 3 — SHPB (deux barres) . . . 187
Chapitre 4 — Impact Direct (DI) . . . 188
Chapitre 5 — Confinement (quasi-œdométrique) . . . 189
Chapitre 6 — Corrélation d'images (DIC) . . . 189
Chapitre 7 — Dispersion élastique . . . 190
Chapitre 8 — Dispersion viscoélastique, calibration bayésienne, amortissement . . . 190
Chapitre 9 — Outils standalone . . . 191
Chapitre 10 — Analyse fissure & vitesse d'onde (Crack & Wave Analysis) . . . 192

Avant-propos

Objet du document

Ce document décrit, chapitre par chapitre, le fonctionnement technique de **shpb_app**, l'application de dépouillement d'essais de caractérisation dynamique des matériaux (barres de Hopkinson / Split-Hopkinson Pressure Bar — SHPB, Impact Direct, essais quasi-statiques, essais quasi-œdométriques de confinement) et de corrélation d'images numériques (DIC). Il couvre dix domaines fonctionnels :

1. la propagation d'incertitudes par méthode de Monte-Carlo et les calculs de covariance associés ;
2. le dépouillement des essais quasi-statiques (QS), pour l'ensemble des géométries d'éprouvette supportées ;
3. le dépouillement des essais SHPB classiques (deux barres) ;
4. le dépouillement des essais à Impact Direct (une seule barre instrumentée) ;
5. le dépouillement des essais quasi-œdométriques (confinement) ;
6. la corrélation d'images numériques (DIC) ;
7. la correction de dispersion des ondes en régime élastique ;
8. la correction de dispersion des ondes en régime viscoélastique, la calibration bayésienne des modèles rhéologiques et l'outil d'amortissement par décroissance libre ;
9. les outils indépendants (« standalone ») : calculateur de coefficient de pont de jauges, synchronisation caméra/vidéo, analyse par lot, calibration bayésienne de lois de comportement en texte libre (portage de MAMBA, sur PyMC 5/PyTensor), et modules techniques annexes ;
10. l'analyse fissure & vitesse d'onde (« Crack & Wave Analysis ») : vitesse des ondes élastiques par suivi de front, détection/suivi de la pointe de fissure, et facteurs d'intensité de contrainte K_I/K_II par plusieurs méthodes indépendantes et vérifiables entre elles (DIC standard en anneau — avec extension optionnelle T-stress —, intégrale d'interaction/J-integral, X-FEM-DIC).

Méthode de rédaction et niveau de détail des équations

Ce document a été produit en lisant intégralement le code source du dossier `core/` de l'application (environ 9 000 lignes réparties sur une trentaine de modules Python), puis en formalisant, pour chaque module, les équations **telles qu'elles sont effectivement implémentées** dans le code — avec les mêmes notations de variables que le code source chaque fois que cela apporte de la traçabilité — accompagnées d'une brève justification physique. Il ne s'agit donc **pas** d'un manuel théorique qui redériverait chaque méthode depuis les équations fondamentales de la mécanique des milieux continus ou du traitement du signal ; c'est une

documentation de vérification, dont l'équation affichée doit pouvoir être retrouvée, terme à terme, dans le fichier et la fonction cités juste à côté.

Chaque section technique est ainsi systématiquement accompagnée d'un renvoi explicite au fichier et à la fonction ou méthode source (`core/fichier.py::fonction`), et de nombreux extraits de code sont reproduits littéralement lorsqu'ils clarifient une formule. Un tableau récapitulatif des renvois fichier/fonction figure à la fin de la plupart des chapitres.

Pourquoi la Monte-Carlo, et pas une propagation analytique – sauf dans un coin de l'application

Point méthodologique transversal à retenir avant la lecture des chapitres suivants : dans l'écrasante majorité de l'application (chapitres 2 à 9), **aucune incertitude n'est calculée par dérivation analytique linéarisée** (type loi de propagation GUM au premier ordre, $\sigma_y^2 = \sum_i (\partial f/\partial x_i)^2 \, \sigma_{x_i}^2$, appliquée à un pipeline physique complet). Toutes les incertitudes affichées dans ces chapitres — sur une courbe contrainte-déformation, sur un coefficient de pont de jauges, sur une contrainte radiale de confinement, sur un champ de déformation DIC — résultent d'un **tirage Monte-Carlo** : chaque grandeur d'entrée incertaine est échantillonnée N fois selon sa loi déclarée (Normale ou Uniforme), le pipeline de calcul complet est rejoué une fois par tirage, et la moyenne/écart-type empiriques des N résultats donnent la courbe moyenne et sa bande d'incertitude. Le chapitre 1 détaille ce mécanisme, commun à la quasi-totalité de l'application.

Exception : l'outil "Crack & Wave Analysis" (chapitre 10) calcule ses incertitudes différemment, par **covariance asymptotique d'un ajustement par moindres carrés** (linéaire exact pour l'ajustement DIC standard en anneau, asymptotique de Gauss-Newton au point convergé pour X-FEM-DIC) — une seule évaluation, pas un ré-échantillonnage. Voir le chapitre 1, §7, pour la mise en perspective des deux paradigmes, et le chapitre 10 pour le détail complet des deux méthodes concernées.

Sur les points de vigilance signalés dans ce document

La lecture attentive du code source a fait apparaître, dans plusieurs modules, des points qu'il est nécessaire de signaler explicitement plutôt que de passer sous silence : incohérences apparentes entre deux chemins de calcul voisins, hypothèses implicites non commentées dans le code, unités ambiguës, fonctionnalités de l'interface qui ne transmettent pas toutes les informations attendues par la fonction de calcul sous-jacente, etc. Chaque chapitre technique se termine par une section **« Limitations, hypothèses et points à vérifier »** qui recense ces observations, formulées **telles qu'observées dans le code**, sans interprétation ni correction de notre part — à charge pour le lecteur de les confirmer ou de les infirmer par comparaison avec l'application d'origine ou avec les auteurs du code. Une synthèse consolidée de l'ensemble de ces points figure également en annexe, en fin de document, pour servir de liste de vérification unique.

Plusieurs modules du code portent eux-mêmes, dans leur en-tête, un avertissement explicite de non-validation indépendante contre des données expérimentales réelles ou contre la sortie de l'application d'origine dont ce code est le portage ; ces avertissements sont systématiquement reproduits dans les chapitres concernés.

Conventions

- Les équations sont données avec les notations du code (indices, exposants `_pa` pour un tableau Monte-Carlo de taille N, suffixes `IB`/`OB` pour barre d'entrée/sortie, etc.) : la section correspondante définit ces notations à leur première occurrence.
- Les renvois de code sont donnés sous la forme `fichier::fonction` ou `fichier::Classe.méthode`, chemins relatifs à la racine du projet `shpb_app/`.
- Le symbole `§` renvoie à une section du *même* chapitre.
- Les extraits de code sont reproduits tels quels depuis le code source, à des fins de vérification directe.

---

1. Propagation d'incertitudes par Monte-Carlo et calculs de covariance

Ce chapitre décrit le socle méthodologique utilisé par tous les modules de dépouillement de `shpb_app` (Quasi-Statique QS, SHPB, Impact Direct DI, capteurs auxiliaires, essais triaxiaux). Tous les renvois de code sont donnés sous la forme `fichier::Classe.méthode` ou `fichier::fonction`, chemins relatifs à `shpb_app/`.

1. Principe général : échantillonnage Monte-Carlo, pas de propagation analytique linéarisée

L'application ne calcule **jamais** l'incertitude finale par dérivation analytique/linéarisation (type "loi de propagation des incertitudes" GUM au premier ordre, $\sigma_y^2 = \sum_i (\partial f/\partial x_i)^2 \sigma_{x_i}^2$). Elle utilise exclusivement une **méthode Monte-Carlo par tirage** : chaque paramètre physique d'entrée incertain (géométrie des barres, masse volumique, module d'Young, coefficient de jauge, etc.) est tiré aléatoirement N fois selon sa loi de distribution déclarée, ce qui produit N jeux de paramètres complets. Le pipeline de calcul complet (conversion ondes → contrainte/déformation pour SHPB/DI, ou charge/déplacement → contrainte/déformation pour QS) est alors **rejoué intégralement N fois**, une fois par jeu de paramètres, produisant N courbes résultat indépendantes. La moyenne et l'écart-type point par point de ces N courbes donnent la courbe moyenne et la bande d'incertitude affichées.

Ceci est explicitement documenté en tête de `core/parallel.py` :

> "Monte-Carlo uncertainty propagation (core/process_shpb.py, core/process_di.py, called from ui/pages/page_process.py): one independent call per Monte-Carlo draw."

1.1 Étape 1 — définition et tirage des paramètres d'entrée

Chaque paramètre physique incertain est représenté par un objet `MCValue` (`core/mc.py`), qui regroupe en une seule structure la moyenne, la dispersion et la distribution du paramètre (`mean value / dispersion / distribution`, commentaire de tête de `core/mc.py`) :

```
# core/mc.py
@dataclass
class MCValue:
    name: str
```

```
    unit: str = ""
    mean: float = 0.0
    dispersion: float = 0.0
    distribution: Distribution = Distribution.NORMAL
```

Le tirage des N réalisations d'un paramètre est effectué par `core/mc.py::MCValue.sample(n, rng)`. Les paramètres d'une barre ou d'une éprouvette sont regroupés dans un `MCParameterSet` (`core/mc.py`), dont la méthode `sample_all(n, rng)` tire chaque paramètre du groupe indépendamment des autres — **aucune corrélation entre paramètres d'entrée n'est modélisée** : le tirage utilise un `numpy.random.Generator` distinct par appel (ou partagé si passé explicitement), et chaque `MCValue.sample()` est indépendant des autres. C'est une hypothèse implicite du code, non commentée explicitement comme telle dans `mc.py`, mais qui découle directement de l'implémentation (pas de matrice de corrélation d'entrée, chaque paramètre a son propre appel `rng.normal`/`rng.uniform`).

Pour les objets `Bar` (`core/models.py::Bar`), l'expansion Monte-Carlo est déclenchée par `Bar.expand(n, rng)`, qui appelle `_mc_array(params, key, n, rng)` (elle-même un simple relais vers `MCValue.sample`) pour chacun des paramètres de barre : `rho_pa`, `C0_pa`, `D_pa`, `e_pa`, `L_pa`, `PG_pa`, `nu_pa`, `CPG_pa`. Le suffixe `_pa` désigne un tableau numpy de longueur N ("population array"), convention héritée du code d'origine (`OB.D_pa`, `IB.rho_pa`, commentaire de tête de `core/models.py`). Le nombre de tirages N vient de `core/project.py::Project.n_montecarlo` (valeur par défaut `500`, champ "Number of Monte-Carlo Draws" de la fiche d'identité de l'analyse sur la page Test Setup). L'expansion globale (barre d'entrée, barre de sortie, éprouvette) est déclenchée par `core/project.py::Project.expand_montecarlo()` :

```python
# core/project.py
def expand_montecarlo(self):
    self.input_bar.expand(self.n_montecarlo)
    self.output_bar.expand(self.n_montecarlo)
    self.specimen.expand(self.n_montecarlo)
```

Un cas particulier existe pour les constantes du modèle viscoélastique (Solide Linéaire Standard) des barres, tirées par `core/models.py::Bar.expand_viscoelastic(n, e1, e1_std, e2, e2_std, n2, n2_std, rng)` — toujours une loi normale, avec repli sur une valeur constante (`np.full(n, e1)`) si l'écart-type fourni est nul.

1.2 Étape 2 — rejeu du pipeline complet pour chaque tirage

Pour SHPB et DI, l'exécution des N tirages est confiée à `core/parallel.py::run_map(func, items, n_workers, context, progress_callback, use_threads)`, appelé depuis `ui/pages/page_process.py` avec `items = list(range(n))` (un indice de tirage par item) et `func = core/process_shpb.py::_process_shpb_task` (resp. `core/process_di.py::_process_di_task`). Chaque appel exécute le pipeline physique complet pour le tirage `d` : il pioche `IB.D_pa[d]`, `IB.rho_pa[d]`, etc. dans les tableaux `_pa` pré-tirés, et produit une courbe contrainte-déformation entière (et toutes les grandeurs dérivées : force, température, vitesse de déformation…) pour ce seul tirage. `n_workers` (réglage "Performance" de la page Test Setup, `Project.n_workers`) détermine si ces N appels tournent en série (`n_workers <= 1`) ou en parallèle sur plusieurs processus (`ProcessPoolExecutor`), sans changer le résultat numérique — seule la vitesse d'exécution change. Le contexte (barres, éprouvette, réglages) est piqué (pickled) **une fois par processus**

travailleur, pas une fois par tirage, pour raison de performance.

Pour QS, le même schéma est utilisé directement dans `ui/pages/page_process.py::_QSTab._on_run` via `run_map(_process_qs_task, qs_items, n_workers=project.n_workers, context=qs_context, ...)`, avec en plus deux coefficients multiplicatifs aléatoires (facteurs de correction, tirés indépendamment par tirage) :

```
rng = np.random.default_rng()
coeff_load_arr = rng.normal(1.0, clu_pct / 100.0, n)
coeff_dd_arr = rng.normal(1.0, meu_pct / 100.0, n)
```

(`clu_pct`/`meu_pct` : pourcentages d'incertitude de calibration de charge et de déplacement — lois normales de moyenne 1.0 et d'écart-type égal au pourcentage/100).

1.3 Étape 3 — agrégation : moyenne et écart-type empiriques point par point

Important : aucune interpolation sur une grille commune n'est effectuée à cette étape. Chaque tirage produisant une courbe de longueur potentiellement différente (le nombre de points d'une courbe contrainte-déformation SHPB/DI/QS dépend de la fenêtre de calcul, elle-même dépendante des valeurs tirées), l'agrégation tronque simplement chaque courbe à la longueur du tirage le plus court, puis calcule moyenne et écart-type **par indice de tableau** (pas par niveau de déformation identique). Ceci est fait à l'identique aux trois emplacements des onglets QS, SHPB, DI de `ui/pages/page_process.py` (lignes ≈813–818, ≈1878–1883, ≈2969–2974) :

```
min_len = min(len(s) for s in all_stress)
stress_arr = np.array([s[:min_len] for s in all_stress])   # (N_valides, min_len)
strain_arr = np.array([s[:min_len] for s in all_strain])   # (N_valides, min_len)
mean_strain = strain_arr.mean(axis=0)
mean_stress = stress_arr.mean(axis=0)
std_stress  = stress_arr.std(axis=0)
```

Formules exactes, pour N' tirages valides (ceux n'ayant pas levé d'exception) tronqués à L points communs, avec $x_{d,i}$ la déformation et $y_{d,i}$ la contrainte du tirage d au point i :

$$\bar{x}_i = \frac{1}{N'} \sum_{d=1}^{N'} x_{d,i}, \qquad \bar{y}_i = \frac{1}{N'} \sum_{d=1}^{N'} y_{d,i}$$

$$s_{y,i} = \sqrt{\frac{1}{N'} \sum_{d=1}^{N'} \left(y_{d,i} - \bar{y}_i \right)^2}$$

C'est l'écart-type **population** (`numpy.ndarray.std` avec `ddof=0` par défaut, pas l'écart-type d'échantillon `ddof=1`) — à noter explicitement, c'est une petite sous-estimation par rapport à l'écart-type non biaisé, négligeable dès que N' est grand (valeur par défaut $N = 500$).

La courbe moyenne tracée utilise l'**axe de déformation lui-même moyenné** ($\bar{x}_i$, pas un axe de référence fixe), et la bande d'incertitude affichée est :

$$[\bar{y}_i - k\,\sigma\,s_{y,i}\,,\ \bar{y}_i + k\,\sigma\,s_{y,i}]$$

où $k\sigma$ = `Project.uncertainty_sigma` $\in \{1, 2, 3\}$ — réglage "Uncertainty Band (σ)" de la fiche d'identité de l'analyse (Test Setup). Le code correspondant (identique sur les 3 onglets) :

```
sigma = self.project.uncertainty_sigma
ax.fill_between(mean_strain, mean_stress - sigma * std_stress, mean_stress + sigma * std_stress, ...
```

Limite à signaler : comme il n'y a pas de resampling sur une grille de déformation commune à ce stade (contrairement au module de covariance, voir §3), la moyenne/écart-type par indice de tableau n'est rigoureusement valide que si toutes les courbes ont sensiblement le même axe de déformation point par point — ce qui est généralement vrai lorsque le nombre de points est fixé par le pas d'échantillonnage temporel du signal brut (commun à tous les tirages) plutôt que par la dynamique du tirage lui-même, mais peut introduire un léger flou si les fenêtres temporelles (donc les longueurs de courbe) varient significativement d'un tirage à l'autre — dans ce cas la troncature à `min_len` aligne les courbes par indice, pas par valeur de déformation.

Toutes les grandeurs par tirage sont en outre agrégées génériquement (pas seulement contrainte/déformation) par `ui/pages/page_process.py::_aggregate_quantities(per_draw)`, qui applique exactement le même principe (troncature à la longueur minimale commune à la grandeur, puis moyenne/écart-type par indice) à **toutes** les clés présentes dans les dictionnaires par tirage (`strain`, `stress`, `strain_rate`, `temperature`, `load`, `displacement`, `nominal_strain`, `force_input`, `force_output`, `inertia_stress`, `friction_stress`, etc.), produisant pour chaque grandeur `<clé>` une paire `<clé>` (moyenne) et `<clé>_std` (écart-type) stockée dans `project.last_results[...]` :

```python
# ui/pages/page_process.py
def _aggregate_quantities(per_draw: list[dict]) -> dict:
    ...
    for key in keys:
        arrays = [np.asarray(d[key]) for d in per_draw if key in d and len(d[key]) > 0]
        min_len = min(len(a) for a in arrays)
        stacked = np.array([a[:min_len] for a in arrays])
        out[key] = stacked.mean(axis=0)
        out[f"{key}_std"] = stacked.std(axis=0)
    return out
```

En parallèle, `ui/pages/page_process.py::_store_raw_draws(project, result_key, per_draw)` conserve **chaque tirage individuellement** (pas seulement moyenne/écart-type) dans `project.last_raw_draws[result_key]` sous forme `{quantité: [tableau_tirage_0, tableau_tirage_1, ...]}`. C'est cette structure brute (et non les moyennes agrégées) qui alimente le module de covariance (§3).

2. Distributions de tirage supportées

Le code ne supporte que **deux** lois, énumérées dans `core/mc.py::Distribution(str, Enum)` : `NORMAL = "Normal"` et `UNIFORM = "Uniform"`. Le tirage est effectué par `core/mc.py::MCValue.sample(n, rng)` :

```python
def sample(self, n, rng=None):
    rng = rng or np.random.default_rng()
    if n <= 1 or self.dispersion == 0:
        return np.full(max(n, 1), float(self.mean))
    if self.distribution == Distribution.NORMAL:
        return rng.normal(self.mean, self.dispersion, size=n)
    return rng.uniform(self.mean - self.dispersion, self.mean + self.dispersion, size=n)
```

2.1 Loi normale (`Distribution.NORMAL`)

Fonction : `numpy.random.Generator.normal(mean, dispersion, size=n)`, équivalent mathématiquement à un tirage standard transformé :

$$x_d = \mu + \sigma \cdot Z_d, \qquad Z_d \sim \mathcal{N}(0,1), \quad d = 1, \ldots, N$$

où μ = `MCValue.mean`, σ = `MCValue.dispersion` (interprété ici comme un écart-type).

2.2 Loi uniforme (`Distribution.UNIFORM`)

Fonction : `numpy.random.Generator.uniform(mean - dispersion, mean + dispersion, size=n)`. Le paramètre `dispersion` est ici traité comme un **demi-intervalle** (demi-étendue) autour de la moyenne, convention explicitement documentée dans le code ("dispersion is treated as a half-range around the mean, matching the 'Dispersion Value' convention used throughout the original GUI") :

$$x_d \sim \mathcal{U}(\mu - \Delta,\ \mu + \Delta), \qquad \Delta = \text{dispersion}$$

soit $x_d = \mu - \Delta + 2\Delta \cdot U_d$ avec $U_d \sim \mathcal{U}(0,1)$.

2.3 Cas dégénéré (valeur fixe)

Si `n <= 1` ou `dispersion == 0`, aucun tirage n'est effectué : `MCValue.sample` renvoie un tableau constant `np.full(max(n,1), mean)` — le paramètre est traité comme déterministe, quelle que soit la distribution déclarée. C'est la manière dont l'utilisateur "désactive" l'incertitude sur un paramètre donné (dispersion = 0).

Remarque de traçabilité : c'est le seul et unique point d'entrée du tirage aléatoire des paramètres physiques dans l'application ; aucune autre distribution (log-normale, triangulaire, bêta, etc.) n'est implémentée dans `core/mc.py`. Par ailleurs, `core/models.py::Bar.expand_viscoelastic` et `core/confinement.py` (voir note en fin de document) appellent directement `rng.normal(...)` sans passer par `MCValue/Distribution` — ils sont donc **toujours** normaux, sans option de loi uniforme, contrairement aux paramètres standards de barre/éprouvette.

3. Covariance / corrélation Monte-Carlo entre points d'une même courbe

Module : `core/covariance.py`, porté depuis `export_res.py` d'origine. Objectif énoncé en tête de fichier : au-delà de l'écart-type point par point déjà affiché partout (qui correspond, en pratique, à la diagonale de cette matrice), calculer la **matrice de covariance complète** entre points d'une même courbe, pour savoir si l'incertitude à un niveau de déformation (ou de temps) donné est corrélée à celle d'un autre niveau — par exemple, un tirage particulier du diamètre de barre décale globalement toute la courbe plutôt que de perturber chaque point indépendamment.

3.1 Nécessité d'une grille commune, et interpolation

Contrairement à l'agrégation moyenne/écart-type de la section 1.3 (qui tronque par indice), le module de covariance **interpole explicitement chaque tirage sur une grille commune** avant tout calcul, car chaque tirage a son propre axe de déformation (la déformation elle-même dépend du tirage Monte-Carlo) :

> "each draw has its own strain axis (since strain is itself Monte-Carlo-dependent), so before stacking draws into one matrix, every draw's quantity is linearly interpolated onto one common strain grid"

Fonction : `core/covariance.py::interpolate_draws_to_common_grid(per_draw_strain, per_draw_quantity, strain_start, strain_end, n_points)`.

1. Construction de la grille commune : `strain_grid = np.linspace(strain_start, strain_end, num=n_points)` — n_points valeurs régulièrement espacées entre `strain_start` (appelé "Reference (start) strain" dans l'UI, correspond au `bay_cut_value` de l'ancien code) et `strain_end`.
2. Pour chaque tirage d, interpolation **linéaire** de sa courbe propre (x_d, y_d) sur cette grille via `scipy.interpolate.interp1d(strain, quantity, kind="linear")`.
3. **Un tirage est intégralement exclu** si son propre axe ne couvre pas entièrement `[strain_start, strain_end]` (`if strain_grid.min() < strain.min() or strain_grid.max() > strain.max(): continue`) — un interpolateur linéaire ne peut être évalué hors de son domaine ; aucune extrapolation n'est tentée.
4. Résultat : `stacked`, tableau `(n_valid_draws, n_points)`, un tirage par ligne, chaque colonne correspondant à la même valeur de déformation (grandeur X) sur tous les tirages.

Côté UI (`ui/pages/page_covariance.py`), `strain_start`/`strain_end`/`n_points` sont les champs "Reference (start) strain" / "End strain" / "Number of points (N)" de la carte "Common grid", avec des valeurs par défaut `0.0`, `0.05`, `50`.

3.2 Formule de la matrice de covariance

Fonction : `core/covariance.py::compute_covariance_matrix(per_draw_arrays)`, appelée avec `per_draw_arrays = stacked` (issu de l'étape précédente). Elle exige au moins 2 tirages valides (`len(per_draw_arrays) < 2` → `None`, une covariance nécessitant au moins 2 observations), transpose `stacked` de `(n_draws, n_points)` vers `(n_points, n_draws)`, puis applique `numpy.cov` :

```
def compute_covariance_matrix(per_draw_arrays):
    if per_draw_arrays is None or len(per_draw_arrays) < 2:
```

```
        return None
    stacked = np.asarray(per_draw_arrays)
    if stacked.ndim == 1:
        return None
    return np.cov(stacked.T)  # (n_points, n_points)
```

`numpy.cov` traite chaque **ligne** de son argument comme une variable et chaque **colonne** comme une observation ; en transposant, chaque point de grille i devient une "variable" observée sur les N' tirages, exactement comme le fait l'original (`np.transpose(...)` avant `np.cov(...)`, cité dans le docstring). La formule mathématique exacte, avec $y_{d,i}$ la valeur du tirage d au point de grille i (après interpolation), N' le nombre de tirages valides retenus, et $\bar{y}_i = \frac{1}{N'} \sum_d y_{d,i}$:

$$C_{ij} = \frac{1}{N' - 1} \sum_{d=1}^{N'} \left(y_{d,i} - \bar{y}_i\right) \left(y_{d,j} - \bar{y}_j\right)$$

C'est ici l'estimateur **non biaisé** (`ddof=1`, comportement par défaut de `numpy.cov`, à distinguer du `ddof=0` utilisé par `ndarray.std` en §1.3 — incohérence mineure entre les deux modules à noter). La diagonale C_{ii} redonne la variance empirique du point i (cohérente en principe avec `<quantité>_std**2` de la section 1.3, à l'interpolation sur grille commune près, qui rend les deux quantités légèrement différentes : §1.3 aligne par indice de tableau, §3 aligne par valeur de déformation interpolée).

3.3 Corrélation : pas de normalisation implémentée dans le code

Le vocabulaire de l'UI et des docstrings emploie indifféremment "covariance" et "corrélation" ("how uncertainty at one strain/time point **correlates** with uncertainty at another"), mais **aucune fonction du code ne normalise la matrice de covariance en matrice de corrélation** (pas d'appel à `numpy.corrcoef`, aucune division par $\sqrt{C_{ii} C_{jj}}$ trouvée dans `core/covariance.py` ni dans `ui/pages/page_covariance.py`). Ce que l'application affiche et exporte est la matrice de covariance brute C_{ij} (unités = carré de l'unité de la grandeur), visualisée en carte de couleur (`ax.imshow(matrix, cmap="viridis", ...)`), avec la diagonale = variance. Une matrice de corrélation normalisée $\rho_{ij} = C_{ij} / \sqrt{C_{ii} C_{jj}}$ pourrait être dérivée de C_{ij} par l'utilisateur mais n'est **pas** calculée en interne — si le document final doit présenter une "matrice de corrélation", il faut le signaler comme une transformation à faire a posteriori, absente du code source actuel.

3.4 Origine des tirages bruts utilisés

Selon la grandeur choisie sur la page Covariance Analysis, `per_draw_strain`/`per_draw_quantity` proviennent de trois sources distinctes, toutes convergeant vers la même paire de fonctions `interpolate_draws_to_common_g` + `compute_covariance_matrix` :

- **Résultat QS/SHPB/DI déjà calculé** : `project.last_raw_draws[result_key]` (rempli par `_store_raw_draws`, §1.3), axe X = `nominal_strain` si disponible sinon `strain`.
- **Capteur personnalisé (Custom Sensor)** : tirages synthétiques générés à la volée par `ui/pages/page_covariance.py` (perturbation autour de la donnée mesurée, dans la bande d'incertitude du capteur — voir §4), avec `N_SYNTHETIC_SENSOR_DRAWS = 200` tirages.

- **Essai triaxial (confinement)** : deux sources possibles, sélectionnées automatiquement par `ui/pages/page_covariance.py::CovariancePage._triax_draws()` selon ce qui est disponible pour le résultat choisi (voir chapitre 5 §5.5/§5.7/§5.8 pour le détail complet) :
  - **Tirages réels** (préférés quand disponibles) : les six grandeurs de confinement basées sur la jauge de cellule (contrainte radiale, p, q, ε_r, ε_v, ε_{eq}) ont été calculées **une fois par tirage, dans la boucle Monte-Carlo elle-même** de l'onglet Processing (`core/confinement.py::per_draw_confinement_quan` chapitre 5 §5.8), à partir de la contrainte/déformation axiale et du signal de cellule réels de CE tirage — un ré-échantillonnage physique déjà présent dans `project.last_raw_draws`, pas une approximation ajoutée après coup. `_triax_draws()` détecte cette disponibilité (clé `confinement_radial_stress` dans les tirages bruts) et les réutilise directement.
  - **Tirages synthétiques de repli** (utilisés si les tirages réels ne sont pas disponibles pour ce résultat, et systématiquement pour les trois grandeurs dérivées de la vélocimétrie laser) : `core/confinement.py::triaxial_synthetic_draws(..., n_draws=200)`, qui perturbe la contrainte axiale et la déflexion de cellule de confinement chacune par un bruit gaussien centré (`rng.normal(0, axial_std)`, `rng.normal(0, defl_std)`) et le module de la cellule par une loi normale relative (`rng.normal(cell_modulus, cell_modulus_std_fraction * cell_modulus)`, fraction par défaut 2 %), pour produire $N = 200$ réalisations synthétiques des grandeurs triaxiales dérivées (pression, déviateur, contrainte radiale, etc.). Ce mécanisme est une propagation Monte-Carlo indépendante, hors du pipeline `Bar.expand`/`MCValue`, qui **ré-échantillonne une gaussienne indépendante autour des courbes déjà moyennées** plutôt que de réutiliser la corrélation réelle tirage-par-tirage — une imitation de l'incertitude, pas une propagation au sens strict (voir chapitre 5 §5.7 pour la discussion complète de cette limite).

3.5 Bibliothèque de calibration (`covariance_library`) — passerelle vers la Calibration bayésienne

Ajout récent : la carte "Save to calibration library" de la page Covariance Analysis (`ui/pages/page_covariance.py`) permet de figer la matrice affichée — ainsi que sa grille commune et sa courbe moyenne, toutes deux calculées en §3.1 mais jusque-là jetées après affichage — dans une entrée nommée, conservée sur le projet plutôt qu'écrite sur disque. C'est le mécanisme d'alimentation en données de l'onglet "Bayesian Calibration" (chapitre 9, §7), qui ne charge plus de dossiers de fichiers `.txt` par essai (comme le faisait le logiciel d'origine, MAMBA) mais pioche directement dans cette bibliothèque.

Structure (`core/project.py::CovarianceLibraryEntry`, dataclass) :

Champ	Contenu
`name`	Nom donné par l'utilisateur, unique dans la bibliothèque (pré-rempli avec le libellé de la matrice ; en cas de collision, un suffixe numérique `_2`, `_3`, … est ajouté automatiquement plutôt que d'écraser l'entrée existante — `_on_save_to_library`)
`source_label`	Provenance (le libellé de la matrice affichée, ex. `SHPB_stress`)

Champ	Contenu
`quantity_key`	Clé dans `QUANTITY_LABELS` (ex. `stress`, `strain`, `pressure`, `volumetric_strain`) — indicative seulement, voir plus bas
`grid`	Grille commune (N points), liste de flottants
`mean`	Courbe moyenne sur cette grille (N points) — `stacked.mean(axis=0)`, calculée en §3.1 mais jamais conservée avant cet ajout
`covariance`	Matrice $N \times N$ (liste de listes) — celle de §3.2
`n_draws_used`, `grid_start`, `grid_end`, `created_at`	Traçabilité (nombre de tirages retenus, bornes de grille utilisées, horodatage ISO)

Le choix de dupliquer entièrement les données (plutôt que de garder une simple référence vers `project.last_raw_draws/last_results`) est délibéré : ces derniers sont recalculés à la demande à partir de l'essai actuellement chargé et **ne survivent pas** au chargement d'un essai différent, alors qu'une bibliothèque de calibration doit précisément accumuler des paires (grille, moyenne, covariance) au fil de plusieurs essais traités successivement dans une même session — traiter l'essai 1, sauvegarder les grandeurs voulues, charger l'essai 2, répéter. Une entrée reste donc utilisable même après que les fichiers de l'essai qui l'a produite ont été déchargés.

`quantity_key` mérite une remarque : c'est une étiquette purement indicative, tirée du dictionnaire `core/covariance.py::QUANTITY_LABELS` (27 entrées : déformation vraie/nominale, contrainte vraie/nominale, vitesse de déformation, température, déplacement, charge, forces d'entrée/sortie de barre, l'ensemble des grandeurs triaxiales — contrainte radiale, pression, déviateur, déformations radiale/volumétrique/équivalente, avec leurs variantes "laser" et "in-loop" — et le signal calibré d'un capteur personnalisé). Rien dans `core/covariance.py` ni `core/calib_library_adapter.py` n'impose qu'une entrée de `quantity_key="strain"` contienne effectivement une déformation : ce champ ne sert qu'à préremplir et filtrer les menus déroulants de la page Bayesian Calibration (chapitre 9, §7.8, carte "Calibration tests"), jamais à valider les données elles-mêmes.

Persistance : `covariance_library` est sauvegardée/restaurée par `core/session.py` (`_covariance_library_to_dict/_cov` enregistrées dans `to_dict/from_dict`), selon le même schéma que `last_correction` (tableaux → listes JSON-natives, reconvertis en `np.ndarray` à la lecture par le code consommateur) — et non selon le schéma "recalculé à la demande, jamais persisté" utilisé pour `last_results/last_raw_draws`, puisque cette bibliothèque n'est peuplée que par une action explicite de l'utilisateur (jamais silencieusement) et est spécifiquement conçue pour survivre à travers plusieurs cycles de chargement d'essai. Elle n'est en revanche **pas** rattachée à `core/setup_profile.py` (les profils de configuration réutilisable portent sur l'équipement/le montage, pas sur des données d'essai calculées ; appliquer un profil à un essai sans rapport ne doit jamais entraîner silencieusement la bibliothèque d'un autre essai).

4. Bande d'incertitude d'un capteur personnalisé (`custom_sensor.py`)

Module : `core/custom_sensor.py::CustomSensor`, porté de `config_custom_sensors.py` (jusqu'à trois capteurs auxiliaires : cellules de charge additionnelles, extensomètres, thermocouples, etc., configurables en

nombre illimité sur la page « Custom Sensors »). Contrairement aux paramètres de barre/éprouvette, **cette bande n'est pas obtenue par tirage Monte-Carlo mais par un calcul direct déterministe** combinant une incertitude relative (pourcentage) et une incertitude absolue déclarées par l'utilisateur.

4.1 Donnée calibrée : facteur multiplicatif, filtre passe-bas, correction thermique

`core/custom_sensor.py::CustomSensor.calibrated_data()` enchaîne, dans cet ordre, jusqu'à trois transformations optionnelles/toujours-actives sur la donnée brute `self.data` :

```python
def calibrated_data(self) -> np.ndarray:
    factor = self.multiplicative_factor if self.multiplicative_factor else 1.0
    data = self.data * factor
    if self.lowpass_enabled and len(self.time) > 1:
        dt = float(np.median(np.diff(self.time)))
        data = apply_lowpass_filter(data, dt, self.lowpass_cutoff_hz)
    if self.thermal_correction_enabled:
        offset = self.thermal_offset_strain()
        n = min(len(data), len(offset))
        data = data.copy()
        data[:n] = data[:n] - offset[:n]
    return data
```

1. **Facteur multiplicatif** (toujours appliqué, `1.0` par défaut si nul/vide) : $y = f \cdot d$.
2. **Filtre passe-bas** (optionnel, `lowpass_enabled`) : filtre de Butterworth zéro-phase (`core/lowpass.py::apply_lowpas` fréquence de coupure `lowpass_cutoff_hz`, 300 kHz par défaut), pour retirer le bruit haute fréquence — un pas de temps invalide ou une coupure au-delà de la fréquence de Nyquist du capteur laisse simplement la donnée non filtrée.
3. **Correction de sortie thermique variable dans le temps** (optionnelle, `thermal_correction_enabled`, §4.4 ci-dessous) : soustrait un décalage de déformation apparente, point par point.

`calibrated_data()` reste **pure/non-mutante** : toujours recalculée depuis `self.data`, jamais réécrite dedans — activer/désactiver n'importe laquelle des deux corrections optionnelles ne peut donc pas accumuler d'effet répété.

4.2 Bande d'incertitude (`uncertainty_band`)

Via `core/custom_sensor.py::CustomSensor.uncertainty_band()` — les deux composantes d'incertitude (pourcentage et absolue) sont **additionnées** pour former la demi-largeur de bande, exactement comme dans le code d'origine (docstring : "combining the percentage and absolute uncertainty additively, exactly matching the original's three `if sensor_unc_check == ...` combinations") :

$$\Delta(t) = u_{\text{abs}} + y(t) \cdot \frac{u_{\%}}{100}$$

$$\text{borne inférieure}(t) = y(t) - \Delta(t), \qquad \text{borne supérieure}(t) = y(t) + \Delta(t)$$

```
def uncertainty_band(self) -> tuple[np.ndarray, np.ndarray]:
    data = self.calibrated_data()
    half_width = self.uncertainty_absolute + data * (self.uncertainty_percent / 100.0)
    return data - half_width, data + half_width
```

où `uncertainty_absolute` (u_{abs}, dans l'unité physique du capteur) et `uncertainty_percent` ($u_{\%}$, en %) sont deux champs indépendants du capteur, pouvant être utilisés seuls ou ensemble (dans l'ancien code, trois combinaisons possibles selon la case cochée : pourcentage seul, absolu seul, ou les deux — ici toujours sommés, les valeurs nulles rendant le terme correspondant neutre). Notez que la part relative est calculée sur la donnée **calibrée** `data` (après facteur multiplicatif, filtre passe-bas et correction thermique le cas échéant), pas sur la donnée brute — la bande d'incertitude, et tout ce qui en dépend en aval (§4.3), reflète donc automatiquement les deux corrections optionnelles quand elles sont actives.

Cette bande demi-largeur $\Delta(t)$ est ensuite réutilisée comme un écart-type synthétique ($\sigma \approx \Delta(t)$, borne_sup – borne_inf sur 2, cf. `cell_defl_std = (cell_hi - cell_lo) / 2` dans `ui/pages/page_covariance.py`) partout où un capteur personnalisé doit être injecté dans un calcul Monte-Carlo en aval (perturbation gaussienne centrée d'écart-type $\Delta(t)$ pour générer des tirages synthétiques, §3.4) — traitement qui **suppose implicitement une loi normale** pour cette bande, alors que `uncertainty_band()` elle-même ne fait aucune hypothèse de distribution : c'est une bande symétrique déterministe, réinterprétée comme un 1σ gaussien uniquement au moment de son usage en aval (`core/confinement.py`, `ui/pages/page_covariance.py::_sensor_synthetic_draws/_triax_draws`).

4.3 Recadrage temporel (`cut_from`)

Le module fournit aussi `core/custom_sensor.py::CustomSensor.cut_from(cut_time)`, un outil de recadrage temporel (rogne tout avant `cut_time` et re-zéro l'axe des temps), sans rapport avec le calcul d'incertitude proprement dit mais utilisé pour préparer les données du capteur avant tirage.

4.4 Correction de sortie thermique variable dans le temps (optionnelle)

À la différence de la correction de sortie thermique statique de l'onglet CPG Calc (chapitre 9 §1.8 — une seule température de fonctionnement, offset **rapporté mais non appliqué** au signal), cette correction est **appliquée directement** dans `calibrated_data()` (§4.1) : elle est donc automatiquement reflétée dans le graphe Overview de la page Custom Sensors, dans `uncertainty_band()` (§4.2), et dans tout traitement Monte-Carlo en aval qui consomme ce capteur (alignement capteur-éprouvette, export, tirages synthétiques de covariance).

Champs de `CustomSensor` associés : `thermal_correction_enabled`, `thermal_unit` ("C"/"F", purement indicatif — n'intervient pas dans le calcul), `thermal_degree` (3 à 7), `thermal_coeffs` (coefficients a_i du polynôme constructeur, en microdéformation/unité^i), `thermal_t_ref` (température de référence/équilibrage du pont), et un profil température-vs-temps indépendant chargé séparément (`thermal_time`, `thermal_temperature`).

`core/custom_sensor.py::CustomSensor.thermal_offset_strain()` :

```
def thermal_offset_strain(self) -> np.ndarray:
    if not self.thermal_correction_enabled or len(self.time) == 0:
```

```
        return np.zeros(len(self.time))
    if len(self.thermal_time) < 2 or len(self.thermal_temperature) < 2:
        return np.zeros(len(self.time))
    coeffs = self.thermal_coeffs[: self.thermal_degree + 1]
    t_profile = np.interp(
        self.time, self.thermal_time, self.thermal_temperature,
        left=self.thermal_temperature[0], right=self.thermal_temperature[-1],
    )
    offset_ue = np.array([
        thermal_output_strain(coeffs, self.thermal_t_ref, t) for t in t_profile
    ])
    return offset_ue * 1e-6  # microdéformation -> déformation
```

Le profil température-vs-temps fourni (potentiellement sur son propre pas de temps/échantillonnage — par exemple un profil d'échauffement adiabatique exporté d'un onglet Processing, ou un relevé de thermocouple ambiant) est interpolé linéairement sur `self.time`, **saturé** (non extrapolé) à la première/dernière température connue en dehors de sa propre plage couverte plutôt que mis à `NaN` — une température inconnue juste avant/après le profil doit retomber sur la valeur connue la plus proche, pas aveugler le signal du capteur à cet instant. Chaque température interpolée est ensuite passée dans le **même** polynôme constructeur que la correction statique de l'onglet CPG Calc (`core/cpg_calc.py::thermal_output_strain`, Tech Note Vishay TN-504-1, éq. 3) :

$$\varepsilon_{T/O}(t) = P(T(t)) - P(T_{\text{ref}}), \qquad P(T) = \sum_i a_i \, T^i$$

et la déformation corrigée finale est $y_{\text{corr}}(t) = y(t) - \varepsilon_{T/O}(t) \times 10^{-6}$.

Décision de conception explicite — aucun modèle de retard thermique : contrairement à une première proposition envisagée (un modèle de diffusion 1D du premier ordre pour représenter le décalage temporel entre la température réelle de l'éprouvette et celle effectivement vue par la jauge, dont l'analyse d'ordre de grandeur donnait une constante de temps $\tau \approx L^2/\alpha$ de l'ordre de 1 à 125 ms selon l'épaisseur du collage/support de la jauge — comparable ou supérieure à la durée d'un essai SHPB typique), la jauge est supposée suivre **directement** le profil de température fourni, sans dynamique intermédiaire. Ce choix a été fait délibérément pour que le même outil reste valide aussi bien pour un essai dynamique rapide avec échauffement adiabatique de l'éprouvette (où un retard serait physiquement plausible mais reste non modélisé ici) que pour un essai quasi-statique lent avec dérive de température ambiante (où un modèle de retard n'aurait de toute façon pas de sens physique, l'équilibre thermique étant atteint en continu). Le profil fourni en entrée doit donc déjà représenter la température que la jauge voit réellement.

Comme pour la correction statique du chapitre 9 §1.8, cette correction n'a de sens physique que pour un capteur dont le signal calibré est lui-même une déformation de jauge (typiquement rôle `specimen_strain_gage` ou `confinement_cell`) — l'application ne le vérifie pas et laisse la case cochable pour tout capteur, sans avertissement.

5. Paramétrage utilisateur du nombre de tirages (N)

`core/project.py::Project.n_montecarlo` (par défaut `500`) est le nombre de tirages N utilisé par `Project.expand_montecarlo()` pour dimensionner tous les tableaux `_pa`, et transmis directement comme borne de boucle (`n = project.n_montecarlo`) aux trois onglets de traitement (`ui/pages/page_process.py`). C'est un champ unique et partagé (pas un réglage par module QS/SHPB/DI), figurant sur la fiche d'identité de l'analyse ("Test Setup"). Le nombre de tirages *effectivement* utilisés dans les moyennes/écarts-types peut être inférieur à N si certains tirages échouent (exception levée dans le pipeline physique, capturée individuellement par `core/parallel.py::run_map` et simplement ignorée dans l'agrégation, avec message diagnostique "Processed X/N draws" / "Draw d failed: …").

Réglage complémentaire, `core/project.py::Project.uncertainty_sigma` (1, 2 ou 3), multiplie l'écart-type empirique pour définir la largeur de la bande affichée/exportée (±1σ/±2σ/±3σ), voir §1.3 — un simple facteur d'échelle appliqué après coup, pas un paramètre du tirage lui-même.

6. Synthèse des points de traçabilité (fichier :: fonction)

Sujet	Fichier :: fonction/méthode
Distributions supportées	`core/mc.py::Distribution` (`NORMAL`, `UNIFORM`)
Tirage d'un paramètre	`core/mc.py::MCValue.sample(n, rng)`
Tirage d'un groupe de paramètres	`core/mc.py::MCParameterSet.sample_all(n, rng)`
Expansion Monte-Carlo d'une barre	`core/models.py::Bar.expand(n, rng)`
Expansion viscoélastique (SLS) d'une barre	`core/models.py::Bar.expand_viscoelastic(...)`
Déclenchement projet (IB/OB/éprouvette)	`core/project.py::Project.expand_montecarlo()`
Nombre de tirages N	`core/project.py::Project.n_montecarlo` (défaut 500)
Facteur d'échelle de la bande affichée	`core/project.py::Project.uncertainty_sigma` (1/2/3)
Exécution parallèle/série des N tirages	`core/parallel.py::run_map(func, items, n_workers, context, progress_callback, use_threads)`
Tirage par draw SHPB	`core/process_shpb.py::_process_shpb_task`
Tirage par draw DI	`core/process_di.py::_process_di_task`
Tirage par draw QS (+ coeffs calib.)	`ui/pages/page_process.py::_QSTab._on_run` (voir `coeff_load_arr`/`coeff_dd_arr`)
Agrégation moyenne/écart-type par indice	`ui/pages/page_process.py::_aggregate_quantities(per_draw`
Stockage des tirages bruts (pour covariance)	`ui/pages/page_process.py::_store_raw_draws(project, result_key, per_draw)`
Interpolation sur grille commune	`core/covariance.py::interpolate_draws_to_common_grid(...`
Matrice de covariance Monte-Carlo	`core/covariance.py::compute_covariance_matrix(per_draw_a`
Page UI Covariance	`ui/pages/page_covariance.py::CovariancePage._on_compute`
Bande d'incertitude capteur custom	`core/custom_sensor.py::CustomSensor.uncertainty_band()`

Sujet	Fichier :: fonction/méthode
Donnée calibrée capteur custom	`core/custom_sensor.py::CustomSensor.calibrated_data()`
Recadrage temporel capteur custom	`core/custom_sensor.py::CustomSensor.cut_from(cut_time)`
Correction thermique variable dans le temps (capteur custom)	`core/custom_sensor.py::CustomSensor.thermal_offset_strai`
Tirages synthétiques triaxiaux (repli)	`core/confinement.py::triaxial_synthetic_draws(...)`
Tirages réels triaxiaux (dans la boucle Monte-Carlo)	`core/confinement.py::per_draw_confinement_quantities(...` orchestré par `ui/pages/page_process.py::_prepare_confinement(...)`
Sélection réel/synthétique par grandeur (page Covariance)	`ui/pages/page_covariance.py::CovariancePage._triax_draws`
Bibliothèque de calibration (grille/moyenne/covariance nommées)	`core/project.py::CovarianceLibraryEntry`, `Project.covariance_library`
Sauvegarde/suppression d'une entrée de bibliothèque	`ui/pages/page_covariance.py::CovariancePage._on_save_to_`
Persistance de la bibliothèque de calibration	`core/session.py::_covariance_library_to_dict/_covariance`

7. Un autre paradigme d'incertitude : covariance asymptotique de moindres carrés

Tout ce qui précède dans ce chapitre repose sur le RÉ-ÉCHANTILLONNAGE : rejouer le pipeline physique N fois avec des entrées tirées, puis agréger empiriquement (moyenne/écart-type point par point, §1.3) ou par une matrice de covariance explicite (§3). C'est le seul paradigme d'incertitude utilisé par les modules SHPB/QS/Impact Direct/Confinement/Dispersion (chapitres 2 à 8).

L'outil "Crack & Wave Analysis" (chapitre 10) introduit un second paradigme, sans rapport avec le tirage Monte-Carlo : la **covariance asymptotique d'un ajustement par moindres carrés**, calculée UNE SEULE FOIS, au point ajusté, à partir de la courbure locale (linéaire ou linéarisée) du problème de moindres carrés lui-même, plutôt que par ré-échantillonnage répété. Pour un ajustement linéaire exact (`core/crack_k1c.py`'s `fit_k_single_frame`, chapitre 10 §4.2, y compris son extension optionnelle T-stress §4.5), c'est la théorie OLS classique et exacte ($\sigma^2 \cdot (X^T X)^{-1}$) ; pour un ajustement non linéaire (`core/xfem_dic.py`'s `fit_xfem_dic_frame`, chapitre 10 §7.7), c'est la même formule appliquée à la linéarisation locale au point convergé (le jacobien ré-assemblé une dernière fois) — l'approche standard, manuel, pour rapporter l'incertitude de paramètre d'un ajustement non linéaire par moindres carrés, distincte d'un intervalle de confiance Monte-Carlo ou bootstrap. Une troisième méthode de calcul de K_I/K_{II} (l'intégrale d'interaction, chapitre 10 §5) n'entre dans NI L'UN NI L'AUTRE de ces deux paradigmes — ce n'est pas un ajustement de moindres carrés du tout, donc pas de covariance ; sa confiance s'évalue par un diagnostic d'indépendance au sous-domaine plutôt qu'une barre d'erreur (chapitre 10 §5.5/§5.7).

Les deux paradigmes ne sont pas directement comparables terme à terme : la covariance de moindres carrés ne capture que le bruit qui affecte directement le résidu de l'ajustement lui-même (bruit de niveau de gris pour X-FEM-DIC, bruit du champ de déplacement pour l'ajustement en anneau), à un seul instant,

alors que le Monte-Carlo de ce chapitre propage l'incertitude de PARAMÈTRES D'ENTRÉE physiques (dimensions, calibration, …) à travers tout un pipeline de traitement. Voir le chapitre 10 pour le détail complet des méthodes de moindres carrés (ajustement DIC standard en anneau, X-FEM-DIC), de la méthode hors covariance (intégrale d'interaction) et de leurs propres limitations.

8. Limitations et hypothèses à signaler explicitement

1. **Indépendance des paramètres d'entrée** : chaque `MCValue` est tiré indépendamment des autres (aucune matrice de corrélation d'entrée) — une éventuelle corrélation physique réelle entre paramètres (ex. diamètre et masse volumique d'une même barre usinée) n'est pas modélisée.
2. **Deux distributions seulement** : Normale et Uniforme (`core/mc.py::Distribution`). Pas de loi log-normale, triangulaire, etc.
3. **Paramètre `dispersion` a un sens différent selon la loi** : écart-type pour Normale, demi-étendue pour Uniforme — la même valeur numérique de `dispersion` ne représente donc pas la même largeur d'incertitude selon la loi choisie.
4. **Agrégation moyenne/écart-type par indice de tableau (§1.3), sans interpolation sur grille commune**, contrairement au module de covariance (§3) qui, lui, interpole explicitement. Les deux écarts-types (`<quantité>_std` de l'agrégation générale vs. diagonale de la matrice de covariance) peuvent donc différer légèrement.
5. **Écart-type biaisé vs. non biaisé** : `ndarray.std()` (§1.3, agrégation générale) utilise `ddof=0` (estimateur biaisé/population), alors que `numpy.cov()` (§3.2, matrice de covariance) utilise `ddof=1` (estimateur non biaisé) — incohérence mineure entre les deux modules, négligeable pour N grand.
6. **Tirages hors domaine silencieusement exclus** : dans `interpolate_draws_to_common_grid`, un tirage dont la courbe ne couvre pas toute la plage `[strain_start, strain_end]` demandée est purement et simplement ignoré (pas d'extrapolation) — le nombre de tirages effectivement utilisés dans la matrice de covariance (`n_used`) peut donc être significativement inférieur au nombre total de tirages réussis (`n_total`), sans avertissement autre que l'indication `n_used/n_total` affichée dans le statut de la page.
7. **Pas de matrice de corrélation normalisée calculée** (§3.3) : seule la covariance brute est produite par le code ; toute matrice de corrélation présentée dans la documentation doit être signalée comme une dérivation externe ($\rho_{ij} = C_{ij}/\sqrt{C_{ii}C_{jj}}$), pas une sortie native de l'application.
8. **Bande d'incertitude capteur custom = combinaison additive déterministe**, pas un tirage Monte-Carlo en soi ; elle n'est traitée comme un écart-type gaussien qu'au moment où elle est réutilisée en amont d'un calcul Monte-Carlo aval (essai triaxial, page Covariance) — hypothèse de normalité introduite à ce stade-là uniquement, pas dans `uncertainty_band()` elle-même.
9. **`quantity_key` d'une entrée de bibliothèque de calibration (§3.5) est une étiquette purement indicative**, jamais validée contre le contenu réel de l'entrée : rien n'empêche d'enregistrer une contrainte sous `quantity_key="strain"`, et rien dans `core/calib_library_adapter.py` (chapitre 9, §7.4) ne le détecterait — seul le filtrage des menus déroulants de la page Bayesian Calibration s'appuie dessus.
10. **Tirages perdus non pondérés/non substitués** : un tirage Monte-Carlo qui échoue (exception dans le pipeline physique) est simplement retiré de la moyenne/écart-type (`n_used/n_total`) — pas de tentative de nouveau tirage, ce qui peut légèrement biaiser l'estimation si les échecs ne sont pas indépendants de la région de l'espace des paramètres tirée (ce point n'est pas discuté dans le code

lui-même, c'est une observation d'analyse).

11. **Deux niveaux de qualité coexistent pour l'incertitude triaxiale/confinement, sans que l'un remplace systématiquement l'autre** : le calcul réel par tirage (§3.4, chapitre 5 §5.8) n'est disponible que si le capteur de cellule de confinement a été recoupé par tirage (SHPB/Impact Direct, case Monte-Carlo cochée) ou provient de l'onglet QS (toujours actif, mais avec un signal de cellule fixe non varié — chapitre 5 §7 limitation 10) ; à défaut, l'application retombe silencieusement (avec message explicite) sur l'enveloppe synthétique de repli, qui ré-échantillonne une gaussienne indépendante plutôt que de propager la corrélation réelle. Les grandeurs dérivées de la vélocimétrie laser restent, elles, toujours calculées par la voie synthétique, quel que soit l'état des autres grandeurs pour ce même résultat.

2. Dépouillement Quasi-Statique (QS) et géométries d'éprouvette

Ce chapitre couvre le mode "Load/Displacement Data" (essai lent en machine d'essai classique, **pas** les barres de Hopkinson) et les géométries d'éprouvette communes à tous les modes de chargement de l'application.

Fichiers sources analysés :

- `core/qs_processing.py` — moteur de calcul QS (`process_qs()`)
- `core/models.py` — géométries d'éprouvette (`Specimen`, `SPECIMEN_FIELD_ORDER`)
- `core/machine_compliance.py` — correction de complaisance machine
- `core/k_value_io.py` — lecture/écriture des coefficients k/sk (éprouvette entaillée)
- `core/curve_cutting.py` — découpe du début de courbe
- `ui/pages/page_process.py`, classe `_QSTab` (≈ lignes 148-800) — pour le vocabulaire utilisateur et pour retrouver **exactement** quels arguments cette page transmet à `process_qs()` (voir §6, limitations)

---

1. Vue d'ensemble de la chaîne de calcul QS

Le module `core/qs_processing.py` indique explicitement dans son en-tête qu'il s'agit d'un **portage direct** de l'ancien code (`process_QS.py: process_QS()`), les formules physiques étant "inchangées" ("*The physics/formulas for each of the four test kinematics are unchanged from the original*").

La fonction centrale est :

```
process_qs(*, type_check, shear_check=0, tens_check=0, comp_check=0,
           displacement, load, timet, specimen, d,
           coeff_load, coeff_dd, t0,
           k_value=None, sk_value=None, sk_check=False) -> QSResult
```

Elle traite **un seul tirage Monte-Carlo `d`** ; l'appelant (la page UI, via `_process_qs_task` et `core/parallel.run_map`) boucle sur les `n` tirages et agrège les résultats (moyenne ± écart-type, voir `_aggregate_quantities()` dans `ui/pages/page_process.py`).

`type_check` sélectionne la cinématique de dépouillement (indépendamment de la géométrie détaillée d'éprouvette — voir §6) :

type_check	Mode
1	Compression (déformation vraie)
2	Tension (déformation vraie)
3	Éprouvette entaillée, Shear-Compression/Tension (coefficients k ajustés)
4	Cisaillement (hat-shaped, double ou simple)

La sortie est un objet `QSResult` (dataclass) contenant : `load`, `displacement` (signaux corrigés du facteur d'incertitude, voir §2.3), `stress`, `strain`, `strain_rate`, `temperature`, et — uniquement pour `type_check` 1 et 2 — `nominal_stress`, `nominal_strain`, `nominal_strain_rate`.

Chaîne amont (avant `process_qs`)

Avant d'appeler `process_qs()`, la page `_QSTab` (`ui/pages/page_process.py`, méthode `_on_run`, lignes ≈720-790) effectue dans cet ordre :

1. Chargement des signaux bruts `timet`, `load`, `disp` (fichiers texte, une valeur par ligne) ;
2. **Découpe optionnelle** du début de courbe (§2.1, appliquée en amont via le bouton "Cut curve beginning", donc *avant* `_on_run`, sur les tableaux `self._timet_raw/_load_raw/_disp_raw`) ;
3. **Correction de complaisance machine optionnelle** (§2.2), appliquée à `disp` juste avant l'appel ;
4. Génération des facteurs d'incertitude Monte-Carlo `coeff_load_arr`, `coeff_dd_arr` (§2.3) ;
5. Appel de `process_qs()` pour chaque tirage `d`.

---

2. Prétraitement des signaux

2.1 Découpe du début de courbe (`core/curve_cutting.py: cut_curve_start`)

Objectif indiqué dans la docstring : retirer une zone de "jeu"/"toe" initiale avant que l'éprouvette ne soit réellement en contact, et re-zéroter le déplacement.

Algorithme exact (`cut_curve_start(timet, load, displacement, cut_displacement)`) :

1. On cherche le **dernier** indice `idx` tel que `displacement[idx] < cut_displacement` (si aucun, `idx = 0`) ;
2. `timet`, `load`, `displacement` sont tronqués à partir de `idx` ;
3. Le déplacement est re-zéroté :

$$d'_i = d_i - d_{\text{idx}}$$

Le point de coupe est choisi interactivement par l'utilisateur sur un graphique charge-déplacement (`TimeWindowPicker`, carte "Cut curve beginning" dans `_QSTab`) ; seul le **début** de la fenêtre sélectionnée compte.

2.2 Correction de complaisance machine (`core/machine_compliance.py: apply_compliance_correction`)

Principe (docstring du module, portage de `process_t.py`, bloc `if corr_file.disp_comp_check == 1 and corr_file.load_comp_check == 1:`) : une éprouvette rigide de calibration (ou le bâti seul) est chargée sur la même plage de charge que l'essai réel ; son propre déplacement (souplesse/complaisance du bâti) est enregistré en fonction de la charge. Cette courbe charge→déplacement-machine sert ensuite à soustraire, à chaque niveau de charge de l'essai réel, la part de déplacement due au bâti lui-même.

Algorithme exact :

1. Les points de calibration `(calib_load, calib_displacement)` sont triés par charge croissante ;
2. Un interpolant **monotone PCHIP** (`scipy.interpolate.PchipInterpolator`, `extrapolate=True`) est construit sur ces points ;
3. Cet interpolant est évalué :
   - au signal de charge mesuré réel : $\delta_{\text{machine}}(F) = \text{PCHIP}(F)$
   - aux points de calibration eux-mêmes, pour obtenir une ligne de base $\delta_0 = \delta_{\text{machine}}(F_{\text{calib},0})$ (première valeur après tri, i.e. la plus petite charge de calibration)
4. Le déplacement machine est recentré : $\delta'_{\text{machine}}(F) = \delta_{\text{machine}}(F) - \delta_0$
5. Le déplacement mesuré est corrigé puis **re-zéroté à son propre minimum** :

$$d_{\text{corr}} = \left(d_{\text{mesuré}} - \delta'_{\text{machine}}(F)\right) - \min\left(d_{\text{mesuré}} - \delta'_{\text{machine}}(F)\right)$$

Cette fonction retourne le déplacement corrigé, qui remplace `disp` avant l'appel à `process_qs()` (les fichiers de calibration charge/déplacement sont stockés dans `project.qs.calib_load` / `project.qs.calib_displacement`).

2.3 Incertitudes de mesure Monte-Carlo (`coeff_load`, `coeff_dd`)

Ces facteurs sont générés **une fois par tirage Monte-Carlo** dans `ui/pages/page_process.py` (`_on_run`, lignes ≈766-768), pas dans `qs_processing.py` lui-même :

$$\text{coeff_load}_d \sim \mathcal{N}\left(1,\ \left(\frac{\text{clu\%}}{100}\right)^2\right), \qquad \text{coeff_dd}_d \sim \mathcal{N}\left(1,\ \left(\frac{\text{meu\%}}{100}\right)^2\right)$$

où `clu_pct` = "Cell Load Uncertainty (%)" et `meu_pct` = "Measurement Uncertainty (%) (displacement/extensometer)" sont saisis par l'utilisateur (carte "Measurement uncertainties" de `_QSTab`). Chaque tirage `d` reçoit donc un `coeff_load[d]` et un `coeff_dd[d]` propres, appliqués **multiplicativement** à la charge et au déplacement bruts dans `process_qs()` (voir toutes les formules ci-dessous, où `coeff_load * load[...]` et `coeff_dd * displacement[...]` apparaissent systématiquement). C'est ce mécanisme qui produit la bande d'incertitude ±1 écart-type affichée sur le graphique "Result".

---

3. Les quatre cinématiques de dépouillement (`type_check`)

3.1 Compression et Tension (`type_check == 1` ou `2`)

Code : `core/qs_processing.py`, bloc `if type_check in (1, 2):` (lignes 56-78).

Section résistante initiale `A0` ("section") :

$$A_0 = \begin{cases} \pi \left(\dfrac{D_0}{2} \right)^2 & \text{si } \texttt{tens_check} = 1 \text{ ou } \texttt{comp_check} = 1 \\ w_i \cdot th_i & \text{sinon} \end{cases}$$

avec `D_pa[d]` (diamètre), `wi_pa[d]` (largeur), `thi_pa[d]` (épaisseur) — champs Monte-Carlo de `Specimen`. **Attention**, voir la limitation détaillée en §6.1 : sur l'onglet QS, `comp_check`/`tens_check` valent toujours 1 dès que `type_check` vaut respectivement 1 ou 2, si bien que la branche "rectangulaire" (`wi_pa * thi_pa`) n'est en pratique **jamais** atteinte depuis cet onglet.

Déformation nominale (ingénieur) :

$$\varepsilon_{n,i} = \text{clamp}\left(\frac{\texttt{coeff_dd} \cdot d_i}{1000\, L_0} \right), \qquad L_0 = \texttt{specimen.L_pa[d]}$$

Le facteur 1000 au dénominateur suggère que le fichier de déplacement est attendu en **mm** alors que `L_pa` (longueur initiale) est en **m** — c'est une déduction à partir du code, non un commentaire explicite. La fonction `clamp` est `_clamp_nominal_strain()` (importée de `core/process_shpb.py`, partagée avec le dépouillement SHPB/DI) : elle borne chaque valeur à l'intervalle $[-0.99999,\ 0.99999]$ uniquement pour éviter un `math domain error` dans la conversion logarithmique qui suit (log(1-v) pour v→1, log(1+v) pour v→-1) ; d'après sa docstring, c'est un filet de sécurité qui ne modifie pas le résultat pour des valeurs déjà dans le domaine valide.

Contrainte nominale (ingénieur), en MPa (en supposant charge en N et longueurs en m) :

$$\sigma_{n,j} = \frac{\texttt{coeff_load} \cdot F_j}{A_0 \cdot 10^6}$$

Déformation vraie (logarithmique) :

$$\varepsilon_t = \begin{cases} -\ln(1 - \varepsilon_n) & \text{compression } (\texttt{type_check} = 1) \\ \ln(1 + \varepsilon_n) & \text{tension } (\texttt{type_check} = 2) \end{cases}$$

Contrainte vraie — formule implémentée (pas la formule "manuel" $\sigma_t = \sigma_n(1+\varepsilon_n)$, mais une correction paramétrée par le coefficient de Poisson du matériau `nu_pa[d]`) :

$$\sigma_t = \begin{cases} \sigma_n \left(1 - 2\,\nu\,\varepsilon_n\right) & \text{compression} \\ \sigma_n \left(1 + 2\,\nu\,\varepsilon_n\right) & \text{tension} \end{cases}$$

Aucun commentaire du code n'explicite l'origine physique de ce facteur $(1 \mp 2\nu\varepsilon_n)$; il est repris tel quel du code porté (`process_QS.py`). Signalé ici comme point à vérifier plutôt qu'interprété.

Vitesse de déformation (différences finies avant, vraie ET nominale) :

$$\dot{\varepsilon}_{t,i} = \frac{\varepsilon_{t,i+1} - \varepsilon_{t,i}}{t_{i+1} - t_i}, \qquad \dot{\varepsilon}_{n,i} = \frac{\varepsilon_{n,i+1} - \varepsilon_{n,i}}{t_{i+1} - t_i}$$

Les signaux de sortie `full_disp`/`full_load` sont simplement `coeff_dd·displacement` et `coeff_load·load` (le signal corrigé de l'incertitude, mais sans mise à l'échelle géométrique).

3.2 Éprouvette entaillée — Shear-Compression/Tension (`type_check == 3`)

Code : lignes 80-124. Cette cinématique n'utilise **pas** une formule directe contrainte = F/A, déformation = ΔL/L, mais un **polynôme de coefficients ajustés `k_value`** (± `sk_value`, voir §4). Cette branche exige une géométrie entaillée correctement configurée : `_require_notched_geometry(specimen, d)` (définie dans `core/process_shpb.py`, partagée avec SHPB/DI) lève une erreur explicite si `specimen.re_pa[d]` (rayon d'entaille) n'est pas strictement positif, pour éviter une division par zéro silencieuse — voir détail en §6.4.

Voir la description complète des équations en §5 (elles dépendent de `k_value`/`sk_value`, donc traitées avec eux).

3.3 Cisaillement (`type_check == 4`)

Code : lignes 125-136. `shear_check` distingue trois sous-géométries.

Hat-shaped (`shear_check == 1`) :

$$S_{as} = \pi\,(R_1 + R_2)\,(H_2 - H_1), \qquad l_0 = R_2 - R_1$$

Double cisaillement (`shear_check == 2`) ou **simple cisaillement** (toute autre valeur, en particulier `shear_check == 3`, ou la valeur par défaut `0`) :

$$S_{as} = n_{\text{plans}} \cdot th_i \cdot L_0, \qquad n_{\text{plans}} = \begin{cases} 2 & \texttt{shear_check} = 2 \text{ (double)} \\ 1 & \text{sinon (simple)} \end{cases} \qquad l_0 = w_i$$

Déformation de cisaillement (ingénieur) et **contrainte de cisaillement**, communes aux trois sous-géométries :

$$\gamma_i = \frac{\texttt{coeff_dd} \cdot d_i}{1000\, l_0}, \qquad \tau_j = \frac{\texttt{coeff_load} \cdot F_j / S_{as}}{10^6} \ \text{(MPa)}$$

$$\dot{\gamma}_i = \frac{\gamma_{i+1} - \gamma_i}{t_{i+1} - t_i}$$

Important : sur l'onglet QS (`_QSTab`), `shear_check` n'est **jamais** transmis explicitement dans le contexte d'appel (`qs_context` dans `ui/pages/page_process.py`, lignes ≈773-781) — voir §6.2. Il prend donc systématiquement sa valeur par défaut `0`, qui tombe dans la branche "simple cisaillement" ($n_{\text{plans}} = 1$) : le hat-shaped et le double cisaillement (facteur 2) ne sont, en l'état de ce code, **pas atteignables depuis l'onglet QS**.

3.4 Modèle de température adiabatique (Taylor-Quinney), commun aux 4 modes

Code : lignes 140-144, appliqué après le calcul de `t_s`/`t_stress` quel que soit `type_check`.

$$c_T = \frac{\xi}{c_p\,\rho} \qquad (\xi = \texttt{specimen.xi_pa[d]},\ c_p = \texttt{specimen.cp_pa[d]},\ \rho = \texttt{specimen.rho_pa[d]})$$

$$T_{i+1} = T_i + c_T \cdot \sigma_{t,i} \cdot 10^6 \cdot (\varepsilon_{t,i+1} - \varepsilon_{t,i}), \qquad T_0 = t_0\ (\texttt{project.thermal.initial_temperature_k})$$

Le facteur 10^6 reconvertit la contrainte (stockée en MPa dans `t_stress`) en Pa, pour que $c_T \cdot \sigma \cdot d\varepsilon$ soit homogène à une température (avec c_p en J/(kg·K) et ρ en kg/m³, $c_p\rho$ est en J/(m³·K), donc $\sigma\, d\varepsilon$ en Pa = J/m³ donne bien des kelvins). `xi` ("Taylor-Quinney Coefficient") est la fraction du travail plastique convertie en chaleur — hypothèse de transformation **adiabatique** (pas de commentaire dans le code justifiant cette hypothèse, mais elle est implicite au nom "Taylor-Quinney" donné dans le commentaire source : *"Adiabatic temperature-rise model (Taylor-Quinney), same for every kinematics"*).

---

4. Géométries d'éprouvette (`core/models.py`)

`SPECIMEN_FIELD_ORDER` (lignes 115-127) définit, pour chaque couple (mode, géométrie), l'ordre exact des champs attendus dans un fichier d'éprouvette — vérifié par les auteurs "ligne à ligne" contre `test_type.py: import_spec_file()`. Les géométries listées dans `COMPRESSION_GEOMETRIES`, `TENSION_GEOMETRIES`, `SHEAR_GEOMETRIES` (lignes 106-108) sont les seules proposées dans l'UI.

Pour chaque géométrie ci-dessous : paramètres (noms exacts du code), formule de section/aire résistante `A0` réellement utilisée par `qs_processing.py`, et particularités.

4.1 Cylindrique (Compression)

- Champs (`SPECIMEN_FIELD_ORDER[("Compression","Cylindrical")]`) : `rho, D, L, e, nu, xi, cp`
- Attributs `Specimen` : `D_pa` (diamètre, m), `L_pa` (longueur initiale, m), `nu_pa`, `xi_pa`, `cp_pa`, `rho_pa`, `e_pa`
- $A_0 = \pi(D_{\text{pa}}/2)^2$ — voir §3.1 (branche `comp_check=1`)
- Déformation vraie de compression $\varepsilon_t = -\ln(1-\varepsilon_n)$, avec $\varepsilon_n = \Delta L/L_0$ (mise à l'échelle mm→m par /1000)

- **Section constante** dans le calcul (`A0` n'évolue pas avec la déformation ; seule la conversion nominale→vraie change, il n'y a pas de recalcul de section "vraie" du type $A = A_0/(1-\varepsilon_n)$ dans ce fichier)

4.2 Carrée/Rectangulaire (Compression)

- Champs : `rho, thickness, L, width, e, nu, xi, cp`
- Attributs : `thi_pa` (épaisseur), `L_pa`, `wi_pa` (largeur)
- $A_0 = w_i \cdot th_i$ (branche "sinon" de la formule §3.1)
- **Non atteignable depuis l'onglet QS** en l'état actuel du code (voir §6.1) : `comp_check` y vaut toujours 1, ce qui force la formule cylindrique $\pi(D/2)^2$ même si la géométrie configurée est "Square/Rectangular". `D_pa` n'étant alors jamais renseigné pour une éprouvette carrée, cela peut produire une section nulle/incorrecte.

4.3 Axisymétrique (Tension)

- Champs : `rho, D, L, e, nu, xi, cp`
- $A_0 = \pi(D_{\text{pa}}/2)^2$ (branche `tens_check=1`, §3.1)
- Déformation vraie de tension $\varepsilon_t = \ln(1+\varepsilon_n)$

4.4 Plate (Tension, "Flat")

- Champs : `rho, thickness, L, width, e, nu, xi, cp`
- $A_0 = w_i \cdot th_i$
- Même remarque qu'en 4.2 : sur l'onglet QS, `tens_check` vaut toujours 1 dès que `type_check=2`, donc cette branche rectangulaire n'est en pratique **jamais utilisée** par l'onglet QS non plus (voir §6.1).

4.5 Éprouvette entaillée (Shear-Compression/Tension, Notched)

- Champs (pas de sous-géométrie, clé `None`) : `rho, D, e, nu, xi, cp, effective_zone, disp_yield, notch_radius, total_length, notch_angle`
- Attributs `Specimen` : `D_pa` (diamètre), `th_pa` ("effective zone thickness"), `re_pa` (rayon d'entaille, "notch_radius"), `NoA_pa` (angle d'entaille, "notch_angle", unité déclarée **deg** dans `Specimen.new()`), `dy_pa` (déplacement au seuil de plasticité, "disp_yield")
- Pas de formule "A0 = …" simple ici : la contrainte et la déformation sont calculées par un ajustement polynomial en fonction de `k_value` (voir §5), utilisant `re_pa`, `NoA_pa`, `th_pa`, `D_pa`, `dy_pa`
- Le champ `total_length` existe dans `SPECIMEN_FIELD_ORDER` mais **n'a pas d'attribut Monte-Carlo dédié dans `Specimen`** pour cette géométrie précise (aucun champ `*_pa` de `Specimen` ne lui correspond explicitement pour le mode Notched — à ne pas confondre avec `leL_pa`, qui est documenté comme spécifique au cisaillement double/simple). Ce paramètre n'apparaît nulle part dans `qs_processing.py` ; son usage éventuel ailleurs dans le code n'a pas été vérifié dans le cadre de cette étude.

4.6 Hat-shaped (Cisaillement)

- Champs : `rho, e, nu, xi, cp, H1, H2, H3, R1, R2, R3, total_length`

- Attributs utilisés par `qs_processing.py` : `R1_pa`, `R2_pa`, `H1_pa`, `H2_pa` uniquement — voir §3.3 : $S_{as} = \pi(R_1 + R_2)(H_2 - H_1)$, $l_0 = R_2 - R_1$
- **`H3_pa`, `R3_pa` et `total_length` ne sont pas utilisés par `qs_processing.py`.** Une recherche dans le reste du code montre que `H3_pa`/`R3_pa` (et `leL_pa`, `wiL_pa`) interviennent bien dans `core/process_shpb.py` et `core/process_di.py` (calcul d'une section équivalente / d'un rapport géométrique `s0` pour le dépouillement par ondes SHPB/Impact Direct), mais **pas** dans le dépouillement quasi-statique.

4.7 Double cisaillement

- Champs : `rho, thickness, L, width, e, nu, xi, cp, head_width, total_length`
- Attributs utilisés par `qs_processing.py` : `thi_pa`, `L_pa`, `wi_pa` (voir §3.3) : $S_{as} = 2\,th_i\,L_0$, $l_0 = w_i$
- `head_width` (→ `wiL_pa`) et `total_length` (→ `leL_pa`) existent dans `Specimen` mais ne sont **pas** utilisés par `qs_processing.py` (utilisés ailleurs, dans `process_shpb.py`/`process_di.py`, pour un calcul de section équivalente propre au dépouillement par ondes).
- Non atteignable seul (avec le facteur ×2) depuis l'onglet QS, voir §3.3 et §6.2.

4.8 Simple cisaillement

- Mêmes champs que 4.7
- $S_{as} = th_i\,L_0$ (un seul plan de cisaillement), $l_0 = w_i$
- Mêmes remarques que 4.7 pour `head_width`/`total_length`
- C'est la **seule** sous-géométrie de cisaillement réellement atteinte par l'onglet QS (par défaut, `shear_check` non transmis → branche "sinon" avec facteur 1), voir §6.2.

---

5. Coefficients k / sk pour éprouvette entaillée

5.1 Format de fichier (`core/k_value_io.py`)

- `load_k_value_file(path)` lit un fichier texte, une valeur par ligne (lignes vides ignorées), et retourne `(k_value, sk_check, sk_value)`.
- 6 lignes obligatoires : `k_value[0..5]` (`N_K = 6`).
- Optionnellement, une ligne `Split = True`/`Split = False` (regex insensible à la casse `split\s*=\s*(true|false)`) suivie de 7 lignes `sk_value[0..6]` (`N_SK = 7`).
- Deux formats acceptés : 6 lignes seules (pas de split — `sk_check` et `sk_value` reviennent alors à `None`, **pas** une valeur par défaut choisie arbitrairement, pour que l'appelant ne touche pas aux réglages déjà affichés à l'écran), ou 14 lignes complètes (6+1+7). Un nombre de lignes intermédiaire (7 à 13) est traité comme une erreur explicite ("partial split/sk section").
- `save_k_value_file()` écrit toujours le format complet à 14 lignes.

5.2 Rôle cinématique de `k_value` (sans split, `sk_check` faux)

Code exact (`core/qs_processing.py`, lignes 90-102), avec $d_i =$ `displacement`$[i]$ (brut) :

$$u_i = \frac{\texttt{coeff_dd} \cdot d_i - dy}{\text{denom}}, \qquad \text{denom} = \frac{2000 \cdot r_e}{\cos(\theta_{\text{notch}})}$$

(r_e = `specimen.re_pa[d]`, θ_{notch} = `specimen.NoA_pa[d]`, dy = `specimen.dy_pa[d]` = déplacement au seuil de plasticité)

$$\varepsilon_{t,i} = k_0\, u_i + k_1\, u_i^2 + k_2\, u_i^3$$

$$\sigma_{t,i} = \frac{k_3\,(1 - k_4\,\varepsilon_{t,i}) \cdot \left(\texttt{coeff_load} \cdot |F_i|\right)^{k_5}}{1000 \cdot th \cdot 1000 \cdot D}$$

(th = `specimen.th_pa[d]`, D = `specimen.D_pa[d]`, tous les deux en m ; le produit $1000\,th \cdot 1000\,D$ équivaut donc à $th_{\text{mm}} \cdot D_{\text{mm}}$).

Remarques explicitement portées par le code (commentaire lignes 84-86) : le décalage - `dy` (déplacement au seuil de plasticité) **n'intervient que dans la branche sans split** (`sk_check` faux) — un commentaire précise que cet offset est appliqué dans cette branche précisément pour correspondre au comportement de `process_QS.py`.

Point non documenté par le code : l'exposant `k_value[5]` appliqué à `|F_i|` rend la relation contrainte–charge **non linéaire** (une loi puissance), contrairement à une simple contrainte = force / section ; la justification physique de cette loi n'apparaît dans aucun commentaire du fichier et n'a pas été retrouvée ailleurs dans les fichiers examinés — à confirmer auprès de l'auteur original si besoin.

5.3 Mécanisme de "split fit" (`sk_check` vrai)

Code exact (lignes 103-118) : pour **chaque échantillon `i`**, le jeu de coefficients utilisé est choisi individuellement selon un seuil de déplacement **brut** (non corrigé par `coeff_dd`) comparé à `sk_value[6]` :

$$\text{coeffs}_i = \begin{cases} k_value & \text{si } d_i \le \texttt{sk_value[6]} \\ sk_value & \text{si } d_i > \texttt{sk_value[6]} \end{cases}$$

Avec ces coefficients (notés génériquement $k_0..k_5$ ci-dessus, pris dans `k_value` ou `sk_value` selon le cas), la formule de déformation **ne soustrait pas `dy`** dans cette branche :

$$u_i = \frac{\texttt{coeff_dd} \cdot d_i}{\text{denom}}, \qquad \varepsilon_{t,i} = k_0\, u_i + k_1\, u_i^2 + k_2\, u_i^3$$

et la contrainte suit exactement la même forme qu'en §5.2 (avec les coefficients du jeu choisi pour l'échantillon `i`).

Point à signaler explicitement (ambiguïté potentielle) : le seuil de comparaison utilise le déplacement **brut** `dd[i]` alors que la formule de déformation elle-même utilise le déplacement **corrigé** `coeff_dd *`

`dd[i]`. Le code ne commente pas ce choix ; il peut être intentionnel (seuil physique fixe indépendant de l'incertitude Monte-Carlo) ou être une incohérence mineure héritée de l'original — non tranché ici, signalé tel quel.

Dans les deux branches (split ou non), le résultat final est **re-zéroté** après coup, quel que soit le chemin emprunté :

$$\varepsilon_{t,i} \leftarrow \varepsilon_{t,i} - \varepsilon_{t,0}$$

(ligne 120, appliquée après le `if/else`, donc y compris pour le cas "split").

5.4 Cohérence avec la géométrie "entaillée" (§4.5)

`_require_notched_geometry()` (importée de `core/process_shpb.py`, réutilisée telle quelle par `process_qs`) refuse explicitement de démarrer le calcul si `specimen.re_pa[d]` n'est pas strictement positif (`> 0`), avec un message clair, précisément pour éviter que `denom` (dénominateur ci-dessus) ne soit nul. Le commentaire de cette fonction explique le scénario d'erreur visé : la cinématique (`type_check`) n'est **pas** liée au mode de test global du projet côté onglet QS (voir §6.1) — si le menu déroulant "Kinematics" de l'onglet QS est resté sur "3 — Notched" pour un essai différent (compression simple, sans rayon d'entaille configuré), sans cette vérification le calcul produirait une cascade de `RuntimeWarning` "divide by zero"/"invalid value" au lieu d'un message d'erreur clair ; la vérification intercepte ce cas et affiche un message explicite.

---

6. Limitations, hypothèses et ambiguïtés explicitement relevées

6.1 Sur l'onglet QS, la géométrie détaillée (cylindrique vs carrée/rectangulaire, axisymétrique vs plate) n'est pas prise en compte

Dans `ui/pages/page_process.py`, `_QSTab._on_run()` (lignes ≈773-781), le contexte transmis à `process_qs()` fixe :

```
comp_check=1 if type_check == 1 else 0,
tens_check=1 if type_check == 2 else 0,
```

c'est-à-dire que `comp_check`/`tens_check` dépendent **uniquement** du menu "Kinematics" (Compression/Tension/Notched/Sh jamais de la géométrie réellement configurée pour l'éprouvette (`project.geometry`, "Cylindrical"/"Square-Rectangular" ou "Axisymmetric"/"Flat"). Or dans `process_qs()`, c'est précisément `comp_check`/`tens_check` qui choisit la formule de section (§3.1) : la branche $\pi(D/2)^2$ est prise dans **tous** les cas de compression/tension sur cet onglet, la branche $w_i th_i$ (rectangulaire/plate) n'étant jamais atteinte depuis lui. Ceci contraste avec les onglets SHPB et Direct Impact du même fichier, qui dérivent bien `comp_check`/`tens_check` de `project.geometry` (ex. ligne 1598 : `comp_check=1 if project.geometry == "Cylindrical" else 2`).

Conséquence pratique : pour dépouiller un essai quasi-statique sur une éprouvette carrée/rectangulaire ou plate avec ce code, `D_pa` (non configuré pour ces géométries) serait utilisé au lieu de `wi_pa`/`thi_pa`, ce qui

produirait une section incorrecte (potentiellement nulle). Ce point mérite vérification/confirmation auprès des développeurs — il pourrait s'agir d'une limitation connue et acceptée de l'onglet QS (peut-être prévu pour n'être utilisé qu'avec des éprouvettes cylindriques/ axisymétriques) plutôt que d'un bug, mais rien dans le code ou les textes d'aide de l'UI ne le dit explicitement.

6.2 Sur l'onglet QS, le sous-type de cisaillement n'est pas sélectionnable

Le `dict qs_context` construit par `_QSTab._on_run()` ne contient **pas** de clé `shear_check` (contrairement aux onglets SHPB/DI, qui la dérivent de `project.geometry` via `shear_map = {"Hat-shaped": 1, "Double shear": 2, "Simple shear": 3}`, lignes 1604-1605 et 2838-2839). `process_qs()` utilise alors sa valeur par défaut `shear_check=0`, qui — d'après la logique de la branche `type_check==4` (§3.3) — tombe systématiquement dans le cas "simple cisaillement" (facteur 1, $l_0 = w_i$). Le hat-shaped et le double cisaillement (facteur 2) sont donc, en l'état actuel du code, **inaccessibles depuis l'onglet QS**, même si l'utilisateur a configuré une éprouvette "Hat-shaped" ou "Double shear" sur la page de configuration d'éprouvette.

6.3 Angle d'entaille : unité « deg » déclarée, mais utilisée directement dans `cos()`

`Specimen.new()` (`core/models.py`, ligne 189) déclare le paramètre `notch_angle` avec l'unité `"deg"`. Or `core/qs_processing.py` (ligne 88, identique dans `process_shpb.py` et `process_di.py`) calcule `math.cos(specimen.NoA_pa[d])` **sans aucune conversion degrés→radians** visible dans le code (recherché également dans `core/mc.py`, qui gère l'échantillonnage Monte-Carlo des paramètres, sans y trouver de conversion d'unité). `math.cos()` de Python attend un argument en **radians**. Ceci est signalé comme une ambiguïté/incohérence potentielle entre l'étiquette d'unité affichée à l'utilisateur et le calcul réellement effectué — non résolue dans le cadre de cette étude ; à vérifier si l'utilisateur est en réalité censé saisir l'angle en radians malgré le libellé "deg", ou s'il s'agit d'un bug hérité du code original.

6.4 Champs géométriques déclarés mais non utilisés en QS

Comme détaillé en §4.5, 4.6, 4.7 : `total_length` (notched), `H3`, `R3`, `total_length` (hat-shaped), `head_width`, `total_length` (double/simple cisaillement) sont tous présents dans `SPECIMEN_FIELD_ORDER` et dans les attributs `Specimen` correspondants, mais **aucun n'est lu par `core/qs_processing.py`**. `H3_pa`/`R3_pa`/`wiL_pa`/`leL_pa` sont en revanche bien utilisés dans `core/process_shpb.py` et `core/process_di.py` pour un calcul de section équivalente propre au dépouillement par ondes — ils ne concernent donc pas la chaîne quasi-statique.

6.5 Absence de recalcul de section en grande déformation

Aucune des quatre cinématiques (`core/qs_processing.py`) ne recalcule la section résistante `A0` en fonction de la déformation courante (pas de correction de type striction/section réduite). `A0` est calculée une seule fois à partir des dimensions initiales de l'éprouvette (`specimen.*_pa[d]`) et reste constante sur toute la courbe pour un tirage Monte-Carlo donné ; seule la conversion nominal→vrai (logarithmique, §3.1) tient compte de l'évolution de la déformation.

6.6 Modèle de température : hypothèse adiabatique non justifiée dans le code

Comme noté en §3.4, l'hypothèse d'adiabaticité (aucune perte thermique pendant l'essai) sous-tend le modèle de Taylor-Quinney appliqué, mais n'est étayée par aucun commentaire explicite au-delà du nom donné à la formule dans le commentaire source.

---

7. Récapitulatif fichier / fonction

Calcul	Fichier	Fonction / méthode
Orchestration QS (1 tirage MC)	`core/qs_processing.py`	`process_qs()`
Worker multi-process	`core/qs_processing.py`	`_process_qs_task()`
Découpe début de courbe	`core/curve_cutting.py`	`cut_curve_start()`
Correction complaisance machine	`core/machine_compliance.py`	`apply_compliance_correction()`
Garde-fou domaine log (déformation vraie)	`core/process_shpb.py`	`_clamp_nominal_strain()` (importée)
Garde-fou géométrie entaillée	`core/process_shpb.py`	`_require_notched_geometry()` (importée)
Lecture fichier k/sk	`core/k_value_io.py`	`load_k_value_file()`
Écriture fichier k/sk	`core/k_value_io.py`	`save_k_value_file()`
Définition géométries/spécimen	`core/models.py`	`Specimen`, `Specimen.new()`, `Specimen.expand()`, `SPECIMEN_FIELD_ORDER`
Génération incertitudes MC (coeff_load/coeff_dd)	`ui/pages/page_process.py`	`_QSTab._on_run()`
Construction du contexte d'appel `process_qs`	`ui/pages/page_process.py`	`_QSTab._on_run()` (variable `qs_context`)
Agrégation moyenne/écart-type multi-tirages	`ui/pages/page_process.py`	`_aggregate_quantities()`

3. Dépouillement SHPB (barres de Hopkinson classiques à deux barres)

> Fichier source principal : `core/process_shpb.py` (fonction `process_shpb()`, 834 lignes). Fichiers connexes : `core/toe_correction.py`, `core/wave_alignment.py`, `core/specimen_sensor_alignment.py`. Vocabulaire utilisateur (cartes, cases à cocher) : `ui/pages/page_process.py`, classe `_SHPBTab`.

Avertissement du code lui-même (à reproduire tel quel)

Le docstring de module de `core/process_shpb.py` (lignes 1-27) précise :

> « Ported as literally as possible (same variable names, same branch order, same formulas) specifically to minimize transcription risk in code this dense — readability was sacrificed for traceability against the original on purpose. […] **NOT YET independently re-validated against real experimental data or the original GUI's output for the full matrix of options (heat × platen × punch × friction × inertia × viscoelastic bars × 4 kinematics)** — only exercised with synthetic signals on the simplest option combination. Cross-check important results against the original application before relying on this for a real analysis, especially with several corrections enabled at once. »

Cet avertissement doit être répercuté dans la documentation utilisateur : le moteur de calcul n'a pas encore été validé indépendamment sur données réelles pour toutes les combinaisons d'options.

---

1. Principe physique et chaîne de calcul générale

1.1 Hypothèses classiques

Le dispositif est composé de deux barres élastiques 1D (barre d'entrée *Input Bar*, IB, et barre de sortie *Output Bar*, OB) instrumentées de jauges de déformation, encadrant l'éprouvette. L'hypothèse fondamentale de la théorie des ondes 1D (propagation sans dispersion dans une barre élastique semi-infinie) permet de relier la déformation de jauge à la force et à la vitesse particulaire au niveau de l'interface barre/éprouvette :

$$F(t) = E\,A_{barre}\,\varepsilon(t), \qquad v(t) = \pm C_0\,\varepsilon(t), \qquad C_0 = \sqrt{E/\rho}$$

où le signe dépend du sens de propagation de l'onde. Le code ne fait jamais explicitement référence à « l'équilibre dynamique » de l'éprouvette dans les commentaires, mais toute la construction (trois signaux indépendants ε_i, ε_r, ε_t, plus les indicateurs `balance_quality_*`, voir §1.6) suit la démarche classique dite **« à trois ondes »** : force et vitesse sont calculées indépendamment aux deux interfaces (entrée et sortie), sans supposer a priori l'égalité $F_{input} = F_{output}$; l'utilisateur vérifie ensuite cet équilibre a posteriori via les indicateurs de qualité d'équilibrage.

1.2 Convention exacte utilisée dans le code — vérifiée ligne à ligne

D'après `process_shpb()` (`core/process_shpb.py`, toutes les branches de kinématique, ex. lignes 502-509 pour compression/traction) :

$$F_{input}(t) = \text{signe} \cdot E_{mat,IB}\,\pi \left(\frac{D_{B,IB}}{2}\right)^2 \left(\varepsilon_i(t) + \varepsilon_r(t)\right) k_{IB}$$

$$F_{output}(t) = \text{signe} \cdot E_{mat,OB}\,\pi \left(\frac{D_{B,OB}}{2}\right)^2 \varepsilon_t(t)\, k_{OB}$$

$$F_{average}(t) = \frac{F_{input}(t) + F_{output}(t)}{2}$$

$$V_{input}(t) = \text{signe} \cdot C0_{mat,IB}\left(\varepsilon_i(t) - \varepsilon_r(t)\right) k_{IB}$$

$$V_{output}(t) = \text{signe} \cdot C0_{mat,OB}\,\varepsilon_t(t)\, k_{OB}$$

où k_{IB} = `IB.CPG_pa[d]` et k_{OB} = `OB.CPG_pa[d]` sont les **coefficients de jauge de déformation** (« SG_coeff », alias historique `CPG`, voir `core/models.py` ligne 44 et 81-85), et $E_{mat,\cdot}/C0_{mat,\cdot}$ sont le module et la célérité « de référence » à l'interface (§2 : identiques aux propriétés ambiantes de la barre sauf correction de gradient thermique).

C'est donc exactement la formulation **« à trois ondes »** classique : - F_{input} utilise les **deux ondes** côté entrée ($\varepsilon_i + \varepsilon_r$, formule dite « 2 ondes »), - F_{output} utilise la **seule onde transmise** (formule dite « 1 onde »), - la vitesse de déformation nominale (voir §1.4) combine les vitesses aux deux interfaces, donc les **trois ondes** $\varepsilon_i, \varepsilon_r, \varepsilon_t$ simultanément.

Le choix de la contrainte finale (§3) sélectionne ensuite F_{input} **OU** F_{output} (jamais $F_{average}$, qui n'est renvoyé que pour diagnostic/tracé dans `SHPBResult.force_average`).

Le signe (`sign`) vaut : - `type_check in (1,2)` (compression/traction) : `sign = -1` en compression, `+1` en traction (ligne 503) ; - `type_check == 3` (cisaillement-compression/traction entaillé) et `type_check == 4` (cisaillement) : `sign` dépend de `options.tenscomp_check` (`-1` si « compression-like », `+1` si « tension-like », lignes 385 et 612) — **pas** de `type_check` directement dans ces deux cas.

1.3 Découpe temporelle des trois ondes (« Cutting waves »)

Avant toute conversion, `process_shpb()` découpe les trois fenêtres temporelles dans les signaux bruts des barres (lignes 240-287) :

- Vitesses d'onde du matériau éprouvette et (si activé) du platen :

$$C0_{\text{éprouvette}} = \sqrt{E_{\text{éprouvette}}/\rho_{\text{éprouvette}}}, \qquad C0_{platen} = \sqrt{E_{platen}/\rho_{platen}} \text{ (si Platens activé)}$$

- Fenêtre incidente : indices `idx_i1`, `idx_i2` = position de `xd1`, `xd2` (bornes choisies par l'utilisateur sur le tracé) dans le vecteur temps, par recherche du plus proche échantillon inférieur (`np.searchsorted(..., side="right") - 1`). Longueur de fenêtre : `ti_pulse = idx_i2 - idx_i1`.
- Fenêtre réfléchie, décalée du temps d'aller-retour jauge → extrémité de barre → jauge :

$$t_{r,1} = x_{d1} + \frac{2\,PG_{IB}}{C0_{IB}} \left(+ \frac{2\,L_{platen}}{C0_{platen}} \text{ si Platens} \right)$$

  où PG_{IB} = `IB.PG_pa[d]`, distance jauge-extrémité de barre côté éprouvette.
- Fenêtre transmise, décalée du temps de trajet jauge d'entrée → éprouvette → jauge de sortie :

$$t_{t,1} = x_{d1} + \frac{PG_{OB}}{C0_{OB}} + \frac{L_{\text{éprouvette}}}{C0_{\text{éprouvette}}} + \frac{PG_{IB}}{C0_{IB}} \left(+ \frac{2\,L_{platen}}{C0_{platen}} \text{ si Platens} \right) + \Delta t_{offset}$$

  où Δt_{offset} = `output_window_offset`, un décalage manuel de réalignement (résiduel, typiquement produit par l'outil d'alignement des ondes, §5).

Les trois tableaux `incident`, `reflected`, `transmitted` sont extraits des signaux bruts (`IB.signal`, `OB.signal`) puis multipliés par `SG_unc_d[d]`, un facteur d'incertitude Monte-Carlo distinct du coefficient de jauge `CPG_pa` proprement dit (les deux sont appliqués successivement : `SG_unc_d[d]` ici, puis `CPG_pa[d]` plus loin dans les formules de force/vitesse et à nouveau explicitement en fin de bloc pour produire les ondes finales de `SHPBResult`). **Le sens exact de cette double multiplication (nature physique de `signal` avant application de `CPG_pa`, vs. rôle de `SG_unc_d`) n'est pas explicité dans les commentaires du code** — signalé ici comme point à clarifier plutôt qu'interprété.

Si l'option « Utiliser le signal corrigé en dispersion » (`disp_ela` ou `disp_visco`) est active, ces trois tableaux sont écrasés par des tableaux déjà découpés et corrigés en amont (`AA[0]`, `AA[1]`, `AA[2]`, provenant vraisemblablement de `core/dispersion.py`) ; le décalage manuel `output_window_offset` est alors ré-appliqué comme un décalage d'échantillons équivalent sur la seule onde transmise (lignes 275-287).

1.4 Vitesse de déformation, déformation nominale, déformation vraie (cas compression/traction)

Pour `type_check in (1, 2)` (`core/process_shpb.py`, lignes 502-604) :

$$\dot{\varepsilon}_{nom}(t) = \frac{V_{input}(t) - V_{output}(t)}{L_{\text{éprouvette}}}$$

Déformation nominale par intégration explicite (règle des rectangles, intégration d'Euler avant) :

$$\varepsilon_{nom}(t_{i+1}) = \varepsilon_{nom}(t_i) + \dot{\varepsilon}_{nom}(t_i)\,(t_{i+1} - t_i), \qquad \varepsilon_{nom}(t_0) = 0$$

(code : boucle `int_sr += n_sr[i]*(time1[i+1]-time1[i])`, lignes 524-528), avec un **écrêtage de sécurité** à chaque pas dans $[-0.99999,\ 0.99999]$ (voir §1.7).

Déplacement relatif (en mm) : $u(t_i) = 1000\, L_{\text{éprouvette}}\, \varepsilon_{nom}(t_i)$.

Déformation vraie (logarithmique), calculée séparément (et non simplement dérivée de ε_{nom} via une formule générique unique — le code bifurque explicitement) :

$$\varepsilon_{true} = -\ln(1 - \varepsilon_{nom}) \quad \text{(compression)}, \qquad \varepsilon_{true} = \ln(1 + \varepsilon_{nom}) \quad \text{(traction)}$$

1.5 Contrainte nominale et contrainte vraie (cas compression/traction)

Section de l'éprouvette :

$S = \pi(D_{\text{éprouvette}}/2)^2$ (cylindrique/axisymétrique), $S = w_i\, e_i$ (carrée/plate, avec $D_{eq} = 2\sqrt{S/\pi}$)

Contrainte nominale (ingénieur), à partir de la force choisie F_F (voir §3) :

$$\sigma_{nom}(t) = \frac{F_F(t)}{S} \times 10^{-6} \quad \text{[MPa]}$$

Contrainte vraie, **correction de Poisson linéaire** (pas d'hypothèse d'incompressibilité stricte) :

$$\sigma_{true} = \sigma_{nom}\,(1 - 2\nu\,\varepsilon_{nom}) \quad \text{(compression)}, \qquad \sigma_{true} = \sigma_{nom}\,(1 + 2\nu\,\varepsilon_{nom}) \quad \text{(traction)}$$

où ν = `specimen.nu_pa[d]` (coefficient de Poisson de l'éprouvette).

1.6 Indicateurs de qualité d'équilibrage (« balance quality »)

Fonction `_balance_quality()` (lignes 746-769), partagée par les trois blocs de kinématique. Trois indicateurs sont calculés :

- **`balance_quality_force`** (bq_1) : rapport des travaux cumulés (intégrale force-déplacement) entrée/sortie —
$$bq_1(t) = \frac{\int F_{output}\, du}{\int F_{input}\, du}$$
(intégration par rectangles cumulés sur le déplacement).
- **`balance_quality_waves`** (bq_2) : rapport des intégrales temporelles des ondes —
$$bq_2(t) = \frac{\int \varepsilon_t\, dt}{\int (\varepsilon_i + \varepsilon_r)\, dt}$$

- **`balance_quality_force_instant`** (bq_3) : rapport instantané point à point —

$$bq_3(t) = \frac{F_{output}(t)}{F_{input}(t)}$$

Ces trois quantités quantifient l'écart à l'hypothèse d'équilibre dynamique ($F_{input} \approx F_{output}$, ou de façon équivalente $\varepsilon_i + \varepsilon_r \approx \varepsilon_t$) ; elles ne modifient pas le résultat, elles servent uniquement de diagnostic.

1.7 Sécurités numériques (pas des corrections physiques)

- `_clamp_nominal_strain()` (lignes 115-133) : écrête la déformation nominale à $[-0.99999,\ 0.99999]$ avant tout passage au logarithme (`-\ln(1-\varepsilon)` en compression, `\ln(1+\varepsilon)` en traction), uniquement du côté où la formule diverge, pour éviter un crash `math domain error` sur toutes les tirages Monte-Carlo à la fois plutôt que de produire une valeur finie (bien qu'extrême) que l'utilisateur peut juger.
- `_robust_dt()` (lignes 192-217) : pas de temps utilisé pour les dérivées de vitesse de déformation, calculé comme la **médiane** des écarts consécutifs de `time1`, ce qui le rend robuste au cas où deux échantillons consécutifs partagent le même horodatage (un simple `time1[2]-time1[1]` serait fragile dans ce cas).

---

2. Variantes de kinématique du même moteur de calcul

`process_shpb()` traite quatre types d'essai (`options.type_check`), choisis via le « Test mode » de la page Test Setup (pas par onglet de traitement individuel — `_require_notched_geometry`, lignes 136-163, avertit explicitement que changer le mode de test affecte tous les onglets déjà traités).

2.1 Compression / traction (`type_check = 1 ou 2`)

Détaillé au §1. C'est le cas « barres de Hopkinson classiques » au sens strict.

2.2 Cisaillement-compression/traction entaillé — « Notched » (`type_check == 3`, lignes 383-497)

Force/vitesse identiques en forme aux formules du §1.2 (mêmes expressions, `sign` piloté par `tenscomp_check`). En revanche :

- **Déplacement** intégré directement en mm à partir de la différence de vitesse, sans passer par une déformation intermédiaire :

$$u(t_{i+1}) = u(t_i) + \left(V_{input}(t_i) - V_{output}(t_i)\right) dt \times 1000$$

- **« Déformation »** ε_{notch} obtenue par un **polynôme d'étalonnage cubique en déplacement normalisé**, propre à la géométrie de l'éprouvette entaillée, avec coefficients utilisateur `k_value[0..5]`

(chargeables depuis un fichier, carte « Notched specimen k-coefficients ») :

$$d_{denom} = \frac{2000\, r_e}{\cos(\theta_{encoche})}, \qquad x = \frac{u - 1000\, d_y}{d_{denom}}$$

$$\varepsilon_{notch} = k_0\, x + k_1\, x^2 + k_2\, x^3$$

$$\sigma_{notch} = \frac{k_3\,(1 - k_4\,\varepsilon_{notch})\,|F_F|^{k_5}}{1000\, e_{zone} \times 1000\, D_{\text{éprouvette}}}$$

où r_e=`specimen.re_pa[d]` (rayon d'entaille), $\theta_{encoche}$=`specimen.NoA_pa[d]` (angle d'entaille), d_y=`specimen.dy_pa[d]` (déplacement au seuil élastique), e_{zone}=`specimen.th_pa[d]` (épaisseur de la zone effective). Si l'option « Split fit at a displacement threshold (sk) » (`options.sk_check`) est activée, le jeu de coefficients bascule de `k_value` à `sk_value` dès que $u > $ `sk_value[6]`.

- Ni déformation nominale ni contrainte nominale distinctes : `n_s = t_s`, `n_stress = t_stress` (alias explicites, lignes 487-497). La vitesse de déformation `t_sr` est calculée par différence finie directe (pas `_robust_dt`, mais `time1[i+1]-time1[i]` à chaque pas).

2.3 Cisaillement pur — « Shear » (`type_check == 4`, lignes 609-723)

Longueur utile l_u = `H2_pa - H1_pa` (éprouvette « chapeau »/hat) ou `wi_pa` (double/simple). Formules de force/vitesse identiques en forme au §1.2 (`sign` de `tenscomp_check`), $L_{\text{éprouvette}}$ remplacé par l_u pour la vitesse de déformation et le déplacement.

Section de cisaillement effective S_{AS} selon la géométrie (`options.shear_check`) :

$$S_{AS} = \pi(R_1 + R_2)(H_2 - H_1) \text{ (hat)}, \quad S_{AS} = 2\, e_i\, L \text{ (double)}, \quad S_{AS} = e_i\, L \text{ (simple)}$$

$$\sigma_{nom} = \sigma_{true} = \frac{F_F}{S_{AS}} \times 10^{-6}, \qquad \varepsilon_{true} = \varepsilon_{nom} \quad \text{(pas de transformation logarithmique)}$$

C'est confirmé indépendamment par `core/toe_correction.py::infer_true_strain_relation()` (voir §4), dont le docstring énonce explicitement : « shear: true = nominal (SHPB shear processing uses no log transform at all — see process_shpb.py, `t_s = list(n_s)`) ».

---

3. Choix de la force utilisée pour la contrainte (`force_check`)

Un seul et même paramètre `options.force_check` (liste déroulante « Force used for stress » sur la page Processing, valeur par défaut = sortie) pilote, **dans chacune des trois branches de kinématique**, quelle force sert à calculer la contrainte nominale, via un petit sélecteur partagé (`_select_stress_force()`, lignes 84-95) :

```
def _select_stress_force(F_input, F_output, F_average, force_check: int):
    if force_check == 1:
        return F_input
    if force_check == 3:
```

```
        return F_average
    return F_output

# ... dans chacune des trois branches (ex. ligne 420 pour le cas entaillé,
# ligne 547 pour compression/traction, ligne 650 pour le cisaillement) :
FF = _select_stress_force(F_input, F_output, F_average, options.force_check)
```

Trois valeurs possibles pour `force_check` (déroulante à trois choix depuis cette version — auparavant binaire, seuls `F_input`/`F_output` étaient sélectionnables) :

1. **`F_input`** (« Input-bar force ») — force côté barre d'entrée uniquement.
2. **`F_output`** (« Output-bar force », **valeur par défaut de l'UI**) — force côté barre de sortie uniquement ; c'est la formule « à une onde », la plus utilisée en pratique une fois l'équilibre dynamique entrée/sortie vérifié (aide contextuelle de `ui/pages/page_process.py`).
3. **`F_average`** (« Average force (input & output) ») — moyenne $(F_{input} + F_{output})/2$, déjà calculée dans les trois branches (`F_average` est systématiquement construite, que ce choix soit sélectionné ou non — voir plus bas) mais auparavant **jamais sélectionnable** pour le calcul de contrainte, seulement exposée en diagnostic (`SHPBResult.force_average`). Alternative courante quand l'équilibre entre F_{input} et F_{output} n'est qu'imparfait ou seulement brièvement atteint, puisqu'elle pondère les deux extrémités de l'éprouvette à égalité plutôt que de se fier à une seule d'entre elles (aide contextuelle, `ui/pages/page_process.py`, lignes 1086-1092).

$F_{average} = (F_{input} + F_{output})/2$ reste calculée dans les trois branches **indépendamment** du choix de `force_check` (elle alimente toujours `SHPBResult.force_average`, pour tracé/diagnostic même quand elle n'est pas utilisée pour la contrainte) — seul change, avec cette évolution, le fait qu'elle puisse *aussi* désormais servir de `FF` pour le calcul de $\sigma_{nom}/\sigma_{true}$ au même titre que F_{input}/F_{output}, sans dupliquer la logique dans chacune des trois branches de kinématique (d'où le passage par `_select_stress_force()`, factorisé une seule fois).

---

4. Corrections physiques disponibles

4.1 Correction de gradient thermique de barre (« heat_check », éprouvette chauffée)

Objectif (docstring de module) : compenser le fait qu'une éprouvette chauffée conduit de la chaleur dans les extrémités de barre, modifiant localement le module d'élasticité (et donc la célérité d'onde) au voisinage de la jauge par rapport au reste de la barre resté à température ambiante.

Fonction `_bar_gradient_wave_speed_modulus()` (lignes 84-112), appelée séparément pour IB et OB avec la température de jauge (`T_gages_value = (gage_temp_ib, gage_temp_ob)`, en K, saisie utilisateur sur la carte « Bar-temperature-gradient parameters ») et la température initiale de l'éprouvette `T0_pa[d]` (page Test Setup) :

Branche `type_bars_check` ∈ {1, 2} (modèle analytique RTE) — utilise deux constantes matériau

`Tm_value = (Tm0, Tm1)` :

$$RTE(T) = \frac{1 - \dfrac{T}{T_{m0}} \exp\Big(T_{m1}(1 - \dfrac{T_{m0}}{T})\Big)}{1 - \dfrac{293}{T_{m0}} \exp\Big(T_{m1}(1 - \dfrac{T_{m0}}{293})\Big)}$$

$$E_{mat} = RTE(T_0) \cdot E_{barre,amb}, \qquad C0_{mat} = \sqrt{E_{mat}/\rho_{barre}}, \qquad E_{surface} = RTE(T_{gage}) \cdot E_{barre,amb}$$

$RTE(T)$ est un facteur de module relatif, normé à 1 à 293 K (température ambiante de référence), fonction de deux paramètres matériau (T_{m0}, T_{m1}) dont la signification physique exacte (p.ex. température de fusion/paramètre de forme) n'est pas documentée dans le code — à faire préciser par le rédacteur si besoin.

Branche `type_bars_check == 3` (tables matériau importées, dépendantes de la température) : $E_{surface}$ = valeur de `e(T)` interpolée à `gage_temp` (table `material_table_ib/_ob`), E_{mat} = `e(T0)`, ρ_{mat} = `rho(T0)`, $C0_{mat} = \sqrt{E_{mat}/\rho_{mat}}$ (repli sur les valeurs ambiantes de la barre si la table ne couvre pas la température demandée).

Cas particulier heat + platens simultanés (lignes 332-337) : quand `plat_check` est également actif, `E_mat`/`C0_mat` (des deux côtés) sont **remplacés par les propriétés du platen** — le commentaire du code l'explique : « the "material on the other side of the bar interface" is the platen here ».

Coefficients de transmission à l'interface « surface de barre chauffée » <-> « matériau de référence » (barre ambiante, ou platen si `plat_check` + `heat_check`) :

$$C0_B = \sqrt{E_{surface}/\rho_{barre}}, \qquad S_B = \pi(D_{barre}/2)^2$$

$$T_{BP} = \frac{2\,C0_B\,E_{surface}\,S_B}{C0_B\,E_{mat}\,S_B + C0_{mat}\,E_{surface}\,S_B}, \qquad T_{PB} = \frac{2\,C0_{mat}\,E_{mat}\,S_B}{C0_B\,E_{mat}\,S_B + C0_{mat}\,E_{surface}\,S_B}$$

(calculés séparément côté entrée `TBP_input`/`TPB_input` et côté sortie `TBP_output`/`TPB_output`.)

Ondes corrigées (lignes 350-352) :

$$\varepsilon_i \mathrel{*}= T_{BP,input}, \qquad \varepsilon_r \mathrel{/}= T_{PB,input}, \qquad \varepsilon_t \mathrel{/}= T_{PB,output}$$

Le module utilisé plus loin dans la correction de poinçonnement (§4.3) reste **toujours** le module propre de la barre après correction thermique (`EB_punchI, EB_punch0 = e_mat_ib, e_mat_ob`, ligne 357), même si `plat_check` a substitué `E_mat_IB`/`E_mat_OB` par les propriétés du platen pour les formules de force/vitesse — l'indentation est en effet supposée se produire au niveau de la surface de la barre.

4.2 Correction de platens (`plat_check`)

Point important, non trivial à la lecture du code : contrairement à ce qu'on pourrait attendre, activer *seulement* « Platens » (sans `heat_check`) **ne modifie pas** l'amplitude des ondes via un coefficient de transmission dédié. En l'absence de correction thermique, `E_mat_IB`/`E_mat_OB`/`C0_mat_IB`/`C0_mat_OB` restent égaux aux propriétés ambiantes propres de la barre (lignes 319-323), strictement identiques au cas

sans aucune correction (lignes 359-361). Les platens n'interviennent alors que par trois canaux :

1. **Le temps de trajet des ondes** (fenêtrage, §1.3) : ajout de `2*plat_Len/plat_C0` au décalage temporel de l'onde réfléchie et de l'onde transmise, pour l'aller-retour/traversée du (des) platen(s).
2. **Le diamètre de poinçonnement** (§4.3) : `punch_diameter_I/punch_diameter_O = plat_Dia[d]` (au lieu du diamètre de l'éprouvette) quand `plat_check` est actif — l'indentation est alors modélisée contre le platen plutôt que directement contre l'éprouvette.
3. **Un terme additif dans la correction de poinçonnement** (§4.3) représentant la compliance élastique propre des platens.

Ce n'est **que si `heat_check` est activé en même temps** que `plat_check` que les propriétés du platen (module, célérité) sont substituées dans les formules de force/vitesse elles-mêmes (§4.1). Le diamètre utilisé pour **F_input/F_output** (`DB_punchI`, `DB_punch0`) et pour les propriétés `nuB_punch`/`EB_punch` de la correction de poinçonnement reste **toujours celui de la barre elle-même**, jamais celui (plus petit) du platen (lignes 289-312) : utiliser le diamètre du platen sous-estimerait `F_input`/`F_output` dès que les platens sont activés, ce qui produirait une contrainte artificiellement plus basse.

4.3 Correction de poinçonnement / indentation (`punch_check`)

Compense la déformation locale (indentation de contact) aux interfaces barre-éprouvette (ou barre-platen), non représentative de la déformation propre de l'éprouvette.

Facteur géométrique elliptique partagé, `_elliptic_punch_factor()` (lignes 772-786), fonction du rapport de diamètres $m = D_{\text{éprouvette ou platen}}/D_{\text{barre}}$ (doit être < 1, sinon `ValueError` explicite : « Punch/platen correction requires the punch or platen diameter to be larger than the specimen diameter ») :

$$E(m) = \int_0^{\pi/2} \sqrt{1 - m^2 \sin^2 x}\, dx, \qquad K(m) = \int_0^{\pi/2} \frac{dx}{\sqrt{1 - m^2 \sin^2 x}}$$

$$H_x(m) = 2 - \Big(m + \frac{1}{m}\Big) E(m) - \Big(m - \frac{1}{m}\Big) K(m)$$

(intégrales elliptiques complètes de première/deuxième espèce, calculées numériquement par `scipy.integrate.quad`.)

Compliance de poinçonnement par interface (côtés entrée `I` et sortie `O`) :

$$K_p = H_x(m) \cdot \frac{16}{3\pi^2} \cdot \frac{1 - \nu_{barre}^2}{D_{\text{poinçon}}\, E_{barre}}$$

avec ν_{barre}, E_{barre} toujours les propriétés propres de la barre (§4.2), $D_{\text{poinçon}}$ = diamètre du platen si `plat_check`, sinon diamètre (ou diamètre équivalent) de l'éprouvette.

Correction de déformation soustraite à la déformation mesurée, aire de l'éprouvette $A = \pi D_{\text{éprouvette}}^2/4$:

$$p_s = 10^6 \left(K_{p,I} + K_{p,O}\right) \sigma_{\text{réf}} \frac{A}{L_{\text{éprouvette}}} \; \underbrace{- \; 2 \frac{\sigma_{\text{réf}}}{E_{platen}} \frac{L_{platen}}{L_{\text{éprouvette}}}}_{\text{si Platens actif}}$$

(trois implémentations quasi identiques selon la kinématique : `_punch_correction_generic` pour compression/traction (lignes 802-812), `_punch_correction_notched` pour l'éprouvette entaillée (lignes 789-799, utilise K_p avec `punch_diameter_I/O` séparés), `_punch_correction_shear` pour le cisaillement (lignes 815-825, dénominateur D_{eq} au lieu de `punch_diameter_I/O` dans K_p). $\sigma_{\text{réf}}$ = contrainte nominale (`n_stress`), sauf pour le cas entaillé où c'est la contrainte déjà calculée `t_stress` qui est passée.

Application : la correction est ensuite soustraite à la déformation nominale (`n_s -= p_s` en compression/traction et cisaillement, `t_s -= p_s` pour l'éprouvette entaillée), puis la contrainte et la déformation vraies sont **recalculées** à partir de la déformation nominale corrigée.

Point à signaler explicitement : dans la branche compression/traction (lignes 565-574), lorsque `punch_check` est actif, le code recalcule systématiquement

```
t_s.append(-math.log(1 - n_s[i]))
t_stress.append(n_stress[i] * (1 - 2 * specimen.nu_pa[d] * n_s[i]))
```

c'est-à-dire **uniquement la formule de compression** (`-ln(1-\varepsilon)`, `1-2\nu\varepsilon`), **quel que soit `type_check`** (compression *ou* traction) — alors que la branche *sans* correction de poinçonnement (lignes 560-563 et 549-550) bifurque correctement entre compression et traction. C'est une incohérence entre les deux chemins de calcul, à vérifier/confirmer avec les auteurs plutôt qu'une interprétation de ma part : en l'état, activer la correction de poinçonnement sur un essai de traction semble appliquer par erreur les formules de contrainte/déformation vraies de compression.

Dans la branche cisaillement (lignes 666-676), la correction de poinçonnement introduit en outre une **division par 2** de la déformation corrigée (`t_s.append(n_s[i] / 2)`) qui n'a pas d'équivalent dans le chemin sans correction (`t_s = list(n_s)`, pas de facteur 2) — à signaler également comme un traitement asymétrique entre les deux chemins.

4.4 Correction d'inertie (`inertia_check`)

Compense les effets d'inertie radiale/axiale propres de l'éprouvette aux forts taux de déformation. Implémentée séparément par kinématique, avec la **même forme fonctionnelle** dans les trois cas :

$$c_1 = \frac{\rho_{\text{éprouvette}}\, D_{eq}^2\, (s_0^2 - 3/16)}{12}, \qquad c_2 = \frac{3\,\rho_{\text{éprouvette}}\, D_{eq}^2}{64}$$

$$\sigma_{inertie}(t) = \frac{c_1\,(\dot{\varepsilon}(t)^2 + \ddot{\varepsilon}(t))}{10^6} + \frac{c_2\,\ddot{\varepsilon}(t)}{10^6}, \qquad \sigma_{true}\ -= \sigma_{inertie}$$

où s_0 est un rapport d'élancement (`L/D` ou équivalent selon la géométrie) et $\ddot{\varepsilon}$ la dérivée seconde de la déformation (« strain-rate-rate », notée `n_srsr`/`t_srsr` dans le code, calculée par différence finie de la vitesse de déformation). La correction est appliquée **uniquement à la contrainte vraie**, jamais à la contrainte nominale.

- **Compression/traction** (lignes 598-604) : D_{eq} = diamètre (ou diamètre équivalent carré/plat) de l'éprouvette, $s_0 = L_{\text{éprouvette}}/D_{eq}$, dérivées calculées avec `_robust_dt()`.
- **Éprouvette entaillée** (lignes 474-486) : géométrie propre — angle figé $\cos(\pi/4)$ (45°, **indépendant** de l'angle d'entaille réel `NoA_pa`, à signaler), $S = D_{\text{éprouvette}}/\cos(\pi/4) \times e_{zone}$, $D = 2\sqrt{S/\pi}$,

$s_{0i} = 2(r_e/\cos(\pi/4))/D$, mêmes formules c_1, c_2 avec D à la place de D_{eq}, dérivées calculées par différence finie directe (pas `_robust_dt`).

- **Cisaillement** (lignes 711-723) : $D_{eq,2}$ recalculé à partir de la géométrie exacte du montage (hat/double/simple), $s_0 = H_3/D_{eq}$ ou $L_{\text{tête}}/D_{eq}$ selon le montage.

4.5 Coefficient de friction (`friction`)

Un coefficient de friction par tirage Monte-Carlo (`friction[d]`), appliqué **de façon identique dans les trois branches de kinématique** dès qu'il est non nul (ex. lignes 578-583 pour compression/traction) :

$$s_0 = \frac{L_{\text{éprouvette}}}{D_{\text{éprouvette}}}, \qquad m = 1 - \frac{f}{3\,s_0}$$

$$\sigma_{nom} \mathrel{*}= m$$

$$\sigma_{true} \mathrel{*}= m \quad \text{(en place)}, \qquad \sigma_{frottement} = \frac{f}{3\,s_0}\,\sigma_{true}^{(\text{après m})}, \qquad \sigma_{true} \mathrel{-}= \sigma_{frottement}$$

Point à signaler : parce que `friction_stress` est calculé **après** que `t_stress` ait déjà été multiplié par `m`, le facteur effectivement appliqué à la contrainte vraie finale est

$$\sigma_{true}^{final} = m \cdot \sigma_{true}^{orig} \cdot (1 - \frac{f}{3s_0}) = m^2 \cdot \sigma_{true}^{orig}$$

soit m^2 **et non** m, alors que la contrainte nominale, elle, n'est réduite que d'un facteur m (elle n'est jamais retouchée par `friction_stress`). C'est un constat direct de lecture du code, pas une réinterprétation : `friction_stress` est exposé séparément dans `SHPBResult.friction_stress` mais utilise la valeur déjà corrigée de `t_stress`, pas la valeur d'origine.

La justification physique usuelle de ce type de correction est la réduction apparente de la contrainte mesurée due au confinement radial induit par le frottement aux interfaces barre/éprouvette (effet de tonnelet) ; le code ne documente pas de référence bibliographique pour la forme exacte $m = 1 - f/(3s_0)$.

---

5. Correction du « pied » / « toe » de courbe (`core/toe_correction.py`)

Corrige un artefact courant de mise en position (faces non parfaitement planes/parallèles, jeu résiduel qui se referme) — une montée initiale lente, de faible raideur, avant que la courbe n'atteigne sa vraie pente élastique. Indépendant de tout problème de calage temporel des ondes (docstring de module, lignes 1-12).

5.1 Détection automatique de la zone linéaire — `detect_linear_region()` (lignes 51-89)

- Fenêtre glissante de largeur `window = max(5, n * window_fraction)` (par défaut 12 % de la longueur totale de courbe), recherchée uniquement dans les premiers `search_fraction` (60 % par défaut) des points — la zone élastique est supposée survenir tôt, avant écoulement/non-linéarité.

- Pour chaque position de fenêtre : régression linéaire `numpy.polyfit` degré 1, calcul du R^2 (`_fit_line()`, lignes 31-37) :
$$R^2 = 1 - \frac{\sum(y - \hat{y})^2}{\sum(y - \bar{y})^2}$$
- Sélection : parmi les fenêtres atteignant `r_squared_threshold` (0.995 par défaut), on garde celle de R^2 **maximal** (pas la plus pentue — le docstring précise explicitement que la pente seule est un critère « facilement trompé »). Si aucune fenêtre n'atteint le seuil, repli sur la fenêtre de R^2 maximal parmi toutes les candidates.
- Dans le cas dégénéré (courbe trop courte pour produire une seule fenêtre candidate), régression sur l'intégralité de la courbe.

5.2 Ajustement manuel — `fit_linear_region()` (lignes 40-48)

Même régression linéaire, mais sur une plage `[start_index, end_index)` choisie manuellement par l'utilisateur (glisser-déposer sur le tracé, carte « Toe (Foot) Correction »).

5.3 Recalage de l'origine — `apply_toe_correction()` (lignes 92-102)

Le point d'intersection de la droite ajustée avec l'axe des contraintes nulles donne le nouveau zéro de déformation :
$$\varepsilon_{offset} = -\frac{\text{ordonnée à l'origine}}{\text{pente}}$$
$$\varepsilon_{\text{corrigée}} = \varepsilon - \varepsilon_{offset}$$
puis (si `trim=True`, comportement par défaut) suppression de tous les points à déformation corrigée négative (le « pied » proprement dit).

5.4 Recalcul de la déformation vraie après recalage — `infer_true_strain_relation()` / `true_strain_from_nominal()` (lignes 105-148)

La correction ci-dessus s'applique directement à la **déformation nominale** (recalage linéaire simple). Comme la relation déformation-vraie/déformation-nominale n'est pas linéaire (compression/traction), la déformation vraie doit être **recalculée** après recalage, pas simplement translatée. `infer_true_strain_relation()` détermine automatiquement (à partir des données déjà produites par `process_shpb()`) laquelle des trois relations s'applique, en comparant à un point représentatif (milieu des indices non nuls) :

- compression : $\varepsilon_{true} = -\ln(1 - \varepsilon_{nom})$
- traction : $\varepsilon_{true} = \ln(1 + \varepsilon_{nom})$
- cisaillement : $\varepsilon_{true} = \varepsilon_{nom}$ (aucune transformation — cohérent avec le §2.3)

`true_strain_from_nominal()` applique ensuite la relation identifiée à la déformation nominale corrigée (avec un `np.clip` de sécurité à 0.999999 côté compression, même logique que `_clamp_nominal_strain` dans `process_shpb.py`).

5.5 Cohérence avec les tirages Monte-Carlo bruts et les grandeurs de confinement (`ui/pages/page_process.py`)

Le §5.3 décrit `apply_toe_correction()`, la primitive de `core/toe_correction.py` qui décale/tronque **un seul couple** `(déformation, contrainte)`. Historiquement, le bouton « Apply » de la carte « Toe (Foot) Correction » n'appelait cette primitive qu'**une seule fois, sur la courbe moyenne déjà agrégée** (`project.last_results["SHPB"]`) : chaque champ tableau de même longueur que la déformation y était retranché du même masque booléen, mais **aucun des tirages Monte-Carlo individuels n'était retouché** — ni `project.last_raw_draws["SHPB"]` (qui alimente la page Covariance Analysis), ni les grandeurs de confinement calculées par tirage (§5.8 du chapitre Confinement) qui vivent, elles, à l'intérieur de chaque tirage brut plutôt que dans l'agrégat. Une fois la correction appliquée, ces deux sources restaient donc décalées d'un nombre de points égal au « pied » retiré — un défaut d'alignement signalé par l'utilisateur et corrigé par ce qui suit.

Mécanisme actuel : `_on_toe_apply()` (`_SHPBTab`, lignes 2314-2385 ; `_DITab`, lignes 3575-3625, logique rigoureusement identique) ne touche plus l'agrégat directement. Il réapplique le même offset ajusté à **chaque tirage Monte-Carlo individuellement**, via `_apply_toe_correction_to_draws()` (fonction module, lignes 147-212), puis ré-agrège :

```
new_per_draw, n_removed = _apply_toe_correction_to_draws(
    self._last_per_draw, strain_key, strain_offset, relation
)
quantities = _aggregate_quantities(new_per_draw)
self.project.last_results["SHPB"] = {"label": "SHPB", **quantities, "toe_n_removed": ...}
_store_raw_draws(self.project, "SHPB", new_per_draw)
```

`self._last_per_draw` est la liste des dictionnaires par tirage déjà produits par le dernier lancement Monte-Carlo (un dict par tirage réussi, exactement ceux passés à `_aggregate_quantities()`/`_store_raw_draws()` en fin de traitement) — **aucune nouvelle simulation n'est relancée** : la correction de toe ne modifie en rien la physique de propagation d'onde (dispersion, équilibre de force, etc.) déjà calculée pour chaque tirage par `process_shpb()`, c'est un pur changement de repère sur l'axe de déformation appliqué *a posteriori*, donc réappliquer ce changement de repère à des résultats déjà calculés est exact, pas approximatif — voir aussi le chapitre Confinement, §5.8.

`_apply_toe_correction_to_draws(per_draw, strain_key, strain_offset, relation)` traite chaque tirage indépendamment :

- Son propre point de franchissement `(déformation_tirage - offset) >= 0` sert de masque de découpe, appliqué à **tous** les champs de ce même tirage partageant la longueur de sa déformation — force, température, déplacement, `sensor_{idx}_signal`, et les grandeurs `confinement_*` (§5.8 du chapitre Confinement) quand elles ont été calculées pour ce tirage. Les champs provenant du dataclass résultat (`SHPBResult`) sont de simples listes Python plutôt que des `ndarray` pour la plupart des grandeurs (seules les grandeurs construites côté UI — confinement, capteurs recoupés — sont déjà des `ndarray`) : la fonction traite les deux types indistinctement (`isinstance(values, (np.ndarray, list))`), condition plus large que le filtre `isinstance(values, np.ndarray)` historique de la correction sur agrégat (§5.3), qui ne s'appliquait qu'à des tableaux déjà empilés par `_aggregate_quantities()`.

- La déformation vraie est **recalculée** (pas simplement translatée) via `true_strain_from_nominal()`, à partir de la relation `infer_true_strain_relation()` inférée **une seule fois** sur la courbe agrégée courante et réutilisée pour tous les tirages (la relation compression/traction/cisaillement est une propriété fixe de la cinématique de l'essai, pas quelque chose qui varie tirage à tirage — voir §5.4).
- Un tirage dont la courbe ne franchit **jamais** l'offset ajusté (réalisation Monte-Carlo aberrante) est **retiré de l'ensemble** plutôt que laissé non corrigé — mélanger un tirage non corrigé à un agrégat par ailleurs corrigé fausserait silencieusement la moyenne/l'écart-type.
- La fonction ne mute jamais ses arguments : elle retourne une **nouvelle** liste de nouveaux dictionnaires, ce qui permet à l'appelant de conserver `self._last_per_draw` d'origine comme sauvegarde exacte pour « Reset » (`_on_toe_reset()`, lignes 2395-2413/3630 ; restaure `self._last_per_draw`, puis régénère `last_results/last_raw_draws` par les mêmes fonctions d'agrégation que ci-dessus, garantissant une restauration bit-à-bit identique au dernier lancement Monte-Carlo).
- La fonction renvoie aussi la **médiane** (pas le nombre retiré du premier tirage seul, plus sensible au bruit propre à un tirage individuel) du nombre de points retirés en tête sur l'ensemble des tirages survivants — accumulée tirage d'application en tirage d'application dans un entier `toe_n_removed`, stocké tel quel (pas un tableau, donc jamais touché par les fonctions d'agrégation) au niveau racine de `project.last_results["SHPB"]`. C'est ce compteur que `ui/pages/page_results_export.py::_refresh_confinement()` relit pour garder aligné le signal **brut** (non recoupé par tirage) d'un capteur de cellule de confinement ou laser, quand ce capteur n'a pas sa case « Include this cut in the Monte-Carlo loop » cochée — voir le chapitre Confinement, §5.5 et §7, pour ce deuxième volet de la correction (`_refresh_confinement`, lignes 922-995).

Plusieurs applications successives de « Apply » (sans « Reset » entre les deux) s'enchaînent sur `self._last_per_draw` tel qu'il se trouve à ce moment — comme le faisait déjà la correction sur agrégat historique — tandis que `self._toe_per_draw_backup` (assigné une seule fois, à la première application suivant un lancement Monte-Carlo) conserve toujours l'ensemble brut d'origine pour « Reset ».

---

6. Outil de vérification de l'alignement des ondes (`core/wave_alignment.py`)

Objectif (docstring de module) : prédire à quoi *devrait* ressembler l'onde transmise si l'éprouvette répondait purement élastiquement, à partir de l'onde incidente+réfléchie mesurée, en n'utilisant que l'**impédance mécanique 1D** et les coefficients de transmission classiques à chaque interface traversée : `Bar_in -> [Platen_in] -> Specimen -> [Platen_out] -> Bar_out`. Explicitement limité : ignore les réverbérations internes à l'éprouvette (supposée mince) et toute réponse inélastique — une prédiction au premier ordre, utile pour recaler temporellement l'onde transmise mesurée sans hypothèse sur le comportement réel (non linéaire) de l'éprouvette.

6.1 Impédance mécanique et coefficient de transmission

`mechanical_impedance()` (lignes 25-29) :

$$Z = \rho\, C_0\, A, \qquad C_0 = \sqrt{E/\rho}, \qquad A = \pi(D/2)^2$$

`transmission_coefficient()` (lignes 32-35), coefficient de transmission classique en contrainte/force d'un milieu d'impédance Z_1 vers un milieu d'impédance Z_2 :

$$T_{1\to 2} = \frac{2Z_2}{Z_1 + Z_2}$$

6.2 Onde transmise théorique — `predicted_transmitted_wave()` (lignes 45-73)

Chaîne d'impédances traversées :

chain $= [Z_{IB},\, Z_{OB}]$ (sans platens), chain $= [Z_{IB},\, Z_{platen},\, Z_{\text{éprouvette}},\, Z_{platen},\, Z_{OB}]$ (avec platens)

Coefficient de transmission total, produit des coefficients à chaque interface successive de la chaîne :

$$T_{total} = \prod_k T_{k\to k+1}$$

Onde transmise théorique (en force) :

$$F_{\text{t,prédite}}(t) = T_{total} \cdot \left(F_i(t) + F_r(t)\right)$$

où $F_i + F_r$ est l'onde incidente+réfléchie déjà convertie en force (fournie en entrée par l'appelant côté UI).

6.3 Auto-alignement par corrélation croisée — `best_alignment_shift()` (lignes 88-129)

Étant donné que les « ondulations » locales de l'onde théorique ne sont qu'une copie mise à l'échelle des ondulations de l'onde incidente+réfléchie (et ne correspondent généralement pas aux ondulations propres de l'onde transmise mesurée — origine physique différente : dispersion, réverbération, réponse matérielle réelle), la comparaison brute point-à-point ne fonctionne pas ; seule la forme globale (montée/plateau/décroissance) est comparable. Méthode :

1. Lissage des deux signaux par moyenne glissante à bords « edge-padded » (`_smooth_edge_padded()`, lignes 76-85), fenêtre `window = max(1, n // 8)`.
2. Normalisation centrée-réduite en énergie (moyenne nulle, norme unité) de chaque signal lissé.
3. Recherche exhaustive du décalage entier `shift` dans $[-N_{max}, +N_{max}]$ (`max_shift_samples`, borné par `n-1`) qui **maximise le produit scalaire** (corrélation croisée normalisée) entre le signal mesuré décalé (`np.roll`, complété par des zéros aux bords découverts) et le signal théorique :

$$\text{shift}^\star = \arg\max_{shift} \ \left\langle \text{mesuré}_{\text{normé}}(\cdot - \text{shift}),\ \text{théorique}_{\text{normé}} \right\rangle$$

Le décalage trouvé peut ensuite être appliqué manuellement (curseur) ou automatiquement, puis « committé » dans `output_window_offset` (§1.3) pour affecter réellement le traitement.

---

7. Alignement temporel d'un capteur monté directement sur l'éprouvette (`core/specimen_sensor_alignment.py`)

Un capteur collé directement sur l'éprouvette (page « Custom Sensors », option « mounted directly on the specimen ») ne commence pas à enregistrer au même instant physique que les jauges de barre : l'onde de chargement doit encore parcourir la distance jauge → éprouvette (et éventuellement traverser des platens, puis la moitié de la longueur de l'éprouvette si le capteur est supposé situé à mi-longueur).

Formule (docstring de module, reproduite dans le code, lignes 15-24 et fonctions lignes 39-60) :

Cas SHPB — `specimen_sensor_delay_shpb()` :

$$\Delta t = \frac{x_{jauge,IB}}{C0_{IB}} \; + \; \frac{L_{platens}}{C0_{platens}} \;(\text{si Platens}) \; + \; \frac{L_{\text{éprouvette}}}{2\,C0_{\text{éprouvette}}}$$

décomposé en trois termes exposés séparément (`DelayBreakdown` : `bar_term`, `platen_term`, `specimen_term`, `total`) : - `bar_term = input_gage_position / c0_input_bar` - `platen_term = platens_length / c0_platens` (0 si pas de platens) - `specimen_term = sample_length / (2 * c0_sample)`

Cas Direct Impact — `specimen_sensor_delay_direct_impact()` : formule strictement identique, mais avec la position/célérité de la **barre de sortie** (`output_gage_position`, `c0_output_bar`) au lieu de la barre d'entrée — cohérent avec la configuration Direct Impact (une seule barre instrumentée, côté sortie).

`sample_wave_speed()` (lignes 63-66) : $C0_{\text{éprouvette}} = \sqrt{E_{\text{éprouvette}}/\rho_{\text{éprouvette}}}$, utilisée quand l'éprouvette n'a pas de célérité mesurée directement.

L'aide contextuelle de `ui/pages/page_process.py` (lignes 944-964) précise que ce délai est calculé **une seule fois**, à partir des propriétés moyennes (pas Monte-Carlo) des barres/platens/éprouvette — l'incertitude Monte-Carlo sur ce temps de trajet est jugée négligeable et n'est pas propagée séparément.

---

8. Récapitulatif des limitations, hypothèses et points d'ambiguïté relevés dans le code

1. **Validation non faite** (docstring de module `process_shpb.py`) : le moteur n'a pas été validé indépendamment sur données réelles ni comparé à la sortie de l'application d'origine pour la matrice complète des options ; seulement testé sur signaux synthétiques dans la combinaison d'options la plus simple.
2. **Platens sans correction thermique** : n'introduit aucun coefficient de transmission d'onde dédié — seuls le fenêtrage temporel et la correction de poinçonnement sont affectés (§4.2). Peut surprendre un utilisateur s'attendant à un effet de type impédance comme dans l'outil d'alignement des ondes (§6).
3. **Correction de poinçonnement + traction** (compression/traction, §4.3) : le chemin de code activé par `punch_check` applique la formule de contrainte/déformation vraies **de compression** même pour un essai en traction — incohérence entre le chemin avec et sans correction de poinçonnement, à confirmer.

4. **Correction de poinçonnement + cisaillement** : introduit une division par 2 de la déformation qui n'existe pas dans le chemin sans correction.
5. **Correction de friction** : le facteur effectivement appliqué à la contrainte vraie finale est m^2 (et non m), du fait de l'ordre des opérations dans le code, alors que la contrainte nominale n'est réduite que d'un facteur m.
6. **Correction d'inertie, cas entaillé** : utilise un angle fixe de 45° ($\cos(\pi/4)$), indépendant de l'angle d'entaille réel configuré par l'utilisateur (`specimen.NoA_pa`).
7. **Bras viscoélastique (`disp_visco`)** : le module complexe fréquentiel est calculé uniquement à partir des paramètres rhéologiques de la barre d'entrée (`IB.e11`, `IB.e22`, `IB.n22`) ; la barre de sortie en réutilise le module réel (`EE11`) pour calculer sa propre célérité — implicite hypothèse que les deux barres partagent la même rhéologie, non signalée explicitement dans le code comme une hypothèse délibérée.
8. **Bras viscoélastique, correspondance fréquence/échantillon** : le module `E(f)` est indexé fréquence par fréquence directement sur les échantillons temporels (`EE11[i]` associé au `i`-ème échantillon de temps), plutôt que via une transformée de Fourier/Fourier inverse complète du signal — traitement non standard pour une correction de dispersion, à valider.
9. **Signification de `SG_unc_d`** : appliqué en plus du coefficient de jauge `CPG_pa` lors de la découpe des ondes, sans que la nature exacte de cette double pondération (incertitude Monte-Carlo distincte vs. redondance) ne soit commentée dans le code.
10. **Choix de mode de test global** : `type_check` (compression/traction/entaillé/cisaillement) est déterminé par le réglage global « Test mode » de la page Test Setup, pas par onglet — `_require_notched_geometry()` avertit explicitement du risque de traiter un essai avec la géométrie d'un mode de test différent de celui effectivement réglé au moment du calcul.
11. **`toe_n_removed` est une médiane, pas un décompte exact par tirage** (§5.5) : depuis que la correction de toe recoupe chaque tirage Monte-Carlo individuellement, le nombre de points retirés en tête peut légèrement différer d'un tirage à l'autre (bruit propre à la réalisation des propriétés barreau/éprouvette de ce tirage). Le compteur `toe_n_removed`, utilisé par `page_results_export.py::_refresh_confinement()` pour recaler le signal **brut** (non recoupé par tirage) d'un capteur de confinement/laser, retient la **médiane** de ce nombre sur l'ensemble des tirages survivants — un bon représentant tant que cette variation inter-tirages reste faible (cas courant), mais une approximation, pas une correspondance exacte tirage-par-tirage pour ce chemin de repli spécifique (le chemin recoupé par tirage, lui, §5.8 du chapitre Confinement, reste exact puisqu'il recoupe directement le signal réel de CE tirage).
12. **Le recalage `toe_n_removed` du signal brut ne corrige qu'un décalage en tête, pas une différence de cadence d'échantillonnage** : la combinaison capteur de confinement/laser brut + courbe contrainte-déformation dans `_refresh_confinement()` reste, en dehors de ce décalage désormais corrigé, une simple troncature position-à-position à la longueur minimale commune (voir chapitre Confinement, §7, point 8) — elle suppose toujours implicitement que les deux signaux partagent le même pas d'échantillonnage, hypothèse indépendante de la correction de toe et non traitée par elle.

4. Dépouillement « Direct Impact » (DI)

Fichier central : `core/process_di.py` (fonctions `process_di()` et `_process_di_task()`). Options : dataclasses `DIOptions` / résultat `DIResult` (mêmes fichier).

---

1. Principe physique et différences avec le SHPB classique

En SHPB (Split Hopkinson Pressure Bar) « classique », l'éprouvette est prise en sandwich entre une **barre d'entrée** (instrumentée, onde incidente + onde réfléchie) et une **barre de sortie** (instrumentée, onde transmise) : la vitesse d'impact du côté entrée est déduite de l'onde incidente mesurée sur la barre d'entrée.

En **Direct Impact (DI)**, il n'y a **qu'une seule barre instrumentée : la barre de sortie (`OB`, Output Bar)**. L'éprouvette est directement impactée (par un projectile/flyer) et sa **vitesse incidente est mesurée indépendamment** (typiquement par barrière laser — voir `velocity_c` dans le code, chargé depuis un fichier « Incident velocity file » dans l'onglet Direct Impact de l'UI). Il n'y a donc :

- **pas d'onde incidente ni d'onde réfléchie** à traiter (pas de séparation d'ondes sur une barre d'entrée) ;
- **pas de barre d'entrée instrumentée** : aucune notion de « choix input/output bar » pour le calcul de force (contrairement à ce que le SHPB pourrait permettre avec deux barres) ; la force n'est calculable que du côté barre de sortie ;
- la **cinématique côté entrée** est directement le profil de vitesse mesuré (`velocity_c`), multiplié par un facteur de calibration, au lieu d'être dérivé par théorie des ondes 1D à partir d'un signal de jauge.

Le fichier `core/process_di.py` est explicitement décrit dans son docstring comme « the single-output-bar counterpart of process_shpb.py: instead of an incident+reflected wave pair on an input bar, the specimen's incident velocity comes from a separate measurement (`velocity_c`, e.g. a laser gate signal), and only the transmitted wave on the output bar is used. » Il est « ported as literally as possible (same variable names, branch order and formulas) to keep it traceable against the original line-for-line » — c'est-à-dire qu'il reproduit fidèlement un script historique (`process_DI.py` / `process_DI()`), y compris ses choix de structure.

---

2. Vue d'ensemble du pipeline (`process_di()`)

Signature : `process_di(options, OB, specimen, velocity_c, time_pulse, xd1, xd2, d, SG_unc_d, f_inc_velocity, T0_pa, friction, k_value=None, sk_value=None, plat_nu=None, plat_rho=None, plat_emod=None, plat_Dia=None, plat_Len=None, Tm_value=None, T_gages_value=None, material_table_ob=None, AA=None)` — appelée une fois par tirage Monte-Carlo `d` (via `_process_di_task()`, le point d'entrée utilisé par `core/parallel.run_map` depuis l'onglet Direct Impact de `ui/pages/page_process.py`).

Étapes, dans l'ordre où elles apparaissent dans le code :

1. **Découpe des signaux** (fenêtre `[xd1, xd2]` sur l'onde transmise ; fenêtre de même durée sur la vitesse incidente) — `process_di()`, bloc « Cutting waves ».
2. **Substitution éventuelle par le signal dispersion-corrigé** (`options.disp_ela` ou `options.disp_visco`) — le signal transmis brut découpé est remplacé par `AA[0]` (résultat de la page Dispersion Corrections).
3. **Correction « platen » (plateaux)**, si `options.plat_check`.
4. **Correction de gradient thermique de barre** (« heat »), si `options.heat_check` — réutilise `core/process_shpb.py::_bar_gradient_wave_speed_modulus()`.
5. **Branche selon le type d'essai** (`options.type_check` : 1 compression, 2 traction, 3 entaillé Shear-Compression/Tension, 4 cisaillement) : intégration vitesse/onde transmise → déplacement, déformation nominale, force, contrainte nominale, contrainte vraie.
6. **Corrections complémentaires optionnelles**, appliquées dans chaque branche : poinçonnement (`punch_check`), friction (`friction[d] != 0`), inertie (`inertia_check`).
7. **Échauffement adiabatique** (`temperature`), calculé après la branche, commun aux 4 types d'essai.
8. **Mise en forme finale** du déplacement (mm) et retour d'un `DIResult`.

`options.type_check` n'est pas choisi directement dans l'onglet Processing : il est dérivé du « Test mode » global, projet-wide, défini sur la page Test Setup (`_test_mode_options()` dans `ui/pages/page_process.py`) — un avertissement explicite à ce sujet est levé par `_require_notched_geometry()` (voir §9).

---

3. Découpe des signaux et vitesse incidente

3.1 Fenêtrage temporel

```
xdt = xd2 - xd1
i_i1 = index_le_plus_proche(time_pulse, xd1)
i_i2 = index_le_plus_proche(time_pulse, xd2)
i_it = index_le_plus_proche(time_pulse, xdt)
time1 = time_pulse[i_i1:i_i2] - xd1
transmitted = OB.signal[i_i1:i_i2] * SG_unc_d[d]
velocity = velocity_c[0:i_it]
```

— `core/process_di.py::process_di()`, bloc « Cutting waves », en s'appuyant sur `core/process_shpb.py::_index_of_`
(recherche du point du vecteur temps le plus proche de la cible ; remplace un ancien lookup par égalité

exacte `time_pulse.index(round(x,6))`, cassant dès que le pas de temps est sub-microseconde — voir docstring de `_index_of_nearest`).

Points clés : - `time1` est réindexé pour démarrer à $t = 0$ à `xd1` (début de la fenêtre d'onde transmise). - `transmitted` = signal brut de la barre de sortie, multiplié par `SG_unc_d[d]` — un facteur multiplicatif d'incertitude par tirage Monte-Carlo, mais **fixé à 1.0 pour tous les tirages** par l'UI (`sg_unc_d = np.ones(n)` dans `_DITab` de `ui/pages/page_process.py`) : le mécanisme Monte-Carlo existe côté `process_di()` mais n'est pas activé (aucune distribution d'incertitude n'y est réellement attachée) depuis l'onglet Direct Impact actuel. - `velocity = velocity_c[0:i_it]` : le tableau de vitesse incidente (déjà recalé par l'UI, voir §6) est tronqué aux `i_it` premiers échantillons — `i_it` étant l'indice de `time_pulse` le plus proche de la **durée** de la fenêtre `xdt = xd2 - xd1`, pas d'un instant absolu. Cela suppose implicitement que `velocity_c` est échantillonné au même pas de temps que `time_pulse`, et que son échantillon d'indice 0 correspond exactement à `t = xd1` (c'est précisément ce que le mécanisme de « cut »/« shift » de l'UI est censé garantir — voir §6).

3.2 Signal dispersion-corrigé (`disp_ela` / `disp_visco`)

```
if options.disp_ela or options.disp_visco:
    transmitted = AA[0][:]
```

— `process_di()`, juste après le découpage. Si l'utilisateur a coché « Use dispersion-corrected signal » dans l'onglet DI (`disp_ela = use_corrected` dans `ui/pages/page_process.py::_DITab`), le signal transmis brut découpé ci-dessus est **entièrement remplacé** par `AA[0]`, c'est-à-dire `result.transmitted_corrected` produit par la page Dispersion Corrections (calculée pour le mode Direct Impact, i.e. `corr["direct_impact"] == True`). Notez que `time1`, lui, reste dérivé de `xd1/xd2/time_pulse` comme au §3.1 — c'est le vecteur temps de la fenêtre choisie côté Corrections (`corr["xd1"], corr["xd2"], corr["time_pulse"]`) qui doit rester cohérent, sans quoi `time1` et `AA[0]` seraient désynchronisés (contrôlé par l'UI qui réutilise `corr["xd1"]`/`corr["xd2"]` comme `xd1`/`xd2` lorsque « Use dispersion-corrected signal » est coché).

3.3 Correction « platen »

```
if options.plat_check:
    plat_C0 = sqrt(plat_emod[d] / plat_rho[d])
    SBO   = pi * (OB.D_pa[d] / 2) ** 2
    S_pla = pi * (plat_Dia[d] / 2) ** 2
    TPB_output = 2*plat_C0*plat_emod[d]*S_pla / (OB.C0_pa[d]*plat_emod[d]*S_pla + plat_C0*OB.e_pa[d]
    transmitted = transmitted / TPB_output
    E_mat_OB, C0_mat_OB, rho_mat_OB = plat_emod[d], plat_C0, plat_rho[d]
else:
    rho_mat_OB = OB.rho_pa[d]
    E_mat_OB = C0_mat_OB = None   # défini plus loin
```

— `process_di()`, bloc « Platen correction ». En notation mathématique, avec $S_{OB} = \pi(D_{OB}/2)^2$ (section de la barre de sortie) et $S_{plat} = \pi(D_{plat}/2)^2$ (section du plateau) :

$$T_{PB} = \frac{2\,C_{0,plat}\,E_{plat}\,S_{plat}}{C_{0,OB}\,E_{plat}\,S_{plat} + C_{0,plat}\,E_{OB}\,S_{OB}}, \qquad \varepsilon^{\text{corrigé}}_{transmitted} = \frac{\varepsilon_{transmitted}}{T_{PB}}$$

C'est un coefficient de transmission d'impédance à l'interface barre/plateau (théorie 1D des ondes à une interface à deux matériaux) : la déformation mesurée sur la barre est divisée par ce coefficient pour remonter à la déformation réelle à l'interface plateau/éprouvette. Si `plat_check` est actif, les propriétés « matériau à l'interface » (`E_mat_OB`, `C0_mat_OB`, `rho_mat_OB`) utilisées plus loin pour la force et le déplacement deviennent **celles du plateau**, pas celles de la barre — **sous réserve de l'interaction avec `heat_check` décrite en §3.5**.

3.4 Correction de gradient thermique de barre (« heat »)

```
if options.heat_check:
    gage_t_ob = T_gages_value[1] if T_gages_value else 293.0
    e_mat_ob, c0_mat_ob, e_surface_ob = _bar_gradient_wave_speed_modulus(
        OB, d, gage_t_ob, T0_pa[d], options.type_bars_check, Tm_value, material_table_ob)
    if not options.plat_check:
        E_mat_OB, C0_mat_OB = e_mat_ob, c0_mat_ob
    C0OB = sqrt(e_surface_ob / OB.rho_pa[d])
    SBO  = pi * (OB.D_pa[d] / 2) ** 2
    TPB_output = 2*C0_mat_OB*E_mat_OB*SBO / (C0OB*E_mat_OB*SBO + C0_mat_OB*e_surface_ob*SBO)
    transmitted = transmitted / TPB_output
    EB_punch = e_mat_ob   # toujours le module de la barre elle-même, corrigé thermiquement

if not options.heat_check:
    E_mat_OB, C0_mat_OB = OB.e_pa[d], OB.C0_pa[d]
```

— `process_di()`, bloc « Bar-temperature-gradient (heat) correction », s'appuyant sur `core/process_shpb.py::_bar_gradi` (fonction **partagée, non dupliquée**, entre SHPB et DI, comme l'indique explicitement le docstring de tête du fichier : « The bar-temperature-gradient helper is shared with core/process_shpb.py since it's already per-bar. »).

`_bar_gradient_wave_speed_modulus(bar, d, gage_temp, t0, type_bars_check, tm_value, table)` calcule, selon `type_bars_check` : - **1 ou 2 (modèle RTE — Relative Thermo-Elastic)** :

$$\mathrm{RTE}(T) = \frac{1 - \dfrac{T}{T_m^{(0)}} \exp\!\left(T_m^{(1)}\left(1 - \frac{T_m^{(0)}}{T}\right)\right)}{1 - \dfrac{293}{T_m^{(0)}} \exp\!\left(T_m^{(1)}\left(1 - \frac{T_m^{(0)}}{293}\right)\right)}$$

appliqué une fois à la température de jauge T_{gage} (donne RTE_{bar}, module « en surface de barre », pour la correction du coefficient de transmission), et une fois à la température initiale de l'éprouvette T_0 (donne RTE_{mat}, module « matériau à l'interface », pour la conversion force/déformation) :

$$E_{mat} = \mathrm{RTE}_{mat} \cdot E_{OB}, \quad C_{0,mat} = \sqrt{E_{mat}/\rho_{OB}}, \quad E_{surface} = \mathrm{RTE}_{bar} \cdot E_{OB}$$

- **3 (table de propriétés importée en fonction de la température)** : interpolation de E et ρ dans `material_table_ob` aux températures T_{gage} (pour $E_{surface}$) et T_0 (pour E_{mat}, ρ utilisée dans $C_{0,mat} = \sqrt{E_{mat}/\rho(T_0)}$).

Puis, dans `process_di()` : le coefficient de transmission à l'interface (chaud/froid) barre-plateau/éprouvette,

$$T_{PB} = \frac{2\,C_{0,mat}\,E_{mat}\,S_{OB}}{C_0^{surface}\,E_{mat}\,S_{OB} + C_{0,mat}\,E_{surface}\,S_{OB}}, \qquad C_0^{surface} = \sqrt{E_{surface}/\rho_{OB}}$$

divise à son tour `transmitted`. `EB_punch` (utilisé uniquement dans la correction de poinçonnement, §5.1) est **toujours** réglé sur `e_mat_ob` (module de la barre, corrigé thermiquement), qu'il y ait ou non un plateau — commentaire explicite du code : « EB_punch is always the bar's own (heat-adjusted) modulus, regardless of plat_check ».

3.5 Attention — Interaction `plat_check` / `heat_check` sur `E_mat_OB` / `C0_mat_OB`

En relisant précisément l'ordre des blocs (§3.3 puis §3.4), on observe ceci : la ligne finale

```
if not options.heat_check:
    E_mat_OB, C0_mat_OB = OB.e_pa[d], OB.C0_pa[d]
```

s'exécute **inconditionnellement dès que `heat_check` est faux, y compris lorsque `plat_check` est vrai**. Or c'est bien `plat_check` (bloc §3.3, ligne `E_mat_OB, C0_mat_OB, rho_mat_OB = plat_emod[d], plat_C0, plat_rho[d]`) qui avait réglé `E_mat_OB`/`C0_mat_OB` sur les propriétés du **plateau**. Conséquence : si `plat_check=True` et `heat_check=False`, `E_mat_OB` et `C0_mat_OB` sont **réécrasés** par les propriétés propres de la barre (`OB.e_pa[d]`, `OB.C0_pa[d]`), alors que `rho_mat_OB` reste, lui, celui du plateau (`plat_rho[d]`, jamais réécrit par ce bloc). Ces trois variables interviennent ensuite dans le calcul de la force (`F_output = π(DB_punch/2)² · rho_mat_OB · C0_mat_OB² · output_m`, branches compression/traction/cisaillement — §4.1/4.2) et dans le calcul du déplacement/de la déformation (`... - C0_mat_OB · int_eps ...`), ce qui revient — dans ce cas précis (`plat_check=True`, `heat_check=False`) — à mélanger la densité du plateau avec le module et la célérité propres de la barre, plutôt qu'un triplet cohérent (E, ρ, C_0) d'un seul matériau. **Ce point est relevé tel qu'observé dans le code — l'agent documentaliste ne peut pas trancher, à la seule lecture de ce fichier, s'il s'agit d'un comportement voulu ou d'un oubli de portage ; il est signalé explicitement pour vérification.**

3.6 Diamètre/propriétés retenus pour `F_output` vs pour la correction de poinçonnement

```
DB_punch = OB.D_pa[d]                          # toujours le diamètre de la BARRE, jamais celui du plat
nuB_punch, EB_punch = OB.nu_pa[d], OB.e_pa[d]   # (EB_punch réécrit en e_mat_ob si heat_check, §3.4)
bar_diameter   = OB.D_pa[d]
punch_diameter = plat_Dia[d] if options.plat_check else None   # None -> diamètre spécimen/D_eq au p
```

Deux commentaires du code explicitent le raisonnement physique retenu :

> « F_output is computed from strain measured by a gage mounted ON THE BAR, using 1D wave theory (F = E_bar * A_bar * strain) — that relationship holds at the bar's own cross-section regardless of what's beyond the bar's end, so DB_punch (the area term) must always be the bar's own diameter, whether or not a platen is used. »

> « Punch (indentation) correction always happens at the BAR surface […] so nuB_punch/EB_punch (used ONLY in the punch correction, never in F_output above) are always the bar's OWN properties; only the "punch diameter" (specimen vs platen) changes […] »

Autrement dit : dans cette implémentation, la **section utilisée pour F = E·A·ε est toujours celle de la barre de sortie**, jamais celle du plateau ; et le module/coefficient de Poisson utilisés dans le calcul du facteur de poinçonnement K_p sont toujours ceux de la barre — seul le **diamètre de contact** (`punch_diameter`) varie entre plateau et éprouvette selon `plat_check`.

3.7 Module et célérité viscoélastiques (`_viscoelastic_modulus_and_speed`, `disp_visco`)

```
def _viscoelastic_modulus_and_speed():
    ft = time1[1] - time1[0]
    NN = 2 * len(transmitted) - 1
    freqi = np.fft.rfftfreq(NN, d=ft)
    E1, E2, n2 = OB.e11[d], OB.e22[d], OB.n22[d]
    E(f) = ( 1/E1 + E2/(E2² + (2π f n2)²) − i·(2π f n2)/(E2² + (2π f n2)²) )⁻¹   # partie réelle con
    C0(f) = sqrt(Re[E(f)] / OB.rho_pa[d])
```

— `process_di.py::_viscoelastic_modulus_and_speed()` (fonction imbriquée locale). C'est un modèle de solide linéaire standard (standard linear solid) en fréquence complexe, avec E_1, E_2, η_2 = `OB.e11`, `OB.e22`, `OB.n22` (paramètres du matériau de barre viscoélastique). Le vecteur de fréquences `freqi = rfftfreq(2·N_transmitted − 1, dt)` est indexé **élément par élément avec les échantillons temporels de `transmitted`** — c.-à-d. que l'échantillon temporel d'indice i se voit assigner directement $E(\text{freqi}[i])$, sans FFT réelle du signal (ce n'est pas une correction de dispersion fréquence-par-fréquence classique, mais l'association directe d'un indice de fréquence à chaque indice temporel). Cette technique reste non standard pour ce type de correction et sa rigueur physique n'est pas validée indépendamment.

Cette fonction n'est appelée **que si `options.disp_visco` est vrai** : elle remplace, dans chaque branche, le module/célérité constants `E_mat_OB`/`C0_mat_OB` par des tableaux `EE11[i]`/`CC22[i]` variables échantillon par échantillon, et **recalcule intégralement** `F_output`, `displacement`, `n_s`/`t_s` avec ces valeurs.

Important pour l'utilisateur : dans l'onglet Direct Impact actuel de l'UI (`ui/pages/page_process.py::_DITab`), `DIOptions(disp_ela=use_corrected, ...)` est construit **sans jamais passer `disp_visco=True`** — aucune case à cocher de l'onglet DI ne pilote ce champ (il reste à sa valeur par défaut `False` du dataclass). **Cette branche viscoélastique du code n'est donc, à ce jour, pas accessible depuis l'interface graphique du Direct Impact** ; elle n'est exécutable qu'en appelant `process_di()` directement avec `disp_visco=True`.

---

4. Chaîne déformation / vitesse de déformation / force / contrainte

Notations : C_{PG} = `OB.CPG_pa[d]` (coefficient de jauge de la barre de sortie, en 1/V — convertit le signal brut en déformation) ; f_v = `f_inc_velocity[d]` (facteur de calibration de la vitesse incidente, §6) ; $\Delta t_k = t_{k+1} - t_k$ (pas réel, non uniforme, du vecteur `time1`).

4.1 Compression / Traction (`type_check` 1 ou 2)

`process_di()`, branche `elif options.type_check in (1, 2):`.

Signe : $s = -1$ (compression) ou $+1$ (traction) — `sign = -1 if options.type_check == 1 else 1`.

$$\varepsilon_t^{brut}(k) = s \cdot \varepsilon_{transmitted}(k) \cdot C_{PG} \quad \texttt{(output_m)}, \qquad V_{in}(k) = v_{incidente}(k) \cdot f_v \quad \texttt{(V_input)}$$

Intégration (Euler explicite, pas réel `time1[i+1]-time1[i]`, en mètres) :

$$u(k) = \sum_{i=0}^{k} V_{in}(i)\,\Delta t_i, \qquad I_\varepsilon(k) = \sum_{i=0}^{k} \varepsilon_t^{brut}(i)\,\Delta t_i$$

Déformation nominale (engineering strain), classique à « une onde » (côté sortie uniquement) :

$$\varepsilon_{nom}(k) = \frac{u(k) - C_{0,mat}^{OB}\, I_\varepsilon(k)}{L_0}$$

(`n_s`, `core/process_di.py::process_di()`), puis **bornée** dans $(-0.99999,\ 0.99999)$ par `core/process_shpb.py::_cla` (fonction **partagée** avec le SHPB), pour éviter une erreur de domaine du logarithme au step suivant en cas de valeur extrême.

Vitesse de déformation nominale et déformation vraie :

$$\dot{\varepsilon}_{nom}(h) = \frac{\varepsilon_{nom}(h{+}1) - \varepsilon_{nom}(h)}{t_1(h{+}1) - t_1(h)}, \qquad \varepsilon_{vrai}(h) = -\ln\!\big(1 - \varepsilon_{nom}(h)\big)$$

(même formule logarithmique quel que soit compression ou traction — le signe est déjà porté par `sign` en amont).

Force sur la barre de sortie (théorie des ondes 1D, impédance $\rho C_0^2 = E$) :

$$F(k) = \pi\Big(\frac{D_{OB}}{2}\Big)^2 \cdot \rho_{mat}^{OB} \cdot \big(C_{0,mat}^{OB}\big)^2 \cdot \varepsilon_t^{brut}(k)$$

Section de l'éprouvette S : $\pi(D/2)^2$ si cylindrique (`comp_check`/`tens_check` == 1), sinon $w_i \cdot th_i$ (rectangulaire, avec diamètre équivalent $D_{eq} = 2\sqrt{S/\pi}$).

Contrainte nominale et contrainte vraie :

$$\sigma_{nom}(k) = \frac{F(k)}{S}\cdot 10^{-6}\ \text{[MPa]}, \qquad \sigma_{vrai}(k) = \begin{cases} \sigma_{nom}(k)\,(1 - 2\nu\,\varepsilon_{nom}(k)) & \text{compression (type_check=1)} \\ \sigma_{nom}(k)\,(1 + 2\nu\,\varepsilon_{nom}(k)) & \text{traction (type_check=2)} \end{cases}$$

C'est une correction de section « vraie » approximative pondérée par le coefficient de Poisson ν de l'éprouvette (et non l'hypothèse d'incompressibilité classique $\sigma_{vrai} = \sigma_{nom}(1 + \varepsilon_{nom})$).

Si `options.disp_visco` : tout ce bloc (déplacement, ε_{nom}, $\dot{\varepsilon}_{nom}$, ε_{vrai}, F) est **recalculé** en remplaçant $C_{0,mat}^{OB}$ par `CC22[i]` et $\rho_{mat}^{OB} \cdot (C_{0,mat}^{OB})^2$ par `rho_mat_OB · CC22[i]²` (algébriquement $= EE11[i]$).

4.2 Cisaillement (`type_check == 4` : hat-shaped / double / simple)

`process_di()`, branche `else:` (fin de la cascade `if/elif/else`).

Même intégration vitesse/onde transmise que le §4.1, mais la **déformation de cisaillement** est normalisée par une largeur de bande de cisaillement ℓ_u au lieu de la longueur L_0 :

$$\ell_u = \begin{cases} R_2 - R_1 & \text{hat-shaped (‘shear_check‘=1)} \\ w_i & \text{double/simple (‘shear_check‘=2 ou 3)} \end{cases}, \qquad \gamma(k) = \frac{u(k) - C_{0,mat}^{OB}\, I_\varepsilon(k)}{\ell_u}$$

Ici $t_s(h) = \gamma(h)$ directement (**pas de conversion logarithmique** — la « déformation vraie » de cisaillement est prise égale à la déformation nominale).

Aire de cisaillement S_{AS} :

$$S_{AS} = \begin{cases} \pi(R_1 + R_2)(H_2 - H_1) & \text{hat-shaped} \\ 2\, th_i\, L & \text{double} \\ th_i\, L & \text{simple} \end{cases}$$

Force identique au §4.1 : $F(k) = \pi(D_{OB}/2)^2\, \rho_{mat}^{OB}\, (C_{0,mat}^{OB})^2\, \varepsilon_t^{brut}(k)$; contrainte nominale = contrainte vraie = $F/S_{AS}/10^6$ (pas de correction Poisson pour le cisaillement).

Attention — Recalcul viscoélastique incomplet pour le cisaillement double/simple. Le recalcul `if options.disp_visco:` de cette branche (lignes ~410-425) utilise **systématiquement** `specimen.R2_pa[d] - specimen.R1_pa[d]` comme largeur ℓ_u et la formule d’aire `SAS = π(R1+R2)(H2-H1)` (géométrie « hat-shaped ») **sans tester `options.shear_check`**, alors que le calcul non-visco correspondant (juste au-dessus) branche correctement `shear_check` (1 → hat, 2/3 → `wi_pa`). Autrement dit, si `disp_visco=True` **et** `shear_check` vaut 2 ou 3 (double/simple), le recalcul viscoélastique utiliserait `R1_pa/R2_pa/H1_pa/H2_pa` (des grandeurs non pertinentes pour une éprouvette double/simple cisaillement) au lieu de `wi_pa/thi_pa/L_pa`. Ce point est relevé tel qu’observé — **et est de toute façon actuellement inatteignable depuis l’UI DI**, puisque `disp_visco` n’y est jamais activé (§3.7).

4.3 Éprouvette entaillée — Shear-Compression/Tension (`type_check == 3`)

`process_di()`, branche `if options.type_check == 3:`.

Préalable : `core/process_shpb.py::_require_notched_geometry(specimen, d)` — lève une erreur explicite si le rayon d’entaille `specimen.re_pa[d]` n’est pas configuré (≤ 0), pour éviter une division par zéro silencieuse plus loin (fonction **partagée** avec le SHPB). Le message d’erreur rappelle que le « Test mode » (qui détermine `type_check`) est **global au projet**, pas propre à un onglet Processing.

$$F(k) = s \cdot E_{mat}^{OB} \cdot \pi\Big(\frac{D_{OB}}{2}\Big)^2 \cdot \varepsilon_{transmitted}(k) \cdot C_{PG}, \qquad \varepsilon_t^{brut}(k) = s \cdot \varepsilon_{transmitted}(k) \cdot C_{PG}$$

avec $s = -1$ si `tenscomp_check==1` (compression) sinon $+1$. **Remarque** : ici F utilise directement E_{mat}^{OB} (pas la forme ρC_0^2 des §4.1/4.2). Les deux formes sont mathématiquement identiques pour un matériau homogène et cohérent ($E = \rho C_0^2$), mais **divergent** dans le cas particulier signalé au §3.5 (`plat_check=True`, `heat_check=False`) où `E_mat_OB` et `rho_mat_OB·C0_mat_OB²` ne correspondent plus au même triplet de propriétés.

Intégration (ici en **millimètres**, `dtime` étant fixé une fois pour toutes à `time1[1]-time1[0]`, contrairement au pas réel variable des §4.1/4.2) :

$$u_{mm}(k) = \sum_{i=0}^{k} V_{in}(i) \cdot dtime \cdot 1000, \qquad I_{\varepsilon}(k) = \sum_{i=0}^{k} \varepsilon_t^{brut}(i) \cdot dtime$$

$$d(k) = u_{mm}(k) - 1000 \cdot C_{0,mat}^{OB} \cdot I_{\varepsilon}(k) \quad (\text{‘displacement‘, en mm})$$

Déformation de cisaillement/entaille, **polynôme cubique calibré** (coefficients `k_value[0..2]`), normalisé par

$$\text{denom} = \frac{2000 \cdot r_e}{\cos(\theta_{entaille})}$$

(`re_pa` = rayon d’entaille, `NoA_pa` = angle d’entaille) :

$$\gamma(k) = k_0 \cdot \xi(k) + k_1 \cdot \xi(k)^2 + k_2 \cdot \xi(k)^3, \qquad \xi(k) = \frac{d(k) - 1000\, d_y}{\text{denom}}$$

(d_y = `specimen.dy_pa[d]`, déplacement au seuil de plastification), puis γ est rebasé pour démarrer à 0 : $\gamma(k) \mathrel{-}= \gamma(0)$.

Contrainte, **calibration empirique** (coefficients `k_value[3..5]`) :

$$\sigma(k) = \frac{k_3 \,(1 - k_4\, \gamma(k)) \cdot |F(k)|^{k_5}}{10^6 \cdot th \cdot D}$$

(th = `specimen.th_pa[d]`, épaisseur de zone effective ; D = diamètre de l’éprouvette, tous deux convertis m→mm via ×1000).

Calibration à deux jeux de coefficients (`sk_check`) : si activé, chaque échantillon utilise `k_value` si `d(k) ≤ sk_value[6]`, sinon `sk_value` — un seuil de déplacement permet de basculer vers un second jeu de coefficients (`k0..k5` + seuil `sk_value[6]`), pour représenter un comportement bi-linéaire/bi-régime de l’entaille.

À la fin de la branche : `n_s, n_stress = t_s, t_stress` — **pas de distinction nominal/vrai** pour l’éprouvette entaillée (contrairement aux branches compression/traction).

Attention — Unité du déplacement final ambiguë pour `type_check == 3`. Juste après le calcul de σ (ligne `displacement = [v / 1000 for v in displacement]`), le code **divise** `displacement` par 1000 (mm → m). Or le commentaire de fin de fonction affirme : « `displacement` is in mm already for type_check == 3 (matches the original, which multiplies by 1000 there) », et la valeur finale exposée dans `DIResult.displacement` (`displacement_mm = displacement if options.type_check == 3 else [1000 * v for v in displacement]`) **n’applique aucune reconversion** pour `type_check==3`. En suivant strictement les unités du code (accumulation en mm au §« intégration » ci-dessus, confirmée par la comparaison à `1000*dy_pa`), la valeur stockée dans `displacement_mm` pour une éprouvette entaillée semble donc être en **mètres**, malgré son nom de champ. Ce point contredit littéralement le commentaire adjacent dans le code lui-même ; il est signalé explicitement pour vérification par l’utilisateur (comparaison chiffrée avec l’application d’origine recommandée) plutôt que corrigé ou réinterprété par cet agent.

---

5. Corrections physiques complémentaires

Les quatre corrections ci-dessous sont **identiques dans leur mécanisme mathématique** au SHPB — mêmes fonctions/formules — mais s'appliquent en Direct Impact **uniquement du côté barre de sortie**, puisqu'il n'existe pas de barre d'entrée instrumentée en DI (pas de moyenne/choix entre force côté entrée et côté sortie, contrairement à ce qu'un SHPB à deux barres peut permettre).

5.1 Poinçonnement / indentation (`punch_check`)

Facteur géométrique elliptique **partagé** avec le SHPB — `core/process_shpb.py::_elliptic_punch_factor(d_ratio)` :

$$H_x(x) = 2 - \Big(x + \frac{1}{x}\Big)E(x) - \Big(x - \frac{1}{x}\Big)K(x), \qquad E(x) = \int_0^{\pi/2} \sqrt{1 - x^2 \sin^2\theta}\, d\theta,\ K(x) = \int_0^{\pi/2} \frac{d\theta}{\sqrt{1 - x^2 \sin^2\theta}}$$

avec $x = d_{ratio} = d_{punch}/d_{barre}$, valide seulement pour $0 < x < 1$ (sinon `ValueError` explicite — le diamètre du poinçon/plateau doit être strictement supérieur à celui de l'éprouvette).

Rigidité de contact :

$$K_p = H_x(d_{ratio}) \cdot \frac{16}{3\pi^2} \cdot \frac{1 - \nu_{barre}^2}{d_{punch} \cdot E_{barre}}$$

où $\nu_{barre} =$ `nuB_punch` et $E_{barre} =$ `EB_punch` sont **toujours** les propriétés de la barre de sortie elle-même (§3.6), et $d_{punch} =$ `punch_diameter` vaut le diamètre du plateau si `plat_check`, sinon le diamètre (ou diamètre équivalent) de l'éprouvette.

Application (schématiquement, avec des formes légèrement différentes selon la branche — coefficients et dénominateurs adaptés à la géométrie de chaque type d'essai) : - Compression/traction (§4.1) : $p_s(k) = 2 \times 10^6\, K_p\, \sigma_{nom}(k)\, \dfrac{\pi D_{eq}^2/4}{L_0}$, puis $\varepsilon_{nom}\ -= p_s$ (reclampée), et $\varepsilon_{vrai}/\sigma_{vrai}$ **recalculés** — mais voir remarque ci-dessous. - Entaillé (§4.3) : $p_s(k) = 2 \times 10^6\, K_p\, \sigma(k)\, \dfrac{\pi D^2/4}{D}$, puis $\gamma\ -= p_s$. - Cisaillement (§4.2) : même forme avec D_{eq} propre à la géométrie de cisaillement.

Attention — Recalcul de `t_stress` avec un signe fixe (branche compression/traction), quel que soit `type_check`. Dans le bloc de recalcul post-poinçonnement de la branche 4.1 (`if options.punch_check:`), la contrainte vraie est recalculée par `t_stress.append(n_stress[i] * (1 - 2 * specimen.nu_pa[d] * n_s[i]))` — **toujours** la forme « compression » `(1 − 2ν·ε)`, alors que le calcul initial de `t_stress` (avant correction de poinçonnement, quelques lignes plus haut dans la même branche) branche correctement sur `(1 − 2ν·ε)` en compression et `(1 + 2ν·ε)` en traction. Autrement dit, si `punch_check=True` **et** `type_check==2` (traction), la contrainte vraie finale utiliserait la formule « compression » au lieu de la formule « traction ». Ce point est relevé tel qu'observé dans le code, sans correction ni interprétation supplémentaire de la part de cet agent.

5.2 Friction (`friction[d] != 0`)

Correction classique (type Bertholf/Cowper) avec s_0 = élancement de l'éprouvette (longueur / diamètre) et f = coefficient de friction :

$$m = 1 - \frac{f}{3s_0}$$

- **Compression/traction (§4.1) et cisaillement (§4.2)** : $\sigma_{nom} \mathrel{*}= m$, $\sigma_{vrai} \mathrel{*}= m$, puis $\sigma_{friction} = \frac{f}{3s_0} \cdot \sigma_{vrai}^{\text{corrigé}}$ est calculée **pour information/affichage seulement** (pas retranchée une seconde fois).
- **Entaillé (§4.3)** : même m et $\sigma_{vrai} \mathrel{*}= m$, **mais** le code retranche **en plus** $\sigma_{friction}$ de σ_{vrai} (`t_stress = [t_stress[i] - friction_stress[i] ...]`, absent des deux autres branches). Comme $\sigma_{friction} = \frac{f}{3s_0} \cdot (m \cdot \sigma_{vrai}^{brut})$, cela revient — pour la seule branche entaillée — à $\sigma_{vrai}^{final} = m^2 \cdot \sigma_{vrai}^{brut}$ (application implicite du facteur m **deux fois**), au lieu de $m \cdot \sigma_{vrai}^{brut}$ pour les autres types d'essai. Cette **asymétrie entre branches est relevée telle qu'observée** dans le code, sans présumer laquelle des deux formes est la formule physiquement voulue.

5.3 Inertie radiale/axiale (`inertia_check`)

Formulation classique à deux termes (accélération radiale + axiale), avec $s_0 = L/D$ (ou D_{eq}) :

$$c_1 = \frac{\rho\, D^2\, (s_0^2 - 3/16)}{12}, \qquad c_2 = \frac{3\,\rho\, D^2}{64}$$

$$\sigma_{inertie}(k) = \frac{c_1\,(\dot{\varepsilon}(k)^2 + \ddot{\varepsilon}(k))}{10^6} + \frac{c_2\,\ddot{\varepsilon}(k)}{10^6}, \qquad \sigma_{vrai} \mathrel{-}= \sigma_{inertie}$$

$\dot{\varepsilon}$ et $\ddot{\varepsilon}$ (vitesse et « accélération » de déformation vraie) sont calculés par différences finies sur un pas de temps **robuste** (`core/process_shpb.py::_robust_dt()`, **partagée** avec le SHPB : médiane des écarts successifs de `time1`, plutôt qu'un pas fixe `time1[2]-time1[1]` qui casse si deux échantillons consécutifs partagent le même horodatage) — **sauf dans la branche entaillée (§4.3), qui utilise `dtime = time1[1]-time1[0]` fixé une fois pour toutes** (pas `_robust_dt`), pour tous ses calculs de $\dot{\gamma}$/$\ddot{\gamma}$.

Pour l'entaillé, la géométrie est projetée à 45° (`cos_a = cos(π/4)`), reflétant la forme caractéristique d'une éprouvette shear-compression/tension chargée à 45°.

5.4 Échauffement adiabatique (température)

Commun aux 4 types d'essai, calculé **après** la branche de type d'essai (`process_di()`, fin de fonction) :

$$\text{coeff}_{temp} = \frac{\xi}{c_p\,\rho}, \qquad T(k{+}1) = T(k) + \text{coeff}_{temp} \cdot \sigma_{vrai}(k) \cdot 10^6 \cdot (\varepsilon_{vrai}(k{+}1) - \varepsilon_{vrai}(k))$$

avec $T(0) = T_0$ (`T0_pa[d]`), ξ = coefficient de Taylor-Quinney de l'éprouvette (`specimen.xi_pa[d]`), c_p = chaleur spécifique (`specimen.cp_pa[d]`), ρ = masse volumique de l'éprouvette. C'est une intégration incrémentale de l'échauffement adiabatique dû au travail plastique (hypothèse adiabatique, fraction ξ du travail plastique convertie en chaleur).

6. Recalage du profil de vitesse incidente (« Cut »/« shift ») et facteur de calibration

Ce mécanisme est entièrement côté UI (`ui/pages/page_process.py::_DITab`) — `process_di()` lui-même ne fait que **consommer** le tableau `velocity_c` déjà recalé (voir §3.1 : `velocity = velocity_c[0:i_it]`, indice 0 supposé synchronisé avec `xd1`).

6.1 Découpe/décalage (« Cut velocity profile at selected start » / « shift by (s) »)

`_DITab._apply_velocity_cut(cut_time)` :

```
time_arr = np.asarray(self._time_pulse)
below = np.where(time_arr < cut_time)[0]
idx = int(below[-1]) if len(below) else 0
self._velocity_c = self._velocity_c[idx:]
```

— trouve l'indice `idx` du dernier échantillon de `time_pulse` **strictement antérieur** à `cut_time`, puis **tronque** les `idx` premiers échantillons du tableau de vitesse incidente. Le tableau résultant redémarre donc son indice 0 à l'instant `cut_time` (approximativement, à la résolution de `time_pulse`), réalignant la vitesse incidente sur l'axe temporel du signal de barre — exactement l'hypothèse que `process_di()` fait ensuite (`velocity_c[0] <-> t = xd1`).

Deux façons d'appeler cette fonction : - **« Cut velocity profile at selected start »** (`_on_cut_velocity`) : utilise le point de départ de la fenêtre « Velocity start cut » sélectionnée à la souris sur le graphique (`self.picker.get_window("velocity_cut")[0]`). - **« shift by (s) » + Apply** (`_on_apply_velocity_shift`) : utilise directement la valeur numérique saisie dans le champ (en secondes) comme `cut_time`.

Les coupes sont **cumulables** : `self._cumulative_velocity_idx` totalise les échantillons retirés au fil de plusieurs appels, et `project.di.velocity_cut_time` mémorise un **instant de coupe équivalent unique**, exprimé sur l'axe temporel **d'origine** (non coupé), pour pouvoir réappliquer la coupe cumulée en une seule fois après un rechargement de session (`self.velocity_cut_time` restauré et réappliqué dans `_DITab.__init__`/au chargement du fichier vitesse). Le bouton **« Reset cut »** (`_on_reset_velocity_cut`) recharge le fichier brut depuis le disque et remet `_cumulative_velocity_idx = 0`.

6.2 Facteur de calibration multiplicatif de la vitesse incidente

Champ UI « Incident-velocity calibration factor » (`self.f_inc_velocity_edit`, défaut `1.0`) → `f_inc_velocity` → étendu en tableau constant `f_inc_arr = np.full(n, f_inc_velocity)` (une valeur par tirage Monte-Carlo, **mais identique pour tous les tirages** — pas de distribution d'incertitude attachée à ce facteur dans l'UI actuelle, comme pour `SG_unc_d`, §3.1) → passé à `process_di(..., f_inc_velocity=f_inc_arr, ...)`.

Appliqué de façon strictement multiplicative, dans **chaque** branche de `process_di()` :

$$V_{in}(k) = v_{incidente}(k) \cdot f_v[d] \qquad \texttt{(V_input = velocity * f_inc_velocity[d])}$$

C'est un facteur d'échelle global sur toute la vitesse incidente mesurée — destiné à compenser un biais de calibration systématique du capteur de vitesse (ex. barrière laser mal étalonnée), sans altérer la forme

temporelle du profil de vitesse.

6.3 Aide auxiliaire pour capteurs personnalisés montés sur l'éprouvette

Sans lien direct avec le profil de vitesse incidente lui-même, mais dans le même onglet : le bouton « **Auto-cut using wave travel time »** (par capteur personnalisé « Custom Sensor » monté sur l'éprouvette, `_DITab._on_align_sensor`) calcule automatiquement un point de coupe pour ce capteur via

$$t_{coupe} = t_{début\ onde\ transmise} - \Delta t_{total}$$

où Δt_{total} = `core/specimen_sensor_alignment.py::specimen_sensor_delay_direct_impact()` — voir §7 ci-dessous — car en Direct Impact, la jauge de la barre de sortie voit l'onde **après** qu'elle a déjà parcouru la barre + le demi-éprouvette (+ éventuellement le plateau) depuis l'instant où l'éprouvette a commencé à se charger.

---

7. Réutilisation de `toe_correction.py` et `specimen_sensor_alignment.py`

Ces deux modules sont **réutilisés tels quels** par le Direct Impact, **sans aucune variante spécifique au DI** dans leur logique de calcul :

- **`core/toe_correction.py`** (`detect_linear_region`, `fit_linear_region`, `apply_toe_correction`) : identique au SHPB, appliqué en aval de `process_di()` dans l'UI (`_DITab`) pour retirer le « toe » (mise en contact progressive) de la courbe contrainte-déformation à l'aide d'une courbe de calibration enregistrée sur une éprouvette rigide/de référence. Depuis la correction décrite au chapitre SHPB §5.5, `_DITab._on_toe_apply()` (lignes 3575-3625) ne se contente plus d'un unique appel `apply_toe_correction(strain, stress, offset, trim=True)` sur la courbe moyenne agrégée : il réapplique le même offset à **chaque tirage Monte-Carlo individuel** via `_apply_toe_correction_to_draws()` (fonction partagée avec `_SHPBTab`, `ui/pages/page_process.py`, lignes 147-212), puis ré-agrège — garantissant que `project.last_raw_draws["DI"]` (page Covariance Analysis) et les grandeurs de confinement par tirage (chapitre Confinement, §5.8, actives sur cet onglet dans les mêmes conditions que sur SHPB) restent cohérentes avec la courbe corrigée, au lieu de rester décalées de l'ancien « pied » retiré uniquement sur l'agrégat. Le mécanisme, ses garanties (aucune nouvelle simulation `process_di()` relancée — c'est un pur changement de repère sur l'axe de déformation, appliqué à des résultats déjà calculés) et ses limites (compteur `toe_n_removed` = médiane inter-tirages, voir chapitre SHPB §8, points 11-12) sont **rigoureusement identiques** entre `_SHPBTab` et `_DITab` — se reporter au chapitre SHPB §5.5 pour le détail complet plutôt que de le dupliquer ici.
- **`core/specimen_sensor_alignment.py`** : contient **deux** fonctions parallèles, `specimen_sensor_delay_shpb()` (deux termes : barre d'entrée + demi-éprouvette [+ plateau]) et **`specimen_sensor_delay_direct_impact()`** (utilisée par le DI) :

```
def specimen_sensor_delay_direct_impact(
    output_gage_position, c0_output_bar, sample_length, c0_sample,
    use_platens=False, platens_length=0.0, c0_platens=0.0,
```

```
):
    bar_term = output_gage_position / c0_output_bar
    specimen_term = sample_length / (2 * c0_sample)
    platen_term = platens_length / c0_platens if use_platens and c0_platens else 0.0
    return DelayBreakdown(bar_term, platen_term, specimen_term, total=bar_term+platen_term+specimen_
```

Mathématiquement **identique** à la version SHPB (mêmes trois termes : position de jauge / célérité de barre, demi-longueur d'éprouvette / célérité d'éprouvette, longueur de plateau / célérité de plateau), la seule différence étant que le DI n'a **qu'un seul terme « barre »** (celui de la barre de sortie) puisqu'il n'y a pas de barre d'entrée à prendre en compte.

Ces deux modules ne sont **pas appelés à l'intérieur de `process_di.py`** lui-même : le toe correction s'applique en aval sur les résultats déjà produits (courbe contrainte-déformation), et l'alignement capteur/spécimen est un outil d'assistance au réglage de la fenêtre de découpe, utilisé en amont par l'UI (`_DITab`), pas une étape du pipeline `process_di()`.

---

8. Table de correspondance équations <-> code

Grandeur / étape	Fichier :: fonction (ou bloc)
Découpe fenêtre transmise + vitesse incidente	`core/process_di.py::process_di()` (bloc « Cutting waves »)
Recherche d'index le plus proche	`core/process_shpb.py::_index_of_nearest()`
Substitution signal dispersion-corrigé	`core/process_di.py::process_di()` (`disp_ela`/`disp_visco`)
Correction platen (coefficient de transmission)	`core/process_di.py::process_di()` (bloc « Platen correction »)
Correction gradient thermique de barre (modèle RTE / table)	`core/process_shpb.py::_bar_gradient_wave_speed_modulus()` (partagée)
Diamètre/propriétés pour F vs poinçonnement	`core/process_di.py::process_di()` (`DB_punch`, `nuB_punch`, `EB_punch`, commentaires dédiés)
Module/célérité viscoélastiques (fréquence par échantillon)	`core/process_di.py::_viscoelastic_modulus_and_speed()` (interne à `process_di()`)
Branche compression/traction	`core/process_di.py::process_di()` (`elif options.type_check in (1, 2):`)
Branche cisaillement	`core/process_di.py::process_di()` (`else:` finale)
Branche entaillée Shear-Comp./Tens.	`core/process_di.py::process_di()` (`if options.type_check == 3:`)
Garde-fou géométrie entaillée	`core/process_shpb.py::_require_notched_geometry()` (partagée)
Bornage déformation nominale	`core/process_shpb.py::_clamp_nominal_strain()` (partagée)

Grandeur / étape	Fichier :: fonction (ou bloc)
Facteur elliptique de poinçonnement H_x	`core/process_shpb.py::_elliptic_punch_factor()` (partagée)
Pas de temps robuste (vitesse/accél. de déformation)	`core/process_shpb.py::_robust_dt()` (partagée)
Échauffement adiabatique	`core/process_di.py::process_di()` (fin de fonction, `coeff_temp/temperature`)
Mise en forme finale (mm) et retour	`core/process_di.py::process_di()` (`displacement_mm`, `DIResult`)
Point d'entrée worker Monte-Carlo	`core/process_di.py::_process_di_task()`
Découpe/décalage du profil de vitesse incidente	`ui/pages/page_process.py::_DITab._apply_velocity_cut()`, `_on_cut_velocity()`, `_on_apply_velocity_shift()`, `_on_reset_velocity_cut()`
Facteur de calibration vitesse incidente	`ui/pages/page_process.py::_DITab` (champ `f_inc_velocity_edit`) → `process_di(f_inc_velocity=...)`
Alignement capteur personnalisé (auto-cut)	`ui/pages/page_process.py::_DITab._on_align_sensor()` + `core/specimen_sensor_alignment.py::specimen_sensor_delay`
Correction de toe (en aval, post-traitement, recoupe chaque tirage Monte-Carlo)	`core/toe_correction.py::apply_toe_correction()`, `true_strain_from_nominal()` (partagées) + `ui/pages/page_process.py::_apply_toe_correction_to_draws` (partagée) via `_DITab._on_toe_apply()` — voir chapitre SHPB §5.5

---

9. Limitations, avertissements et points à vérifier

Avertissement explicite en tête de fichier (`core/process_di.py`, docstring du module) :

> « NOT YET independently re-validated against real experimental data across the full option matrix (heat x platen x punch x friction x inertia x viscoelastic bar x 4 kinematics) — exercised with synthetic and real signals only on the simplest option combination so far. Cross-check important results against the original application before relying on this for a real analysis. »

C'est-à-dire : **seule la combinaison d'options la plus simple** a été testée avec des signaux réels/synthétiques ; le croisement complet chaleur × plateau × poinçonnement × friction × inertie × barre viscoélastique × 4 cinématiques d'essai **n'a pas été revalidé indépendamment**. À documenter clairement à l'utilisateur final comme une réserve méthodologique majeure sur ce module.

Autres points relevés dans le code lui-même ou déduits d'une lecture précise, à faire vérifier par l'utilisateur :

1. **§3.5** — Lorsque `plat_check=True` et `heat_check=False`, `E_mat_OB/C0_mat_OB` sont réécrasés par les propriétés de la **barre** alors que `rho_mat_OB` reste celle du **plateau** : triplet (E, ρ, C_0) potentiellement incohérent utilisé dans le calcul de force et de déplacement.
2. **§4.3** — La formule de déplacement final `displacement_mm` pour l'éprouvette entaillée (`type_check==3`) semble, par un suivi strict des unités dans le code, exprimée en **mètres** malgré son nom et malgré le commentaire adjacent affirmant qu'elle est « déjà en mm ».
3. **§4.2** — Le recalcul viscoélastique (`disp_visco`) de la branche cisaillement utilise inconditionnellement la géométrie « hat-shaped » (`R1/R2/H1/H2`), même pour `shear_check = double` ou `simple`. Sans conséquence pratique tant que `disp_visco` reste inaccessible depuis l'UI DI (point 5 ci-dessous).
4. **§5.1** — Le recalcul de la contrainte vraie après correction de poinçonnement (branche compression/traction) utilise toujours la formule « compression » (`1−2ν·ε`), même en traction (`type_check==2`).
5. **§5.2** — La correction de friction retranche `friction_stress` une seconde fois dans la seule branche entaillée (équivalent à appliquer le facteur m au carré), contrairement aux branches compression/traction et cisaillement.
6. **§3.7 / §4.2** — `options.disp_visco` n'est **jamais activé depuis l'onglet Direct Impact de l'UI actuelle** (`ui/pages/page_process.py::_DITab`) : seul `disp_ela` (case « Use dispersion-corrected signal ») y est exposé. La branche viscoélastique du code est présente et fonctionnelle en tant qu'API mais actuellement inatteignable par un utilisateur de l'application.
7. **§3.1** — `SG_unc_d` et `f_inc_velocity` sont des paramètres per-tirage Monte-Carlo côté `process_di()`, mais l'UI DI leur assigne systématiquement une valeur **constante** (`np.ones(n)` / `np.full(n, f_inc_velocity)`) : ni l'incertitude sur le gain de jauge de la barre de sortie, ni celle sur le facteur de calibration de la vitesse incidente, ne sont actuellement échantillonnées par tirage.
8. **`_require_notched_geometry`** rappelle explicitement que `type_check` dépend du « Test mode » **global au projet** (page Test Setup), pas d'un choix par onglet Processing : traiter un essai entaillé puis, sans changer le Test mode, un essai compression/traction/cisaillement sur un autre onglet, réutilise silencieusement la branche entaillée avec des paramètres d'éprouvette non pertinents — sauf que le garde-fou lève désormais une erreur explicite si le rayon d'entaille est nul.
9. **§7** — Le recoupage par tirage de la correction de toe (`_apply_toe_correction_to_draws()`) hérite des mêmes deux nuances que sur l'onglet SHPB (chapitre SHPB, §8, points 11-12) : le compteur `toe_n_removed` utilisé pour recaler un capteur de confinement/laser resté en signal brut est une **médiane** sur l'ensemble des tirages, pas un décompte exact tirage par tirage ; et ce recalage ne corrige qu'un décalage en tête, pas une éventuelle différence de cadence d'échantillonnage entre le capteur et la courbe contrainte-déformation.

Ces neuf points sont rapportés tels qu'observés dans le code source, sans réinterprétation ni correction de la part de cet agent : ils doivent être vérifiés (et, le cas échéant, confirmés ou infirmés) par comparaison avec l'application d'origine avant d'être présentés comme des défauts avérés dans la documentation finale.

5. Essai quasi-œdométrique (confiné) : contrainte radiale, pression hydrostatique et déviateur

Fichier source principal : `core/confinement.py` (482 lignes, lu intégralement). Fichiers de contexte : `ui/pages/page_special_tests.py` (paramétrage utilisateur), `ui/pages/page_process.py` (boucles Monte-Carlo SHPB / Impact Direct / Quasi-Statique, qui calculent elles-mêmes les grandeurs de confinement par tirage — voir §5.8), `ui/pages/page_results_export.py` (carte « Confinement / Triaxial Stresses », qui préfère ces données réelles par tirage quand elles existent), `ui/pages/page_covariance.py` (page Covariance Analysis, même préférence), `core/project.py` (dataclass `ConfinementSettings`, avec les constantes de `corr_file.py`).

---

1. Principe physique de l'essai

L'essai quasi-œdométrique (ou essai confiné) est un essai de compression (SHPB, impact direct ou quasi-statique) dans lequel l'éprouvette est placée dans une **cellule de confinement** rigide qui empêche (ou limite fortement) sa déformation radiale libre. La cellule elle-même est instrumentée par une **jauge de déformation** collée sur sa paroi externe, qui mesure la **déformation orthoradiale** de la cellule sous l'effet de la pression exercée par l'éprouvette qui tend à se dilater radialement pendant la compression axiale.

Ce déplacement/cette déformation orthoradiale de la cellule est ensuite convertie, via une calibration (linéaire ou non-linéaire, voir §2-3), en **contrainte radiale** appliquée par l'éprouvette sur la paroi de la cellule — qui est, par hypothèse du montage (confinement cylindrique), assimilée à la contrainte radiale (= orthoradiale, par symétrie de révolution) régnant dans l'éprouvette.

À partir de ce couple (contrainte axiale, contrainte radiale) — la contrainte axiale provenant du dépouillement standard SHPB/Impact Direct/Quasi-Statique déjà réalisé sur un autre onglet — on calcule les invariants de contrainte usuels d'un chemin triaxial : la **pression hydrostatique p** et le **déviateur de contrainte q** (§5), ainsi que les déformations radiale, volumétrique et équivalente (von Mises) de l'éprouvette (§4-5).

Ce module regroupe la fonction `confine_show()` (et le bloc « laser stress ») ainsi que les constantes de calibration (notamment `corr_file.nonlinear_check`, qui sélectionne le modèle linéaire/non-linéaire de

cellule) — voir `core/confinement.py`, lignes 1-21.

Dans l'UI (`ui/pages/page_special_tests.py`, Card « Confinement (Quasi-Oedometric Test) »), l'essai confiné nécessite : - de cocher « This is a quasi-oedometric (confined) test » (→ `project.confinement.enabled`) ; - d'assigner à un Custom Sensor le rôle « Confinement cell strain gage » (Custom Sensors page) — c'est ce capteur qui fournit le signal `cell_deflection/cell_defl` utilisé dans tous les calculs ci-dessous ; - de choisir le modèle de cellule (« Linear » ou « Nonlinear », `project.confinement.nonlinear_enabled`) et de renseigner les constantes de calibration correspondantes.

---

2. Modèle LINÉAIRE de cellule

2.1 Formule

Implémentation : `core/confinement.py::linear_cell_radial_stress()` (lignes 40-54).

```python
def linear_cell_radial_stress(
    axial_strain, cell_deflection, cell_modulus, C1, C2, ref_strain,
):
    e_cell_deff = cell_modulus * cell_deflection
    blend = (eps / ref_strain) * C2 + (1 - eps / ref_strain) * C1
    return e_cell_deff * blend / 1e6
```

En notant ε_a la déformation axiale (`axial_strain`, en fait la déformation nominale/technique issue du résultat SHPB/DI/QS — voir `strain_key = "nominal_strain" if "nominal_strain" in result else "strain"` dans `page_results_export.py` ligne 814), δ_{cell} le signal de déflexion/déplacement orthoradial calibré de la cellule (`cell_deflection`), E_v le module de cellule (`cell_modulus`, en Pa), ε_{ref} la déformation de référence (`ref_strain`), la contrainte radiale (en MPa, du fait de la division par 10^6) vaut :

$$E_{cell,deff} = E_v \cdot \delta_{cell}$$

$$\sigma_r = \frac{E_{cell,deff}}{10^6}\left[\frac{\varepsilon_a}{\varepsilon_{ref}}\,C_2 + \left(1 - \frac{\varepsilon_a}{\varepsilon_{ref}}\right) C_1\right]$$

soit, en un seul bloc :

$$\sigma_r(\varepsilon_a) = \frac{E_v\,\delta_{cell}}{10^6}\left[C_1 + \frac{\varepsilon_a}{\varepsilon_{ref}}\,(C_2 - C_1)\right]$$

2.2 Interprétation — C1, C2 et le mécanisme de transition

- C_1 (« low-strain calibration constant », UI : « C1 (low-strain calibration constant) ») est le coefficient de calibration appliqué au régime **bas niveau de déformation axiale** ($\varepsilon_a \to 0$) ;

- C_2 (« high-strain calibration constant ») est le coefficient appliqué au régime **haut niveau de déformation** ($\varepsilon_a \to \varepsilon_{ref}$) ;
- E_v (« Cell modulus Ev (Pa) ») est un module effectif de la cellule qui, multiplié par le signal de déflexion `cell_deflection`, donne un terme homogène à une contrainte, `E_cell_deff` (nommé `Ecell_deff` dans le code original) ;
- ε_{ref} (« Reference strain ») est la déformation axiale à laquelle la seconde courbe de calibration (voir §3) a été établie ; ici elle sert de **pivot de transition linéaire** entre C_1 et C_2.

Le terme `blend = (eps/ref_strain)*C2 + (1-eps/ref_strain)*C1` est donc une **interpolation linéaire (affine) en fonction de** $\varepsilon_a/\varepsilon_{ref}$ entre C_1 (poids 1 à $\varepsilon_a = 0$) et C_2 (poids 1 à $\varepsilon_a = \varepsilon_{ref}$). Le code n'impose **aucun clamp** : pour $\varepsilon_a > \varepsilon_{ref}$, `blend` continue de s'extrapoler linéairement au-delà de C_2 (extrapolation, pas de saturation), et de même pour $\varepsilon_a < 0$. Ceci n'est pas documenté explicitement comme une limitation dans le code, mais mérite d'être signalé : la formule suppose implicitement que l'essai reste dans la plage $[0, \varepsilon_{ref}]$ de la calibration.

Les vecteurs `axial_strain` et `cell_deflection` sont tronqués à la longueur commune la plus courte (`n = min(len(axial_strain), len(cell_deflection))`) avant tout calcul.

---

3. Modèle NON-LINÉAIRE de cellule

3.1 Vue d'ensemble

Implémentation : `core/confinement.py::nonlinear_cell_radial_stress()` (lignes 143-160), avec la fonction utilitaire `_eval_poly_no_constant()` (lignes 90-97).

Cette fonction est structurée en deux étapes séparées : - `fit_nonlinear_calibration()` (lignes 100-119) : ajuste les deux polynômes de calibration degré 6 (ce que fait l'étape 1 ci-dessous) et les retourne (`poly_ini`, `poly_esp`) ; - `nonlinear_cell_radial_stress_precomputed()` (lignes 122-140) : évalue le modèle (l'étape 3 ci-dessous) à partir de polynômes déjà ajustés, sans refaire le `np.polyfit`.

`nonlinear_cell_radial_stress()` elle-même (lignes 143-160) enchaîne les deux — `fit_nonlinear_calibration(...)` puis `nonlinear_cell_radial_stress_precomputed(...)` — et donne le même résultat pour tout appelant (`_refresh_confinement`, `triaxial_synthetic_draws`). L'intérêt de cette séparation : une boucle Monte-Carlo qui évalue ce modèle des centaines de fois par tirage (§5.8) ajuste les deux polynômes **une seule fois**, avant la boucle, au lieu de refaire un `np.polyfit` (l'opération la plus coûteuse du modèle) à chaque tirage.

Le modèle non-linéaire repose sur **deux courbes de calibration numérique** (typiquement issues de simulations numériques de la cellule de confinement), chacune reliant la déformation orthoradiale de la cellule à la contrainte radiale :

- une courbe « **ini** » : calibration à la **hauteur initiale** de l'éprouvette (L_{ini}) — fichiers `Orthoradial displacement, initial (.txt)` / `Radial stress, initial (.txt)` sur la page Special Tests, correspondant aux paramètres `ortho_disp_ini`, `radial_ini` ;

- une courbe « **esp** » : calibration à la **hauteur de référence** (L_{esp}, hauteur de l'éprouvette lorsque sa déformation axiale vaut ε_{ref}) — fichiers `Orthoradial displacement, at reference strain (.txt)` / `Radial stress, at reference strain (.txt)`, paramètres `ortho_disp_esp`, `radial_esp`.

3.2 Étape 1 — Normalisation en déformation orthoradiale et ajustement polynomial

```
r_cell = cell_diameter / 2
ortho_def_ini = np.asarray(ortho_disp_ini) / r_cell
ortho_def_esp = np.asarray(ortho_disp_esp) / r_cell
radial_ini_mpa = np.abs(np.asarray(radial_ini)) / 1e6
radial_esp_mpa = np.abs(np.asarray(radial_esp)) / 1e6

ff1 = np.polyfit(ortho_def_ini, radial_ini_mpa, 6)
ff2 = np.polyfit(ortho_def_esp, radial_esp_mpa, 6)
```

Le **diamètre de cellule** D_{cell} (`cell_diameter`, paramètre « Cell diameter Dcell (m) ») sert uniquement à convertir le **déplacement** orthoradial mesuré/tabulé (en mètres) en **déformation** orthoradiale sans dimension, par division par le rayon de cellule :

$$r_{cell} = \frac{D_{cell}}{2}, \qquad \varepsilon^{ini}_{ortho} = \frac{u^{ini}_{ortho}}{r_{cell}}, \qquad \varepsilon^{esp}_{ortho} = \frac{u^{esp}_{ortho}}{r_{cell}}$$

Les contraintes radiales tabulées des deux courbes de calibration sont prises en **valeur absolue** puis converties en MPa (division par 10^6) : $\sigma_r^{ini} = |\sigma^{ini}_{r,\text{fichier}}|/10^6$, $\sigma_r^{esp} = |\sigma^{esp}_{r,\text{fichier}}|/10^6$.

Chaque paire (déformation orthoradiale, contrainte radiale) est ensuite ajustée par un **polynôme de degré 6** (`np.polyfit(..., 6)`), donnant deux jeux de 7 coefficients `ff1` (courbe « ini ») et `ff2` (courbe « esp »).

3.3 Étape 2 — Évaluation du polynôme sans terme constant

`_eval_poly_no_constant()` (lignes 57-64) évalue le polynôme de degré 6 **en omettant volontairement son terme constant** (`coeffs[6]`) :

$$P(x) = c_0x^6 + c_1x^5 + c_2x^4 + c_3x^3 + c_4x^2 + c_5x$$

Le docstring précise explicitement que ceci **force la courbe à passer par l'origine** (déformation orthoradiale nulle → contrainte radiale nulle), jugé « physiquement sensé » (« physically sensible ») pour cette calibration — c'est une hypothèse/choix explicite du portage, à noter comme telle dans la documentation.

3.4 Étape 3 — Interpolation entre les deux courbes selon la hauteur courante de l'éprouvette

```
l_ini = specimen_initial_length
l_esp = (1 - ref_strain) * l_ini
dhv = (1 - strain) * l_ini   # hauteur courante de l'éprouvette, déformation nominale (technique)
```

```
drad1 = _eval_poly_no_constant(ff1, defl) * (dhv - l_esp) / (l_ini - l_esp)
drad2 = _eval_poly_no_constant(ff2, defl) * (l_ini - dhv) / (l_ini - l_esp)
return drad1 + drad2
```

Notations : L_{ini} = hauteur (longueur) initiale de l'éprouvette (`specimen_initial_length`), ε_{ref} = déformation de référence, $\varepsilon_a(t)$ = déformation axiale nominale courante (`axial_nominal_strain`), $\delta_{cell}(t)$ = signal de déflexion de la cellule courant (`cell_deflection`).

Hauteur de référence (hauteur de l'éprouvette à $\varepsilon_a = \varepsilon_{ref}$) :

$$L_{esp} = (1 - \varepsilon_{ref})\, L_{ini}$$

Hauteur courante de l'éprouvette (calculée à partir de la déformation technique/nominale, donc en compression positive dans cette convention — `# current specimen height, from the nominal (engineering) strain`) :

$$L(t) = (1 - \varepsilon_a(t))\, L_{ini}$$

La contrainte radiale à l'instant t est alors une **interpolation linéaire pondérée entre les deux polynômes de calibration** P_{ini} (via `ff1`) et P_{esp} (via `ff2`), évalués tous deux au même signal de déflexion courant $\delta_{cell}(t)$, avec des poids qui dépendent de la position de $L(t)$ entre L_{esp} et L_{ini} :

$$\sigma_r(t) = P_{ini}(\delta_{cell}(t)) \cdot \frac{L(t) - L_{esp}}{L_{ini} - L_{esp}} \; + \; P_{esp}(\delta_{cell}(t)) \cdot \frac{L_{ini} - L(t)}{L_{ini} - L_{esp}}$$

Remarques : - Quand $L(t) = L_{ini}$ (début d'essai, $\varepsilon_a = 0$), le poids de P_{ini} vaut $\dfrac{L_{ini} - L_{esp}}{L_{ini} - L_{esp}} = 1$ et celui de P_{esp} vaut 0 : on retombe exactement sur la courbe de calibration « initiale ». - Quand $L(t) = L_{esp}$ (i.e. $\varepsilon_a = \varepsilon_{ref}$), le poids de P_{ini} vaut 0 et celui de P_{esp} vaut 1 : on retombe exactement sur la courbe de calibration « à la hauteur de référence ». - Pour ε_a hors de $[0, \varepsilon_{ref}]$, la pondération est une **extrapolation linéaire** (les deux poids somment toujours à 1 mais l'un devient négatif) — même remarque de prudence que pour le modèle linéaire (§2.2) : aucune saturation/clamp n'est appliquée dans le code. - Les deux polynômes P_{ini}, P_{esp} sont évalués au **même** signal `defl` (déflexion de cellule courante), donc c'est bien uniquement la **pondération** entre les deux courbes qui varie avec la hauteur courante — pas leur argument. - Les tableaux `axial_nominal_strain` et `cell_deflection` sont tronqués à la longueur commune la plus courte avant le calcul (`n = min(...)`, ligne 93).

Cette fonction (ou son couple précompute/évalue) est appelée depuis quatre endroits : `ui/pages/page_results_export.py:` (ligne 933, `nonlinear_cell_radial_stress()`, pour la courbe moyenne affichée) avec `specimen_length = self.project.specimen.params["L"].mean` comme `specimen_initial_length` ; `triaxial_synthetic_draws()` (§5.7, approximation synthétique de repli) et `nonlinear_confinement_envelope()`, de la même façon ; et `core/confinement.py::per_draw_confinement_quantities()` (§5.8), qui utilise le couple `fit_nonlinear_calibration()`/`nonlinear_cell_radial_stress_precomputed()` pour évaluer ce

modèle une fois par tirage Monte-Carlo sans re-fitter les polynômes à chaque fois.

---

4. Constantes alpha00 et alpha0^(l/2) — calibration de la déformation radiale

Implémentation : `core/confinement.py::radial_strain()` (lignes 219-240).

4.1 Relation implicite

```
a_term = (2/3) * (1 - eps_axial*(1 + eps_axial/2)) * alpha00 * eps_cell
b_term = alpha0_l2 * eps_cell / 3
eps_rad = np.zeros(n)
for _ in range(n_iter):     # n_iter = 50 par défaut
    eps_rad = a_term + b_term * (1 + eps_rad) ** 2
```

En notant ε_a la déformation axiale (**signée**, voir §4.3), ε_{cell} le signal de la jauge de cellule (`cell_strain`, ici c'est en fait le paramètre `cell_deflection/cell_defl` passé tel quel — le nom du paramètre dans la fonction est `cell_strain`), α_{00} (`alpha00`) et $\alpha_0^{l/2}$ (`alpha0_l2`) les deux constantes de calibration (issues, d'après le docstring, « d'une simulation numérique de la cellule de confinement », saisies sur la page Special Tests), la déformation radiale ε_r est définie de façon **implicite** (elle apparaît des deux côtés de l'équation) :

$$\varepsilon_r = \frac{2}{3}\left[1 - \varepsilon_a\left(1 + \frac{\varepsilon_a}{2}\right)\right]\alpha_{00}\,\varepsilon_{cell} \;+\; \frac{\alpha_0^{l/2}}{3}\,(1+\varepsilon_r)^2\,\varepsilon_{cell}$$

4.2 Résolution numérique

Le code de `core/confinement.py` résout cette équation par **itération à point fixe simple** (« fixed-point iteration », commentaire ligne 220-222), initialisée à $\varepsilon_r^{(0)} = 0$ et répétée `n_iter = 50` fois (valeur par défaut, paramétrable) :

$$\varepsilon_r^{(k+1)} = \underbrace{\frac{2}{3}\left[1 - \varepsilon_a\left(1 + \tfrac{\varepsilon_a}{2}\right)\right]\alpha_{00}\,\varepsilon_{cell}}_{\text{terme constant}} + \frac{\alpha_0^{l/2}\,\varepsilon_{cell}}{3}\left(1+\varepsilon_r^{(k)}\right)^2$$

Aucun critère de convergence explicite n'est vérifié (pas de test de tolérance/écart entre itérations) : le code effectue systématiquement 50 itérations et retient le dernier résultat, sans garantie formelle de convergence dans le code lui-même — à signaler comme limitation implicite (aucune vérification de stabilité numérique du point fixe n'est faite).

4.3 Convention de signe — point d'attention explicite du code

Le docstring et le code appelant (`page_results_export.py`, lignes 848-857) signalent explicitement une **inversion de convention de signe** entre le reste de l'application et ces formules de déformation radiale/volumétrique/équivalente :

> « This app's own strain/stress convention keeps compression positive, but the radial/volumetric/equivalent strain formulas (Special Tests page) use the opposite sign convention: axial strain negative in compression, radial strain positive (the specimen widens) — negate axial strain going into them only, leaving the stress-side calculations above untouched. »

Concrètement, avant d'appeler `radial_strain()`, `volumetric_strain()` et `equivalent_strain()`, le code fait :

```
axial_strain_signed = -strain   # (ou -axial_nominal_strain dans triaxial_synthetic_draws)
```

c'est-à-dire $\varepsilon_a^{signed} = -\varepsilon_a^{compression>0}$. Ceci **ne s'applique pas** au calcul de la contrainte radiale (`linear_cell_radial_stress / nonlinear_cell_radial_stress`) ni au calcul des invariants de contrainte (`stress_invariants`), qui utilisent la convention « compression positive » standard de l'application, sans changement de signe.

---

5. Grandeurs dérivées : contrainte radiale, pression, déviateur, déformations

5.1 Invariants de contrainte : pression hydrostatique et déviateur

Implémentation : `core/confinement.py::stress_invariants()` (lignes 108-114), retournant une dataclass `StressInvariants(pressure, deviator)` (lignes 102-105).

En notant σ_a la contrainte axiale (`axial_stress`, issue du résultat SHPB/DI/QS déjà calculé) et σ_r la contrainte radiale calculée en §2 ou §3 :

$$p = \frac{\sigma_a + 2\sigma_r}{3} \qquad \text{(pression hydrostatique)}$$

$$q = |\sigma_a - \sigma_r| \qquad \text{(déviateur de contrainte)}$$

Les tableaux sont tronqués à la longueur commune la plus courte avant calcul (`n = min(len(axial_stress), len(radial_stress))`). Notez que q est défini ici comme un **écart absolu simple** $|\sigma_a - \sigma_r|$ et non comme l'invariant de von Mises généralisé $q = \sqrt{\frac{3}{2} s_{ij} s_{ij}}$ complet — dans le cas axisymétrique à deux contraintes principales distinctes (σ_a, $\sigma_r = \sigma_r$ répétée deux fois), cette expression coïncide bien avec $q_{vM} = |\sigma_a - \sigma_r|$, donc c'est cohérent, mais cela mérite d'être noté explicitement dans la doc car ce n'est pas écrit sous la forme classique $\sqrt{3/2\, s : s}$ dans le code.

5.2 Déformation radiale

Voir §4 — `core/confinement.py::radial_strain()`, formule implicite résolue par point fixe.

5.3 Déformation volumétrique

Implémentation : `core/confinement.py::volumetric_strain()` (lignes 243-248).

$$\varepsilon_v = (1+\varepsilon_a)(1+\varepsilon_r)^2 - 1$$

où ε_a est la déformation axiale **signée** (convention négative en compression, §4.3) et ε_r la déformation radiale calculée en §4. Cette formule est la définition classique de la déformation volumétrique finie à partir des élongations principales $1+\varepsilon_a$ (axiale) et $(1+\varepsilon_r)$ répétée deux fois (les deux directions radiales), sous hypothèse de symétrie de révolution.

5.4 Déformation équivalente (von Mises)

Implémentation : `core/confinement.py::equivalent_strain()` (lignes 251-256).

$$\varepsilon_{eq} = \frac{2}{3}\left|\ln(1+\varepsilon_a) - \varepsilon_r\right|$$

où $\ln(1+\varepsilon_a)$ est la déformation logarithmique (vraie) axiale déduite de la déformation technique signée ε_a, et ε_r la déformation radiale (§4). Cette formule combine une mesure de déformation axiale « vraie » (logarithmique) et une déformation radiale technique — l'usage mixte log/technique est tel quel dans le code, sans commentaire justificatif additionnel dans le docstring au-delà de la formule elle-même.

5.5 Résumé — chaîne de calcul complète

D'après `ui/pages/page_results_export.py` (méthode `_refresh_confinement`, lignes 865 et suivantes), la séquence de calcul pour un essai confiné est :

1. Lecture de la déformation/contrainte axiale nominale du résultat SHPB/DI/QS sélectionné (`nominal_strain`/`nominal_stress`, ou `strain`/`stress` à défaut) — lignes 911-915.
2. Lecture du signal du capteur ayant le rôle `confinement_cell`, via `_sensor_signal_for_result()` — qui préfère déjà le signal **coupé/rééchantillonné par tirage Monte-Carlo** (`sensor_{idx}_signal`, voir §5.8) quand ce résultat a été calculé avec « Include this cut in the Monte-Carlo loop » coché pour ce capteur, et retombe sinon sur le signal brut/coupé manuellement — ligne 922. **Recalage post-correction de toe (ligne 936-940)** : quand ce signal retombe sur le brut (capteur non recoupé par tirage), il n'a lui-même jamais été touché par une éventuelle correction de toe (chapitre SHPB, §5.5) appliquée à la courbe `strain`/`axial` lue au point 1 — sans intervention, ses points de tête resteraient donc décalés du nombre de points que la correction de toe a retiré en tête de cette courbe. `_refresh_confinement()` relit l'entier `toe_n_removed` (accumulé par `_on_toe_apply()`, voir chapitre SHPB §5.5) déposé à la racine du résultat, et retire ce même nombre de points en tête du signal de cellule brut (`cell_defl = cell_defl[cut:]`, avec `cut = min(toe_n_removed, len(cell_defl))`) **avant** l'appel au point 3 ci-dessous — uniquement quand `cell_used_mc` est faux, puisque le chemin recoupé par tirage (§5.8) est déjà correctement aligné par construction (voir chapitre SHPB §5.5). Le même recalage est appliqué indépendamment au signal laser optionnel (point 6 ci-dessous, lignes 987-993) lorsque lui aussi provient du brut.
3. Calcul de la contrainte radiale moyenne σ_r via `linear_cell_radial_stress()` (§2) ou `nonlinear_cell_radial_stress()` (§3) selon `conf.nonlinear_enabled` — lignes 942-963 (à partir des courbes MOYENNES ci-dessus, indépendamment du calcul par tirage du §5.8).

4. Calcul des invariants p, q via `stress_invariants()` (§5.1) — ligne 966.
5. Inversion de signe de la déformation axiale (`axial_strain_signed = -strain`) puis calcul de ε_r, ε_v, ε_{eq} via `radial_strain()`, `volumetric_strain()`, `equivalent_strain()` (§4-5.3-5.4) — lignes 974-977.
6. Option laser : si un capteur de rôle `laser_velocity` est présent, contrainte axiale indépendante via `laser_stress()` (§5.6) puis invariants p_{laser}, q_{laser} recalculés avec la même σ_r — lignes 979-1002, 1069-1075 environ (voir la limitation 9 en §7 : le laser reste, par choix explicite, en dehors du calcul en boucle Monte-Carlo du §5.8).
7. Bande d'incertitude pour les grandeurs 3-5 (gage) : **réelle** si le résultat courant a été calculé avec le §5.8 (`confinement_radial_stress_std` présent), sinon **synthétique** via `triaxial_synthetic_draws()` (§5.7) — lignes 1020-1046 environ. Bande d'incertitude pour la variante laser (6) : toujours synthétique, via `triaxial_synthetic_draws()`, quel que soit le cas ci-dessus.

Les quatre graphiques affichés (carte « Confinement / Triaxial Stresses ») sont : contraintes confinement vs. déformation axiale, déformations radiale/volumétrique/équivalente vs. déformation axiale nominale, chemin de contrainte (p, q), et pression p vs. déformation volumétrique ε_v (analogue œdométrique). Seule la provenance de la bande d'incertitude affichée en pointillé/transparence sur chacun d'eux varie (réelle ou synthétique, selon le point 7 ci-dessus) ; les courbes moyennes et le tracé restent les mêmes dans les deux cas. Le message affiché sous les graphiques (`confine_status`) précise explicitement, pour la partie gage, si l'incertitude provient d'une vraie propagation par tirage ou de l'approximation synthétique — voir §5.8.

5.6 Contrainte axiale alternative par vélocimétrie laser

Implémentation : `core/confinement.py::laser_stress()` (lignes 29-37) — indépendante du signal de jauge du barreau de sortie, utile en complément/vérification :

$$\sigma_a^{laser} = \frac{D_{ob}^2}{D_{spe}^2} \cdot \frac{1}{2}\, \rho_{ob}\, C_{0,ob}\, \frac{v_{laser}}{10^6} \quad \text{(MPa)}$$

avec D_{ob} diamètre du barreau de sortie (`ob_diameter`), D_{spe} diamètre de l'éprouvette (`specimen_diameter`), ρ_{ob} masse volumique du barreau de sortie (`ob_rho`), $C_{0,ob}$ célérité des ondes dans le barreau de sortie (`ob_c0`), v_{laser} vitesse de surface mesurée par laser (`laser_velocity`). Le docstring précise que cette formule correspond exactement à `(D0**2/Dspe**2) * 0.5 * rho0 * C0 * laser_velocity / 1e6` de l'implémentation d'origine.

5.7 Enveloppe synthétique de repli (« imitation ») — `triaxial_synthetic_draws`

Implémentation : `core/confinement.py::confinement_envelope()` (linéaire, lignes 201-240), `nonlinear_confinement_envelope()` (non-linéaire, lignes 243-284), et `triaxial_synthetic_draws()` (toutes grandeurs, tirages individuels, lignes 398-482).

`triaxial_synthetic_draws()` sert de **repli** pour la bande d'incertitude affichée sur la carte Confinement : elle n'est utilisée que lorsqu'aucune donnée réelle par tirage n'est disponible pour la grandeur concernée (voir §5.8) — ce qui reste systématiquement le cas pour les trois grandeurs « laser », par choix explicite (voir la limitation 9 en §7).

`confinement_envelope/nonlinear_confinement_envelope` (dataclass `ConfinementEnvelope`, lignes 186-198) sont, elles, du **code mort** : elles ne sont appelées nulle part dans l'application (vérifié par recherche exhaustive) — seule `triaxial_synthetic_draws` est utilisée en pratique, aussi bien par `page_results_export.py` que par `page_covariance.py`.

Principe (`n_draws = 200` tirages par défaut) : à chaque tirage synthétique, on perturbe - la contrainte axiale par un bruit gaussien d'écart-type égal à son incertitude déjà reportée (`axial_stress_std`), - la déflexion de cellule par un bruit gaussien d'écart-type = moyenne de son incertitude reportée (`cell_deflection_std`), - et, **modèle linéaire uniquement**, le module de cellule E_v par un bruit gaussien d'écart-type = `cell_modulus_std_fraction * cell_modulus` (2 % par défaut, `cell_modulus_std_fraction`, correspondant au commentaire « matching the original's `E_cd = np.random.normal(E_c, 0.02*E_c, N)` »),

puis on recalcule σ_r, p, q, ε_r, ε_v, ε_{eq}, et les variantes « laser » de σ_a, p, q, pour ce tirage synthétique. `triaxial_synthetic_draws` retourne les **tirages individuels** (`list_of_per_draw_arrays` par grandeur), au même format que le calcul réel du §5.8 (`project.last_raw_draws`) — ce qui permet à `page_results_export.py`/`page_covariance.py` de traiter les deux sources de façon interchangeable.

Pourquoi c'est une approximation, pas une vraie propagation d'incertitude : chaque tirage synthétique perturbe la contrainte axiale et la déflexion de cellule par un bruit gaussien **frais et indépendant**, sans lien avec un tirage Monte-Carlo réel du dépouillement SHPB/DI/QS — alors qu'en réalité, la contrainte axiale ET le signal de cellule (quand ce dernier est coupé par tirage, §5.8) proviennent tous les deux de la **même** réalisation aléatoire des propriétés du barreau/de l'éprouvette/de l'alignement du capteur pour un tirage donné, et sont donc corrélés. `triaxial_synthetic_draws` détruit cette corrélation réelle en tirant deux bruits indépendants — d'où la description de cette approche comme une « imitation » plutôt qu'une véritable propagation d'incertitude, ce qui motive le calcul réel décrit en §5.8.

Note de robustesse implémentée dans `triaxial_synthetic_draws()` (lignes ~421-427) : si le premier point de déformation nominale est très proche de zéro (`< 1e-6` en valeur absolue) sans être exactement 0, il est forcé à `0.0` — le commentaire du code explique que ceci évite qu'un résidu de flottant issu du traitement amont empêche la page Covariance Analysis de trouver un tirage démarrant exactement à la « Reference (start) strain = 0.0 » demandée par l'utilisateur. Le même traitement est repris à l'identique pour les tirages réels du §5.8, côté `page_covariance.py`.

5.8 Calcul réel en boucle Monte-Carlo (SHPB / Impact Direct / Quasi-Statique)

Implémentation : `core/confinement.py::per_draw_confinement_quantities()` (lignes 327-395) ; branchement dans `ui/pages/page_process.py` (`_prepare_confinement()`, lignes 92-126, partagée par les trois onglets ; usage dans `_SHPBTab._on_run`, `_DITab._on_run`, `_QSTab._on_run`).

Principe : plutôt que de reposer uniquement sur σ_r, p, q, ε_r, ε_v, ε_{eq} calculées une seule fois à partir des courbes MOYENNES du résultat (dans `_refresh_confinement`, §5.5) avec une incertitude approximée par des tirages synthétiques indépendants (§5.7), ces six grandeurs sont calculées **une fois par tirage Monte-Carlo, à l'intérieur même de la boucle de dépouillement SHPB/Impact Direct/Quasi-Statique** (`ui/pages/page_process.py`), à partir de la contrainte/déformation axiale **réelle de CE tirage** combinée au signal de cellule de confinement **réel de ce même tirage** (quand ce dernier existe — voir plus bas). Le mécanisme d'agrégation générique déjà utilisé pour toutes les autres grandeurs par tirage

(`_aggregate_quantities()`/`_store_raw_draws()`, lignes 71-101 de `page_process.py`) prend alors automatiquement en charge la moyenne, l'écart-type ET l'ensemble brut des tirages, sans code spécifique supplémentaire — exactement comme `strain`, `stress`, ou un capteur `sensor_{idx}_signal` existant.

`per_draw_confinement_quantities(axial_strain, axial_stress, cell_deflection, conf, specimen_initial_length, poly_ini=None, poly_esp=None)` calcule, pour un seul tirage : la contrainte radiale (`linear_cell_radial_stress()` ou `nonlinear_cell_radial_stress_precomputed()` selon `conf.nonlinear_enabled` — voir §3.1 pour le précompute des polynômes `poly_ini`/`poly_esp`), les invariants (`stress_invariants()`), puis — avec la même inversion de signe qu'en §4.3 — les trois déformations dérivées. Elle retourne un dict à clés préfixées `confinement_` (`confinement_radial_stress`, `confinement_pressure`, `confinement_deviator`, `confinement_radial_strain`, `confinement_volumetric_strain`, `confinement_equivalent_strain`), fusionné directement dans le dict `draw` de la boucle appelante.

`_prepare_confinement(project)` (lignes 92-126 de `page_process.py`, partagée par les trois onglets) détermine si ce calcul est possible pour ce lancement : confinement activé (`project.confinement.enabled`), un Custom Sensor de rôle `confinement_cell` activé présent, et — pour le modèle non-linéaire — des courbes de calibration valides (sans quoi elle retourne `None` et le calcul par tirage est simplement omis pour ce lancement). Quand le modèle non-linéaire est utilisé, elle appelle aussi `fit_nonlinear_calibration()` **une seule fois avant la boucle** (voir §3.1), plutôt que de re-fitter les deux polynômes à chaque tirage.

Disponibilité du signal de cellule réel par tirage — trois cas, selon l'onglet :

- **SHPB / Impact Direct** : le calcul du §5.8 n'est actif QUE si le capteur de rôle `confinement_cell` a AUSSI la case « Include this cut in the Monte-Carlo loop » cochée (mécanisme « Specimen-Mounted Sensor Alignment », voir le chapitre SHPB/Impact Direct) — c'est ce qui donne à ce capteur un signal réellement recoupé/rééchantillonné pour chaque tirage (`draw[f"sensor_{idx}_signal"]`), à partir des propriétés barreau/éprouvette effectivement tirées pour ce tirage-là. Si la case n'est pas cochée, `_prepare_confinement` détecte l'absence de ce signal par tirage et le calcul du §5.8 est simplement désactivé pour ce lancement — l'onglet Résultats & Export retombe alors sur l'enveloppe synthétique du §5.7, avec un message explicite invitant à cocher la case.
- **Quasi-Statique (QS)** : cet onglet n'a — et n'a pas besoin — d'aucun mécanisme de recoupe par tirage (`cut_in_montecarlo`) : il n'y a pas d'onde à propager dans un essai quasi-statique, donc pas de délai de propagation à recaler tirage par tirage — le capteur de cellule est déjà, par construction, sur la même base de temps que la charge/le déplacement. Le calcul du §5.8 y est donc actif dès que confinement est activé et qu'un capteur `confinement_cell` existe, mais il combine la contrainte/déformation axiale RÉELLE de chaque tirage (déjà Monte-Carlo-variée via les incertitudes Cell Load/Measurement de cet onglet) avec le signal de cellule **brut, identique pour tous les tirages** (pas de variation Monte-Carlo propre à ce signal sur cet onglet — choix explicite, voir la limitation 10 en §7). L'incertitude qui en résulte est donc réelle mais partielle : elle capture la variabilité côté axial, pas celle du canal de cellule lui-même.

Dans tous les cas, le message affiché à l'issue du traitement (`status_label`, sur la page Processing) précise explicitement si le calcul du §5.8 a été effectué ou non pour ce lancement, et pourquoi. De même, `_refresh_confinement()` (§5.5) et la page Covariance Analysis (`_triax_draws()`, `ui/pages/page_covariance.py` lignes 336-436) inspectent la présence de `confinement_radial_stress_std`/`confinemen` dans le résultat/les tirages bruts pour savoir, grandeur par grandeur, s'ils peuvent utiliser cette donnée

réelle ou doivent retomber sur l'approximation synthétique du §5.7 — et l'affichent honnêtement dans leur message de statut respectif.

Cohérence avec une correction de toe appliquée après coup : les six grandeurs `confinement_*` calculées par ce §5.8 vivent à l'intérieur du dictionnaire `draw` de chaque tirage Monte-Carlo, à la même longueur que la déformation axiale de ce tirage — exactement le format générique que la correction de toe retaille déjà pour toute autre grandeur par tirage (`ui/pages/page_process.py::_apply_toe_correction_to_draws()`, voir chapitre SHPB §5.5). Une correction de toe appliquée après un lancement Monte-Carlo avec ce §5.8 actif (« Include this cut in the Monte-Carlo loop » coché) recoupe donc automatiquement ces six grandeurs, tirage par tirage, sans code spécifique supplémentaire — `project.last_raw_draws["SHPB"/"DI"]` et l'agrégat `project.last_results[...]` restent alors mutuellement cohérents, et la page Covariance Analysis hérite de la même correction. Ce n'était **pas** le cas avant cette correction (la correction de toe ne touchait que l'agrégat) : c'est précisément le défaut d'alignement (« décalage ») documenté et corrigé au chapitre SHPB, §5.5. Le chemin de repli brut (capteur non recoupé par tirage, `cell_used_mc` faux dans `_sensor_signal_for_result()`) reste, lui, en dehors de ce mécanisme par tirage — c'est ce que corrige séparément le recalage par `toe_n_removed` décrit au §5.5 ci-dessus, dans `_refresh_confinement()`.

---

6. Récapitulatif des correspondances fonctions <-> grandeurs

Grandeur / calcul	Fonction (fichier `core/confinement.py`)
Contrainte radiale, modèle linéaire	`linear_cell_radial_stress()`
Contrainte radiale, modèle non-linéaire (fit + évaluation en un appel)	`nonlinear_cell_radial_stress()` (utilise `_eval_poly_no_constant()`)
Contrainte radiale, modèle non-linéaire — fit des calibrations SEUL, une fois par lancement	`fit_nonlinear_calibration()` (lignes 100-119)
Contrainte radiale, modèle non-linéaire — évaluation SEULE à partir de polynômes déjà fittés	`nonlinear_cell_radial_stress_precomputed()` (lignes 122-140)
Contrainte axiale par laser (optionnel)	`laser_stress()`
Pression hydrostatique p, déviateur q	`stress_invariants()` → `StressInvariants`
Enveloppe d'incertitude, modèle linéaire	`confinement_envelope()` → `ConfinementEnvelope` (code mort, jamais appelée dans l'application)
Enveloppe d'incertitude, modèle non-linéaire	`nonlinear_confinement_envelope()` → `ConfinementEnvelope` (code mort, jamais appelée dans l'application)
Déformation radiale (alpha00, alpha0^(l/2))	`radial_strain()`
Déformation volumétrique	`volumetric_strain()`
Déformation équivalente (von Mises)	`equivalent_strain()`
Tirages Monte-Carlo synthétiques de repli (toutes grandeurs, y compris laser ; pour Covariance Analysis et le repli de `_refresh_confinement`)	`triaxial_synthetic_draws()` (§5.7)

Grandeur / calcul	Fonction (fichier `core/confinement.py`)
Six grandeurs de confinement pour UN tirage Monte-Carlo réel (contrainte radiale, p, q, ε_r, ε_v, ε_{eq})	`per_draw_confinement_quantities()` (lignes 327-395, §5.8)

Point d'entrée UI qui orchestre l'ensemble : `ui/pages/page_results_export.py::_refresh_confinement()` (carte « Confinement / Triaxial Stresses ») — calcule l'affichage final à partir des données déjà agrégées (réelles si disponibles, sinon synthétiques). En amont, côté traitement, `ui/pages/page_process.py::_prepare_confinemer` (lignes 92-126, partagée par les trois onglets Processing) est le point d'entrée qui décide si le calcul réel par tirage (§5.8) est possible pour ce lancement et précompute les polynômes non-linéaires le cas échéant. Paramétrage utilisateur des constantes (C_1, C_2, E_v, ε_{ref}, D_{cell}, α_{00}, $\alpha_0^{l/2}$, courbes de calibration non-linéaires) : `ui/pages/page_special_tests.py`. Structure de données des paramètres : `core/project.py::ConfinementSettings` (dataclass, lignes 108-134).

---

7. Limitations, hypothèses et avertissements relevés dans le code

1. **Absence de clamp aux bornes de calibration** : dans les deux modèles (linéaire §2.2 et non-linéaire §3.4), la pondération entre régimes/courbes est une interpolation **linéaire non bornée** ; hors de la plage $[0, \varepsilon_{ref}]$ elle devient une extrapolation. Ce n'est pas signalé comme un avertissement explicite dans le code, mais c'est une conséquence directe de la formule et doit être signalé à l'utilisateur.
2. **Polynôme de calibration non-linéaire forcé à l'origine** : `_eval_poly_no_constant()` élimine délibérément le terme constant du fit `np.polyfit` de degré 6, ce qui force $\sigma_r(\varepsilon_{ortho} = 0) = 0$ — hypothèse justifiée dans le docstring comme « physiquement sensée », mais qui altère la qualité de l'ajustement par rapport à un fit libre à 7 coefficients.
3. **Convergence de l'itération à point fixe non vérifiée** : `radial_strain()` effectue systématiquement `n_iter=50` itérations sans test de convergence (pas de critère d'arrêt anticipé, pas de vérification que le résidu diminue) — limitation implicite du code, à ne pas présenter comme une méthode garantissant la convergence.
4. **Convention de signe inversée pour les déformations dérivées** : les fonctions `radial_strain()`, `volumetric_strain()`, `equivalent_strain()` attendent une déformation axiale **négative en compression** (opposée à la convention « compression positive » du reste de l'application), d'où la négation explicite (`axial_strain_signed = -strain`) faite par l'appelant avant de les invoquer — documenté explicitement en commentaire dans `page_results_export.py` (lignes 848-853). Ceci ne s'applique PAS aux calculs de contraintes (`sigma_r`, `p`, `q`), qui restent en convention « compression positive ».
5. **Déviateur** $q = |\sigma_a - \sigma_r|$: forme simplifiée à deux contraintes principales (axisymétrie), correcte dans ce cadre mais non écrite sous la forme générale $\sqrt{3/2\, s_{ij} s_{ij}}$ — à préciser dans la doc pour éviter toute confusion avec un calcul 3D général.
6. **Déformation équivalente mixte log/technique** : $\varepsilon_{eq} = \frac{2}{3}|\ln(1+\varepsilon_a) - \varepsilon_r|$ combine une déformation axiale vraie (logarithmique) avec une déformation radiale technique — combinaison reprise telle quelle du code d'origine, sans justification supplémentaire dans le docstring actuel.

7. **Module de cellule perturbé à 2 % fixe** : dans les enveloppes Monte-Carlo du modèle linéaire, l'écart-type de E_v est fixé à 2 % de sa valeur (`cell_modulus_std_fraction`, valeur par défaut de `ConfinementSettings`), configurable mais avec cette valeur par défaut explicitement calée sur l'original (`E_cd = np.random.normal(E_c, 0.02*E_c, N)`). Dans le modèle non-linéaire, aucune incertitude équivalente sur les courbes de calibration n'est modélisée (seules la contrainte axiale et la déflexion de cellule sont perturbées).
8. **Troncature systématique à la longueur minimale** : toutes les fonctions de `core/confinement.py` tronquent silencieusement leurs tableaux d'entrée à `min(len(...))` sans avertissement si les longueurs diffèrent — comportement à connaître pour l'interprétation des résultats en cas de signaux de longueurs différentes (contrainte axiale vs. signal de cellule vs. signal laser). Cette troncature reste **position-à-position, pas temporelle** : elle suppose implicitement un même pas d'échantillonnage entre les deux signaux combinés. Pour le chemin brut (capteur non recoupé par tirage), `_refresh_confinement()` corrige désormais le décalage en tête qu'introduirait une correction de toe (`toe_n_removed`, voir chapitre SHPB §5.5 et §5.5 ci-dessus) avant cette troncature — mais une différence de cadence d'échantillonnage entre le capteur et la courbe contrainte-déformation, elle, n'est ni détectée ni corrigée, avec ou sans correction de toe.
9. **Calcul réel par tirage (§5.8) conditionné, sur SHPB/Impact Direct, à une case à cocher indépendante** : le calcul en boucle Monte-Carlo des grandeurs de confinement n'est activé sur ces deux onglets que si le capteur `confinement_cell` a séparément sa case « Include this cut in the Monte-Carlo loop » (mécanisme Specimen-Mounted Sensor Alignment) cochée. Rien n'empêche techniquement de laisser confinement activé sans cocher cette case : dans ce cas, le calcul par tirage est silencieusement omis (pas d'erreur), et toute la chaîne avale (statut du traitement, `_refresh_confinement`, Covariance Analysis) retombe sur l'approximation synthétique du §5.7 — avec un message explicite, mais l'utilisateur doit lire ce message pour s'en apercevoir.
10. **Signal de cellule non Monte-Carlo-varié sur l'onglet Quasi-Statique** : contrairement à SHPB/Impact Direct, le calcul par tirage sur QS combine une contrainte/déformation axiale réellement tirée avec un signal de cellule de confinement **fixe, identique pour tous les tirages** de ce lancement (choix explicite : il n'existe pas de mécanisme de recoupe par tirage propre à QS, voir §5.8). L'incertitude qui en résulte pour les grandeurs de confinement sur cet onglet est donc réelle mais partielle : elle ne capture pas la variabilité propre du canal de cellule (bruit de mesure, incertitude de calibration du capteur), seulement celle transmise par la variabilité de la contrainte/déformation axiale.
11. **Grandeurs par vélocimétrie laser toujours synthétiques, jamais calculées en boucle** : `axial_stress_laser`, `pressure_laser` et `deviator_laser` ne sont, par choix de conception explicite, jamais calculées par `per_draw_confinement_quantities()` dans la boucle Monte-Carlo (§5.8) — même quand les six autres grandeurs de confinement le sont pour ce même lancement. Elles proviennent systématiquement de l'enveloppe synthétique du §5.7, quelle que soit la disponibilité de données réelles par tirage pour les grandeurs basées sur la jauge de cellule.
12. **`toe_n_removed` est une médiane inter-tirages, pas un décompte exact** : depuis que la correction de toe recoupe chaque tirage Monte-Carlo individuellement (chapitre SHPB, §5.5), le nombre de points retirés en tête peut légèrement différer d'un tirage à l'autre. Le compteur `toe_n_removed` relu par `_refresh_confinement()` (§5.5 ci-dessus) pour recaler un capteur de confinement/laser resté en signal brut retient la **médiane** de ce nombre sur l'ensemble des tirages survivants — une approximation raisonnable tant que cette variation inter-tirages reste faible, mais pas une

correspondance exacte pour ce chemin de repli spécifique (le chemin recoupé par tirage du §5.8, lui, reste exact puisqu'il recoupe directement le signal réel propre à chaque tirage).

6. Corrélation d'Images Numériques (DIC)

Le module DIC (*Digital Image Correlation*) de `shpb_app` permet de suivre, sur une séquence d'images ou une vidéo, des points caractéristiques d'un mouchetis (*speckle*) déposé sur l'éprouvette, afin d'en déduire des champs de déplacement, de déformation, de vitesse, d'accélération et une force dynamique de la région d'intérêt (ROI). Le module comprend également un estimateur prédictif de l'incertitude de mesure.

Fichiers source concernés :

- `core/dic_preprocessing.py` — prétraitement des images (crop, mise à l'échelle 8 bits, downscale, décimation temporelle).
- `core/dic_correlation.py` — définition de la ROI et du maillage de corrélation (ce fichier ne contient **pas** l'algorithme de suivi lui-même, malgré son nom : `ROI`, `Ellipse`, `MeshGrid`, `build_mesh*`).
- `core/dic_processing.py` — c'est ce fichier qui contient l'algorithme de suivi/corrélation proprement dit (`run_dic`, `_track_subset`, `_subpixel_peak`, `_grid_strain`, `_apply_neighbor_consistency_filte` et les structures de résultat (`DICFrameResult`, `DICResult`).
- `core/dic_kinematics.py` — dérivation cinématique (vitesse, accélération, déformation équivalente de von Mises, taux de déformation, force dynamique) et conversion en déformation vraie (`nominal_to_true_strain`).
- `core/dic_uncertainty.py` — modèle complet, prédictif, d'incertitude de mesure DIC.
- `core/dic_result_io.py` — relecture d'un export CSV (`_on_export_results`) en `DICResult`, pour rouvrir un run passé sans le recalculer (§4.8).
- `core/dic_render.py` — rendu graphique d'un champ DIC sur une image (triangulation de Delaunay), commun à l'affichage live et à l'export vidéo.
- `core/camera_clock.py` — outil séparé (« Camera / Video Sync ») de dépouillement du fichier d'horloge caméra ; n'intervient pas directement dans le calcul DIC mais est lié au sujet de la synchronisation temporelle.
- `core/project.py` (dataclass `DICSettings`, lignes ~170–299) — tous les paramètres persistés (valeurs par défaut citées ci-dessous).
- `ui/pages/page_image_correlation.py` — orchestration UI, vocabulaire, formules de temps effectif (`_effective_fps`, `_dic_time_s`).

Avertissement général du module (`core/dic_processing.py`, docstring d'en-tête) : ce n'est *pas* un solveur DIC à sous-image déformable complet (contrairement à des logiciels commerciaux comme VIC-2D ou Ncorr, qui optimisent aussi la rotation/le cisaillement/l'étirement local de chaque sous-image pendant l'appariement). Chaque sous-image (« subset ») est ici suivie comme un gabarit rigide en translation pure,

localisé par `cv2.matchTemplate`, avec un raffinement sous-pixel parabolique à 3 points. Ceci est jugé suffisant pour les déformations modérées, à dominante uniaxiale, typiques d'un essai SHPB / impact direct / quasi-statique, à un coût d'implémentation bien moindre qu'une optimisation complète de forme de sous-image.

---

1. Vue d'ensemble du pipeline

1. **Prétraitement spatial** (`core/dic_preprocessing.preprocess_frame`) : crop → conversion 8 bits affichable → downscale.
2. **Décimation temporelle** (`core/dic_preprocessing.decimated_frame_indices`) : sélection d'une frame sur N.
3. **Définition de la ROI et du maillage de corrélation** (`core/dic_correlation.py`) : rectangle / polygone / ellipse, grille régulière de points espacés de `step`, chacun centre d'une sous-image `subset_size×subset_size`.
4. **Suivi/corrélation** (`core/dic_processing.run_dic`) : appariement par corrélation croisée normalisée (NCC), frame par frame, mode « Fixed » ou « Moving », avec filtres de qualité (seuil de corrélation, cohérence de voisinage).
5. **Champ de déformation local** (`core/dic_processing._grid_strain`) : gradient du champ de déplacement par différences finies sur la grille régulière.
6. **Cinématique dérivée** (`core/dic_kinematics.compute_kinematics`) : vitesse, accélération, déformation/taux de déformation équivalents de von Mises, force dynamique ROI.
7. **Conversion en déformation vraie** (`core/dic_kinematics.nominal_to_true_strain`), optionnelle, à la publication vers Results & Export.
8. **Correction temporelle caméra/jauges** (`time_scale`, `time_offset_s`, gérés dans `page_image_correlation.py`) — pas de synchronisation automatique.
9. **Modèle d'incertitude** (`core/dic_uncertainty.py`), optionnel, propagé du déplacement jusqu'à la force dynamique.

---

2. Prétraitement (`core/dic_preprocessing.py`)

Trois étapes indépendantes et optionnelles, **toujours appliquées dans cet ordre** (docstring du module) :

1. **Crop** (`crop_frame`) à un rectangle `ROI` (défini dans `core/dic_correlation.py`) dessiné sur l'image brute (pleine résolution), retaillé aux bornes de l'image (`ROI.clamped`).
2. **Étirement de contraste** vers une image 8 bits affichable/analysable (`frame_source.to_display_uint8`).
3. **Downscale** (`downscale_frame`) par un facteur entier `factor ≥ 1` (1 = pas de downscale), par moyenne de zone (`cv2.INTER_AREA`) — interpolation appropriée pour réduire une image sans phénomène d'aliasing.

Fonction d'orchestration : `preprocess_frame(raw_frame, crop, downscale_factor)`, qui retourne un tableau `uint8` contigu. **Point important** : la ROI et le maillage de corrélation, ainsi que tout le moteur

de corrélation, travaillent entièrement dans les coordonnées pixel de cette image *prétraitée* — jamais dans celles de l'image brute.

La **décimation temporelle** (traiter 1 frame sur N plutôt que toutes) est un concept indépendant, géré par :

`decimated_frame_indices(n_frames, reference_frame, step)`

qui retourne la liste `[reference_frame, reference_frame+step, reference_frame+2*step, ...]` jusqu'à la fin de la séquence. La frame de référence (état non déformé, déplacement nul) est toujours incluse en premier ; le DIC ne suit la déformation que vers l'avant dans le temps — aucune frame antérieure à `reference_frame` n'est jamais traitée.

Valeurs par défaut (`core/project.py`, `DICSettings`) : `downscale_factor = 1`, `frame_step = 1`.

---

3. ROI et maillage de corrélation (`core/dic_correlation.py`)

Ce fichier ne fait *que* construire et valider la grille de points à corréler ; la corrélation elle-même est un traitement séparé, ultérieur (`core/dic_processing.py`).

3.1 Formes de ROI

- **`ROI`** : rectangle (x, y, largeur, hauteur) en coordonnées pixel de la frame de référence (prétraitée). `clamped()` le contraint à l'intérieur d'une frame `frame_width × frame_height`.
- **`Ellipse`** : ovale axis-aligned (centre `(cx, cy)`, demi-axes `rx`, `ry`) — adapté à une éprouvette cylindrique vue de profil, pour ne pas gaspiller de points de corrélation sur les coins hors éprouvette. `bounding_roi()` donne le plus petit rectangle englobant.
- **Polygone** : liste de sommets `(x, y)`, pour des formes d'éprouvette qu'un rectangle n'épouse pas bien (haltère/dogbone, etc.).

3.2 Construction de la grille — `build_mesh(roi, subset_size, step)`

Points espacés de `step` pixels, centrés à `subset_size/2` pixels du bord de la ROI (pour que chaque sous-image reste entièrement à l'intérieur de la ROI) :

$$x_0 = \text{roi.x} + \frac{\text{subset_size}}{2}, \qquad x_1 = \text{roi.x} + \text{roi.width} - \frac{\text{subset_size}}{2}$$

(idem en y), puis :

```
xs = np.arange(x0, x1 + 1e-9, step)
ys = np.arange(y0, y1 + 1e-9, step)
```

Si `x1 < x0` ou `y1 < y0` (ROI trop petite pour même un seul subset), une grille vide (`n_points == 0`) est retournée — c'est à l'appelant de détecter ce cas et d'en avertir l'utilisateur plutôt que de traiter cela comme une erreur silencieuse.

3.3 ROI non rectangulaires

- `build_mesh_from_polygon(vertices, subset_size, step)` : construit d'abord la grille régulière sur la boîte englobante du polygone (`build_mesh`), puis masque (`MeshGrid.mask`) tout point dont le **centre** tombe hors du polygone (test via `matplotlib.path.Path.contains_points`).
- `build_mesh_from_ellipse(ellipse, subset_size, step)` : même principe sur `ellipse.bounding_roi()`, avec le test d'appartenance

$$\left(\frac{x-c_x}{r_x}\right)^2+\left(\frac{y-c_y}{r_y}\right)^2\leq 1$$

Avertissement explicite du code : ce masquage ne teste que le *centre* de chaque point — une sous-image proche de la frontière peut donc déborder légèrement hors de la ROI ; le masque ne sert qu'à décider quels points comptent « dans la ROI » pour les statistiques/résultats, pas à découper précisément le contour de chaque sous-image.

Un point masqué (hors ROI) est traité par `core/dic_processing.py` exactement comme un point dont la sous-image tombe hors de l'image : exclu du suivi, NaN dans tous les champs de résultat.

3.4 Structure `MeshGrid`

- `xs`, `ys` : coordonnées 1D des colonnes/lignes de la grille.
- `subset_size`, `step` (en pixels).
- `mask` : tableau booléen (ou `None` pour une ROI rectangulaire simple, qui n'a jamais de trous).
- `n_points` = `n_points_x * n_points_y` (grille de la boîte englobante), `n_valid_points` = nombre de points réellement dans la ROI (`np.count_nonzero(mask)` si masque, sinon = `n_points`).

Valeurs par défaut (`DICSettings`) : `subset_size_px = 21`, `step_size_px = 7`.

---

4. Algorithme de suivi/corrélation (`core/dic_processing.py`)

4.1 Principe général

Chaque point du maillage est suivi comme un **gabarit rigide** (sous-image de `subset_size × subset_size` pixels, découpée **une fois pour toutes dans la frame de référence**, jamais re-découpée dans une frame ultérieure — ceci évite que l'erreur de corrélation d'une étape ne se propage/dérive sur le long terme). Ce gabarit est recherché dans chaque frame traitée par corrélation croisée normalisée.

4.2 Corrélation croisée normalisée et fenêtre de recherche

Fonction clé : `_track_subset(frame, template, center_x, center_y, search_radius)`.

Une fenêtre de recherche est définie autour du centre `(center_x, center_y)`, étendue de `search_radius` pixels **au-delà du template de chaque côté** :

$$sx_0 = \text{round}\left(center_x - \frac{t_w}{2} - r\right), \qquad sx_1 = \text{round}\left(center_x + \frac{t_w}{2} + r\right)$$
$$sy_0 = \text{round}\left(center_y - \frac{t_h}{2} - r\right), \qquad sy_1 = \text{round}\left(center_y + \frac{t_h}{2} + r\right)$$

(avec t_w, t_h dimensions du template, r = `search_radius`), puis rognées aux bornes de la frame. Si la fenêtre rognée est plus petite que le template dans une direction (`sx1 - sx0 < tw` ou `sy1 - sy0 < th`), le point est déclaré perdu pour cette frame : retour `(center_x, center_y, NaN)`.

La corrélation elle-même est calculée par **OpenCV** :

```
corr_map = cv2.matchTemplate(search_window, template, cv2.TM_CCOEFF_NORMED)
```

c'est-à-dire le coefficient de corrélation croisée normalisée (NCC), défini par OpenCV comme :

$$R(x,y) = \frac{\sum_{x',y'} (T(x',y') - \bar{T})(I(x+x',y+y') - \bar{I}(x,y))}{\sqrt{\sum_{x',y'} (T(x',y') - \bar{T})^2 \cdot \sum_{x',y'} (I(x+x',y+y') - \bar{I}(x,y))^2}}$$

où T est le template (sous-image de référence), $\bar{T}$ sa moyenne, I la fenêtre de recherche, $\bar{I}(x,y)$ la moyenne locale du patch de I à la position candidate (x,y). $R \in [-1,1]$, 1.0 = correspondance parfaite. Le pic (position et valeur maximale) est trouvé par `cv2.minMaxLoc(corr_map)`.

4.3 Raffinement sous-pixel — `_subpixel_peak`

Interpolation parabolique à 3 points, indépendamment en x et en y, autour du pic entier `(py, px)` :

$$\delta x = \frac{1}{2} \frac{c_0 - c_2}{c_0 - 2c_1 + c_2}$$

avec c_0, c_1, c_2 les valeurs de corrélation aux abscisses $px-1, px, px+1$ (idem en y pour δy), le décalage n'étant appliqué que si $|c_0 - 2c_1 + c_2| > 10^{-12}$ (évite une division par ~0). Le code **retombe sur la position entière du pic** (décalage nul) si le pic est situé sur le bord de la carte de corrélation (pas de voisin d'un côté pour ajuster une parabole).

La nouvelle position du centre de la sous-image dans la frame courante est :

$$x_{new} = sx_0 + px + \delta x + \frac{t_w}{2}, \qquad y_{new} = sy_0 + py + \delta y + \frac{t_h}{2}$$

Limitation documentée (`core/dic_uncertainty.py`, section « WHAT THIS MODULE DOES NOT MODEL ») : ce raffinement parabolique à 3 points possède un léger biais d'interpolation près des déplacements de pixel entier, de l'ordre de quelques centièmes de pixel dans le pire cas — biais non modélisé dans l'estimation d'incertitude.

4.4 Modes de suivi « Fixed » vs « Moving »

Paramètre `tracking_mode` de `run_dic` (`"fixed"` par défaut, `"moving"` sinon — toute autre valeur est traitée comme `"fixed"`). Ce qui change est uniquement **où est centrée la fenêtre de recherche** de chaque frame :

- **`"fixed"`** : toujours centrée sur la position d'origine du point dans la frame de référence (`xx0, yy0`). Simple et immunisé contre la propagation d'un mauvais suivi antérieur, mais `search_radius` doit à lui seul couvrir le déplacement total du point sur toute la séquence traitée.
- **`"moving"`** : centrée sur la position trouvée dans la **frame précédemment traitée** (`cur_x, cur_y`). Permet à un `search_radius` restreint de suivre un déplacement cumulé important, au prix qu'une mauvaise correspondance dans une frame décale la fenêtre de recherche (et donc toutes les frames suivantes) loin de la vraie position du point.

Valeurs par défaut : `search_radius_px = 15`, `tracking_mode = "fixed"`.

4.5 Filtre de qualité de corrélation — `correlation_threshold`

Si `correlation_threshold > 0`, un point est **définitivement abandonné** (« dropped for good », NaN pour cette frame et toutes les suivantes) dès la première frame où sa NCC de pic tombe sous ce seuil :

```
if correlation_threshold > 0 and not (c >= correlation_threshold):
    active[iy, ix] = False
```

Le test `not (c >= threshold)` plutôt que `c < threshold` capture aussi le cas `c = NaN` (fenêtre de recherche entièrement sortie de l'image). Par défaut `correlation_threshold = 0.0` : aucun point n'est jamais abandonné pour raison de qualité seule.

Cause typique documentée d'une perte de point en cours de séquence : le rétrécissement de la section de l'éprouvette (striction/*necking*), qui fait qu'un point initialement sur l'éprouvette se retrouve sur le fond, que le template de référence ne matche alors plus du tout.

4.6 Filtre de cohérence de voisinage — `_apply_neighbor_consistency_filter`

Analogue numérique du « hourglass control » des éléments finis (terme employé dans le docstring). Pour chaque point encore actif d'une frame donnée, on compare son déplacement $(u, v) = (x_{new} - x_0,\ y_{new} - y_0)$ à la **médiane** des déplacements de ses voisins immédiats actifs (connectivité à 4, haut/bas/gauche/droite sur la grille) :

$$\text{deviation} = \sqrt{\left(u - \text{median}(u_{voisins})\right)^2 + \left(v - \text{median}(v_{voisins})\right)^2}$$

Si `deviation > max_neighbor_deviation_px`, le point est retiré définitivement (`active = False`, position et corrélation mis à NaN pour cette frame). Un point isolé (tous ses voisins déjà exclus) n'est comparé à rien et n'est jamais rejeté par ce filtre. Désactivé par défaut (`max_neighbor_deviation_px = 0.0`).

But documenté : capter le cas d'un point qui a bien matché (donc que `correlation_threshold` ne rejette pas) mais a « accroché » un motif de mouchetis voisin ressemblant, légèrement décalé du vrai — plus

probable quand la fenêtre de recherche est large. Un tel point isolé rend visiblement le maillage déformé « en dents de scie » (*jagged*) tout autour de lui.

4.7 Boucle principale `run_dic`

1. Vérifie que `frame_indices` n'est pas vide et que `mesh.n_points > 0`.
2. Prend `xx0, yy0 = mesh.grid_xy()` comme positions de référence.
3. Découpe une sous-image (template) `subset_size × subset_size` centrée sur chaque point valide (`mesh.mask`) de la frame de référence ; exclut (`valid = False`) tout point dont la sous-image déborde de la frame.
4. Calcule les ingrédients d'incertitude (`estimate_noise_sigma`, `subset_sssig_xy`, voir §7) une seule fois, sur les sous-images de référence — échec silencieux (best-effort, n'interrompt jamais un run par ailleurs réussi ; erreur consignée dans `uncertainty_error`).
5. `active = valid.copy()` : masque qui ne perd des points, jamais n'en regagne, au fil de la séquence.
6. Pour chaque frame traitée (dans l'ordre, séquentiellement — la parallélisation via `n_workers > 1` ne s'applique qu'*au sein* d'une même frame, jamais entre frames, car le mode « moving » centre chaque frame sur le résultat de la précédente) :
   - Frame de référence : position/corrélation fixées à `(xx0, yy0, 1.0)`.
   - Autre frame : suivi de chaque point actif (`_track_subset`, éventuellement réparti sur plusieurs threads via `core/parallel.run_map` — threads car `cv2.matchTemplate` relâche le GIL), puis application du seuil de corrélation, puis du filtre de cohérence de voisinage.
   - `u, v = new_x - xx0, new_y - yy0`.
   - Calcul du champ de déformation local `_grid_strain` (§5).
7. Retourne un `DICResult` (maillage, indices de frames, liste de `DICFrameResult`, plus les ingrédients d'incertitude de la frame de référence).

4.8 Export et réimport des résultats (CSV) — `core/dic_result_io.py`

`DICResult` n'est **jamais persisté** dans le fichier de session (docstring de `DICSettings`, `core/project.py`) : un run DIC est gardé en mémoire sur la page Image Correlation elle-même, « re-run on demand », pour garder le fichier de session léger. Le seul moyen que ce module offre de faire *sortir* un run de la page est le bouton **« Export results (.csv)… »** (`page_image_correlation.py::_on_export_results`) : un fichier avec une ligne par (frame, point de la grille), colonnes `frame_index, time_s, point_row, point_col, x0, y0, x, y, u, v, u_mm, v_mm, exx, eyy, exy, correlation`.

Le bouton **« Import results (.csv)… »**, à côté, relit ce même format pour reconstruire un `DICResult` sans avoir à relancer la corrélation — utile pour comparer plusieurs runs, reprendre un dépouillement sur une autre machine, ou simplement rouvrir un ancien résultat après avoir fermé la session. `core/dic_result_io.load_dic_result_csv(path)` fait la lecture ; `page_image_correlation.py::_on_import_result` appelle ensuite exactement le même `_on_run_finished(result)` qu'un vrai run de corrélation, pour bénéficier de toute la chaîne d'affichage/export/publication existante sans code dupliqué.

Ce qui est reconstruit à l'identique (aucune perte, mêmes unités, même convention NaN-pour-point-exclu que l'original) : le maillage régulier de points (`x0`/`y0` → `mesh.xs`/`mesh.ys`, avec `step` retrouvé exactement à partir de l'espacement médian de la grille), et chaque frame — position suivie, déplacement, corrélation,

et les champs de déformation (`exx`/`eyy`/`exy`, un tableau entier ramené à `None` si toute la frame était vide au moment de l'export — même convention que `DICFrameResult`).

Ce qui n'est *pas* dans le fichier, donc absent après réimport : les réglages du run (taille de sous-image, forme/emplacement de la ROI, rayon de recherche, mode de suivi, seuils de qualité, prétraitement crop/downscale/décimation) et les ingrédients d'incertitude bruts (`noise_sigma`, `sssig_x`, `sssig_y`). Un résultat réimporté a donc systématiquement `noise_sigma = sssig_x = sssig_y = None` — exactement l'état que `compute_uncertainty()` traite déjà comme « pas d'estimation d'incertitude disponible » pour toute autre raison (§9.9), donc aucune enveloppe d'incertitude n'est proposée pour un résultat réimporté, sans code spécial à écrire pour ce cas. `mesh.mask` est laissé à `None` (un point masqué par une ROI non rectangulaire et un point qui n'a simplement jamais été suivi sont tous deux `NaN` sur toute la séquence dans l'export — indiscernables, et traités de façon identique en aval) ; `mesh.subset_size` reçoit `step` comme valeur de substitution inerte (`core/dic_uncertainty.py` ne lit jamais `mesh.subset_size` sans avoir d'abord vérifié `noise_sigma is not None`, condition toujours fausse ici).

Récupération de deux étalonnages, quand le fichier les contient (colonnes non vides à l'export) :

- **Taille de pixel** (`mm_per_working_px`) : les colonnes `u_mm`/`v_mm` ne sont écrites que si une taille de pixel était renseignée à l'export (`scale = self._mm_per_working_px()`). Quand elles sont présentes, le rapport `u_mm / u` (et `v_mm / v`) redonne ce même facteur d'échelle — déjà « mm par pixel *de travail* », c'est-à-dire incluant le downscale du run original. La valeur retenue est la **médiane** de ce rapport groupée sur les deux axes et sur tous les points/frames disponibles (plutôt qu'une seule ligne, que l'arrondi `%.6g`/`%.4g` de l'export pourrait fausser).
- **Fréquence caméra effective** (`fps`) : la colonne `time_s` n'est écrite que si une fréquence caméra était renseignée (`fps = self._effective_fps()`) ; `frame_index` est lui toujours exact. Le rapport `(frame_index - ref_index) / time_s`, sur toutes les frames hors référence, redonne cette même fréquence effective (fréquence caméra × `time_scale`), là aussi par médiane sur toutes les lignes disponibles.

Dans les deux cas, seul le **produit** de deux réglages est récupérable, jamais leur décomposition d'origine (`pixel_size_mm × downscale_factor` ; `frame_rate_hz × time_scale`) — parce que c'est ce produit, et lui seul, qui intervient dans tout calcul en aval (§8). À l'import, la page place donc la valeur récupérée dans un seul champ et neutralise l'autre facteur (`downscale = 1` ; `time_scale = 1.0`, `time_offset_s = 0.0`), ce qui reproduit exactement les mêmes calculs quelle qu'ait été la décomposition d'origine. Un `time_offset_s` non nul au moment de l'export n'est pas récupérable du tout (c'est une constante additive, invisible dans un rapport) et est supposé nul. Quand une colonne de calibration est absente ou vide, la valeur correspondante n'est **pas** récupérée (`None`) plutôt qu'approximée — le message de statut affiché après l'import le dit explicitement, plutôt que de laisser croire à un étalonnage qui n'existe pas.

Ce que l'import ne touche jamais : les curseurs *subset size*, *step*, *frame step* et *search radius* de la page — leurs gestionnaires de changement invalident tous le résultat courant (`_invalidate_results()`), ce qui effacerait immédiatement le résultat venant d'être importé. L'import n'a d'ailleurs besoin d'aucune vidéo/séquence chargée au préalable : la page fonctionne aussi bien à partir d'une page vierge (l'aperçu image de fond, sur les graphes de résultat, est alors simplement absent — un « nice-to-have », jamais requis pour afficher les champs eux-mêmes).

---

5. Champ de déformation local — `_grid_strain` (`core/dic_processing.py`, lignes 188–202)

```python
def _grid_strain(xx0, yy0, u, v):
    dx = np.gradient(xx0, axis=1)
    dy = np.gradient(yy0, axis=0)
    dudx = np.gradient(u, axis=1) / dx
    dudy = np.gradient(u, axis=0) / dy
    dvdx = np.gradient(v, axis=1) / dx
    dvdy = np.gradient(v, axis=0) / dy
    return dudx, dvdy, 0.5 * (dudy + dvdx)  # exx, eyy, exy
```

C'est-à-dire, sur la grille régulière (espacement nominal = `step`, en x comme en y) :

$$\varepsilon_{xx} = \frac{\partial u}{\partial x}, \qquad \varepsilon_{yy} = \frac{\partial v}{\partial y}, \qquad \varepsilon_{xy} = \frac{1}{2}\left(\frac{\partial u}{\partial y} + \frac{\partial v}{\partial x}\right)$$

où u, v sont les composantes du champ de déplacement (`x - x_référence`, `y - y_référence`). Les dérivées sont calculées par **différences finies centrées** (`np.gradient`), avec un schéma **décentré (one-sided)** aux bords de la grille (comportement natif de `np.gradient`). ε_{xy} ci-dessus est la déformation de cisaillement **tensorielle** (moitié du cisaillement ingénieur $\gamma_{xy} = 2\varepsilon_{xy}$) — convention rappelée explicitement dans `core/dic_kinematics.py`.

Nature de l'approximation — précision explicite du code (docstring de `_grid_strain`) : *« First-order (small-strain) engineering strain field from the displacement field […] A quick, local estimate — good for spotting where/when strain concentrates — not a substitute for a proper large-deformation DIC strain computation. »* Il s'agit donc :

- d'une estimation du **gradient de déplacement au premier ordre** (linéarisée, hypothèse des petites déformations), donc structurellement proche d'une **déformation nominale/ingénieur** (petits déplacements relatifs entre points voisins de la grille), **pas** de la déformation logarithmique/vraie utilisée par les autres moteurs de dépouillement de l'application (moteur SHPB/QS standard) ;
- une méthode locale par différences finies sur un maillage régulier, sans reconstruction globale (pas de lissage/régularisation par éléments finis ou polynomiaux comme dans certains logiciels DIC commerciaux) ;
- un point NaN dans une frame (perte de suivi, exclu par le filtre qualité ou de voisinage) reste NaN dans `exx/eyy/exy` en ce point ; comme `np.gradient` utilise un schéma centré, la perte d'un point ne contamine que les gradients de ses voisins immédiats (différence centrée touchant ce point), pas toute la grille (précisé dans le docstring de `core/dic_kinematics.py`).

`u.shape[0] < 2 ou u.shape[1] < 2` → retourne `(None, None, None)` (grille trop petite pour calculer un gradient).

5.1 Déformation équivalente de von Mises — `core/dic_kinematics.von_mises_strain`

$$\varepsilon_{eq} = \sqrt{\frac{2}{3}\left(\varepsilon_{xx}^2 + \varepsilon_{yy}^2\right) + \frac{4}{3}\,\varepsilon_{xy}^2}$$

C'est la formule standard $\varepsilon_{eq} = \sqrt{\frac{2}{3}\,\varepsilon_{ij}^{dev} : \varepsilon_{ij}^{dev}}$ réduite au cas **déformation plane** avec cisaillement ingénieur $\gamma_{xy} = 2\varepsilon_{xy}$.

Hypothèse de déformation plane explicitée (docstring du module `dic_kinematics.py`) : la déformation hors-plan ε_{zz} est prise nulle, car une seule caméra 2D ne la mesure jamais directement (une vraie DIC 3D/stéréo donnerait ε_{zz} directement). C'est l'hypothèse la plus simple et standard en DIC 2D, appliquée telle quelle et documentée au point d'usage plutôt que dissimulée.

La même formule appliquée au tenseur des **taux** de déformation (`exx_rate, eyy_rate, exy_rate`, obtenus par différentiation temporelle, voir §6) donne le **taux de déformation équivalent de von Mises**.

---

6. Cinématique dérivée — `core/dic_kinematics.compute_kinematics`

6.1 Axe temporel

```
t_s = [(fi - ref_index) / fps for fi in frame_indices]
```

`fps` = nombre de frames **effectivement traitées** par seconde (fréquence caméra déjà divisée par la décimation `frame_step`, voir §2) ; ValueError si `fps` n'est pas strictement positif.

6.2 Vitesse et accélération

Conversion optionnelle pixel → mm via `mm_per_px` (sinon tout reste en px). Puis différentiation temporelle par différences finies (centrées à l'intérieur, décentrées aux bords), via `np.gradient(stack, t_s, axis=0)` (fonction `_gradient_over_time`), qui gère un pas de temps non uniforme directement à partir du tableau de coordonnées `t_s` :

$$v_x = \frac{dx}{dt}, \quad v_y = \frac{dy}{dt}, \quad v_{mag} = \sqrt{v_x^2 + v_y^2}$$

$$a_x = \frac{dv_x}{dt}, \quad a_y = \frac{dv_y}{dt}, \quad a_{mag} = \sqrt{a_x^2 + a_y^2}$$

Unités : `mm/s`, `mm/s²` si `mm_per_px` fourni, sinon `px/s`, `px/s²` (`Kinematics.length_unit`).

Propagation des NaN (docstring du module) : `np.gradient` étant un schéma aux différences centrées, une valeur NaN à une frame ne contamine que le(s) point(s) immédiatement voisin(s) dans le temps (ses voisins de différence centrée), jamais toute la séquence, ni les autres points spatiaux.

6.3 Force dynamique ROI — hypothèse de section constante

```
a_mag_m_s2 = a_mag_stack * 0.001  # mm/s^2 -> m/s^2
dynamic_force_stack = rho_kg_m3 * area_m2 * a_mag_m_s2  # F = rho * A * a, en Newtons
```

soit :

$$F_{dyn} = \rho \cdot A \cdot a_{mag}$$

avec ρ en kg/m³, A (`area_m2`) en m², a_{mag} convertie en m/s². **Analyse dimensionnelle explicitée dans le code** :

$$\left[\frac{kg}{m^3}\right] \times [m^2] \times \left[\frac{m}{s^2}\right] = \left[\frac{kg \cdot m}{s^2}\right] = [N]$$

`dynamic_force` n'est calculé que si `rho_kg_m3`, `area_m2` **et** `mm_per_px` sont tous les trois fournis (une densité/aire physique a besoin d'une accélération physique pour se combiner).

Interprétation physique et hypothèse de section constante (docstring de `compute_kinematics`) : $\rho \times A$ représente la « masse par unité de longueur » d'une éprouvette prismatique de section constante A (par exemple A = largeur × épaisseur pour une éprouvette plate — voir les champs de calibration de la page Image Correlation : `specimen_width_mm`, `specimen_thickness_mm`, combinés dans `core/project.py`, commentaire ligne ~210-213 : « F = rho * (w * e) * a »). C'est l'application directe de la seconde loi de Newton $F = m \cdot a$, avec la masse exprimée comme densité × aire de section.

Limites explicitement notées : - L'aire de section est supposée **constante** le long de l'éprouvette et ne rétrécit pas localement (pas de prise en compte de la striction/necking). - a_{mag} est évaluée **ponctuellement** (le champ varie donc sur la ROI) ; la lecture la plus pertinente est la moyenne ROI, interprétée comme la force d'inertie d'un segment de section uniforme accélérant comme la moyenne de la ROI. - C'est néanmoins présenté comme une « vraie » force dimensionnellement cohérente, pas une combinaison arbitraire d'unités.

6.4 Lissage — `smooth_series`

Moyenne glissante centrée, NaN-aware, sur une série 1D (typiquement une moyenne ROI par frame — utilisée pour lisser l'accélération/force dynamique avant tracé) : `window` est arrondi au nombre impair supérieur si pair ; chaque point de sortie moyenne les valeurs finies de ses `window` voisins centrés, de sorte qu'une plage de NaN ne s'étende jamais au-delà de sa propre couverture. `window <= 1` retourne une copie inchangée.

---

7. Conversion en déformation vraie — `nominal_to_true_strain` (`core/dic_kinematics.p` lignes 204–229)

$$\varepsilon_{true} = \ln(1 + \varepsilon)$$

implémenté via `np.log1p(e)` pour la stabilité numérique.

Conditions de validité, explicitées dans le docstring :

- C'est la conversion 1D standard (ratio d'étirement $\lambda = 1 + \varepsilon$, $\varepsilon_{true} = \ln \lambda$), **exacte** pour un allongement/raccourcissement réellement **uniaxial** 1D.
- **Approximation** dans les autres cas : un état de déformation 2D/3D n'a pas de « déformation vraie » scalaire unique bien définie comme un simple changement de longueur 1D. Son application à une composante de déformation normale (`exx`, `eyy`), ou à la déformation équivalente/effective toujours non négative `eps_eq`, est une pratique standard qui se réduit au cas 1D exact dans ce dernier cas.
- **Jamais appliquée à `exy`** (déformation de cisaillement) : le cisaillement n'a pas d'interprétation en ratio d'étirement sur laquelle cette formule repose. Les appelants ne convertissent donc jamais `exy` (voir `ui/pages/page_image_correlation.py`, `_publish_dic_strain`, et `core/project.py` : `strain_convert_to_true` est ignoré/jamais appliqué quand `strain_definition == "exy"`).
- **Indéfini (NaN)** partout où $1+\varepsilon \leq 0$ (plus de 100 % de raccourcissement nominal), plutôt que de lever une exception — physiquement, ce point a été écrasé à une longueur nulle ou négative, ce qu'aucune mesure de déformation ne peut décrire de façon significative.

Formule de taux associée, `nominal_to_true_strain_rate(rate, nominal_strain)` :

$$\dot{\varepsilon}_{true} = \frac{d}{dt}\big[\ln(1+\varepsilon)\big] = \frac{\dot{\varepsilon}}{1+\varepsilon}$$

(même domaine de validité et même comportement NaN hors domaine que ci-dessus).

Par défaut, `DICSettings.strain_convert_to_true = True` — comportement documenté (`core/project.py`) : ce choix a été retenu par défaut parce qu'une comparaison non convertie contre la « Strain (true) » du moteur standard serait la plus surprenante/la moins utile des deux options pour le graphique « Multi-Strain Comparison ».

---

8. Time scale / Time offset — synchronisation caméra <-> jauges

Il n'y a aucune synchronisation automatique caméra/signaux de barres dans l'application — ceci est explicitement indiqué à plusieurs endroits du code et de l'interface (`ui/pages/page_image_correlation.py`, ex. lignes ~906–913, ~2514–2518) : *« there is no camera-to-bar-signal synchronization in this app »*.

Mécanisme (`core/project.py`, `DICSettings.time_scale` / `time_offset_s`, lignes 261–284) :

- La caméra et l'acquisition des jauges/barres (DAQ) sont **deux horloges indépendantes et non synchronisées**. De plus, la cadence de capture réelle de la caméra peut légèrement différer de la valeur nominale entrée par l'utilisateur (`frame_rate_hz`).
- **`time_scale`** corrige ce dernier point :

  $$\text{effective_fps} = \text{frame_rate_hz} \times \text{time_scale}$$

  Valeur par défaut 1.0 (fréquence entrée inchangée) ; p. ex. 1.02 signifie que la caméra a réellement tourné 2 % plus vite que la valeur entrée.

- **`time_offset_s`** décale ensuite le temps DIC (ajouté **après** la mise à l'échelle, donc n'affecte pas les dérivées temporelles — un décalage constant ne change pas une dérivée) :

$$t_{DIC}(i) = \frac{i - i_{ref}}{\text{effective_fps}} + \text{time_offset_s}$$

  implémenté dans `_dic_time_s` (`page_image_correlation.py`, ligne ~1584–1597) et pour le burn-in vidéo dans `_format_timestamp` (ligne ~3121–3137).
- Ces deux paramètres doivent être réglés **manuellement** par l'utilisateur, en faisant correspondre une caractéristique visible dans les deux séries temporelles (par exemple le début et la durée d'un même événement) — aucune détection automatique n'est fournie.
- Ils s'appliquent partout où cette page construit un axe temporel : tous les graphes en fonction du temps, vitesse/accélération/taux de déformation (qui dépendent de `dt`, donc `time_scale` corrige aussi le `dt` physique utilisé pour ces dérivées), la série « DIC Strain » publiée, tout export CSV avec une colonne `time_s`, et l'horodatage incrusté dans la vidéo exportée.
- **Avertissement** (commentaire ligne ~943-947) : si l'enregistrement caméra couvre une durée plus courte ou plus longue que celle réellement écoulée, `time_scale` peut aussi compenser cela.

Remarque annexe : le module `core/camera_clock.py` (outil séparé « Camera / Video Sync ») propose des fonctions pour dépouiller un fichier d'horloge caméra externe (détection de train d'impulsions d'exposition `detect_pulse_train`, détection de front montant d'un signal de trigger `detect_rising_edge`, extrapolation des temps de frame `estimate_frame_times`, localisation de la frame de trigger `locate_trigger_by_signal` / `locate_trigger_by_frame_number`) — mais ceci est un outil d'analyse de fichier séparé, non branché automatiquement sur `time_scale`/`time_offset_s` du pipeline DIC lui-même ; la synchronisation avec les signaux de barres reste, dans le pipeline DIC, un réglage manuel de l'utilisateur.

---

9. Modèle d'incertitude de mesure DIC (`core/dic_uncertainty.py`)

9.1 Positionnement du modèle — ce qu'il EST et n'est PAS

Le docstring d'en-tête du module est très explicite et doit être repris intégralement dans la documentation :

> *« There is no single, universally "correct" way to state a DIC measurement's uncertainty […] DIC's uncertainty depends on the speckle pattern's own texture, the camera's noise, the correlation mesh's own settings (subset size, step), and how much the material actually deforms during the test. »*

Le modèle combine **trois mécanismes d'erreur distincts**, en s'appuyant sur la littérature de métrologie DIC citée dans le code : Sutton et al. (ouvrage de référence DIC) ; Bornert et al., *« Assessment of Digital Image Correlation Measurement Errors: Methodology and Results »*, Exp Mech 2009 ; Wang, Sutton et al., *« Quantitative Error Assessment in Pattern Matching »*, Strain 2009 ; Lecompte et al. 2006 (effets du motif de mouchetis / de la taille de sous-image).

Ce que ce module n'est PAS (section « THE RIGOROUS ALTERNATIVE THIS DOES NOT

REPLACE ») : pour un budget d'incertitude validé de qualité métrologique (p. ex. pour un rapport devant énoncer une incertitude certifiée), la pratique standard est une **calibration EMPIRIQUE** que ce module ne remplace pas : faire tourner le pipeline DIC identique (même taille de subset, même step, même motif de mouchetis) sur (a) deux images du **même** spécimen non déformé (essai « zéro » / corps rigide) pour mesurer directement le vrai plancher de bruit incluant tous les effets réels à la fois, et/ou (b) une image de référence déformée synthétiquement avec un champ de déplacement/déformation **connu**, pour mesurer le biais et la dispersion réels contre une vérité terrain. Cela donne un nombre empirique unique (remplaçant les termes 1-2 ci-dessous) validé pour **ce montage précis**, alors que tout ce qui suit est un **modèle prédictif sans vérité terrain pour se vérifier lui-même**.

Ce que ce module NE modélise PAS (section « WHAT THIS MODULE DOES NOT MODEL ») : la distorsion/calibration d'objectif, le mouvement hors-plan (3D) dans un montage DIC 2D, les vibrations/flou de mouvement au-delà de ce qui se reflète déjà dans un coefficient de corrélation plus faible, et le **biais d'interpolation sous-pixel** fixe que certains schémas de corrélation présentent près des déplacements de pixel entier (l'ajustement parabolique à 3 points de cette application a un tel biais, de l'ordre de quelques centièmes de pixel dans le pire cas — voir `_subpixel_peak`, §4.3). `_MIN_SIGMA_PX = 0.005` px est un plancher fixé précisément pour reconnaître qu'aucun système réel ne descend sous une petite fraction de pixel, quelle que soit la qualité du mouchetis.

9.2 Mécanisme 1 — Plancher de précision de recalage sub-pixel (photométrique)

Estimation du bruit de l'image — `estimate_noise_sigma(image)` : estimateur rapide de bruit d'Immerkaer (J. Immerkaer, *« Fast Noise Variance Estimation »*, CVIU 1996) : convolution avec un noyau laplacien discret choisi pour que le contenu lisse de l'image s'annule, ne laissant essentiellement que du bruit :

```
kernel = [[1, -2, 1], [-2, 4, -2], [1, -2, 1]]
conv = convolve2d(img, kernel, mode="valid")
sigma = sqrt(pi/2) * mean(|conv|) / 6
```

soit :

$$\sigma_{noise} = \sqrt{\frac{\pi}{2}} \cdot \frac{\overline{|I * K|}}{6}$$

Peu coûteux (une seule petite convolution), ne nécessite aucune image d'étalonnage (dark-frame/flat-field), au prix d'être une estimation approximative — un motif de mouchetis réellement texturé a un vrai contenu haute fréquence que cet estimateur ne peut pas parfaitement distinguer du bruit, ce qui tend à en faire une estimation **légèrement pessimiste** (trop élevée) plutôt qu'optimiste — direction jugée plus sûre pour un plancher d'incertitude. Retourne 0.0 si l'image fait moins de 3×3 pixels.

SSSIG (Sum of Squared Subset Intensity Gradients) — `subset_sssig_xy(patch)`, calculé sur la sous-image de **référence uniquement** :

$$\text{SSSIG}_x = \sum (\partial I/\partial x)^2, \qquad \text{SSSIG}_y = \sum (\partial I/\partial y)^2$$

(gradients obtenus par `np.gradient`). Une sous-image de mouchetis riche et contrastée donne un SSSIG

élevé (pic de corrélation net et bien localisé), une sous-image plate/faible contraste donne un SSSIG faible (pic peu profond, mal localisé).

Plancher de type Cramér-Rao — `crlb_sigma_px(noise_sigma, sssig)` :

$$\sigma_{u,floor} = \max\left(\frac{\sigma_{noise}}{\sqrt{\text{SSSIG}_x}},\ \sigma_{min}\right), \qquad \sigma_{v,floor} = \max\left(\frac{\sigma_{noise}}{\sqrt{\text{SSSIG}_y}},\ \sigma_{min}\right)$$

avec $ _{min} = $ _MIN_SIGMA_PX = 0.005 px. NaN partout où `sssig` est lui-même NaN (point exclu/non suivi). C'est la même formule fermée utilisée par des logiciels DIC industriels (ex. VIC-2D de Correlated Solutions) pour publier une carte d'erreur « u/v » par point. **C'est un plancher (borne inférieure), pas toute l'histoire** — d'où les termes 2 et 3.

9.3 Mécanisme 2 — Dégradation liée à la qualité de corrélation

`correlation_inflation(correlation, corr_floor=0.5, exponent=2.0)` : facteur multiplicatif d'inflation appliqué au plancher de registration, croissant à mesure que la NCC de pic de cette frame s'éloigne d'une correspondance parfaite (1.0) :

$$\text{inflation} = \left(\frac{1}{\max(\text{corr_floor},\ \min(1, C))}\right)^{exponent}$$

où C est la corrélation NCC de la frame, saturée (clip) entre `corr_floor` (0.5 par défaut) et 1.0. `exponent = 2.0` est un **choix de modélisation** (non dérivé de premiers principes, précisé explicitement dans le docstring) : garde l'inflation modeste pour une correspondance bonne-mais-imparfaite (NCC = 0.98 → inflation ≈ 1.04) tout en pénalisant visiblement une correspondance médiocre (NCC = 0.7 → inflation ≈ 2).

Distinction explicite avec `correlation_threshold` : ceci est un modèle de dégradation lisse pour un point *toujours suivi*, pas un substitut au seuil `correlation_threshold` de `run_dic` (§4.5), qui abandonne un point purement et simplement une fois sa correspondance devenue peu fiable.

Mécanisme 2 — motivation physique : les mécanismes réels qui dégradent une correspondance mais que le plancher photométrique du mécanisme 1 ne capture pas intrinsèquement : mouvement hors-plan, changement d'éclairage, décorrélation partielle, flou de mouvement.

9.4 Mécanisme 3 — Distorsion du subset (moyennage intra-subset)

`distortion_term_px(local_strain_mag, subset_size_px, k=0.5)` :

$$\sigma_{distortion} = k \cdot |\varepsilon_{local}| \cdot \text{subset_size_px}$$

Chaque subset étant suivi comme **un seul gabarit rigide en translation** (§4.1), si le vrai champ de déplacement varie sensiblement **à l'intérieur** d'un subset (déformation locale grande par rapport à la taille du subset), la position unique rapportée par la corrélation est un mélange à travers cette variation, pas la position exacte d'un point matériel unique. `k = 0.5` **approxime** un gradient de déplacement linéaire à

travers le subset : la correspondance se stabilise près du centre du subset, donc l'écart au pire cas au bord du subset, $|\varepsilon| \times \text{subset_size}/2$, est une estimation d'ordre de grandeur raisonnable de l'erreur de position résultante — **choix de modélisation, pas une dérivation rigoureuse** (précisé explicitement).

`local_strain_mag` (calculé dans `compute_uncertainty`, ligne 432) :

$$|\varepsilon_{local}| = \sqrt{\varepsilon_{xx}^2 + \varepsilon_{yy}^2 + 2\varepsilon_{xy}^2}$$

(norme de Frobenius du tenseur de déformation local `_grid_strain`, ou 0 si les déformations ne sont pas disponibles pour cette frame).

C'est la réponse directe du modèle à la question « qu'en est-il du niveau de distorsion des éléments » — la règle classique de la littérature sur le mouchetis : « garder le subset petit par rapport au gradient de déformation ».

9.5 Combinaison des trois termes en quadrature

`combine_displacement_uncertainty(sigma_floor_px, inflation, distortion_px)` :

$$\sigma_u = \sqrt{\left(\sigma_{u,floor} \times \text{inflation}\right)^2 + \sigma_{distortion}^2}$$

(idem pour σ_v avec $\sigma_{v,floor}$). **Hypothèse d'indépendance, explicitement discutée comme simplification** (docstring d'en-tête, point « All three are combined in quadrature ») : les trois termes sont supposés non corrélés — simplification, les mécanismes 1 et 2 en particulier ne sont pas totalement indépendants en réalité — mais les traiter comme tels donne une enveloppe utilisable et, si quelque chose, **plutôt conservatrice** (les termes ne s'annulent pas partiellement entre eux).

9.6 Propagation aux déformations

`strain_uncertainty_from_displacement(sigma_u_px, sigma_v_px, step_px)` — propagation via la même formule de différences centrées que `_grid_strain` (`np.gradient` sur une grille régulière d'espacement `step_px`) :

Pour un point intérieur, $\varepsilon_{xx} = \dfrac{u_{i,j+1} - u_{i,j-1}}{2\,\text{step}}$; si les deux échantillons de u ont un bruit indépendant de variance σ_u^2 chacun :

$$\text{Var}(\varepsilon_{xx}) = \frac{2\sigma_u^2}{(2\,\text{step})^2} = \frac{\sigma_u^2}{2\,\text{step}^2} \quad \Rightarrow \quad \sigma_{\varepsilon_{xx}} = \frac{\sigma_u}{\text{step}\sqrt{2}}$$

et de même $\sigma_{\varepsilon_{yy}} = \sigma_v/(\text{step}\sqrt{2})$. Pour le cisaillement, $\varepsilon_{xy} = \frac{1}{2}(\partial u/\partial y + \partial v/\partial x)$ combine une dérivée en y de u et une dérivée en x de v, chacune avec le **même** σ_u/σ_v propre au point (l'incertitude de registration d'un point ne dépend pas de la direction dans laquelle il est ensuite différencié), additionnées en quadrature et divisées par deux :

$$\sigma_{\varepsilon_{xy}} = \frac{1}{2} \cdot \frac{\sqrt{\sigma_u^2 + \sigma_v^2}}{\text{step}\sqrt{2}}$$

Approximation documentée : la formule utilise l'espacement nominal `step_px` partout, y compris aux bords du maillage où `np.gradient` utilise en réalité une différence à un seul côté (pas la formule à deux côtés supposée ici) — approximation qui **sous-estime légèrement** l'incertitude réelle des points de bord (une différence à un seul côté du même pas est plus bruitée, pas moins, que la version à deux côtés).

Incertitude de la déformation équivalente de von Mises — `von_mises_strain_uncertainty` : propagation d'erreur linéarisée (premier ordre) au travers de la formule de `von_mises_strain` :

$$\sigma_{\varepsilon_{eq}}^2 = \left(\frac{\partial \varepsilon_{eq}}{\partial \varepsilon_{xx}}\right)^2 \sigma_{\varepsilon_{xx}}^2 + \left(\frac{\partial \varepsilon_{eq}}{\partial \varepsilon_{yy}}\right)^2 \sigma_{\varepsilon_{yy}}^2 + \left(\frac{\partial \varepsilon_{eq}}{\partial \varepsilon_{xy}}\right)^2 \sigma_{\varepsilon_{xy}}^2$$

avec $\frac{\partial \varepsilon_{eq}}{\partial \varepsilon_{xx}} = \frac{2}{3}\frac{\varepsilon_{xx}}{\varepsilon_{eq}}$, $\frac{\partial \varepsilon_{eq}}{\partial \varepsilon_{yy}} = \frac{2}{3}\frac{\varepsilon_{yy}}{\varepsilon_{eq}}$, $\frac{\partial \varepsilon_{eq}}{\partial \varepsilon_{xy}} = \frac{4}{3}\frac{\varepsilon_{xy}}{\varepsilon_{eq}}$. **Hypothèse simplificatrice explicite** : ε_{xx}, ε_{yy}, ε_{xy} sont supposées avoir des erreurs indépendantes, alors qu'en réalité elles partagent le même bruit de u/v sous-jacent — même type d'approximation utilisée partout ailleurs dans le module. NaN (plutôt qu'une division par zéro) là où $\varepsilon_{eq} = 0$.

9.7 Propagation temporelle (taux, vitesse, accélération, force)

`time_derivative_uncertainty(sigma_value, dt_s)` — même principe, différentiation en temps au lieu de l'espace, en supposant un `dt_s` à peu près uniforme :

$$\sigma_{\dot{x}} = \frac{\sigma_x}{\text{dt}\sqrt{2}}$$

Simplification explicitement signalée : enchaîner cet appel deux fois (position → vitesse → accélération) traite le bruit de chaque étage comme indépendant du précédent, ce qui n'est pas exactement vrai (des étages consécutifs réutilisent des échantillons de position qui se chevauchent) — signalé dans le code, non corrigé.

Utilisé pour $\sigma_{v_x}, \sigma_{v_y}$ (à partir de σ_u, σ_v), puis $\sigma_{a_x}, \sigma_{a_y}$ (à partir de $\sigma_{v_x}, \sigma_{v_y}$), et pour les incertitudes des taux de déformation $\sigma_{\dot{\varepsilon}_{xx}}, \sigma_{\dot{\varepsilon}_{yy}}, \sigma_{\dot{\varepsilon}_{xy}}$.

Incertitude de la force dynamique — même conversion mm/s² → m/s² (×0.001) que `compute_kinematics` applique à a_{mag} avant $F = \rho A a$:

$$\sigma_F = \rho \cdot A \cdot \sigma_{a_{mag}} \times 0.001$$

(propagation linéaire directe, ρ et A étant traités comme exacts).

Incertitude des grandeurs « magnitude » — `_hypot_uncertainty(x, y, sigma_x, sigma_y)`, utilisée pour la magnitude du déplacement, de la vitesse et de l'accélération : propagation à travers $\text{mag} = \sqrt{x^2 + y^2}$:

$$\sigma_{mag} = \sqrt{\left(\frac{x}{mag}\sigma_x\right)^2 + \left(\frac{y}{mag}\sigma_y\right)^2}$$

(repli à 0 au point dégénéré $mag = 0$).

9.8 Nombre de points effectif et incertitude de moyenne ROI

`effective_independent_count(n_active_points, roi_area_px2, subset_size_px)` : les points DIC voisins **ne sont pas des mesures indépendantes** — leurs subsets se chevauchent quand `step < subset_size` (réglage courant), et même sans chevauchement, des points voisins sur un matériau continu sont physiquement liés, de sorte que leurs erreurs de suivi ont tendance à évoluer ensemble plutôt qu'à s'annuler par moyennage comme un bruit indépendant le ferait.

Le modèle approxime la **longueur de corrélation** comme égale à **une largeur de subset**, c'est-à-dire traite la ROI comme couverte par :

$$n_{eff} = \frac{A_{ROI}}{\text{subset_size}^2}$$

« patches » indépendants — correction standard (quoique grossière, précisé explicitement) pour du bruit spatialement corrélé lorsqu'on moyenne sur une surface (cf. le concept de « taille d'échantillon effective » en statistique spatiale). Bornée à $[1, n_{active_points}]$, de sorte que ce calcul **ne revendique jamais une précision meilleure** que le nombre brut de points suivis, ni un nombre effectif nul/négatif.

`roi_mean_uncertainty(sigma_field, n_eff)` : erreur-type d'une moyenne ROI, combinant le RMS des incertitudes ponctuelles avec la réduction standard en $1/\sqrt{n}$, mais sur le nombre **effectif** plutôt que le nombre brut de points :

$$\sigma_{\overline{X}} = \frac{\sqrt{\frac{1}{N}\sum_i \sigma_i^2}}{\sqrt{n_{eff}}}$$

(RMS des sigmas ponctuels finis, la bonne façon de combiner plusieurs sigmas *différents* en un seul représentatif pour des points de qualité variable, divisé par $\sqrt{n_{eff}}$). NaN si aucun point n'a de sigma fini.

9.9 Orchestration — `compute_uncertainty`

Construit un objet `DicUncertainty` complet à partir d'un `DICResult` (a besoin de `noise_sigma`/`sssig_x`/`sssig_y`) et, optionnellement, d'un `Kinematics` déjà calculé (pour les champs taux/vitesse/accélération/force). Retourne `None` si `result` n'a aucune estimation de bruit/SSSIG (cas best-effort, voir §4.7). Les mêmes `mm_per_px`/`rho_kg_m3`/`area_m2` que ceux utilisés pour `compute_kinematics` doivent être passés, afin que les bandes d'incertitude correspondent bien aux valeurs qu'elles encadrent.

Structure `DicUncertainty` — un tableau par frame traitée, même agencement que `DICFrameResult`/`Kinematics` (`sigma_u`, `sigma_v`, `sigma_disp_mag`, `sigma_exx`, `sigma_eyy`, `sigma_exy`, `sigma_eps_eq`, puis, si `kin`

fourni : `sigma_vx/vy/v_mag`, `sigma_ax/ay/a_mag`, `sigma_exx_rate/eyy_rate/exy_rate/eps_eq_rate`, `sigma_dynamic_force`).

Paramètres de réglage exposés (transmis à l'UI, `core/project.py DICSettings`) : `show_uncertainty` (bool, défaut `True`), `uncertainty_sigma_k` (multiple du sigma 1-σ affiché en enveloppe, défaut 1.0). Commentaire explicite du code sur le choix `show_uncertainty = True` par défaut : *« a DIC result plotted with no indication of its own uncertainty invites over-reading it »* (un résultat DIC tracé sans aucune indication de sa propre incertitude invite à le surinterpréter). Et sur `uncertainty_sigma_k` : 1.0 → ±1σ (~68 % de couverture *si* les erreurs sous-jacentes étaient gaussiennes, **ce qui n'est pas vérifié** — précisé explicitement), 2.0 → ±2σ (~95 %).

---

10. Rendu visuel (`core/dic_render.py`) — résumé bref

Fonction unique `render_field(fig, ax, background, x, y, values, field_label, vmin, vmax, cmap, timestamp_text, text_color, lost_track_color, max_edge_px)`, partagée entre l'affichage live (page Image Correlation) et l'export vidéo (`core/dic_video_export.py`).

- Dessine sur une **triangulation de Delaunay fraîche** des positions **réelles** des points dans la frame courante (`ax.tricontourf`), plutôt que sur la grille quadrillée fixe de la frame de référence (`contourf` sur `mesh.grid_xy()`) — **motif documenté** : garder la connectivité d'un quadrilatère fixe pendant que ses quatre coins bougent indépendamment peut le faire se replier sur lui-même dès que des points voisins divergent, même légèrement (très visible une fois qu'une éprouvette strictionne fortement — une tache de couleur en dents de scie, en pointe, se recoupant elle-même, au lieu d'un champ lisse). Une triangulation de Delaunay fraîche d'un nuage de points n'a jamais ce problème par construction.
- Un point sans position fiable dans cette frame (perte de suivi, masqué, ou rejeté par un filtre qualité) est simplement omis de la triangulation, plutôt que maintenu à une position de substitution — une position de substitution était elle-même source du même artefact de repliement.
- Si moins de 3 points valides (ou ensemble dégénéré, ex. points colinéaires), repli sur un simple nuage de points coloré (`ax.scatter`) plutôt que la triangulation.
- `vmin`/`vmax` explicites fixent l'échelle de couleur (utilisé pour la cohérence frame-à-frame d'une vidéo exportée) ; laissés à `None`, l'échelle s'ajuste automatiquement aux données de chaque frame (correspond à `color_scale_mode = "auto"` de `DICSettings`, l'alternative étant `"fixed"` avec `color_scale_min`/`color_scale_max`).
- **Masquage des triangles « pont » au-dessus d'une zone exclue** (`max_edge_px`, `_long_edge_mask`) : une triangulation de Delaunay ne triangule jamais que l'**enveloppe convexe** des points fournis, sans aucune notion de trou en son sein. Une ROI non rectangulaire (polygone/ellipse) ou une zone manuellement exclue laisse un vrai creux concave sans aucun point suivi à l'intérieur (`MeshGrid.mask`) — mais les points juste de part et d'autre du creux restent valides, donc une triangulation naïve les relie tout de même par des triangles longs et artificiels, que `tricontourf` colorie ensuite comme s'il s'agissait de données réelles (un dégradé lisse mais entièrement fictif comblant l'entaille/le trou exclu). Quand `max_edge_px` est fourni, tout triangle dont une arête dépasse cette longueur est éliminé de la triangulation avant le tracé (`Triangulation.set_mask`), pour que la zone exclue apparaisse comme un vrai vide plutôt qu'une zone colorée fabriquée. La

valeur recommandée, `default_max_edge_px(mesh.step) = mesh.step × 3` (utilisée par les deux appelants — canvas Résultats en direct et export vidéo), laisse une marge nette au-dessus de la diagonale d'une triangulation de grille régulière non déformée (~1,41 × step) et d'une zone modérément déformée (jusqu'à ~2 × step), tout en éliminant un pont qui traverse toute une entaille/un trou. **Limite connue et acceptée** : juste au coin rentrant (concave) d'une zone exclue, un petit triangle peut encore avoir chacune de ses arêtes sous ce seuil tout en recouvrant un fragment de la zone exclue — arrondir un coin concave prend une arête bien plus courte que ponter tout le trou. Ce masquage élimine le pontage massif sur l'intérieur d'un trou, pas ce dernier arrondi, visuellement mineur, tout contre son bord.

Ce module est essentiellement graphique et ne modifie ni ne recalcule aucune grandeur physique.

---

11. Récapitulatif des correspondances fichier → fonction

Sujet	Fichier	Fonction(s)
Crop / contraste / downscale	`core/dic_preprocessing.py`	`crop_frame`, `downscale_frame`, `preprocess_frame`
Décimation temporelle	`core/dic_preprocessing.py`	`decimated_frame_indices`
ROI rectangle / ellipse / polygone	`core/dic_correlation.py`	`ROI`, `Ellipse`, `bounding_roi`
Maillage de corrélation	`core/dic_correlation.py`	`build_mesh`, `build_mesh_from_polygon`, `build_mesh_from_ellipse`, `MeshGrid`
Suivi NCC + sous-pixel	`core/dic_processing.py`	`_track_subset`, `_subpixel_peak`
Boucle de suivi complète, Fixed/Moving, filtres	`core/dic_processing.py`	`run_dic`, `_track_chunk`, `_apply_neighbor_consistency_filter`
Champ de déformation local	`core/dic_processing.py`	`_grid_strain`
Export / réimport des résultats (CSV)	`core/dic_result_io.py`, `ui/pages/page_image_correlation.py`	`load_dic_result_csv`, `_on_export_results`, `_on_import_results`
Vitesse / accélération	`core/dic_kinematics.py`	`compute_kinematics`, `_gradient_over_time`
Déformation/taux équivalents von Mises	`core/dic_kinematics.py`	`von_mises_strain`
Force dynamique ROI	`core/dic_kinematics.py`	`compute_kinematics` (bloc `dynamic_force`)
Conversion en déformation vraie	`core/dic_kinematics.py`	`nominal_to_true_strain`, `nominal_to_true_strain_rate`
Lissage temporel	`core/dic_kinematics.py`	`smooth_series`

Sujet	Fichier	Fonction(s)
Time scale / offset	`core/project.py` (`DICSettings`), `ui/pages/page_image_correlation.py`	`_effective_fps`, `_dic_time_s`, `_format_timestamp`
Bruit image / SSSIG	`core/dic_uncertainty.py`	`estimate_noise_sigma`, `subset_sssig_xy`
Plancher de registration (méc. 1)	`core/dic_uncertainty.py`	`crlb_sigma_px`
Dégradation qualité (méc. 2)	`core/dic_uncertainty.py`	`correlation_inflation`
Distorsion subset (méc. 3)	`core/dic_uncertainty.py`	`distortion_term_px`
Combinaison quadratique	`core/dic_uncertainty.py`	`combine_displacement_uncertainty`
Propagation vers déformations	`core/dic_uncertainty.py`	`strain_uncertainty_from_displacement`, `von_mises_strain_uncertainty`
Propagation temporelle (taux, v, a, F)	`core/dic_uncertainty.py`	`time_derivative_uncertainty`, `_hypot_uncertainty`
Points effectifs / moyenne ROI	`core/dic_uncertainty.py`	`effective_independent_count`, `roi_mean_uncertainty`
Orchestration incertitude	`core/dic_uncertainty.py`	`compute_uncertainty`, `DicUncertainty`
Rendu du champ (Delaunay)	`core/dic_render.py`	`render_field`
Masquage des triangles-pont sur une zone exclue	`core/dic_render.py`	`default_max_edge_px`, `_long_edge_mask`
Horloge caméra (outil séparé)	`core/camera_clock.py`	`detect_pulse_train`, `locate_trigger_by_signal`, `locate_trigger_by_frame_number`, `estimate_frame_times`

---

12. Synthèse des limitations et avertissements relevés dans le code

1. Suivi rigide en translation par subset (pas de subset déformable) — approprié aux déformations modérées, à dominante uniaxiale (`core/dic_processing.py`, en-tête).
2. `_grid_strain` : estimation locale, petites déformations, premier ordre — pas un calcul DIC grande déformation rigoureux (`_grid_strain`, docstring).
3. La conversion `eps_true = ln(1+e)` n'est exacte qu'en 1D uniaxial ; approximation en 2D/3D ; jamais appliquée au cisaillement `exy` (`nominal_to_true_strain`, docstring).
4. Force dynamique ROI : hypothèse de section constante (pas de striction), évaluée ponctuellement (`compute_kinematics`, docstring).
5. Aucune synchronisation automatique caméra/jauges — réglage manuel via `time_scale`/`time_offset_s` (`ui/pages/page_image_correlation.py`, plusieurs endroits).
6. Modèle d'incertitude **prédictif**, non calibré empiriquement — ne remplace pas un essai « zéro »/corps rigide ou une déformation synthétique connue pour une incertitude certifiée (`core/dic_uncertainty.py`, en-tête).

7. Le modèle d'incertitude ne couvre pas : distorsion optique, mouvement hors-plan, vibration/flou au-delà de ce que reflète déjà la NCC, biais d'interpolation sous-pixel de l'ajustement parabolique (`core/dic_uncertainty.py`, en-tête).
8. Les trois termes d'incertitude sont combinés en quadrature en supposant l'indépendance — simplification reconnue, tendanciellement conservatrice (`core/dic_uncertainty.py`, en-tête).
9. Propagation d'erreur (déformation, taux, force) suppose des erreurs indépendantes entre composantes qui partagent en réalité le même bruit sous-jacent de u/v — simplification assumée (`strain_uncertainty_from_displacement`, `von_mises_strain_uncertainty`, `time_derivative_uncertainty`).
10. Nombre de points effectif : approximation grossière de la longueur de corrélation (= une largeur de subset) pour corriger le moyennage ROI du bruit spatialement corrélé (`effective_independent_count`, docstring).
11. Coverage ±1σ/±2σ supposée gaussienne mais non vérifiée (`core/project.py`, commentaire `uncertainty_sigma_k`).
12. Ce modèle d'incertitude DIC est **distinct** de l'incertitude formelle calculée par l'outil "Crack & Wave Analysis" (chapitre 10) sur les facteurs d'intensité de contrainte K_I/K_II : ce dernier est une covariance de moindres carrés (OLS exacte pour l'ajustement DIC standard en anneau, asymptotique de Gauss-Newton pour X-FEM-DIC — chapitre 10, §4.2 et §6.7), appliquée à un ajustement de champ near-tip après coup ou à une re-corrélation directe des images, pas une extension ou un raffinement du modèle prédictif de ce chapitre.
13. Un résultat réimporté depuis un export CSV (§4.8) n'a **jamais** d'enveloppe d'incertitude (`noise_sigma`/`sssig_x`/`sssig_y` non recouvrables depuis le fichier) et perd les réglages du run d'origine (taille de sous-image, ROI, rayon de recherche, seuils, prétraitement) — seuls le maillage, les champs suivis par frame, et, quand les colonnes correspondantes étaient renseignées à l'export, la taille de pixel et la fréquence caméra effective sont récupérés (`core/dic_result_io.py`, docstring du module).
14. Le masquage des triangles-pont sur une zone exclue (§10, `max_edge_px`) élimine le pontage massif sur l'intérieur d'un trou (ROI non rectangulaire, zone exclue), mais un petit triangle rasant tout contre un coin concave de cette zone peut encore survivre au masquage — arrondir un coin concave prend une arête plus courte que le seuil, contrairement à un pont traversant tout le trou (`core/dic_render.py::_long_edge_mask`, docstring).

7. Correction de dispersion des ondes — régime élastique

Fichier source de référence : `core/dispersion.py` (858 lignes). En-tête du module (lignes 1–24) :

> « Pochhammer-Chree dispersion correction. Ported from `FFT_dispersion.py` / `FFT_dispersion_ID.py` / `FFT_dispersion_visco.py` / `FFT_dispersion_visco_ID.py`. »

Le module gère **quatre combinaisons** (deux axes indépendants), toutes dans le même fichier :

	Barres élastiques	Barres viscoélastiques
Montage 2 barres (SHPB classique)	`elastic_correction()`	`viscoelastic_correction()`
Montage barre de sortie seule (Impact Direct, "_ID")	`elastic_correction_direct_impact()`	`viscoelastic_correction_direct_impact()`

Ce chapitre couvre exclusivement la colonne **« Barres élastiques »** (`elastic_correction` et `elastic_correction_direct_impact`, plus les fonctions communes `_bar_elastic_wave_speed` et `_solve_pochhammer_branch`/`_solve_pochhammer_branches`). Le cas viscoélastique (modèle rhéologique du type Solide Linéaire Standard, module complexe, `core/rheology_models.py`, `_solve_visco_branch`, `_visco_phase_correction`, `viscoelastic_correction*`) est traité dans le chapitre « dispersion viscoélastique » ; il n'est mentionné ici que pour expliquer **comment l'application bascule** entre les deux régimes.

Un avertissement du module (lignes 19–23) doit être reproduit tel quel dans la documentation destinée à l'utilisateur :

> « NOTE: this module is numerically faithful to the original but has not been re-validated against real experimental data (no test data / no way to run sympy's `nsolve` root search in this environment) — treat the physics as "ported, not re-verified" and sanity-check against a known-good original run before relying on it for a real analysis. »

C'est-à-dire : le portage préserve fidèlement l'algorithme numérique d'origine, mais n'a pas été re-vérifié sur un cas expérimental connu dans l'environnement de portage.

---

1. Principe physique : pourquoi une barre finie disperse l'onde

La théorie élémentaire des barres (théorie 1D, dite « théorie de Bernoulli/onde de compression pure ») suppose une onde de déformation axiale uniforme sur toute la section de la barre, se propageant sans déformation de forme à la célérité constante

$$c_0 = \sqrt{E/\rho}$$

où E est le module d'Young du matériau de la barre et ρ sa masse volumique. C'est cette relation qui est utilisée dans l'application pour tout calcul « non dispersif » (temps de vol, dépouillement standard) — voir `core/models.py` et le champ `C0_pa` d'un `Bar`.

En réalité, une barre a un rayon fini a. Le champ de déplacement dans une barre cylindrique de section finie n'est **pas** uniforme sur la section : la propagation d'une onde de compression harmonique s'accompagne nécessairement d'un mouvement radial (effet Poisson dynamique, inertie radiale). Ce couplage axial/radial fait que la vitesse de phase réelle $C_p(\omega)$ d'une composante harmonique de pulsation ω dépend de la fréquence et du rapport rayon/longueur d'onde : c'est le phénomène de **dispersion géométrique** (à ne pas confondre avec la dispersion matérielle des barres viscoélastiques, traitée ailleurs).

Conséquence pratique pour les barres de Hopkinson : un pulse de déformation mesuré par une jauge collée à une distance z de l'interface barre/éprouvette n'a **pas la même forme temporelle** que le pulse qui atteint réellement l'interface (ou qui atteindrait un autre point de la barre), car chaque composante fréquentielle du pulse s'est propagée à une vitesse de phase légèrement différente et a subi un déphasage cumulatif différent sur la distance z. Plus la distance jauge-interface est grande et/ou plus le pulse contient de hautes fréquences (front raide, choc), plus cette distorsion parasite (oscillations, étalement) est importante. La correction de dispersion consiste donc à recalculer, pour chaque composante fréquentielle du signal mesuré, le déphasage (et l'atténuation/amplification d'amplitude) qu'elle aurait subi si on l'avait « rembobinée » ou « avancée » exactement de la distance z le long de la barre, en utilisant la vraie relation de dispersion $C_p(\omega)$ de la barre — et non la vitesse constante c_0 de la théorie 1D — puis à recomposer un signal corrigé représentatif du point voulu (typiquement l'interface avec l'éprouvette).

---

2. Théorie de dispersion implémentée : équation de Pochhammer-Chree

Le nom utilisé explicitement dans le code (docstring du module, ligne 1, et docstring de `_solve_pochhammer_branch`, `elastic_correction`) est **« Pochhammer-Chree »**, la théorie exacte de propagation des ondes axisymétriques longitudinales dans un cylindre élastique isotrope infini (solution exacte de l'élasto-dynamique 3D en coordonnées cylindriques, mode fondamental longitudinal L(0,1)). Aucune référence à Bancroft ou Love n'apparaît dans le code lui-même ; c'est bien l'équation caractéristique de Pochhammer-Chree, sous sa forme classique faisant intervenir les fonctions de Bessel J_0 et J_1, qui est résolue.

2.1 Variables et notations exactes du code (`core/dispersion.py::_solve_pochhammer_branch`, lignes 162–197)

Pour chaque branche (incidente, réfléchie ou transmise), avec :

- a = `radius` : rayon de la barre correspondante ($D_{pa}/2$, où D_{pa} est le diamètre moyen de la barre),
- c_0 = `c0` : célérité élastique 1D de la barre, $c_0 = \sqrt{E/\rho}$ (issue de `_bar_elastic_wave_speed`, voir §2.3),
- ν = `nu` : coefficient de Poisson de la barre,
- f = `freq` : vecteur des fréquences (Hz) issu de la FFT (voir §4),
- C_p = `Cp` : vitesse de phase inconnue à résoudre, pour la fréquence courante.

le code pose :

$$\beta = \frac{1-2\nu}{1-\nu}$$

$$x = \left(\frac{C_p}{c_0}\right)^2 (1+\nu)$$

$$\eta = \frac{2\pi f}{C_p} \qquad \text{(c'est le nombre d'onde } k = \omega/C_p\text{, nommé } \texttt{nl} \text{ dans le code)}$$

$$h = \eta\sqrt{\beta x - 1} \qquad\qquad k = \eta\sqrt{2x-1}$$

(attention : dans le code, la variable `k_expr/k` ci-dessus n'est **pas** le nombre d'onde axial — c'est une des deux quantités radiales caractéristiques de l'équation de Pochhammer-Chree, analogue au q classique de la littérature ; h correspondant au p classique).

2.2 Équation caractéristique résolue

L'équation de dispersion résolue numériquement, fréquence par fréquence, est exactement (extraite littéralement de `_solve_pochhammer_branch`, lignes 179–182) :

$$(x-1)^2\, a\, h\, \frac{J_0(ah)}{J_1(ah)} \;-\; (\beta x - 1)\left[x - a\, k\, \frac{J_0(ak)}{J_1(ak)}\right] \;=\; 0$$

avec J_0, J_1 les fonctions de Bessel de première espèce d'ordre 0 et 1 (`sympy.besselj`). C'est la forme standard de l'équation caractéristique de Pochhammer-Chree pour le mode longitudinal axisymétrique fondamental d'un cylindre élastique isotrope infini, écrite en variables réduites x (proportionnelle au carré du rapport de vitesses) et h, k (grandeurs radiales dépendant de $\eta = \omega/C_p$, de a et de ν via β).

L'inconnue résolue à chaque fréquence est C_p (via x, h, k, qui en dépendent tous) : c'est donc bien la **relation de dispersion** $C_p(\omega)$, ou de façon équivalente $\omega(k)$, de la barre réelle (rayon fini), par opposition à la relation non-dispersive $C_p = c_0 =$ cste de la théorie 1D.

2.3 Vitesse de barre c0 et coefficient de Poisson : `_bar_elastic_wave_speed()`

`_bar_elastic_wave_speed()` (lignes 113–159) fournit le couple (c_0, ν) utilisé comme point de départ de la résolution :

- **Cas standard** (`heat_check=False`) : $c_0 = \overline{C0_{pa}}$ et $\nu = \overline{\nu_{pa}}$ sont simplement les moyennes (sur le tirage Monte-Carlo) des propriétés de barre saisies par l'utilisateur — pas de calcul supplémentaire.
- **Cas avec correction thermique** (`heat_check=True`) : si un gradient de température le long de la barre est actif (essais à température non ambiante), le module recalcule un module d'Young effectif E_{mat} (et donc $c_0 = \sqrt{E_{mat}/\rho}$) à partir d'un profil de température exponentiel le long de la barre,

$$T(z) = T_{amb} + (T_0 - T_{amb})\,\exp\left(-2\sqrt{\frac{h}{k_{therm}\,\overline{D}}}\,z\right), \qquad z \in [0, \overline{PG}],\ h = 20$$

(`h` ici est un coefficient d'échange convectif constant codé en dur, à ne pas confondre avec le `h` de Pochhammer-Chree du §2.1), puis soit une loi empirique de type Ramberg-Osgood-like « `rte` » (types de barre 1/2, avec conductivité $k_{therm} = 40$ ou 220 W/m/K selon le matériau), soit une table de propriétés importée dépendant de la température (type de barre 3, via `table.value("e"/"rho"/"nu", ...)`). Ce mécanisme, bien que présent dans `core/dispersion.py`, relève de la thermique de barre plutôt que de la dispersion élastique proprement dite ; il n'affecte le calcul de dispersion que par les valeurs (c_0, ν) qu'il fournit en entrée de `_solve_pochhammer_branch`.

---

3. Méthode numérique de résolution : root-search fréquence par fréquence

3.1 Algorithme (`_solve_pochhammer_branch`, lignes 162–197)

Pour chaque « branche » (onde incidente, réfléchie ou transmise), la résolution se fait par **méthode de continuation** :

1. **Initialisation à $f = 0$ (DC)** : `cp_sol = [c0]`, `m1 = [1.0]`, `m2 = [1.0]` — à fréquence nulle, la vitesse de phase de Pochhammer-Chree tend exactement vers c_0 (limite basse fréquence de la théorie 1D), et les facteurs correctifs sont pris égaux à 1 (pas de correction).
2. **Boucle sur les bins de fréquence positifs** `i = 1 … len(freq)-1` (le bin `freq[0]=0` est sauté puisqu'il est déjà couvert par l'initialisation) :
   - L'équation caractéristique du §2.2, à la fréquence `freq[i]`, est résolue pour l'inconnue C_p par **`sympy.nsolve`** — un solveur de racine **numérique** (Newton-Raphson multidimensionnel/scalaire de `sympy`, avec arithmétique éventuellement en précision arbitraire), initialisé (`x0`) avec **la solution trouvée à la fréquence précédente**, `cp_sol[-1]`.
   - C'est donc une **méthode de continuation (« marching »)** : chaque solution sert de germe (« seed ») à la suivante, en supposant que $C_p(\omega)$ varie continûment (pas de saut brutal entre bins de fréquence adjacents) le long de la branche de dispersion suivie. Aucune borne explicite (intervalle de recherche) n'est fournie à `nsolve` — seul un point de départ ; c'est la continuité physique de la courbe de dispersion, combinée au bon comportement de convergence local de

Newton, qui garantit (en pratique) que la racine trouvée reste sur la même branche (le mode fondamental) d'un bin à l'autre.

- La solution retenue est prise en valeur absolue : `cp_sol.append(float(Abs(dsol)))` — le résultat de `nsolve` peut être légèrement complexe (résidu numérique), et seul le module est conservé comme vitesse de phase physique.

3. **Facteurs correctifs `m1`, `m2`** : une fois C_p trouvée pour la fréquence courante, le code recalcule x_a, η_a, h_a, k_a à partir de cette solution puis évalue :

$$M_1 = \frac{2\left(1 + \dfrac{1-\beta x}{x-1}\right)}{a\,h\,\dfrac{J_0(ah)}{J_1(ah)} + a\,k\,\dfrac{J_0(ak)}{J_1(ak)} \cdot \dfrac{1-\beta x}{x-1}}$$

(partie réelle conservée : `float(complex(dm1).real)`), et

$$M_2 = \left(\frac{C_p}{c_0}\right)^2$$

Le rapport M_2/M_1 sert ensuite à corriger l'**amplitude** de chaque composante fréquentielle (§4). Le code ne documente pas explicitement l'origine physique de M_1 : structurellement, cette forme correspond à un facteur de correction reliant la déformation axiale mesurée en surface de barre (ce que lit réellement une jauge collée à l'extérieur de la barre) à une grandeur moyenne sur la section — un type de correction classiquement associé à Bancroft/Davies dans la littérature sur la dispersion des barres de Hopkinson — **mais cette interprétation n'est pas confirmée par un commentaire du code** et doit être considérée comme une lecture prudente, pas une certitude. 4. Un callback `progress_cb(i, total)` est appelé après chaque fréquence résolue (utilisé par l'UI pour la barre de progression, cf. `page_corrections.py`, message « Running dispersion correction… (this involves a root-search per frequency and can take a while) »).

3.2 Parallélisation entre branches (`_solve_pochhammer_branches`, lignes 209–239)

Les branches incidente/réfléchie/transmise sont **indépendantes entre elles** (chacune a son propre rayon a, éventuellement son propre c_0, ν pour la barre d'entrée vs. la barre de sortie), mais **chaque branche est intrinsèquement séquentielle en interne** (continuation d'une fréquence à l'autre). `_solve_pochhammer_branches` permet, si `n_workers > 1`, d'exécuter les (jusqu'à 3) branches en parallèle dans des processus séparés (via `core/parallel.run_map`) — un gain borné à ~3× au mieux, jamais un parallélisme fréquence-par-fréquence. Avec `n_workers <= 1` (par défaut), c'est un simple appel séquentiel équivalent au code d'origine, avec suivi de progression fréquence par fréquence.

---

4. Algorithme complet de correction du signal (FFT → correction par bin → IFFT)

Implémenté dans `elastic_correction()` (montage 2 barres, lignes 309–404, fonction interne `_corrected` lignes 373–385) et de façon identique (une seule branche) dans `elastic_correction_direct_impact()` (lignes 407–449).

4.1 Découpage temporel des fenêtres (avant FFT)

Le signal mesuré est d'abord découpé en fenêtres temporelles correspondant respectivement à l'onde incidente, réfléchie (barre d'entrée IB) et transmise (barre de sortie OB), à partir des bornes `xd1,xd2` que l'utilisateur positionne sur le graphe « Pulse window(s) » (`ui/pages/page_corrections.py`). Les fenêtres réfléchie et transmise sont positionnées automatiquement à partir de la fenêtre incidente en ajoutant les temps de vol correspondants (même durée `ti_pulse = idx2-idx1` que la fenêtre incidente pour les trois) :

$$t_{\text{réfléchi,1}} = t_{d1} + \frac{2\,\overline{PG}_{IB}}{\overline{C0}_{IB}} \left[+ \frac{2\,L_{plateau}}{\overline{C0}_{plateau}} \text{ si plateau/pulse-shaper actif} \right]$$

$$t_{transmis,1} = t_{d1} + \frac{\overline{PG}_{OB}}{\overline{C0}_{OB}} + \frac{\overline{L}_{\text{éprouvette}}}{c_{0,\text{éprouvette}}} + \frac{\overline{PG}_{IB}}{\overline{C0}_{IB}} \left[+ \frac{2\,L_{plateau}}{\overline{C0}_{plateau}} \right]$$

avec $c_{0,\text{éprouvette}} = \sqrt{\overline{E}_{\text{éprouvette}}/\overline{\rho}_{\text{éprouvette}}}$ (célérité élastique de l'éprouvette, utilisée uniquement pour l'estimation de temps de vol, pas pour l'analyse contrainte-déformation). $\overline{PG}$ désigne la position moyenne de jauge (distance jauge–interface) de la barre concernée. C'est un temps de vol **non dispersif** (basé sur $C0$ constant), ce qui est cohérent : il ne sert qu'à positionner grossièrement les fenêtres, la correction de dispersion proprement dite étant appliquée ensuite dans le domaine fréquentiel. Un garde-fou `_require_window()` (lignes 44–74) lève une erreur explicite si une fenêtre résolue est vide ou trop étroite (moins de 2 échantillons d'écart), plutôt que de laisser une `IndexError` cryptique remonter plus loin dans le calcul FFT/root-search.

4.2 Passage au domaine fréquentiel (FFT)

Pour la fenêtre de longueur n et de pas d'échantillonnage dt :

```
yi = np.fft.rfft(incident)          # idem pour reflected -> yr, transmitted -> yt
freq = np.fft.rfftfreq(n, d=dt)
ai, pi = np.abs(yi), np.angle(yi)   # amplitude et phase par bin de fréquence
```

C'est une **FFT réelle** (`rfft`), donc seules les fréquences positives (DC à Nyquist) sont représentées, chaque bin étant caractérisé par une amplitude `ai[i]` et une phase `pi[i]`.

4.3 Correction par composante fréquentielle : `_corrected()` (lignes 373–385)

Pour chaque bin de fréquence f_i, avec z la distance de propagation signée à corriger (voir §4.4), c_0 la célérité 1D de la barre concernée, $C_p(f_i)$ la vitesse de phase de Pochhammer-Chree résolue au §3, et

$M_1(f_i), M_2(f_i)$ les facteurs correctifs associés :

Correction de phase :

$$\phi'(f_i) \;=\; \phi(f_i) \;-\; \left(2\pi f_i \,\frac{z}{c_0}\right)\left(\frac{c_0}{C_p(f_i)} - 1\right)$$

(code : `phase[i] - (2*cmath.pi*freq[i]*z/c0) * (c0/cp_sol[i] - 1))`

Interprétation : $\dfrac{\omega z}{c_0}$ est le déphasage qu'accumulerait une onde **non dispersive** (théorie 1D, vitesse constante c_0) sur la distance z ; en le multipliant par $\left(\frac{c_0}{C_p} - 1\right)$, le terme soustrait est exactement la **différence** entre le déphasage réel dispersif ($\omega z / C_p(\omega)$) et le déphasage non-dispersif ($\omega z / c_0$) sur cette même distance — autrement dit $\phi' = \phi - \omega z\left(\frac{1}{C_p(\omega)} - \frac{1}{c_0}\right)$. La phase mesurée est donc « re-projetée » à la position décalée de z le long de la barre en tenant compte de la vraie physique dispersive.

Correction d'amplitude et reconstruction spectrale :

$$\hat{X}'(f_i) \;=\; \frac{M_2(f_i)}{M_1(f_i)}\,|X(f_i)|\;e^{\,i\,\phi'(f_i)}$$

(code : `(m2[i]/m1[i]) * amp[i] * np.exp(1.0j * n_phase[i]))`

Retour au domaine temporel (IFFT) :

```
sig = np.fft.irfft(fft_new, n=n)
```

Le paramètre explicite `n=n` (longueur de la fenêtre d'origine, potentiellement impaire puisque les fenêtres sont positionnées librement par l'utilisateur et non forcées à une taille fixe) est **indispensable** : `np.fft.irfft` ne peut pas déduire seul une longueur impaire à partir du spectre réel seul, et suppose par défaut une longueur paire — ce qui, pour une fenêtre de longueur impaire, produirait un signal reconstruit numériquement faux sur toute sa longueur (pas seulement décalé d'un échantillon). Le code le documente explicitement en commentaire (lignes 376–383) et renvoie vers le mécanisme identique dans `core/visco_calibration.py::_predict_pulse` pour un exemple numérique détaillé.

4.4 Distance signée z et convention de signe des trois ondes

Les trois appels effectifs dans `elastic_correction` (lignes 387–389) sont :

```
new_incident    = -np.absolute(_corrected(ai, pi, freq,  z1, c01, cp_i, m1i, m2i))
new_reflected   =  np.absolute(_corrected(ar, pr, freq, -z1, c01, cp_r, m1r, m2r))
new_transmitted = -np.absolute(_corrected(at, pt, freq, -z2, c02, cp_t, m1t, m2t))
```

où $z_1 = \overline{PG}_{IB}$, $z_2 = \overline{PG}_{OB}$ (distances jauge–interface moyennes de la barre d'entrée et de la barre de sortie). Deux mécanismes distincts sont à noter :

1. **Le signe de z dans la correction de phase distingue le sens de propagation à corriger** : $+z_1$ pour l'onde incidente (mesurée avant qu'elle n'atteigne l'interface, donc « avancée » vers l'interface), $-z_1$ pour l'onde réfléchie (mesurée après un aller-retour, la correction se fait donc dans le sens opposé),

$-z_2$ pour l'onde transmise (mesurée après avoir déjà traversé de l'interface vers la jauge de sortie). Ce choix de signe est cohérent avec la construction des fenêtres temporelles du §4.1 (mêmes distances, mêmes signes de propagation implicites).

2. **Le signe final imposé après `np.absolute(...)`** (`-`, `+`, `-` respectivement pour incident/réfléchi/transmis) **force la convention de signe physique attendue** de chaque pulse (typiquement : incident et transmis en compression, réfléchi en traction — ou l'inverse selon le câblage des jauges du projet, mais dans tous les cas une convention fixe et cohérente) — indépendamment de toute ambiguïté de signe global que la reconstruction FFT/amplitude-phase pourrait introduire (une amplitude `|X|` combinée à une phase reconstruite ne préserve pas nécessairement le signe d'origine du signal temporel). C'est un mécanisme purement algorithmique de correction de sens, pas un phénomène physique de dispersion en soi.

Après l'IFFT, si le signal corrigé ressort avec un échantillon de moins que la fenêtre d'origine (résidu de troncature `rfft`/`irfft` sur une taille impaire), le dernier échantillon est dupliqué pour compenser (lignes 391–396) : `new_incident = np.concatenate((new_incident, [new_incident[-1]]))`, etc.

4.5 Cas « barre de sortie seule » (Impact Direct) : `elastic_correction_direct_impact()`

Structure rigoureusement identique, mais une seule branche (`transmitted`), une seule fenêtre temporelle (positionnée directement par l'utilisateur, pas dérivée d'une fenêtre incidente), et une seule correction appliquée avec $z = -z_2$ (lignes 407–449) :

```
n_phase = [pt[i] - (2*cmath.pi*freq[i]*(-z2)/c02) * (c02/cp_t[i] - 1) for i in range(len(pt))]
fft_new = [(m2t[i]/m1t[i]) * at[i] * np.exp(1.0j*n_phase[i]) for i in range(len(n_phase))]
new_transmitted = -np.absolute(np.fft.irfft(fft_new, n=n))
```

Ici la sélection de fenêtre se fait par recherche du sample le plus proche dans un vecteur `time_pulse` explicite (`_index_of_nearest`, lignes 77–96), et non par `np.searchsorted` sur `timet` comme dans le montage à deux barres — différence purement liée au format des données d'entrée du montage Impact Direct, sans impact sur la physique de correction elle-même.

---

5. Fenêtrage fréquentiel et bornage de la bande passante de la correction

5.1 Absence de fenêtrage temporel (apodisation) avant FFT — toujours vrai

Une lecture attentive du fichier ne montre **aucun fenêtrage temporel de type Hann/Hamming/Tukey avant la FFT**, ni de détection/garde-fou explicite d'une divergence de la recherche de racine à haute fréquence. Ces deux points restent vrais indépendamment du mécanisme du §5.2 ci-dessous (qui agit en aval, sur le spectre déjà corrigé, pas sur le signal brut en amont de la FFT) :

- Le seul mécanisme qui pourrait limiter l'impact du bruit haute fréquence côté recherche de racine est **indirect** : `nsolve` étant initialisé par continuation depuis la solution précédente, une divergence ou un mauvais comportement de la recherche de racine à haute fréquence (zone où la courbe de dispersion de Pochhammer-Chree devient très non linéaire, proche de la limite de Rayleigh) pourrait faire échouer ou dévier la continuation — mais **aucune détection ni garde-fou de ce cas n'est codé** dans

`_solve_pochhammer_branch` (contrairement à `_solve_visco_branch` qui, lui, a un `try/except ValueError` de repli — voir chapitre viscoélastique).

- Il n'y a pas non plus de fenêtrage temporel (apodisation) appliqué à `incident/reflected/transmitted` avant `rfft` : le signal brut découpé (rectangulaire, sans transition douce aux bords) est directement transformé. Ceci peut introduire une fuite spectrale (spectral leakage) si les bords de la fenêtre ne coïncident pas avec des zéros du signal — le code ne mentionne ni ne corrige ce point.

5.2 Limitation de la bande passante propre de la correction — `_bandwidth_rolloff_weight()` (lignes 92-153)

Motivation, telle que documentée dans le code (`core/dispersion.py`, docstring de `_bandwidth_rolloff_weight`) : le gain propre de la correction (le facteur $M_2(f)/M_1(f)$ du §4.3) n'est **pas borné par 1** — à haute fréquence il dépasse couramment 1, puisqu'il existe pour **annuler** la dispersion géométrique de la barre, pas pour supprimer quoi que ce soit. Un signal de jauge réel porte toujours un bruit électronique/de quantification large-bande, négligeable aux fréquences réellement utiles du pulse (basses) mais pas rigoureusement nul aux fréquences hautes — un gain de correction supérieur à 1 dans cette zone amplifie donc ce bruit plutôt qu'un quelconque signal transmis par la barre, mécanisme documenté comme la cause des bursts d'oscillations haute fréquence qu'une correction de Pochhammer-Chree peut produire sur des données réelles. Le code précise explicitement que la courbe C_p/C_0 (§7, diagnostic « Result ») reste lisse sur la même plage : ce n'est donc **pas** la recherche de racine elle-même qui diverge ou se comporte mal, l'amplification du bruit est **inhérente** à ce type de correction, quelle que soit son implémentation — pas un bug de la résolution.

Mécanisme : un poids lisse (cosinus surélevé), entre 0 et 1, multiplié dans le spectre **déjà corrigé** (pas le signal brut) juste avant l'IFFT — donc en aval du §4.3, pas un filtrage préalable comme évoqué (et toujours absent) au §5.1 :

```
def _bandwidth_rolloff_weight(freq, radius, c0_ref, max_normalized_freq, rolloff_fraction=0.25):
    if max_normalized_freq is None or max_normalized_freq <= 0 or radius <= 0 or c0_ref <= 0:
        return np.ones_like(freq)          # désactivé -- comportement d'origine, gain non borné
    f_cut = max_normalized_freq * c0_ref / radius
    f_start = max(0.0, 1.0 - rolloff_fraction) * f_cut
    w = np.ones_like(freq)                  # 1.0 en dessous de f_start
    in_taper = (freq > f_start) & (freq < f_cut)
    w[in_taper] = 0.5 * (1.0 + np.cos(np.pi * (freq[in_taper] - f_start) / (f_cut - f_start)))
    w[freq >= f_cut] = 0.0                  # 0.0 au-delà de f_cut
    return w
```

$$w(f) = \begin{cases} 1 & f \le f_{start} \\ \dfrac{1 + \cos\left(\pi \dfrac{f - f_{start}}{f_{cut} - f_{start}}\right)}{2} & f_{start} < f < f_{cut} \\ 0 & f \ge f_{cut} \end{cases}, \qquad f_{cut} = f_{max} . \frac{c_0}{a}, \quad f_{start} = (1 - 0{,}25)\, f_{cut}$$

avec f_{max} = « Max normalized frequency (f·radius/C0) » (`max_normalized_freq`, page « Dispersion Corrections », carte « Correction bandwidth limit »), a le rayon de la barre concernée, c_0 sa célérité — **la même grandeur normalisée $f \cdot a/c_0$ déjà tracée en abscisse de la courbe C_p/C_0** (§7), donc directement lisible sur le même graphique de diagnostic que celui déjà utilisé pour la correction. La fraction de transition (`rolloff_fraction`, 0,25 par défaut, non exposée dans l'UI) donne une coupure **progressive** (cosinus surélevé) plutôt qu'un seuil dur, pour ne pas réintroduire une nouvelle discontinuité spectrale (source classique de ringing de type Gibbs) en remplacement de l'ancienne absence de bornage.

Branchement dans `elastic_correction()`/`elastic_correction_direct_impact()` (lignes 366-473/475-517, nouveau paramètre `max_normalized_freq: float = 0.6` des deux fonctions) : le poids est calculé **séparément pour chaque branche** (incidente, réfléchie, transmise — ou transmise seule en Impact Direct), avec le rayon/la célérité propres à CETTE branche (barre d'entrée pour incidente/réfléchie, barre de sortie pour transmise), puis multiplié dans le spectre reconstruit juste avant l'IFFT, à l'intérieur même de `_corrected()` (§4.3) :

```
fft_new = np.array([(m2[i] / m1[i]) * amp[i] * np.exp(1.0j * n_phase[i]) for i in range(len(n_phase)
fft_new = fft_new * _bandwidth_rolloff_weight(np.asarray(freq), radius, c0, max_normalized_freq)
sig = np.fft.irfft(fft_new, n=n)
```

Réglage UI (`ui/pages/page_corrections.py`, carte « Correction bandwidth limit », lignes 161-201) : case à cocher « Limit correction bandwidth » (`self.bandwidth_limit_check`, **cochée par défaut** — `project.dispersion_bandwidth_limit_enabled: bool = True`, `core/project.py` ligne 353) et champ numérique « Max normalized frequency (f·radius/C0) » (`self.bandwidth_limit_edit`, valeur par défaut `project.dispersion_bandwidth_limit = 0.6`, ligne 358). Décocher la case revient à passer `max_normalized_freq=0.0` à toutes les fonctions de correction — `_bandwidth_rolloff_weight` retourne alors un tableau de 1 partout, c'est-à-dire exactement le comportement d'origine (gain non borné, §5.1 ci-dessus inchangé). `_max_normalized_freq_for_run()` (lignes 487-494) encapsule cette bascule et alimente les quatre appels de correction (élastique/viscoélastique × 2 barres/Impact Direct, lignes 870/883/908/920) — `no_correction`/`no_correction_direct_impact` n'y sont, logiquement, pas concernés (aucune physique de dispersion à borner en l'absence de correction). Persisté dans une session (`core/session.py`, lignes 128-129/191-193) et dans un profil de setup (`core/setup_profile.py`, lignes 77-78/144-147), au même titre que `dispersion_correction_mode`/`dispersion_setup_mode`.

Le défaut de 0,6 est un choix pragmatique, pas une borne dérivée de premiers principes — le docstring de `_bandwidth_rolloff_weight` le dit explicitement : une valeur ronde, choisie pour se situer confortablement au-delà de la zone où l'énergie réelle d'un pulse SHPB typique s'est déjà éteinte, tout en restant dans la plage habituellement inspectée sur la courbe C_p/C_0 elle-même — un point de départ, pas un optimum prouvé pour toute combinaison barre/pulse ; à ajuster (champ éditable) selon le cas réel, à la baisse si du ringing persiste, à la hausse seulement avec une raison spécifique de faire confiance à la correction plus loin dans le spectre.

Complémentaire, pas redondant, avec le filtre passe-bas du signal brut (`Bar.lowpass_enabled`/`lowpass_cutoff_hz`, cases « Input/Output Bar low-pass filter » de la même page, désactivé par défaut, 300 kHz par défaut — `core/lowpass.py`) : ce filtre-là agit **avant** la correction, sur le signal brut de jauge, à une fréquence de coupure fixe en Hz choisie par l'utilisateur ; le bornage de bande passante ci-dessus agit **après**, sur le gain de la correction elle-même, à une fréquence de coupure normalisée $f \cdot a/c_0$ automatiquement mise à

l'échelle de la géométrie/matériau de chaque barre. Les deux peuvent être combinés — l'un ne rend pas l'autre inutile, puisqu'ils bornent des choses différentes (le contenu du signal mesuré, contre le gain que la correction elle-même peut appliquer).

---

6. Bascule élastique / viscoélastique dans le module

La sélection entre les quatre combinaisons du tableau en tête de chapitre se fait **en dehors** de `core/dispersion.py`, dans `ui/pages/page_corrections.py` (aide contextuelle citée : « Bar material model » : *No dispersion correction*, *Elastic bars (standard Pochhammer-Chree)*, *Viscoelastic bars* ; et « Setup » : montage 2 barres vs. Impact Direct). Selon les choix `project.dispersion_correction_mode` / `project.dispersion_setup_mode`, la page appelle directement l'une des fonctions suivantes de `core/dispersion.py` (voir `page_corrections.py`, méthode `_on_run`, lignes ~660–740) :

- `no_correction` / `no_correction_direct_impact` (aucune physique de dispersion, simple découpe de fenêtres),
- `elastic_correction` / `elastic_correction_direct_impact` (ce chapitre),
- `viscoelastic_correction` / `viscoelastic_correction_direct_impact` (chapitre séparé — modèle rhéologique du type Solide Linéaire Standard ou autre `RheologyModel`, module complexe $E^*(\omega)$, atténuation, etc. → **voir chapitre dispersion viscoélastique**).

Le module ne « bascule » donc pas en interne entre les deux physiques par un paramètre ou un `if` central : ce sont des fonctions distinctes, choisies en amont par l'UI, qui partagent uniquement des utilitaires communs (découpage de fenêtres, `_require_window`, `_index_of_nearest`, structure `DispersionResult`, et le principe général FFT → correction par bin → IFFT). L'équation caractéristique elle-même diffère fondamentalement entre les deux (Pochhammer-Chree « réel », ν et c_0 constants, pour l'élastique ; Pochhammer-Chree « complexe » avec module $E^*(\omega)$ dépendant du modèle rhéologique, pour le viscoélastique — voir `_solve_visco_branch`, hors périmètre de ce chapitre).

---

7. Résultats exposés (`DispersionResult`, lignes 99–110)

Pour diagnostic/tracé (page « Dispersion Corrections »), en plus des signaux bruts et corrigés, le module expose :

- `freq_over_c` : le paramètre de fréquence réduite $f \cdot a/c_0$ pour chaque branche (grandeur adimensionnée classique des courbes de dispersion de Pochhammer-Chree, souvent notée Ω ou fa/c_0 dans la littérature) — ligne 401 : `[f * a1 / c01 for f in freq]`.
- `phase_velocity_ratio` : le rapport $C_p(f)/c_0$ pour chaque branche — ligne 402 : `[c / c01 for c in cp_i]`.

Ces deux grandeurs forment ensemble la courbe de dispersion normalisée C_p/c_0 en fonction de fa/c_0, présentation standard pour visualiser à quel point la correction s'écarte de l'hypothèse 1D ($C_p/c_0 = 1$) sur la bande de fréquence du pulse mesuré.

- `temperature_profile` : profil $(z, T(z))$ le long de chaque barre, uniquement si `heat_check` est actif (voir §2.3) — sans lien direct avec la dispersion géométrique elle-même, mais influence c_0, ν en entrée.

---

8. Synthèse des correspondances code

Élément théorique	Fichier / fonction
Vitesse barre 1D $c_0 = \sqrt{E/\rho}$, ν, éventuelle correction thermique	`core/dispersion.py::_bar_elastic_wave_speed()`
Équation caractéristique de Pochhammer-Chree (Bessel J_0/J_1)	`core/dispersion.py::_solve_pochhammer_branch()`, lignes 179–182
Root-search par continuation (`sympy.nsolve`, seed = solution précédente)	`core/dispersion.py::_solve_pochhammer_branch()`, lignes 178–196
Facteurs correctifs d'amplitude M_1, M_2	`core/dispersion.py::_solve_pochhammer_branch()`, lignes 188–194
Parallélisation des branches incident/réfléchi/transmis	`core/dispersion.py::_solve_pochhammer_branches()`
Découpe temporelle des fenêtres, temps de vol non dispersifs	`core/dispersion.py::elastic_correction()` (lignes 329–352) / `_cut_two_bar_windows()`
FFT du pulse fenêtré	`core/dispersion.py::elastic_correction()`, lignes 354–359
Correction phase + amplitude par bin	`core/dispersion.py::elastic_correction()::_corrected()`, lignes 373–385
IFFT (reconstruction, `n=n` critique)	`core/dispersion.py::_corrected()`, ligne 384
Convention de signe forcée par onde	`core/dispersion.py::elastic_correction()`, lignes 387–389
Cas Impact Direct (barre de sortie seule)	`core/dispersion.py::elastic_correction_direct_impact()`, lignes 407–449
Bascule élastique/viscoélastique/aucune (UI)	`ui/pages/page_corrections.py`, combo « Bar material model », méthode `_on_run`

---

9. Limitations et avertissements identifiés dans le code

- **Non re-vérifié numériquement** : avertissement explicite en tête de module (§ ci-dessus), à rappeler dans la documentation utilisateur — le portage est jugé fidèle à l'algorithme d'origine mais n'a pas pu être re-testé contre un cas de référence connu.
- **Pas de garde-fou sur la convergence de `nsolve`** en régime élastique haute fréquence (contrairement au chemin viscoélastique qui, lui, a un repli `try/except`) : une racine mal trouvée ou une divergence de la continuation à haute fréquence ne serait pas détectée automatiquement.

- **Aucun filtrage/fenêtrage spectral** : toutes les fréquences jusqu'à Nyquist sont corrigées et recomposées ; aucune atténuation des hautes fréquences bruitées.
- **Interprétation physique de M_1 non documentée dans le code** : sa forme est cohérente avec un facteur de correction d'amplitude du type Bancroft/Davies (surface vs. moyenne de section), mais ce n'est pas confirmé par un commentaire du code — à traiter avec prudence dans la documentation finale (formulation proposée : « facteur correctif empirique de la théorie de Pochhammer-Chree, dont le rôle est de corriger l'amplitude de chaque composante fréquentielle en plus de sa phase »).
- **Temps de vol de découpe des fenêtres non dispersifs** : le positionnement des fenêtres réfléchie/transmise (§4.1) utilise $C0$ constant même dans le cas élastique corrigé — seule la correction spectrale elle-même est dispersive, pas le placement des fenêtres temporelles.
- Le paramètre `n=n` de `np.fft.irfft` est signalé dans le code lui-même (lignes 376–383) comme un point de correction **critique et non optionnel** pour des fenêtres de longueur impaire (cas courant ici, fenêtres positionnées librement par l'utilisateur) — un oubli produirait un signal reconstruit numériquement faux sur toute sa longueur, pas seulement décalé.

8. Correction de dispersion viscoélastique, calibration bayésienne et amortissement

Ce chapitre couvre trois blocs du code étroitement liés :

1. les **modèles rhéologiques** (`core/rheology_models.py`) qui fournissent le module complexe $E^*(f)$ d'une barre en matériau viscoélastique (polymère) ;
2. la **calibration** de ces modèles à partir de deux pulses de décroissance libre mesurés en deux points d'une même barre, en mode recherche bayésienne (`core/bayes_visco_calibration.py`, avec son surrogate de Processus Gaussien `core/gaussian_process.py`) ou en mode « Test parameters » à paramètres fixés (`core/visco_calibration.py`), ainsi que l'**auto-alignement** des deux pulses (`core/pulse_alignment.py`) ;
3. l'outil indépendant **« Free-Decay Damping »** (`core/decay.py`) qui estime un coefficient d'amortissement par la méthode de largeur de bande à mi-puissance.

Le mécanisme général de correction de dispersion par FFT (décomposition en fréquences, correction de phase/amplitude par bin, IFFT) est traité dans le chapitre consacré à la dispersion élastique ; on ne le redétaille pas ici. Ce chapitre se concentre sur **comment $E^*(f)$ est calculé selon le modèle rhéologique choisi**, et sur les algorithmes de calibration/analyse qui en dépendent.

8.1 Pourquoi un modèle viscoélastique ?

Dans la correction de dispersion « élastique » standard (barres métalliques), le module d'Young E de la barre est une constante réelle, indépendante de la fréquence : seule la géométrie (rayon, coefficient de Poisson) introduit une dispersion, via l'équation de Pochhammer-Chree.

Pour des barres en matériau **viscoélastique** (polymères type PMMA, polycarbonate, nylon — utilisées notamment pour tester des matériaux mous à basse impédance), le module d'élasticité n'est plus une constante mais un **module complexe dépendant de la fréquence** :

$$E^*(\omega) = E'(\omega) + i\,E''(\omega), \qquad \omega = 2\pi f$$

où $E'(\omega)$ (partie réelle, « module de stockage ») représente la raideur élastique instantanée et $E''(\omega)$ (partie

imaginaire, « module de perte ») représente la dissipation visqueuse. Cette dépendance en fréquence se répercute directement sur la vitesse de phase complexe utilisée par le solveur de dispersion,

$$c(\omega) = \sqrt{E^*(\omega)/\rho}$$

(voir `core/visco_calibration.py::_complex_modulus_and_speed`, ligne 78-87), qui alimente ensuite exactement la même recherche de racine de l'équation de Pochhammer-Chree (fonctions de Bessel) que la correction élastique, mais avec des modules de cisaillement/Lamé complexes $\mu(\omega)$, $\lambda(\omega)$ au lieu de réels.

Le docstring de `core/rheology_models.py` précise que **seul le fait de fournir $E^*(f)$ change d'un modèle à l'autre** : la recherche de racine de Pochhammer-Chree par bin de fréquence, la propagation FFT/IFFT du pulse et les modules de Lamé qui en dérivent sont génériques et ne dépendent pas du modèle rhéologique choisi (« that seam is what makes it possible to offer several models here without duplicating the (expensive, delicate) dispersion physics per model »).

Le coefficient de Poisson `nu` de la barre est explicitement **séparé** des paramètres de chaque modèle rhéologique : c'est une propriété toujours présente de la barre, indépendante du modèle de relaxation utilisé pour son module (`core/rheology_models.py`, lignes 13-15).

Limitation documentée explicitement : à ce jour, seul le modèle *Standard Linear Solid* (SLS) est implémenté dans `core/dispersion.py::viscoelastic_correction` / `viscoelastic_correction_direct_impact` (l'option « Viscoelastic bars » de la page Dispersion Corrections). Les trois autres modèles (Kelvin-Voigt, Generalized Maxwell 2 branches, Burgers) ne sont disponibles **que** pour la page de calibration (`core/visco_calibration.py`, `core/bayes_visco_calibration.py`) ; le bouton « Apply to Dispersion Corrections » de la page de calibration (`ui/pages/page_visco_calib.py`) est désactivé pour ces trois modèles. Par ailleurs, l'expansion Monte-Carlo de barre (`core/models.py::Bar.expand_viscoelastic`) est un chemin de code **séparé et non lié** à ces quatre modèles : il reste câblé en dur sur le module SLS et n'est pas concerné par ce chapitre (note du docstring, lignes 56-61).

Bornage de la bande passante de la correction (`_bandwidth_rolloff_weight`) : `viscoelastic_correction` et `viscoelastic_correction_direct_impact` reçoivent, comme leurs équivalents élastiques, un paramètre `max_normalized_freq: float = 0.6` — voir le chapitre « dispersion élastique » §5.2 pour le mécanisme complet (fenêtre en cosinus surélevé 0→1 multipliée dans le spectre reconstruit avant `irfft`, motivation anti-amplification du bruit haute fréquence, réglage sur la page Corrections). La seule différence dans le cas viscoélastique est le choix de la vitesse de référence c_0 qui normalise la fréquence : une barre viscoélastique n'a pas de vitesse d'onde unique et constante comme une barre élastique, donc chaque branche (incidente, réfléchie, transmise) utilise sa **propre** vitesse en fréquence nulle issue du module complexe de cette branche, `abs(c_i[0])` / `abs(c_r[0])` / `abs(c_t[0])`, plutôt qu'un `c0` unique partagé. Concrètement (`core/dispersion.py`) :

```
w_i = _bandwidth_rolloff_weight(freq, a1, abs(c_i[0]), max_normalized_freq)
w_r = _bandwidth_rolloff_weight(freq, a1, abs(c_r[0]), max_normalized_freq)
w_t = _bandwidth_rolloff_weight(freq, a2, abs(c_t[0]), max_normalized_freq)
```

chaque poids étant ensuite multiplié dans le spectre correspondant juste avant `np.fft.irfft(..., n=n)`. Dans `viscoelastic_correction_direct_impact` (une seule branche transmise), seul `w_t` existe, construit de la même façon à partir de `abs(c_t[0])`. Le réglage (case à cocher + valeur, page Dispersion Corrections)

est partagé avec le cas élastique : c'est le même `project.dispersion_bandwidth_limit_enabled` / `project.dispersion_bandwidth_limit` et le même `_max_normalized_freq_for_run()` qui alimentent les quatre fonctions de correction (élastique deux-barres, élastique Direct Impact, viscoélastique deux-barres, viscoélastique Direct Impact) — voir chapitre « dispersion élastique » §5.2 pour le câblage UI complet ; il n'est **pas** documenté ni exposé séparément pour le cas viscoélastique.

8.2 Les quatre modèles rhéologiques (`core/rheology_models.py`)

Le module définit une structure générique `RheologyModel` (dataclass, ligne 101) portant une liste de `ParamSpec` (clé, libellé, unité, bornes de recherche bayésienne, valeur par défaut pour le mode « Test parameters ») et une fonction `complex_modulus(freq, params) -> E*(f)`. Quatre modèles sont enregistrés dans le dictionnaire `MODELS` (clés `MODEL_KELVIN_VOIGT`, `MODEL_SLS`, `MODEL_GENERALIZED_MAXWELL_2`, `MODEL_BURGERS`), listés par ordre de complexité croissante dans le docstring du module (lignes 24-43) : Kelvin-Voigt (2 paramètres) → Standard Linear Solid (3 paramètres) → Generalized Maxwell 2 branches (5 paramètres) → Burgers (4 paramètres).

8.2.1 Kelvin–Voigt (`_kelvin_voigt_modulus`, ligne 120)

Schéma rhéologique : un ressort E monté **en parallèle** avec un amortisseur η (`short_formula = "E || eta"`).

Paramètres : E (Pa), η (Pa·s).

Module complexe :

$$E^*(\omega) = E + i\,\omega\,\eta$$

Code (ligne 120-123) :

```python
def _kelvin_voigt_modulus(freq, params):
    e, eta = params["e"], params["eta"]
    w = 2 * np.pi * freq
    return e + 1j * w * eta
```

Limitation notée dans le code (`formula_html`, lignes 216-223) : l'amortissement de ce modèle croît sans borne avec la fréquence, et il ne distingue pas de module instantané/relaxé séparé ; c'est décrit comme « une base grossière plutôt qu'un bon ajustement au spectre de relaxation d'un polymère réel ».

8.2.2 Standard Linear Solid / Zener (`_sls_modulus`, ligne 126) — seul modèle branché sur la page Dispersion Corrections

Schéma rhéologique : un ressort E_1 **en série** avec une cellule de Kelvin-Voigt (E_2 en parallèle avec n_2) — la « représentation à 3 paramètres de Kelvin » (`short_formula = "E1 -- (E2 || n2)"`).

Traçabilité de la formule (docstring de `_sls_modulus`, lignes 127-138) : la formule reprend telle quelle celle de `calib_visco_model.py` / `core/dispersion.py::_standard_linear_solid_modulus`. `ui/rheology_diagram.py::draw_sls` dessine cette topologie « Kelvin série », cohérente avec ce qui est réellement calibré.

Paramètres : E_1 (Pa), E_2 (Pa), n_2 (Pa·s).

Équation constitutive (compliance $J(\omega)$, puis module $E^*(\omega) = 1/J(\omega)$) :

$$J(\omega) = \frac{1}{E_1} + \frac{E_2}{E_2^2 + (\omega n_2)^2} - i\,\frac{\omega n_2}{E_2^2 + (\omega n_2)^2}$$

$$E^*(\omega) = \frac{1}{J(\omega)}$$

Code (ligne 139-143) :

```
e1, e2, n2 = params["e1"], params["e2"], params["n2"]
w = 2 * np.pi * freq
j1 = (1.0 / e1) + (e2 / (e2**2 + (w * n2)**2))
j2 = (w * n2) / (e2**2 + (w * n2)**2)
return 1.0 / (j1 - 1j * j2)
```

C'est un modèle à **un seul temps de relaxation** (Debye). Le docstring et la `polymer_note` du modèle avertissent que ce spectre unique « colle souvent trop étroitement » au comportement réel du PMMA/PC/nylon, dont le spectre de perte est plus large et moins symétrique.

8.2.3 Generalized Maxwell / Wiechert, 2 branches (`_generalized_maxwell_2_modulus`, ligne 146)

Schéma rhéologique : un ressort à long terme E_∞ en parallèle avec **deux** bras de Maxwell (E_1–n_1 et E_2–n_2), chacun réglé sur un temps de relaxation propre $\tau_k = n_k/E_k$ (`short_formula = "Einf || (E1-n1) || (E2-n2)"`).

Paramètres : E_∞ (Pa), E_1 (Pa), n_1 (Pa·s), E_2 (Pa), n_2 (Pa·s).

Équation constitutive :

$$\tau_k = \frac{n_k}{E_k} \quad (k = 1, 2)$$

$$E^*(\omega) = E_\infty + \sum_{k=1}^{2} E_k \cdot \frac{i\,\omega\,\tau_k}{1 + i\,\omega\,\tau_k}$$

Code (ligne 146-154) :

```
e_inf, e1, n1, e2, n2 = params["e_inf"], params["e1"], params["n1"], params["e2"], params["n2"]
w = 2 * np.pi * freq
```

```
tau1, tau2 = n1 / e1, n2 / e2
return e_inf + e1*(1j*w*tau1)/(1+1j*w*tau1) + e2*(1j*w*tau2)/(1+1j*w*tau2)
```

Avec deux temps de relaxation, ce modèle donne un spectre plus large et asymétrique, décrit dans la `polymer_note` comme « généralement plus proche des données réelles de PMMA/PC/nylon ».

8.2.4 Burgers (`_burgers_modulus`, ligne 157)

Schéma rhéologique : un élément de Maxwell (ressort E_1 en série avec amortisseur n_1) monté **en série** avec une cellule de Kelvin-Voigt (ressort E_2 en parallèle avec amortisseur n_2) — combinaison directe des deux modèles les plus simples (`short_formula = "(E1-n1) -- (E2 || n2)"`).

Paramètres : E_1 (Pa), n_1 (Pa·s), E_2 (Pa), n_2 (Pa·s).

Équation constitutive (compliance, puis inversion) :

$$J(\omega) = \frac{1}{E_1} + \frac{1}{i\,\omega\,n_1} + \frac{1}{E_2 + i\,\omega\,n_2}, \qquad E^*(\omega) = \frac{1}{J(\omega)}$$

Code (ligne 189-193) :

```
def _modulus_at(f):
    w = 2 * np.pi * f
    j_maxwell = (1.0 / e1) + 1.0 / (1j * w * n1)
    j_kelvin_voigt = 1.0 / (e2 + 1j * w * n2)
    return 1.0 / (j_maxwell + j_kelvin_voigt)
```

Singularité à $f = 0$ et traitement spécial : contrairement aux trois autres modèles, le module statique (fréquence nulle) de Burgers est **exactement nul** (le bras de Maxwell « s'écoule » librement sous contrainte soutenue). Ceci pose un problème documenté en détail dans le docstring de `_burgers_modulus` (lignes 158-184) :

- `core/visco_calibration.py::_predict_pulse` calcule une vitesse de référence $c[0] = \sqrt{E(0)/\rho}$ réutilisée, **inchangée**, dans sa formule de correction de phase pour *tous* les bins fréquentiels, et l'étape finale (`cmath.rect`) exige un résultat exactement réel — une partie imaginaire non nulle même infime provoque un crash (« can't convert complex to float »).
- Une référence à fréquence nulle n'est de toute façon pas physiquement pertinente pour un pulse SHPB de l'ordre de la milliseconde.

Solution implémentée : le bin DC (`freq[0] == 0.0`) est évalué non pas à $f = 0$ mais à la fréquence fondamentale non nulle la plus basse du pulse, `freq[1]` (déjà utilisée ailleurs dans `_predict_pulse` comme germe de la recherche de racine), et **seule la partie réelle** de ce résultat est conservée comme rigidité de référence (la partie imaginaire, perte, est ignorée) :

```
e_complex = _modulus_at(_safe_freq(freq))
if freq.size > 1 and freq[0] == 0.0:
    e_complex[0] = _modulus_at(freq[1]).real
```

`_safe_freq` (ligne 115-117) remplace toute fréquence nulle par `_EPS_FREQ = 1e-6` Hz pour éviter une division par zéro littérale dans `1/(i·ω·n1)` — **mais ce garde-fou n'est appliqué qu'à l'intérieur de `_burgers_modulus`**, jamais aux trois autres modèles, dont le module DC est déjà fini et exactement réel ; le module précise explicitement (lignes 73-88) qu'appliquer ce nudge aux autres modèles provoquerait le même crash de conversion complexe→réel.

Ce module a un module à long terme réellement nul, ce qui lui permet de représenter l'élasticité instantanée + l'élasticité retardée (fluage) + un véritable écoulement visqueux — pertinent pour des thermoplastiques comme le nylon sous charge soutenue ou répétée (`polymer_note`).

8.2.5 Tableau récapitulatif

Modèle	Clé (`MODEL_*`)	Paramètres (unités)	$E^*(\omega)$
Kelvin–Voigt	`kelvin_voigt`	E (Pa), η (Pa·s)	$E + i\omega\eta$
Standard Linear Solid	`standard_linear_solid`	E_1, E_2 (Pa), n_2 (Pa·s)	$1/\left[\frac{1}{E_1} + \frac{E_2}{E_2^2+(\omega n_2)^2} - i\frac{\omega n_2}{E_2^2+(\omega n_2)^2}\right]$
Generalized Maxwell (2 branches)	`generalized_maxwell_2`	E_∞, E_1, E_2 (Pa), n_1, n_2 (Pa·s)	$E_\infty + \sum_{k=1,2} E_k \frac{i\omega\tau_k}{1+i\omega\tau_k}$, $\tau_k = n_k/E_k$
Burgers	`burgers`	E_1, E_2 (Pa), n_1, n_2 (Pa·s)	$1/\left[\frac{1}{E_1} + \frac{1}{i\omega n_1} + \frac{1}{E_2+i\omega n_2}\right]$

8.3 Propagation du pulse à travers le modèle de barre viscoélastique (`core/visco_calibration.py`)

Cette fonction est le cœur commun aux deux modes de calibration (Bayésien et « Test parameters ») : elle prédit le pulse 2 (mesuré plus loin sur la barre) à partir du pulse 1 mesuré et d'un jeu de paramètres candidat.

8.3.1 Module complexe, vitesse et modules de Lamé (`_complex_modulus_and_speed`, ligne 78)

Pour chaque fréquence f du spectre FFT du pulse 1 :

$$\mu(f) = \frac{E^*(f)}{2(1+\nu)}, \qquad \lambda(f) = \frac{\nu\, E^*(f)}{(1+\nu)(1-2\nu)}, \qquad c(f) = \sqrt{\frac{E^*(f)}{\rho}}$$

où ν est le coefficient de Poisson (paramètre séparé, cf. §8.1) et ρ la masse volumique de la barre.

8.3.2 Recherche de racine de Pochhammer-Chree par bin de fréquence (`_predict_pulse`, ligne 90)

Le pulse 1 est transformé par FFT réelle (`np.fft.rfft`), donnant amplitude et phase par bin. Pour chaque fréquence non nulle f_v ($v = 1, \ldots$), avec a le rayon de la barre, le code résout numériquement (via `sympy.nsolve`, inconnue B) l'équation :

$$p = \sqrt{\frac{\rho\,\omega_v^2}{\lambda_v + 2\mu_v} - B^2}, \qquad q = \sqrt{\frac{\rho\,\omega_v^2}{\mu_v} - B^2}$$

$$\frac{2p}{a}(q^2 + B^2)\,J_1(pa)\,J_1(qa) - (q^2 - B^2)^2\,J_0(pa)\,J_1(qa) - 4B^2pq\,J_0(qa)\,J_1(pa) = 0$$

(J_0, J_1 : fonctions de Bessel de première espèce, `sympy.besselj`). La racine complexe trouvée donne le nombre d'onde $k = \Re(B)$ (utilisé pour la vitesse de phase $c_p = \omega_v/k$) et l'atténuation atn $= |\Im(B)|$.

Solution de repli (`except ValueError`) : si `nsolve` échoue sur cette formulation, le code résout une seconde forme de l'équation de Pochhammer-Chree, cette fois pour l'inconnue c_p directement, avec :

$$\beta = \frac{1 - 2\nu}{1 - \nu}, \qquad \xi = \left(\frac{c_p}{c_v}\right)^2 (1 + \nu), \qquad k = \frac{\omega_v}{c_p}$$

$$h_e = k\sqrt{\beta\xi - 1}, \qquad k_e = k\sqrt{2\xi - 1}$$

$$(\xi - 1)^2\,a\,h_e\,\frac{J_0(ah_e)}{J_1(ah_e)} - (\beta\xi - 1)\left(\xi - ak_e\,\frac{J_0(ak_e)}{J_1(ak_e)}\right) = 0$$

Dans ce cas de repli, l'atténuation n'est **pas** recalculée : elle est reportée depuis le bin précédent (`atn.append(atn[-1])`), et le nombre d'onde complexe équivalent reconstruit comme $B = k - i \cdot$ atn. *Le code ne documente pas explicitement la dérivation physique de cette seconde formulation ni les conditions précises qui déclenchent le repli ; elle est rapportée ici telle qu'implémentée, sans interprétation supplémentaire.*

Le premier bin de fréquence non nul ($v = 0$) est initialisé par extrapolation linéaire simple : `k_sol = [0, (2π·freq[1])/c[1]]` (ligne 112), servant de germe (`k_sol[-1]`) au premier `nsolve`.

8.3.3 Correction de phase, atténuation et amortissement, puis reconstruction temporelle

Pour chaque bin w :

$$\Delta\varphi_w = \varphi_w - \frac{2\pi f_w\,z_1}{c_0}\left(\frac{c_0}{|c_{p,w}|} - 1\right)$$

$$\hat{Y}_w = \exp\!\left(-\frac{C_{\text{damp}}}{c_0}\,z_1\right) \cdot A_w \cdot \exp(-\text{atn}_w\,z_1) \cdot \exp(i\,\Delta\varphi_w)$$

où z_1 est la distance entre les deux points de mesure du pulse, C_{damp} le facteur d'amortissement de la barre (« Damping factor/C », en rad/m — typiquement issu de l'outil Free-Decay Damping, §8.6), $c_0 = c(f{=}0)$ la vitesse de référence, A_w et φ_w l'amplitude/phase du spectre du pulse 1. Le pulse 2 prédit est obtenu par FFT inverse réelle avec la longueur explicitement forcée (`np.fft.irfft(new_pulse_fft, n=n)`)

— le commentaire (lignes 155-171) explique que sans ce `n=n` explicite, une fenêtre de pulse de longueur impaire (cas fréquent puisque la fenêtre vient d'un glisser-déposer utilisateur) produirait un signal reconstruit **numériquement faux**, pas seulement tronqué (écart mesuré jusqu'à ~2 dans un signal d'ordre 1 pour une fenêtre de 139 échantillons), à cause de l'hypothèse implicite de `irfft` sur la parité de la longueur d'origine.

8.3.4 Score de comparaison : Pearson vs R² (`SCORE_FUNCS`, ligne 223)

Le pulse prédit et le pulse 2 mesuré sont d'abord recalés sur leur intervalle commun (`_align_pulses`, ligne 176, règle de fenêtrage `time2 ≥ 0` / `time1 < max(time2)`, inchangée depuis l'implémentation d'origine). Deux métriques sont proposées (`SCORE_ORDER = ("r2", "correlation")`, défaut `DEFAULT_METRIC = "r2"`) :

Corrélation de Pearson (`_pearson_score`, ligne 201) :

$$r = |\,\mathrm{corr}(\text{mesuré}, \text{prédit})\,| = \left| \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\sqrt{\sum_i (y_i - \bar{y})^2}} \right|$$

(via `scipy.stats.pearsonr`, valeur absolue prise). Le docstring du module insiste : cette mesure est invariante par transformation affine de l'un ou l'autre signal — c'est précisément son angle mort ici : un candidat dont la forme est correcte mais dont l'amplitude absolue (amortissement, module global) est fausse peut quand même obtenir un score proche de 1.

Coefficient de détermination R² (`_r2_score`, ligne 207), critère par défaut :

$$SS_{res} = \sum_i (y_i - \hat{y}_i)^2, \qquad SS_{tot} = \sum_i (y_i - \bar{y})^2$$

$$R^2 = \mathrm{clip}\left(1 - \frac{SS_{res}}{SS_{tot}},\ -1,\ 1\right)$$

(y = pulse mesuré, $\hat{y}$ = pulse prédit). Le code note explicitement que R² est borné à $[-1, 1]$ (alors qu'un R² classique est non borné inférieurement) « pour ne pas laisser un candidat pathologique dominer l'échelle de longueur ajustée du Processus Gaussien surrogate pendant l'optimisation bayésienne » (ligne 212-215).

Les fonctions publiques `predict_pulse` (ligne 242) et `pulse_correlation` (ligne 251) sont les points d'entrée réutilisés par `core/bayes_visco_calibration.py` ; `test_parameters` (ligne 259) est le point d'entrée du mode « Test parameters » de la page : il évalue un unique jeu $(\nu, \text{modèle}, \text{paramètres})$ fixé par l'utilisateur.

Avertissement explicite du module (docstring, lignes 43-47) : « NOT YET independently re-validated against real experimental data or the original GUI's output […] ported as literally as possible from the original for traceability; cross-check results against the original application before relying on this for a real calibration. »

8.4 Calibration bayésienne (`core/bayes_visco_calibration.py`)

8.4.1 Pourquoi une recherche bayésienne plutôt qu'une grille

Le docstring du module (lignes 8-38) justifie ce choix : une recherche par grille sur les n paramètres d'un modèle (jusqu'à 6 dimensions pour Generalized Maxwell, ν inclus) coûterait `count_1 × count_2 × … × count_n` évaluations, chacune impliquant une recherche de racine `sympy.nsolve` par bin de fréquence — coûteuse. L'optimisation bayésienne remplace cela par : (1) un plan d'expérience initial en Latin Hypercube Sampling (LHS), (2) un surrogate Processus Gaussien peu coûteux à ajuster/évaluer, (3) une proposition itérative du point suivant maximisant l'« Expected Improvement » sur le surrogate, (4) un arrêt anticipé par patience.

8.4.2 Évaluation d'un point candidat

Un point candidat est d'abord un vecteur de l'hypercube unité $[0,1]^d$ (espace de travail du GP et de la recherche d'acquisition), ramené aux unités physiques par `_from_unit` (ligne 112) :

$$x_i = \begin{cases} \exp\left(\ln(\mathrm{lo}_i) + u_i\,[\ln(\mathrm{hi}_i) - \ln(\mathrm{lo}_i)]\right) & \text{si le paramètre } i \text{ est en échelle log} \\ \mathrm{lo}_i + u_i\,(\mathrm{hi}_i - \mathrm{lo}_i) & \text{sinon} \end{cases}$$

Le coefficient de Poisson ν est cherché en échelle **linéaire** (plage physique étroite), tous les modules/viscosités du modèle en échelle **log** (grandeurs strictement positives et « multiplicatives » — ligne 229-233).

`eval_point(u)` (ligne 246) convertit u en paramètres réels, appelle `predict_pulse(pulse1, time1, nu, model, params, density, radius, z1, damp)` (§8.3.2-8.3.3) puis note le résultat avec `pulse_correlation(predicted, pulse2, time1, time2, metric)` (§8.3.4). **Toute exception est interceptée et le point est noté 0.0** plutôt qu'exclu (« scored, not excluded, so the GP still learns "avoid this region" » — lignes 254-255, 148-149), afin que le surrogate apprenne aussi la forme des régions non évaluables. La fonction garde en mémoire (`nonlocal`) le meilleur pulse prédit rencontré (`best_pulse_seen`) et son score (`best_corr_seen`), pour éviter un appel supplémentaire coûteux en fin de recherche (voir §8.4.6).

8.4.3 Échantillonnage initial (Latin Hypercube Sampling)

`latin_hypercube(n, dim, rng)` (`core/gaussian_process.py`, ligne 129) génère `n_init` points initiaux dans $[0,1]^{\mathrm{dim}}$: chaque dimension est découpée en `n` intervalles égaux, chacun est touché exactement une fois avec un jitter aléatoire à l'intérieur, et les intervalles sont mélangés indépendamment par dimension :

```
for d in range(dim):
    cut = (np.arange(n) + rng.uniform(size=n)) / n
    rng.shuffle(cut)
    samples[:, d] = cut
```

Cela répartit les premières évaluations — les plus informatives — uniformément sur toute la boîte de recherche, sans les `n_per_dim ** n_dims` points d'une grille ni les larges trous laissés par un tirage purement uniforme pour un `n` faible.

Ces `n_init` points peuvent être évalués en parallèle si `n_workers > 1` (via `core/parallel.run_map` et le worker `_eval_point_task`, ligne 127) : ils sont indépendants les uns des autres, contrairement à chaque point de la boucle adaptative qui suit.

8.4.4 Boucle adaptative : surrogate Processus Gaussien + Expected Improvement

Pour chaque itération (jusqu'à `n_iter`, ligne 301) :

1. Un nouveau `GaussianProcess` (§8.5) est ajusté sur **tous** les points (U, y) évalués jusqu'ici (`gp.fit(U, y)`).
2. `_propose_next(gp, best_so_far, dim, rng)` (ligne 167) cherche le point de $[0,1]^{\text{dim}}$ maximisant l'**Expected Improvement (EI)** :

$$\mathrm{EI}(x) = (\mu(x) - f_{\text{best}} - \xi)\,\Phi(z) + \sigma(x)\,\varphi(z), \qquad z = \frac{\mu(x) - f_{\text{best}} - \xi}{\sigma(x)}$$

où $\mu(x)$, $\sigma(x)$ sont la moyenne et l'écart-type prédits par le GP (§8.5.3), $f_{\text{best}} = \max(y)$ le meilleur score déjà observé, Φ/φ la CDF/densité de la loi normale standard, et $\xi = 0.01$ une marge d'exploration (`_expected_improvement`, ligne 153 ; « without it, EI can get stuck refining right around the current best instead of also checking still-uncertain regions »). $\sigma(x)$ est planché à 10^{-9} pour éviter une division par zéro.

Cette maximisation se fait en deux temps (ligne 167-192) : un balayage aléatoire large et bon marché (`n_random = max(2000, 400·dim)` candidats, car cela n'appelle que le GP, jamais le modèle direct coûteux), suivi d'un polissage local par `scipy.optimize.minimize` (méthode `L-BFGS-B`, bornes $[0,1]^{\text{dim}}$) depuis les `n_restarts = max(8, dim+6)` meilleurs candidats. 3. Le point proposé u_{next} est évalué avec le **vrai** modèle direct coûteux (`eval_point`), le score est ajouté à `(U, y)`. 4. Critère de convergence/patience : le compteur `no_improve` est remis à 0 si `new_best - best_so_far > convergence_tol` (défaut 10^{-4}), sinon incrémenté ; la recherche s'arrête dès que `no_improve >= convergence_patience` (défaut 10 itérations sans amélioration), ou après `n_iter` itérations (défaut 40) si la patience n'est jamais atteinte.

Le docstring de `calibrate_bayesian` (lignes 218-226) précise explicitement que **seule la phase LHS initiale est parallélisable** : la boucle adaptative ne l'est pas, car proposer plusieurs points depuis un même GP non encore mis à jour changerait quels points la recherche visite réellement, pas seulement sa vitesse.

8.4.5 Moyenne, écart-type et corrélation a posteriori des paramètres

Une fois la recherche terminée, un **dernier** GP est ajusté sur tous les points évalués. La posterior est alors estimée par échantillonnage dense et bon marché de la moyenne prédite du surrogate (jamais du modèle direct coûteux), lignes 346-378 :

1. `n_posterior = 20000` points U_{post} tirés uniformément dans $[0,1]^{\text{dim}}$; `mean_post, _ = gp.predict(U_post)`.
2. Pondération par un **softmax** de la moyenne prédite, avec une « température » adaptative :

$$T = \max(\mathrm{std}(\mu_{\text{post}}),\ 10^{-6}) \times 0.3$$

$$w_i = \frac{\exp\big((\mu_{\text{post},i} - \max_j \mu_{\text{post},j})/T\big)}{\sum_k \exp\big((\mu_{\text{post},k} - \max_j \mu_{\text{post},j})/T\big)}$$

3. Les points sont reconvertis en unités physiques ($X_{\text{post}} = \text{from_unit}(U_{\text{post}})$), puis moyenne/écart-type pondérés par paramètre :

$$\bar{x}_k = \sum_i w_i \, x_{i,k}, \qquad \sigma_k = \sqrt{\sum_i w_i \, (x_{i,k} - \bar{x}_k)^2}$$

4. **Matrice de corrélation entre paramètres** (`param_correlation`) — la corrélation de Pearson **pondérée** entre chaque paire de paramètres sur ce même échantillon posterior :

$$\text{Cov}_{jk} = \sum_i w_i \, (x_{i,j} - \bar{x}_j)(x_{i,k} - \bar{x}_k), \qquad \rho_{jk} = \text{clip}\left(\frac{\text{Cov}_{jk}}{\sigma_j \, \sigma_k}, \, -1, \, 1\right)$$

(diagonale forcée à 1). Le docstring du module (lignes 51-61) précise le sens de ce diagnostic : une valeur proche de ± 1 entre deux paramètres signifie que la mesure (les deux pulses) contraint surtout une combinaison des deux (par ex. leur produit) plutôt que chacun individuellement, et qu'**aucun budget de recherche supplémentaire ne corrige cela** — seule une mesure différente ou additionnelle le pourrait. « It says nothing about the parameters being physically coupled, only that *this* dataset cannot tell them apart. »

Le score moyen/écart-type globaux (`score_mean`, `score_std`) sont la moyenne/écart-type pondérés de `mean_post` lui-même (lignes 377-378).

Le résultat final expose aussi la trace brute des points réellement évalués (`evaluated_params`, `evaluated_scores` — distincte de l'échantillon posterior dense synthétique) et l'historique du meilleur score par évaluation (`history_best`, pour le tracé de convergence).

8.4.6 Pulse prédit retourné : réutilisation, pas de recalcul

`predicted_pulse` (utilisé pour le tracé de comparaison) **n'est pas** recalculé à la moyenne posterior — le docstring (lignes 380-401) explique que ce point moyen (moyenne pondérée en espace linéaire même pour des paramètres en échelle log) peut tomber dans une région où la recherche de racine Pochhammer-Chree (`nsolve`) converge très lentement, en particulier pour Generalized Maxwell/Burgers, provoquant un blocage apparent de l'interface (barre de progression déjà à 100 % pendant qu'un appel non suivi tourne encore). Le pulse prédit du **meilleur point réellement évalué pendant la recherche** (`best_pulse_seen`, cf. §8.4.2) est réutilisé à la place ; un recalcul n'a lieu que si absolument toutes les évaluations ont échoué.

8.5 Le Processus Gaussien surrogate (`core/gaussian_process.py`)

Implémentation « from scratch », volontairement légère en dépendances (numpy + scipy uniquement), documentée comme « both sufficient and easy to audit/traceable » pour un problème de quelques dizaines de points en au plus quelques dimensions (docstring, lignes 1-11).

8.5.1 Noyau de covariance

Noyau exponentiel-carré anisotrope (ARD — Automatic Relevance Determination), une longueur de corrélation ℓ_d par dimension :

$$k(x, x') = \sigma_f^2 \exp\left(-\frac{1}{2} \sum_d \left(\frac{x_d - x'_d}{\ell_d} \right)^2 \right)$$

(`_kernel`, ligne 56-61). La matrice de covariance d'entraînement ajoute un terme de bruit i.i.d. et un « jitter » numérique de stabilité :

$$K = k(X, X) + (\sigma_n^2 + 10^{-10})\, I$$

8.5.2 Ajustement des hyperparamètres (maximisation de la vraisemblance marginale)

La cible y est standardisée avant ajustement : $y_c = (y - \bar{y})/\mathrm{std}(y)$. Les hyperparamètres $\theta = (\ln \ell_1, \dots, \ln \ell_d,\ \ln \sigma_f,\ \ln \sigma_n)$ sont trouvés en minimisant la **log-vraisemblance marginale négative** :

$$\mathrm{NLL}(\theta) = \tfrac{1}{2}\, y_c^\top K^{-1} y_c + \tfrac{1}{2} \ln |K| + \tfrac{n}{2} \ln(2\pi)$$

(`_neg_log_marginal_likelihood`, ligne 63-76), résolue via une factorisation de Cholesky (`scipy.linalg.cho_factor/cho` pour la stabilité et le coût. L'optimisation utilise `scipy.optimize.minimize` (`L-BFGS-B`) avec `n_restarts` (défaut 6) initialisations aléatoires dans des bornes fixées : $\ln \ell_d \in [-3, 2]$, $\ln \sigma_f \in [-2, 2]$, $\ln \sigma_n \in [-4, 0]$ (commentées ligne 86-92 : le bruit peut aller jusqu'à l'écart-type complet du signal standardisé, car le solveur `sympy.nsolve` par fréquence peut légèrement varier entre jeux de paramètres voisins, et un peu de marge de bruit évite au GP de sur-ajuster ce « wobble »).

8.5.3 Prédiction (moyenne/variance a posteriori)

Pour de nouveaux points X_* (`predict`, ligne 117-126), avec $K_s = k(X_*, X)$ et $\alpha = K^{-1} y_c$ (déjà calculé à l'ajustement via Cholesky) :

$$\mu_*(X_*) = K_s\, \alpha \cdot \sigma_y + \bar{y}$$

$$\sigma_*^2(X_*) = \max\Big(\sigma_f^2 - \mathrm{diag}(K_s\, K^{-1}\, K_s^\top),\ 10^{-12} \Big) \cdot \sigma_y^2$$

(le code exploite $k(x_*, x_*) = \sigma_f^2$ pour un noyau stationnaire, ligne 124). Ce sont les formules classiques de régression par Processus Gaussien, avec dé-standardisation finale vers l'échelle originale du score.

`latin_hypercube` (§8.4.3) est également défini dans ce même module.

8.6 Auto-alignement des pulses par corrélation sub-échantillon (`core/pulse_alignment.p`

8.6.1 Pourquoi

Le docstring (lignes 1-39) explique le problème : un glisser-déposer manuel des deux fenêtres de pulse est imprécis précisément là où cela compte le plus — le point de **départ** de la fenêtre du pulse 2. Une imprécision de quelques échantillons sur ce point ne modifie pas l'analyse du pulse 1 lui-même (sa fenêtre ne fixe que la résolution fréquentielle de la FFT) mais fausse directement toute comparaison avec le pulse 2 réellement mesuré : la recherche bayésienne « compenserait » alors ce décalage arbitraire en biaisant les paramètres ajustés, sans rapport avec une propriété réelle du matériau.

8.6.2 Algorithme (`align_pulse2`, ligne 50)

La **largeur** de la fenêtre du pulse 2 telle que glissée par l'utilisateur est conservée à l'identique (choix légitime de l'utilisateur) ; seul son **point de départ** est recherché automatiquement :

1. Un signal gabarit (`template`) est extrait à partir de la fenêtre du pulse 1 (valeur absolue du signal brut, cohérent avec la construction de `pulse1`/`pulse2` dans `_on_run` de la page). `ref = template[:m]` avec $m = \min(\text{largeur}_1, \text{largeur}_2)$ en échantillons — quand les deux largeurs diffèrent, la comparaison se fait sur la partie de tête commune, cohérent avec la façon dont `core.visco_calibration._align_pulses` réconcilie déjà des longueurs différentes ailleurs.
2. Une marge de recherche autour du point de départ indiqué par l'utilisateur (`margin`, par défaut la moitié de la largeur de la fenêtre du pulse 1) définit un intervalle d'indices `[lo, hi]` candidats.
3. Pour chaque décalage entier candidat i_2 dans cet intervalle, le score est **systématiquement la corrélation de Pearson** (`SCORE_FUNCS["correlation"]`, c.-à-d. `_pearson_score`, §8.3.4) entre le candidat et le gabarit `ref` — **et non** le critère (R² ou Pearson) choisi pour la calibration elle-même. Le docstring justifie ce choix (lignes 28-39) : le pulse 2 est *censé* être atténué par rapport au pulse 1 (c'est justement la physique de dispersion/amortissement que la calibration cherche à mesurer), donc juger l'alignement par correspondance d'amplitude absolue (R²) pénaliserait à tort le bon décalage simplement parce que le pulse 2 est physiquement plus petit — exactement l'inverse de la raison pour laquelle R² a été préféré à la corrélation pour l'objectif de calibration lui-même. Ici, l'invariance d'échelle de la corrélation est au contraire l'outil approprié : elle ignore l'atténuation attendue et se concentre uniquement sur la correspondance de forme.
4. Le meilleur décalage entier `best_i2` (score maximal) est ensuite raffiné à précision **sub-échantillon** par interpolation parabolique à 3 points autour de l'optimum entier :

$$\text{offset} = \frac{1}{2} \cdot \frac{y_0 - y_2}{y_0 - 2y_1 + y_2}$$

où y_0, y_1, y_2 sont les scores aux décalages `best_i2 - 1, best_i2, best_i2 + 1`. Le raffinement n'est appliqué que si $|\text{offset}| < 1$ (« a real peak refines by less than one whole sample ») et si le meilleur décalage n'est pas au bord de l'intervalle de recherche (une parabole à 3 points passant par un optimum au bord n'a pas de sens). 5. Le nouveau départ est `new_start = time[best_i2] + offset·dt`, la fin étant translatée de la même quantité pour conserver la largeur d'origine.

La fonction retourne `None` si le voisinage du point indiqué ne contient pas assez de signal pour effectuer la

recherche (par ex. trop proche du bord de l'enregistrement).

8.7 Outil standalone « Free-Decay Damping » (`core/decay.py`)

Cet outil (fonction unique `compute_decay(ftime, fmeasure)`, ligne 31) estime le coefficient d'amortissement d'une barre à partir d'un signal de décroissance libre (par ex. un enregistrement d'impact/coup de marteau), par la **méthode de largeur de bande à mi-puissance (half-power / -3 dB bandwidth)** sur le pic du spectre FFT. Le docstring précise que la logique est un portage de `decay.py`, sans l'interface graphique Tkinter d'origine.

8.7.1 Localisation précise du pic et des points à mi-puissance

1. FFT réelle du signal : `yi = rfft(fmeasure)`, `freq = rfftfreq(n, dt)`, `amplitude = |yi|`.
2. Interpolation cubique du spectre d'amplitude (`scipy.interpolate.interp1d(freq, amplitude, kind="cubic")`) sur une grille fine `freq_fine` (5× plus de points que la grille FFT native), pour localiser le pic et les points à mi-puissance avec une précision supérieure à la résolution native de la FFT (« exactly as in the original implementation », ligne 49-50).
3. Pic : `peak_idx = argmax(amp_fine)` → fréquence propre $f_n = \text{freq_fine}[\text{peak_idx}]$, amplitude max A_{max}.
4. Seuil de mi-puissance (−3 dB) :

$$A_{\text{demi-puissance}} = \frac{A_{\text{max}}}{\sqrt{2}}$$

5. Recherche des deux fréquences encadrant le pic où le spectre franchit ce seuil : `lower_idx` = premier indice (en balayant depuis le début du tableau) où l'amplitude dépasse le seuil ; `upper_idx` = premier indice après `lower_idx` où l'amplitude retombe sous le seuil (ou dernier indice du tableau si jamais atteint). D'où $f_l = \text{freq_fine}[\text{lower_idx}]$, $f_u = \text{freq_fine}[\text{upper_idx}]$.

Remarque : cette recherche balaie l'ensemble du spectre depuis l'indice 0, pas seulement un voisinage du pic — elle est donc sensible à un spectre bruité ou multi-pics (voir avertissement UI ci-dessous), sans garde-fou explicite dans le code contre un franchissement de seuil parasite avant le pic principal.

8.7.2 Facteur d'amortissement

Avec la largeur de bande relative $\text{ratio} = (f_u - f_l)/f_n$:

$$\zeta = \sqrt{0.5 - \sqrt{\frac{1}{4 + 4\,\text{ratio}^2 - \text{ratio}^4}}}$$

(`damping_ratio`, ligne 70 — c'est le ratio d'amortissement adimensionnel « curve damping factor »). Le facteur d'amortissement complet, exprimé dans les unités utilisées ailleurs dans l'application pour le paramètre « Damping factor/C » (`core/visco_calibration.py`'s `damp`, cf. §8.3.3) est :

$$C = \zeta \cdot 2\pi f_n$$

(`full_damping_factor`, ligne 78). **Remarque sur les unités** : le commentaire du champ `full_damping_factor` dans `DecayResult` (ligne 23) l'annote « rad/m, per original units convention », alors que dimensionnellement $\zeta \cdot 2\pi f_n$ (avec f_n en Hz) est une pulsation, donc en **rad/s**, et non en rad/m. Le code ne documente pas de conversion supplémentaire par une vitesse d'onde à cet endroit ; la valeur `full_damping_factor` calculée ici est réutilisée telle quelle comme paramètre `damp` (rad/m) dans `core/visco_calibration.py` (§8.3.3) et dans `core/dispersion.py`. **Ceci est signalé comme point ambigu du code, non résolu par la documentation source, à vérifier avant toute utilisation quantitative rigoureuse** — il ne s'agit pas d'une extrapolation de notre part mais d'une incohérence dimensionnelle apparente entre le commentaire et le calcul tels qu'écrits.

L'enveloppe exponentielle de décroissance ajustée et tracée (`exponential_envelope`) est :

$$s_{\text{env}}(t) = \max(|s|) \cdot \exp(-\zeta \cdot 2\pi f_n \cdot t)$$

8.7.3 Utilisation en aval

`ui/pages/page_decay.py` sauvegarde automatiquement `full_damping_factor` dans `project.viscoelastic.damping_fa` (ligne 131), réutilisable via le bouton « Use value from Free-Decay Damping tool » sur la page Dispersion Corrections et sur la page de calibration viscoélastique (`ui/pages/page_visco_calib.py`, `_on_use_decay_damping`).

L'aide contextuelle de la page (`page_decay.py`, lignes 80-84) avertit : « A narrow, clean peak gives a more reliable estimate than a noisy or split one — if the spectrum looks off, check that the recorded decay signal is long enough and free of extra vibration sources. »

8.8 Synthèse des renvois fichier/fonction

Sujet	Fichier	Fonction/méthode
4 modèles rhéologiques, $E^*(f)$	`core/rheology_models.py`	`_kelvin_voigt_modulus`, `_sls_modulus`, `_generalized_maxwell_2_modulus`, `_burgers_modulus`
Module/vitesse/Lamé par fréquence	`core/visco_calibration.py`	`_complex_modulus_and_speed`
Propagation FFT/Pochhammer-Chree/IFFT du pulse	`core/visco_calibration.py`	`_predict_pulse` / `predict_pulse` (wrapper public)
Scores Pearson / R^2	`core/visco_calibration.py`	`_pearson_score`, `_r2_score`, `SCORE_FUNCS`

Sujet	Fichier	Fonction/méthode
Recalage temporel avant score	`core/visco_calibration.py`	`_align_pulses`
Évaluation à paramètres fixes	`core/visco_calibration.py`	`test_parameters`
Recherche bayésienne complète	`core/bayes_visco_calibration.py`	`calibrate_bayesian`
Conversion hypercube unité → paramètres	`core/bayes_visco_calibration.py`	`_from_unit`
Expected Improvement	`core/bayes_visco_calibration.py`	`_expected_improvement`
Proposition du point suivant	`core/bayes_visco_calibration.py`	`_propose_next`
Noyau, ajustement, prédiction du GP	`core/gaussian_process.py`	`GaussianProcess._kernel`, `.fit`, `.predict`
Échantillonnage LHS	`core/gaussian_process.py`	`latin_hypercube`
Auto-alignement pulse 2	`core/pulse_alignment.py`	`align_pulse2`
Analyse de décroissance libre / largeur mi-puissance	`core/decay.py`	`compute_decay`
Correction dispersion viscoélastique appliquée (SLS uniquement)	`core/dispersion.py`	`viscoelastic_correction`, `viscoelastic_correction_direct_impact`
Bornage de bande passante (vitesse de référence par branche)	`core/dispersion.py`	`_bandwidth_rolloff_weight` (mécanisme détaillé au chapitre « dispersion élastique » §5.2)

8.9 Récapitulatif des limitations et avertissements relevés dans le code

- Seul le modèle **Standard Linear Solid** est actuellement branché sur la correction de dispersion de la page Dispersion Corrections ; Kelvin-Voigt, Generalized Maxwell (2 branches) et Burgers ne sont disponibles que pour la calibration (`core/rheology_models.py`, lignes 30-54).
- L'expansion Monte-Carlo de barre viscoélastique (`core/models.py::Bar.expand_viscoelastic`) reste câblée en dur sur le module SLS, indépendamment des quatre modèles ici décrits (`core/rheology_models.py`, lignes 56-61).
- La formule du module SLS correspond à la topologie « ressort en série avec cellule de Kelvin-Voigt », comme documenté dans le code (`core/rheology_models.py`, lignes 127-138).
- Le modèle Burgers a un module statique exactement nul, ce qui impose un traitement spécial de la fréquence nulle (évaluation à `freq[1]`, partie réelle seule conservée) pour éviter un crash de conversion complexe → réel documenté comme effectivement reproduit (`core/rheology_models.py`, lignes 71-88, 158-184).
- Dans `_predict_pulse`, une seconde formulation de l'équation de Pochhammer-Chree est utilisée en repli quand la première échoue à converger ; dans ce cas l'atténuation n'est pas recalculée mais reportée du bin précédent, sans justification physique détaillée dans le code (`core/visco_calibration.py`, lignes 133-146).
- `core/visco_calibration.py` porte un avertissement explicite de non-validation : le module n'a pas encore été revalidé indépendamment contre des données expérimentales réelles ou la sortie de

l'application d'origine (docstring, lignes 43-47).

- La boucle adaptative de la recherche bayésienne n'est délibérément **pas** parallélisée (contrairement à la phase LHS initiale), car cela changerait quels points sont réellement visités (`core/bayes_visco_calibration.py`, lignes 218-226).
- La matrice de corrélation entre paramètres (posterior) indique une non-identifiabilité de la mesure, pas un couplage physique réel entre paramètres, et aucun budget de recherche supplémentaire ne peut la résoudre (`core/bayes_visco_calibration.py`, lignes 51-61).
- Le pulse prédit final n'est pas recalculé à la moyenne posterior (risque de convergence très lente du solveur de racine dans cette région), mais réutilisé depuis le meilleur point réellement évalué durant la recherche (`core/bayes_visco_calibration.py`, lignes 380-401).
- Dans `core/pulse_alignment.py`, le critère d'alignement est toujours la corrélation de Pearson, indépendamment du critère choisi pour la calibration elle-même (R² par défaut), par choix délibéré documenté (lignes 28-39).
- Dans `core/decay.py`, l'unité annotée pour `full_damping_factor` (« rad/m ») ne correspond pas dimensionnellement à la formule $\zeta \cdot 2\pi f_n$ (qui donne des rad/s pour f_n en Hz) — incohérence relevée ici sans qu'aucune conversion supplémentaire ne soit visible dans le code environnant ; point à vérifier auprès de l'auteur/utilisateur avant usage quantitatif.
- Dans `core/decay.py`, la recherche des points de mi-puissance balaie l'ensemble du spectre depuis l'indice 0 (pas seulement un voisinage du pic), sans garde-fou contre un spectre bruité/multi-pics ; l'aide contextuelle de la page avertit qu'un pic étroit et propre est nécessaire à une estimation fiable.
- Le bornage de bande passante (`_bandwidth_rolloff_weight`, §8.1) utilise pour chaque branche viscoélastique sa propre vitesse en fréquence nulle (`abs(c_i[0])/abs(c_r[0])/abs(c_t[0])`) comme référence de normalisation, faute d'une vitesse d'onde unique dans un milieu viscoélastique ; la valeur par défaut (0,6) reste, comme pour le cas élastique, une heuristique pragmatique non validée par une étude paramétrique dédiée.

9. Outils standalone (CPG, synchronisation vidéo, analyse par lot, calibration bayésienne)

Ce chapitre documente les outils indépendants du pipeline principal de dépouillement, accessibles séparément dans l'application : le calculateur de coefficient de pont de jauges (CPG), la synchronisation caméra/vidéo, l'analyse par lot (batch), et la calibration bayésienne de lois de comportement (portage de MAMBA, §7). Il couvre aussi, brièvement, le sous-échantillonnage commun à toute l'application, l'abstraction de lecture vidéo/séquence d'images, et l'export vidéo annoté DIC.

L'outil de calcul de l'amortissement par décroissance libre (Free-Decay Damping, `core/decay.py`) est traité dans le chapitre consacré à la dispersion viscoélastique et n'est pas repris ici.

---

1. Calculateur de coefficient de pont de jauges (CPG)

Fichier source : `core/cpg_calc.py` **Page UI :** `ui/pages/page_cpg_calc.py`

1.1 Contexte et rôle

Le coefficient CPG ("Coefficient Pont de Gauge") convertit la tension brute mesurée en sortie d'un pont de jauges de contrainte (après gain d'amplificateur) en déformation :

$$\varepsilon = CPG \times V_{bridge}$$

`CPG` est la valeur que stocke le paramètre Monte-Carlo `SG_coeff` (alias `CPG_pa`) d'une barre (`Bar`, voir `core/models.py`). Le module `core/cpg_calc.py` calcule ce coefficient — et son incertitude — à partir des grandeurs physiques du montage, pour les trois configurations usuelles de pont de Wheatstone. Le code est explicitement documenté comme un **portage aussi littéral que possible** de l'application d'origine (`test_type.py`, fonctions `CPG_comp`/`CPG_comp_value`), **non revalidé indépendamment** contre la sortie de l'application d'origine (docstring du module, lignes 18-20).

Deux corrections optionnelles, propres à la jauge elle-même plutôt qu'au montage en pont, peuvent être activées en plus du calcul de base : la correction de sensibilité transverse (§1.7) et la correction de sortie

thermique (§1.8).

Trois types de montage sont supportés (`BridgeType`, `core/cpg_calc.py` lignes 29-39) :

- **Quarter-bridge** : 1 jauge active, 3 résistances de complétion fixes.
- **Half-bridge** : 2 jauges actives (axiale + latérale), exploitant le coefficient de Poisson de la barre pour un terme de compensation de déformation latérale.
- **Full-bridge** : 4 jauges actives (2 axiales + 2 latérales).

Seuls Half- et Full-bridge utilisent le coefficient de Poisson de la barre (`USES_POISSON_RATIO`, lignes 43-47) — Quarter-bridge n'a pas de jauge latérale à compenser.

1.2 Entrées physiques

Toutes les entrées sont des paramètres Monte-Carlo (moyenne + écart-type), lus par `compute_cpg()` (`core/cpg_calc.py` lignes 58-99) :

Symbole	Signification	Unité
GF	Facteur de jauge (Gage Factor)	sans dimension
V_{ex}	Tension d'excitation du pont (Input Tension)	V
$Gain$	Gain de l'amplificateur	sans dimension
T	Température de fonctionnement de la jauge	K
$SG_{T,\%}$	Coefficient de température du facteur de jauge, saisi en %/K	%/K
ν_b	Coefficient de Poisson de la barre (Half/Full uniquement)	sans dimension

Une exception `ValueError` est levée si GF, V_{ex}, $Gain$ ou T vaut zéro (ligne 75-76), et pour Half/Full si ν_b vaut zéro (ligne 88-89, car ν_b intervient au dénominateur).

La température de référence à laquelle le facteur de jauge est spécifié est codée en dur :

$$T_{ref} = 293 \text{ K} \quad (\text{`_REFERENCE_TEMPERATURE`, ligne 49})$$

1.3 Terme thermique commun

Le coefficient de température du facteur de jauge est d'abord converti de %/K en fraction (ligne 78) :

$$SG_T = \frac{SG_{T,\%}}{100}$$

Un terme "thermal" commun aux trois montages est ensuite calculé (`compute_cpg`, ligne 79) :

$$V_{th} = GF \cdot \left[1 + SG_T \cdot (T - T_{ref})\right] \cdot V_{ex}$$

Ce terme représente la tension effective de pont, corrigée de la dérive thermique du facteur de jauge.

1.4 Formule du coefficient CPG par type de pont

Quarter-bridge (`core/cpg_calc.py`, lignes 81-82) :

$$CPG_{quarter} = \frac{4}{V_{th} \cdot Gain}$$

Half-bridge (`factor = 4`, lignes 90-91) :

$$CPG_{half} = \frac{4}{V_{th}\,(1 + \nu_b)\,Gain}$$

Full-bridge (`factor = 2`, lignes 90-91) :

$$CPG_{full} = \frac{2}{V_{th}\,(1 + \nu_b)\,Gain}$$

Ces trois formules ont pour unité l'inverse d'un volt (1/V), affiché tel quel dans l'interface (`page_cpg_calc.py` ligne 416 : `"Coefficient = {value:.6g} ± {std:.6g} (1/V)"`).

1.5 Propagation d'incertitude

L'écart-type du coefficient CPG est obtenu par propagation d'erreur en quadrature sur les écarts-types **relatifs** de chaque entrée (hypothèse implicite d'indépendance statistique des entrées et de linéarisation au premier ordre — non documentée explicitement dans le code, mais standard pour ce type de calcul).

Quarter-bridge (lignes 83-86) :

$$\left(\frac{\sigma_{CPG}}{CPG}\right)^2 = \left(\frac{\sigma_{GF}}{GF}\right)^2 + \left(\frac{\sigma_{V_{ex}}}{V_{ex}}\right)^2 + \left(\frac{\sigma_{Gain}}{Gain}\right)^2 + \left(\frac{\sigma_T}{T}\right)^2$$

Half-/Full-bridge (lignes 92-96), terme additionnel en ν_b :

$$\left(\frac{\sigma_{CPG}}{CPG}\right)^2 = \left(\frac{\sigma_{GF}}{GF}\right)^2 + \left(\frac{\sigma_{V_{ex}}}{V_{ex}}\right)^2 + \left(\frac{\sigma_{Gain}}{Gain}\right)^2 + \left(\frac{\sigma_T}{T}\right)^2 + \left(\frac{\sigma_{\nu_b}}{\nu_b}\right)^2$$

Dans les deux cas (ligne 98) :

$$\sigma_{CPG} = CPG \cdot \sqrt{\sum \left(\frac{\sigma_{x_i}}{x_i}\right)^2}$$

Point à noter (ambiguïté potentielle du code) : le terme de température utilise directement σ_T/T (l'écart-type relatif de la température elle-même), alors que la dépendance de CPG à T passe par le facteur $\left[1 + SG_T(T - T_{ref})\right]$ et n'est donc **pas** strictement proportionnelle à T (sauf si $SG_T = 0$). La formule appliquée est une propagation en quadrature "par facteur multiplicatif" simplifiée, cohérente avec un portage littéral du code d'origine, mais qui ne correspond pas à une propagation d'erreur rigoureuse à travers le terme $\left[1 + SG_T(T - T_{ref})\right]$ (qui donnerait un facteur $\frac{SG_T \sigma_T}{1+SG_T(T-T_{ref})}$ plutôt que σ_T/T). Ceci n'est pas documenté comme une approximation volontaire dans le code ; il s'agit vraisemblablement d'un héritage direct de la formule de l'application d'origine. De même, l'incertitude sur $SG_{T,\%}$ lui-même n'est pas prise en compte (aucun `sg_t_std` en entrée) : le coefficient de température du facteur de jauge est traité comme exact.

1.6 Interface / synchronisation

La page `ui/pages/page_cpg_calc.py` permet de sélectionner le type de pont (affichage/masquage du champ "Bar Poisson Ratio" selon `USES_POISSON_RATIO`), de saisir les entrées ci-dessus (moyenne + écart-type), de calculer via `compute_cpg()`, puis — bouton "Sync to bar's Strain Gage Bridge Coefficient" — d'écrire directement le résultat (`mean`, `dispersion`, distribution `NORMAL`) dans le paramètre `SG_coeff` de la barre d'entrée ou de sortie du projet (`_on_sync`, lignes 472-488).

1.7 Correction de sensibilité transverse (optionnelle)

Fonction : `transverse_sensitivity_correction()` (`core/cpg_calc.py`)

Une jauge de contrainte répond, dans une proportion moindre, à la déformation perpendiculaire à son axe de mesure — sa sensibilité transverse K_t, indiquée sur la fiche technique du fabricant (typiquement autour de 0.5-1 %, la valeur "TRANSVERSE SENSITIVITY" de l'emballage). Cette sensibilité biaise la déformation indiquée par rapport à la déformation axiale réelle.

Pour une jauge axiale unique sous un champ de contrainte uniaxial — le cas usuel de cette application (jauge collée le long de l'axe d'une barre) — la correction s'appuie sur deux relations de la note technique Vishay TN-509 ("Errors Due to Transverse Sensitivity in Strain Gages") :

1. Le facteur de jauge annoncé par le fabricant F est lui-même calibré sur une éprouvette en traction uniaxiale, où la déformation transverse vaut $-\nu_0\,\varepsilon_a$ (ν_0 le coefficient de Poisson de calibration, 0.285 pour les jauges Micro-Measurements/Vishay) : $F = F_a(1 - \nu_0 K_t)$, avec F_a le facteur de jauge axial "vrai" de la jauge.
2. Sur la barre mesurée, sous contrainte uniaxiale avec un coefficient de Poisson réel ν (le champ "Bar Poisson Ratio" déjà présent sur cette page, réutilisé tel quel), la déformation transverse réelle vaut $-\nu\,\varepsilon_a$.

En combinant les deux, la déformation axiale réelle se déduit de la déformation indiquée par :

$$\varepsilon_a = \varepsilon_{ind} \cdot \frac{1 - \nu_0 K_t}{1 - \nu K_t}$$

Puisque $\varepsilon = CPG \times V_{bridge}$, appliquer ce même facteur multiplicatif directement au coefficient CPG

produit une déformation corrigée pour n'importe quelle tension de pont mesurée par la suite :

$$CPG_{corr} = CPG \cdot \frac{1 - \nu_0 K_t}{1 - \nu K_t}$$

Le facteur est appliqué de façon identique quel que soit le type de pont sélectionné, en approximation de premier ordre (cohérent avec le reste de ce module, documenté mais non revalidé indépendamment). La case "Bar Poisson Ratio", habituellement masquée en Quarter-bridge (§1.1), redevient visible dès que cette correction est activée, puisqu'elle réutilise ce même champ pour ν.

L'incertitude est propagée en combinant, en quadrature relative, l'incertitude déjà propagée sur CPG (§1.5) avec celle du facteur de correction lui-même — cette dernière obtenue par dérivation analytique du facteur par rapport à K_t et ν (le coefficient de calibration ν_0 est traité comme une constante exacte, sans incertitude propre) :

$$\frac{\partial(\text{facteur})}{\partial K_t} = \frac{\nu - \nu_0}{(1 - \nu K_t)^2} \qquad \frac{\partial(\text{facteur})}{\partial \nu} = \frac{(1 - \nu_0 K_t)\, K_t}{(1 - \nu K_t)^2}$$

1.8 Correction de sortie thermique (optionnelle, informative)

Fonction : `thermal_output_strain()` (`core/cpg_calc.py`)

Distincte du coefficient de température du facteur de jauge (§1.2, qui corrige la SENSIBILITÉ de la jauge), la "sortie thermique" ("thermal output") est la déformation apparente qu'une jauge rapporte du seul fait d'un changement de température, sans aucune charge mécanique — causée par la différence de dilatation thermique entre la grille de la jauge et le matériau de l'éprouvette. Le fabricant fournit, pour chaque combinaison jauge/matériau (la jauge étant appairée à un matériau donné, par ex. acier 1018), un polynôme de température dont les coefficients sont imprimés sur l'emballage, typiquement du 3ème au 7ème ordre.

La carte "Thermal Output Correction" permet de saisir le degré du polynôme (menu déroulant, 3 à 7), l'unité de température (°C ou °F, pour correspondre au jeu de coefficients utilisé — le fabricant fournit généralement les deux), les coefficients $a_0 \ldots a_n$ eux-mêmes (en microdéformation par unité^i), ainsi que la température de référence (celle à laquelle le pont a été équilibré/mis à zéro) et la température de fonctionnement (celle du test).

Selon la note technique Vishay TN-504-1 ("Strain Gage Thermal Output and Gage Factor Variation with Temperature"), équation 3, la déformation apparente à corriger est la valeur du polynôme à la température de fonctionnement MOINS sa valeur à la température de référence — pas seulement sa valeur à la température de fonctionnement seule, puisque toute sortie thermique déjà présente au moment de la mise à zéro est déjà annulée par cette mise à zéro elle-même :

$$\varepsilon_{T/O} = P(T_{op}) - P(T_{ref}) \qquad P(T) = \sum_{i=0}^{n} a_i\, T^i$$

Contrairement à la correction de sensibilité transverse (§1.7), cette correction porte sur la déformation elle-même (un décalage additif), pas sur le coefficient CPG (un facteur multiplicatif) : elle n'est donc **pas** repliée

dans le coefficient synchronisé vers `SG_coeff`. Le résultat est affiché à titre informatif (en microdéformation, avec sa tension de décalage équivalente $\varepsilon_{T/O} \times 10^{-6}/CPG$), à appliquer manuellement par l'utilisateur — typiquement pertinent lorsque la mise à zéro du pont a eu lieu à une température différente de celle du test lui-même.

Cette correction statique suppose une température de fonctionnement **fixe** pendant tout l'essai. Pour un cas où la température varie réellement pendant l'essai (échauffement adiabatique d'une éprouvette lors d'un essai dynamique rapide, ou dérive de température ambiante lors d'un essai quasi-statique lent), la page Custom Sensors propose une version **variable dans le temps** de cette même correction, appliquée directement au signal du capteur plutôt que seulement rapportée — voir chapitre 1 §4.4.

---

2. Synchronisation caméra / vidéo

Fichier source : `core/camera_clock.py` **Page UI :** `ui/pages/page_video_sync.py` **Abstraction vidéo :** `core/frame_source.py` (voir §6)

2.1 Contexte

Cet outil permet de localiser, dans une vidéo ou séquence d'images haute vitesse, la frame exacte exposée à l'instant de déclenchement (trigger) du système d'acquisition, et de lui associer un temps absolu sur la même base de temps que le reste des signaux enregistrés de l'essai. Il s'appuie sur un fichier d'horloge ("clock file") enregistré en parallèle par la caméra, contenant :

- un vecteur de temps absolu (même base que les autres signaux de l'essai) ;
- le signal d'exposition ("clock") de la caméra : une impulsion par frame ;
- optionnellement, un signal de trigger : un front marquant l'instant où le déclenchement externe a été reçu.

2.2 Lecture du fichier d'horloge

`read_clock_file()` (`core/camera_clock.py`, lignes 36-88) lit un fichier texte délimité par espace/virgule/point-virgule/tabulation, en détectant automatiquement :

- le délimiteur (celui qui découpe les 10 premières lignes non vides/non-commentées en un nombre de champs constant et maximal — un délimiteur qui coïncide accidentellement avec un séparateur décimal, par ex. une virgule française, est ainsi écarté au profit du délimiteur qui donne le plus de colonnes cohérentes, lignes 57-65) ;
- une éventuelle ligne d'en-tête (si la première ligne n'est pas entièrement numérique) ;
- le format décimal à virgule française (remplacement `,` → `.` avant conversion en `float`, ligne 101).

Les lignes non numériques (hors en-tête) sont simplement ignorées plutôt que de provoquer une erreur.

`decimate_for_display()` (lignes 107-142) réduit le nombre de points affichés (~20000 par défaut) pour le tracé, en conservant le min et le max de chaque bin plutôt qu'un sous-échantillonnage uniforme — ceci pour ne pas manquer d'impulsions étroites sur fond plat lors de l'affichage. **Cette réduction n'affecte que**

le tracé : la détection de pics et de trigger s'exécute toujours sur les tableaux pleine résolution (docstring, lignes 118-120).

2.3 Détection du train d'impulsions d'exposition

Fonction : `detect_pulse_train()` (`core/camera_clock.py`, lignes 151-174).

Le signal d'exposition de la caméra est une impulsion périodique (une par frame). La détection utilise `scipy.signal.find_peaks` avec deux critères :

Distance minimale entre pics acceptés — calculée en nombre d'échantillons à partir d'un intervalle d'échantillonnage médian dt (`_estimate_dt`, lignes 145-148, médiane des écarts de temps positifs) :

$$d_s = \begin{cases} 0.5 \times T_{expected} & \text{si une période attendue } T_{expected} \text{ est fournie} \\ 2 \times dt & \text{sinon} \end{cases}$$

$$d_{samples} = \max\left(1,\ \text{round}\left(\frac{d_s}{dt}\right)\right)$$

La période attendue $T_{expected}$ (`expected_period_s`), quand elle est connue (fréquence d'enregistrement saisie, fps de la vidéo chargée, ou durée du fichier d'horloge divisée par le nombre de frames chargées — voir `_expected_period_s()` dans `page_video_sync.py`, lignes 502-525), fixe l'écart minimal entre deux pics acceptés à **la moitié** de cette période. Sans elle, seul un écart minimal de sécurité ($2\,dt$) est imposé, ce qui expose les signaux bruités à une sur-détection (docstring, lignes 158-161).

Prominence minimale — si non fournie, fixée à 30 % de l'étendue crête-à-crête du signal (lignes 169-171) :

$$\text{prominence} = 0.3 \times (\max(signal) - \min(signal))$$

Les indices d'échantillons des pics détectés et leurs temps correspondants sont retournés.

2.4 Estimation du temps absolu de chaque frame

Fonction : `estimate_frame_times()` (lignes 177-207).

Pour chaque frame $0..n_{frames} - 1$ de la vidéo/séquence chargée :

1. Les $n_{known} = \min(n_{\text{peaks_détectés}},\ n_{frames})$ premières frames reçoivent directement le temps du pic d'exposition détecté correspondant (association par rang, frame i <-> pic i).
2. Au-delà (frames sans impulsion détectée en correspondance, généralement en fin de séquence), le temps est **extrapolé à période constante** :
   - la période utilisée est $T_{expected}$ si fournie, sinon la médiane des écarts entre les temps des pics détectés eux-mêmes :

$$T_{\text{période}} = \text{median}(\text{diff}(peak_times[:n_{known}]))$$

- pour chaque frame $i \geq n_{known}$:

$$t_i = t_{n_{known}-1} + (i - (n_{known} - 1)) \times T_{\text{période}}$$

3. Si aucune période n'est disponible (aucun pic détecté, et aucune période fournie), les temps non déterminables restent `NaN`.

C'est cette fonction qui alimente le repère "time ≈ …" affiché sous la frame courante et la ligne verticale tracée sur chaque canal de mesure affiché.

2.5 Détection d'un front montant (signal de trigger à palier unique)

Fonction : `detect_rising_edge()` (lignes 210-221) : premier échantillon où le signal passe de "au niveau ou en-dessous" à "au-dessus" d'un seuil (par défaut, le point médian entre min et max du signal) — utile pour un signal de trigger qui bascule une seule fois (échelon), par opposition à un signal impulsionnel périodique.

$$\text{seuil} = \frac{\min(signal) + \max(signal)}{2} \quad \text{(si non fourni explicitement)}$$

Retourne l'index du premier échantillon `> seuil` (via `np.argmax` sur le masque booléen), ou `None` si le signal ne dépasse jamais le seuil.

2.6 Méthode 1 — Localisation du trigger par signal de trigger

Fonction : `locate_trigger_by_signal()` (lignes 253-280).

1. Détection du train d'impulsions d'exposition (§2.3) sur le signal d'exposition.
2. Détection du front montant du signal de trigger (§2.5), donnant `trigger_time`.
3. Association de l'index d'échantillon du trigger (`trigger_idx`) à l'un des index d'échantillons des pics d'exposition détectés (`peak_indices`), via `_nearest_index()` (lignes 224-236), selon le paramètre `match` (trois choix, exposés dans l'UI comme "Match the trigger edge to") :
   - **`"nearest"`** (par défaut) : le pic d'exposition dont l'index est le plus proche de `trigger_idx` (comparaison des deux candidats immédiatement avant/après par recherche binaire `np.searchsorted`, et sélection du plus proche en valeur absolue).
   - **`"after"`** : le **premier** pic d'exposition dont l'index est ≥ `trigger_idx` (borné à la dernière frame si le trigger survient après le dernier pic détecté).
   - **`"before"`** : le **dernier** pic d'exposition dont l'index est ≤ `trigger_idx` (borné à la première frame si le trigger survient avant le premier pic).

   Remarque d'implémentation : la correspondance est effectuée en espace d'**index d'échantillon** (`peak_indices`, `trigger_idx` — des entiers, pas des temps), ce qui suppose implicitement un échantillonnage à peu près régulier du fichier d'horloge pour que la proximité en index reflète correctement la proximité en temps ; ce n'est pas explicitement vérifié par le code.

Si aucun front montant n'est trouvé dans le signal de trigger, ou si aucune impulsion d'exposition n'est détectée, un avertissement est ajouté et `frame_index`/`frame_time` restent `None` (lignes 268-276). Un

avertissement est également émis si le nombre de pics détectés diffère du nombre de frames réellement chargées (lignes 261-265), invitant l'utilisateur à vérifier visuellement la détection sur le graphique.

2.7 Méthode 2 — Localisation du trigger par numéro de frame connu + fréquence

Fonction : `locate_trigger_by_frame_number()` (lignes 283-335).

Utilisée quand le fichier d'horloge ne contient pas de colonne de trigger, mais que le numéro de frame déclenchée est connu directement (rapporté par la caméra elle-même), avec sa fréquence d'enregistrement f (Hz).

- Si `expected_period_s` n'est pas fourni mais que f l'est, il est déduit : $T_{expected} = 1/f$ — utilisé comme indice pour la détection des pics (§2.3).
- L'impulsion d'exposition ciblée est `target = frame_number + frame_offset`, où `frame_offset` (par défaut 0) compense d'éventuelles impulsions enregistrées avant la première frame effectivement chargée (ex. tampon circulaire pré-trigger non sauvegardé) : l'impulsion utilisée pour la frame `frame_number` est ainsi la `target`-ième impulsion détectée, et non nécessairement la `frame_number`-ième.

Cas 1 — l'impulsion cible est dans la plage détectée ($0 \leq target < n_{peaks}$) :

$$t_{frame} = peak_times[target] \qquad \text{(méthode : "exposure pulse count")}$$

Cas 2 — l'impulsion cible tombe hors des impulsions détectées, et f est connue — extrapolation à fréquence constante :

$$t_{base} = \begin{cases} peak_times[0] - \dfrac{frame_offset}{f} & \text{si au moins un pic a été détecté} \\ time[0] & \text{sinon} \end{cases}$$

$$t_{frame} = t_{base} + \frac{target}{f} \qquad \text{(méthode : "frequency extrapolation")}$$

Un avertissement est systématiquement ajouté dans ce cas pour signaler que le temps a été extrapolé plutôt que lu directement sur le vecteur de temps (lignes 319-323).

Cas 3 — hors plage et fréquence non fournie : aucun temps n'est calculable ; un avertissement invite à saisir la fréquence de la caméra (lignes 326-329), et `frame_time` reste `None`.

2.8 Résultat retourné

`TriggerResult` (lignes 239-250) rassemble : `frame_index`, `frame_time`, `trigger_time` (front brut du signal de trigger, quand applicable), `n_peaks_detected`, `peak_times`/`peak_sample_indices` complets, la méthode utilisée (chaîne libre : `"trigger signal"`, `"exposure pulse count"` ou `"frequency extrapolation"`), et une liste d'avertissements — affichés tels quels dans l'interface pour que l'utilisateur puisse juger de la fiabilité du résultat.

2.9 Limitations notées dans le code

- La correspondance trigger<->pic (méthode signal) se fait en espace d'index, pas en temps réel — implicite hypothèse de régularité d'échantillonnage (non vérifiée par assertion).
- Sans période attendue, la détection de pics peut sur-détecter sur un signal bruité (docstring `detect_pulse_train`).
- `estimate_frame_times` renvoie `NaN` pour toute frame dont le temps ne peut ni être lu ni être extrapolé.
- Le nombre de pics détectés qui diffère du nombre de frames chargées ne bloque jamais le calcul, seulement un avertissement (l'utilisateur doit vérifier le graphique).

---

3. Analyse par lot (Batch Analysis)

Fichier source : `core/batch_analysis.py` **Page UI :** `ui/pages/page_batch_analysis.py`

3.1 Contexte

Combine plusieurs essais déjà dépouillés (chacun exporté en CSV/TXT depuis la page "Results & Export", toute quantité calculée étant une colonne nommée par son nom interne) en un résumé statistique unique sur un axe X commun. Portage littéral de `analysis_resume.py` (`analysis_tests()`) — **non revalidé indépendamment** contre l'application d'origine (docstring, lignes 35-37). X et Y ne sont pas figés à déformation/contrainte : n'importe quelle colonne exportée peut être choisie pour l'un ou l'autre (ex. `confinement_pressure` en X, `confinement_deviator` en Y).

Trois bandes de variabilité sont calculées autour de la courbe moyenne :

- **"Experimental"** : RMS de l'incertitude Monte-Carlo propre à chaque essai (colonne `<Y>_std`, si présente dans le fichier exporté — sinon zéro).
- **"Material"** : dispersion essai-à-essai autour de la moyenne, ajustée comme une loi normale sur le ratio (valeur de cet essai / valeur moyenne) à chaque niveau de X — sauf si remplacée par un pourcentage fixe imposé par l'utilisateur.
- **"Total"** : combinaison quadratique des deux précédentes.

3.2 Chargement des fichiers

`load_export_header()` (lignes 88-109) lit uniquement la ligne d'en-tête d'un fichier exporté (délimiteur déduit de l'extension : `,` pour `.csv`, tabulation pour `.txt` — `_sniff_delimiter`, lignes 80-85), pour construire les listes déroulantes X/Y de l'UI. `_load_export_columns()` (lignes 112-136) charge les colonnes demandées (`x`, `y`, et `y_std` optionnelle) d'un fichier. Une colonne `<y_column>_std`, si présente dans l'en-tête, est automatiquement utilisée comme incertitude "Experimental" de cet essai sur Y — aucun sélecteur séparé n'est nécessaire (`run_batch_analysis_from_files`, lignes 160-164).

3.3 Correction isotherme (échauffement adiabatique)

Paramètres : `IsothermalSettings` (lignes 50-55) — `xi` (coefficient de Taylor-Quinney), `cp` (chaleur spécifique, J/(kg·K)), `rho` (masse volumique, kg/m³), `alpha` (sensibilité en température, MPa/K). Appliquée,

si activée, au Y brut de **chaque essai individuellement, avant tout rééchantillonnage** (`_combine_tests`, lignes 240-248) — n'est pertinente que lorsque Y est une contrainte et X une déformation (docstring module, ligne 19).

Pour un essai de longueur n (tableaux `x`, `y`), le code calcule (lignes 243-247) :

$$k = \frac{\xi}{c_p\,\rho}$$

Élévation de température cumulée ΔT, par intégration en escalier (rectangle à gauche) — $\Delta T_0 = 0$ puis, pour $g = 0..n-2$:

$$\Delta T_{g+1} = \Delta T_g + k \cdot y_g \cdot (x_{g+1} - x_g) \cdot 10^6$$

(le facteur 10^6 suggère une conversion de y exprimé en MPa vers Pa, pour obtenir une densité d'énergie en J/m³ cohérente avec $c_p\rho$ en J/(kg·K)·kg/m³ — non commentée explicitement dans le code).

Correction isotherme appliquée à Y :

$$y_g^{corr} = y_g + \alpha \cdot \Delta T_g \qquad \text{pour } g = 0..n-2$$

et pour le dernier point de l'essai (indice $n-1$) :

$$y_{n-1}^{corr} = y_{n-2} + \alpha \cdot \Delta T_{n-1}$$

Point à signaler explicitement : la valeur corrigée du **dernier** point réutilise y_{n-2} (l'avant-dernière valeur **brute**, non corrigée) au lieu de y_{n-1} (la dernière valeur brute). Ceci ressort de la ligne 247 telle qu'écrite (`[... for g in range(len(y)-1)] + [y[-2] + iso.alpha*dT[-1]]`) et a très vraisemblablement pour origine un portage littéral d'une particularité (bug ou choix délibéré non documenté) du code d'origine ; ce n'est pas expliqué dans les commentaires et n'a pas été réinterprété ici. À vérifier auprès de l'auteur d'origine si une clarification est nécessaire pour la documentation finale.

3.4 Rééchantillonnage sur une grille commune

Décidé essai par essai selon la monotonie de X (`_backtrack_fraction`, lignes 178-190) : la fraction de l'étendue de X sur laquelle X redescend sous son propre maximum courant.

$$f_{backtrack} = \frac{\max_i \left(\text{running_max}(x)_i - x_i\right)}{\max(x) - \min(x)}$$

Si $f_{backtrack} \leq 0.15$ (`_BACKTRACK_TOLERANCE`, ligne 197) pour **tous** les essais, X est jugée "quasi-monotone" (bruit de mesure typique de quelques %) et la méthode **`"target_x"`** est utilisée ; sinon (ex. chemin de contrainte avec décharge/rechargement), la méthode **`"index_fraction"`** est utilisée.

Méthode `"target_x"` (X se comporte comme une déformation, lignes 253-268) :

- Grille commune de n_{min} points (n_{min} = longueur du plus court essai), sur 95 % du X maximal du **plus petit** essai (le plus petit des X maximaux parmi tous les essais) :

$$x_{max}^{target} = 0.95 \times \min_{\text{essais}} \big(\max(x) \big) \qquad x_i^{target} = i \cdot \frac{x_{max}^{target}}{n_{min}},\ i = 0..n_{min} - 1$$

- Pour chaque essai et chaque point cible x^{target}, le code cherche le **dernier index dont X est strictement inférieur à la cible** (recherche simple, pas d'interpolation linéaire — reproduit fidèlement l'algorithme "last index below target" de l'application d'origine, docstring lignes 255-256) :

$$\text{indice retenu} = \max\{\, i : x_i < x^{target} \,\}$$

Cas particulier noté dans le code : si aucun index de X n'est strictement inférieur à la cible (typiquement pour $x^{target} = 0$, la première cible), l'indice **1** est utilisé par défaut (`below[-1] if len(below) else 1`, ligne 266) plutôt que l'indice 0 — cette valeur de repli n'est pas commentée dans le code et constitue potentiellement une particularité héritée de l'application d'origine plutôt qu'un choix délibérément documenté. - La valeur Y (et Y_std) retenue à chaque point cible est directement celle de l'échantillon à l'indice trouvé, arrondie à 6 décimales (`round(..., 6)`) — il ne s'agit donc pas d'une interpolation linéaire mais d'un plus-proche-voisin "en escalier vers le bas".

Méthode `"index_fraction"` (X non monotone, lignes 269-284, via `_resample_by_fraction`, lignes 200-213) :

- Chaque essai est rééchantillonné sur n_{min} points **également espacés le long de son propre ordre d'enregistrement** (fraction d'index $0..1$), par **interpolation linéaire** de X et de Y contre cette fraction :

$$\text{frac}_i = \frac{i}{n-1},\ i = 0..n-1 \qquad \text{target_frac}_j = \frac{j}{n_{min}-1},\ j = 0..n_{min}-1$$

$$x_j^{res} = \text{interp}(\text{target_frac}_j;\ \text{frac},\ x) \qquad y_j^{res} = \text{interp}(\text{target_frac}_j;\ \text{frac},\ y)$$

- Chaque point reste dans la plage réellement enregistrée de cet essai (aucune extrapolation).
- L'axe X commun rapporté est alors la **moyenne**, point par point, des x^{res} de tous les essais :

$$x_j^{target} = \frac{1}{N_{tests}} \sum_k x_{k,j}^{res}$$

3.5 Courbe moyenne et bandes de variabilité

Une fois tous les essais rééchantillonnés sur la grille commune (matrices y_{arr}, y_std_{arr} de forme (N_{tests}, n_x)) — `_combine_tests`, lignes 287-317 :

Courbe moyenne :

$$\bar{y}(x) = \frac{1}{N_{tests}} \sum_{k=1}^{N_{tests}} y_k(x)$$

Variabilité "Experimental" — RMS des écarts-types Monte-Carlo propres à chaque essai, à chaque point de la grille :

$$\sigma_{exp}(x) = \sqrt{\frac{1}{N_{tests}} \sum_{k} \sigma_{y,k}(x)^2}$$

Variabilité "Material" — à chaque point de la grille, le ratio $r_k = y_k(x)/\bar{y}(x)$ est calculé pour chaque essai k, puis une loi normale est ajustée sur l'ensemble de ces ratios (`scipy.stats.norm.fit`), donnant un écart-type σ_{fit} ; la variabilité "Material" est alors :

$$\sigma_{mat}(x) = n_\sigma \cdot \sigma_{fit}(x) \cdot \bar{y}(x)$$

où n_σ est le paramètre `sigma` (nombre d'écarts-types Monte-Carlo du projet, `sigma=self.project.uncertainty_sigma` passé depuis l'UI).

Option "Fixed Y variability override" (`fixed_pct`, si défini par l'utilisateur) : remplace **entièrement** le calcul ci-dessus par un pourcentage fixe de la moyenne (ligne 303-304), utile quand le nombre d'essais est trop faible pour que la dispersion mesurée soit statistiquement significative (aide contextuelle de l'UI) :

$$\sigma_{mat}(x) = \frac{fixed_pct}{100} \times \bar{y}(x)$$

Variabilité "Total" — combinaison quadratique :

$$\sigma_{total}(x) = \sqrt{\sigma_{mat}(x)^2 + \sigma_{exp}(x)^2}$$

Variabilités relatives (pour la vue "Relative Uncertainty" de l'UI), en pourcentage de la moyenne :

$$\sigma_{total,\%}(x) = 100 \cdot \frac{\sigma_{total}(x)}{\bar{y}(x)}, \quad \sigma_{exp,\%}(x) = 100 \cdot \frac{\sigma_{exp}(x)}{\bar{y}(x)}, \quad \sigma_{mat,\%}(x) = 100 \cdot \frac{\sigma_{mat}(x)}{\bar{y}(x)}$$

(calculées avec `np.errstate(divide="ignore", invalid="ignore")`, donc des `inf`/`NaN` peuvent apparaître silencieusement si $\bar{y}(x) = 0$ en un point).

3.6 Export

`export_batch_result()` (lignes 321-329) écrit dans un dossier : `x.txt`, `mean_y.txt`, `material_variability_y_std.txt`, `total_y_std.txt`, `experimental_variability_y_std.txt`, et `axes.txt` (noms des colonnes X/Y utilisées).

3.7 Limitations et remarques notées dans le code

- Non revalidé indépendamment contre l'application d'origine.
- Le seuil de bascule monotone/non-monotone (15 %) est un choix empirique du code, pas dérivé d'un critère physique.
- La méthode `"target_x"` utilise une recherche de "dernier point sous la cible" (pas d'interpolation), donc introduit une erreur de discrétisation de l'ordre du pas d'échantillonnage de chaque essai — comportement délibérément reproduit à l'identique de l'application d'origine.
- Cas particulier de repli à l'indice 1 (plutôt que 0) quand aucun point de X n'est sous la cible (§3.4), non documenté dans le code source.
- La correction isotherme traite le dernier point de la courbe de façon asymétrique par rapport aux autres (réutilisation de y_{n-2}, §3.3) — comportement noté ici mais non expliqué dans les commentaires du code source.

---

4. Sous-échantillonnage (`core/downsampling.py`)

Fichier source : `core/downsampling.py` (20 lignes)

Fonction unique `downsample(values, n)` (lignes 14-20) : **décimation simple** (garde 1 échantillon sur n), sans moyennage ni filtrage anti-repliement. Pour un tableau de longueur L et un facteur $n > 1$:

$$\text{count} = \left\lfloor \frac{L}{n} \right\rfloor \qquad \text{values}^{downsampled} = \left[\text{values}[i \cdot n] \text{ pour } i = 0..\text{count} - 1\right]$$

Si $n \leq 1$, le tableau est retourné inchangé. Nommé `N_cut` dans l'application d'origine (`process_t.py`), reproduit à l'identique : `[arr[int(N*y)] for y in range(int(len(arr)/N))]` (docstring, lignes 1-8). Appliqué uniformément à tous les signaux bruts d'un essai (signaux de barres, temps/charge/déplacement, vitesse, capteurs personnalisés) afin qu'ils restent synchronisés entre eux après réduction — utilisé pour accélérer le traitement de fichiers de mesure très volumineux.

---

5. Abstraction vidéo / séquence d'images (`core/frame_source.py`)

Fichier source : `core/frame_source.py` (191 lignes) — module technique, sans calcul physique.

Fournit une interface commune (`FrameSource` : `n_frames`, `get_frame(i)`, `fps`) pour deux sources : `VideoFrameSource` (fichier MP4/MKV/AVI décodé à la volée via OpenCV, avec comptage de frames par scan de secours si l'index du conteneur ne le fournit pas — `_count_frames_by_scanning`) et `ImageSequenceFrameSource` (dossier d'images TIFF/JPEG/PNG/BMP, triées par ordre naturel des noms de fichiers via `_natural_sort_key` pour que "frame2" précède "frame10"). La fonction `to_display_uint8()` convertit toute frame (8/16-bit, niveaux de gris ou RGB) en image 8-bit affichable, avec étirement de contraste entre les 0,5e et 99,5e percentiles des valeurs de la frame par défaut (les capteurs

16-bit haute vitesse n'utilisent typiquement qu'une petite plage de la dynamique complète). Ce module n'implique aucune formule physique de dépouillement.

---

6. Export vidéo annoté DIC (`core/dic_video_export.py`)

Fichier source : `core/dic_video_export.py` (179 lignes) — module de rendu, sans calcul physique nouveau (les champs affichés sont ceux déjà calculés par la corrélation d'images, voir `core/dic_processing.py/core/dic_r`

Génère une vidéo MP4 (codec `mp4v` via OpenCV) à partir de la séquence de résultats DIC déjà traitée : chaque frame combine le fond de la caméra avec les isocontours du champ choisi tracés à la position **réellement suivie** de chaque point (maillage déformé, pas la grille de référence fixe), sur fond blanc et à résolution fixe (1920×1440 px par défaut), avec option d'un second panneau affichant en direct l'évolution temporelle de la moyenne ROI du champ. L'échelle de couleur (`vmin`/`vmax`) et les axes du panneau temporel sont des constantes fournies par l'appelant (non recalculées par frame) afin que les frames consécutives restent visuellement comparables. Le rendu utilise une figure matplotlib Agg pure (non Qt), ce qui le rend sûr à exécuter hors du thread principal.

---

7. Calibration bayésienne de lois de comportement — portage de MAMBA (`core/calib_*.py`, `ui/pages/page_bayes_calib.py`)

7.1 Contexte : MAMBA et le choix de portage

MAMBA est un logiciel Tkinter + pymc3 antérieur, indépendant de HYDRA, qui ajuste une équation de loi de comportement en texte libre (en déformation `e`, vitesse de déformation `v`, température `T`, et jusqu'à 3 variables supplémentaires optionnelles `a`/`b`/`d`) à des données contrainte-déformation expérimentales par calibration bayésienne — simple (mise en commun/« pooled ») ou hiérarchique (effets aléatoires par essai, avec mise en commun partielle), avec une vraisemblance multi-normale utilisant une matrice de covariance mesurée. L'onglet "Bayesian Calibration" (section "Calibration" de la barre latérale) en est un portage dans HYDRA, sur **PyMC 5 + PyTensor** plutôt que l'ancien pymc3/Theano de MAMBA — un choix assumé malgré son coût (bibliothèques plus lourdes, empaquetage PyInstaller plus délicat, question explicitement mise de côté pour plus tard).

Contrairement à MAMBA, qui charge des dossiers de fichiers `.txt` par essai, chaque variable de l'équation (et la grandeur modélisée/de sortie) est assignée, par essai de calibration, à une paire (grille, courbe moyenne, matrice de covariance) préalablement enregistrée dans la bibliothèque de calibration de la page Covariance Matrix (chapitre 1, §3.5) — c'est ce qui permet de construire un jeu de calibration hiérarchique multi-essais entièrement à l'intérieur d'une session HYDRA : traiter un essai, en sauvegarder les grandeurs utiles, traiter le suivant, etc.

Plusieurs limites relevées dans le code d'origine de MAMBA ont été corrigées plutôt que reproduites à l'identique — reprises en détail au fil de cette section et récapitulées en §7.10 : un analyseur d'équation fragile (texte découpé deux fois, de façon incohérente, puis passé à `eval()`) remplacé par un arbre `ast`

validé une seule fois ; une “validation” qui ne réévaluait en réalité jamais le modèle aux conditions de l’essai de validation (elle ne faisait que superposer l’ajustement de calibration) ; une covariance de vraisemblance figée sur la moyenne inter-essais même quand la covariance par essai était disponible ; l’absence totale de diagnostic de convergence (R-hat/ESS) ; un `target_accept` du échantillonneur codé en dur et caché à l’utilisateur.

7.2 Analyseur et évaluateur d’équation (`core/calib_equation.py`)

MAMBA analysait le texte de l’équation **deux fois, indépendamment** : une fois par découpage de chaîne ad hoc pour détecter les noms de paramètres libres (`def_model.py::recup_expr` — code mort, l’expression `sympy` construite n’était en réalité jamais réutilisée), et une fois par substitution littérale `str.replace()` de chaque nom détecté par une chaîne indexée, passée à `eval()` (`cal_MCMC`) — fragile (un nom de paramètre substring d’un autre token pouvait corrompre la substitution silencieusement ; une notation scientifique comme `1e-3` risquait de voir son `e` confondu avec la variable de déformation) et dépendant d’une ancienne interopérabilité Theano/numpy qui ne se transpose pas à PyTensor.

`core/calib_equation.py` analyse le texte **une seule fois**, dans un arbre `ast` (`ast.parse(expr, mode="eval")`) restreint à une liste blanche explicite — puisqu’il s’agit de texte libre saisi par l’utilisateur, cette liste blanche est une véritable frontière de sécurité, pas seulement un choix de style : `^` est d’abord traduit en `**` (syntaxe de MAMBA et du manuel utilisateur, où `^` est la puissance et `log` désigne le logarithme naturel, pas base 10) ; seuls les nœuds `Name/Constant/BinOp/UnaryOp/Call/Load` sont acceptés (`_validate_and_collect`, lignes 115-178) ; les seuls opérateurs binaires autorisés sont `+ - * /` et la puissance (`ast.Add/Sub/Mult/Div/Pow`), les seuls opérateurs unaires `+/-` (lignes 84-85) ; un appel de fonction (`ast.Call`) doit cibler un nom reconnu (`KNOWN_FUNCS`), sans argument nommé, avec exactement un argument positionnel (lignes 156-171). Toute autre construction (indexation, accès d’attribut, lambda, compréhension, comparaison, etc.) est rejetée avec un message explicite.

Variables réservées (`RESERVED_VARS`, ligne 66) : `e` (déformation), `v` (vitesse de déformation), `T` (température), et jusqu’à 3 variables optionnelles `a/b/d` — exactement l’ensemble de MAMBA. Fonctions reconnues (`KNOWN_FUNCS`, lignes 71-75, également l’ensemble exact de MAMBA, nommé `lll` dans le code d’origine) : `sqrt cos sin tan cosh sinh tanh acos asin atan erf exp log log10 asinh acosh atanh`, plus la constante `pi` (traitée à part, injectée automatiquement dans l’espace de noms si l’équation l’utilise et que l’appelant ne l’a pas déjà fournie, `evaluate`, ligne 274). Les **paramètres libres** (les quantités que la Calibration bayésienne va effectivement ajuster) sont détectés automatiquement comme les `Name` restants une fois retirées les variables réservées et `pi` — une seule source de vérité, contrairement aux deux passes indépendantes et parfois incohérentes de MAMBA.

L’évaluateur (`evaluate(parsed, namespace, backend)`, lignes 254-276, s’appuyant sur `_eval_node`, lignes 211-251) est délibérément **agnostique du type des valeurs** : l’arbre est ré-évalué en dispatchant chaque opérateur arithmétique via la surcharge Python ordinaire (`+ - * / **`), qui fonctionne identiquement pour un tableau numpy ou pour une variable tenseur PyTensor, et chaque fonction reconnue via un dictionnaire `backend` fourni par l’appelant — `NUMPY_BACKEND` (lignes 94-100, `np.sqrt`, `np.arccos` pour `acos`, `scipy.special.erf` pour `erf` faute d’équivalent numpy natif, etc.) pour l’aperçu numérique, ou l’équivalent PyTensor construit à la volée par `core/calib_pymc_model.py` (§7.5) pour le modèle de calibration lui-même. Une seule fonction d’évaluation, deux backends interchangeables, aucune logique dupliquée à maintenir synchronisée — c’est ce mécanisme qui permet à la carte “Per-test calibration

fit" (§7.9) de réévaluer le modèle a posteriori sans code numérique séparé de celui utilisé pendant l'échantillonnage.

7.3 Trois traitements de prior par paramètre libre (`core/calib_priors.py`)

`ParamPrior` (dataclass, lignes 41-111) porte, pour un paramètre libre donné, l'un des trois traitements mutuellement exclusifs de MAMBA (`PRIOR_KINDS`, ligne 27) :

- **`fixed`** : constante non calibrée (`fixed_value`), utilisée telle quelle partout où le paramètre apparaît dans l'équation.
- **`uniform`** (mise en commun/« pooled » — une seule valeur partagée par tous les essais de calibration) : `pm.Uniform(name, uniform_lo, uniform_hi)`.
- **`normal_hier`** (hiérarchique — un véritable effet aléatoire par essai, avec mise en commun partielle) : l'hyperprior Normal–Chi²-inverse de MAMBA, reproduite formule pour formule (`core/calib_pymc_model.py::build_model`, §7.5) :

$$\mu \sim \mathcal{N}(\mu_0, s_0) \qquad X \sim \chi^2_{\nu_0} \qquad \sigma = \sqrt{\frac{s_0^2\,\nu_0}{X}} \qquad \theta_j \sim \mathcal{N}(\mu, \sigma) \text{ pour chaque essai } j$$

avec $\mu_0/s_0/\nu_0$ (`hier_mu0/hier_s0/hier_nu0`) les trois nombres saisis par l'utilisateur. MAMBA saisissait quatre nombres (`mu0, s0, S0, nu0`) mais réinjectait en réalité le même `s0` à la fois dans l'hyperprior Normal sur μ et dans le calcul de σ dérivé du Chi² — un `S0` séparé n'était donc jamais un paramètre réellement indépendant ; ce champ redondant n'a pas été reporté ici, un seul `hier_s0` en tient lieu.

`validate()` (lignes 79-96) vérifie que `uniform_hi > uniform_lo` et que `hier_s0 > 0`/`hier_nu0 > 0` selon le type choisi ; `validate_priors(dict)` (lignes 114-123) vérifie en plus que chaque clé du dictionnaire correspond bien au `name` de son propre `ParamPrior`. `point_value()` (lignes 98-111) fournit une valeur numérique unique et représentative (le point fixe, le milieu de l'intervalle Uniforme, ou μ_0 pour l'hyperprior) utilisée par le chemin d'aperçu numérique avant tout échantillonnage réel. Cette classe est volontairement distincte de `core/bayes_visco_calibration.py::PriorSpec` (chapitre 8, §8.4, utilisée par l'autre page de calibration bayésienne de l'application, celle du modèle de barre viscoélastique) — un concept différent (bornes d'un optimiseur à surrogate Processus Gaussien, sans traitement hiérarchique), volontairement non mutualisé.

7.4 De la bibliothèque de covariance au jeu de données d'un essai (`core/calib_library_adapter.py`)

Module pur numpy/stdlib (aucune dépendance PySide6/pymc), qui transforme l'assignation d'une ligne de la carte "Calibration tests" (une `CovarianceLibraryEntry` par rôle de variable, plus une pour la grandeur de sortie) en un `CalibTestDataset` (dataclass, lignes 38-51 : `label`, `grid`, `variables` — dictionnaire rôle → tableau interpolé —, `observed`, `covariance`, `is_validation`, et deux champs de traçabilité `output_entry_name/variable_entry_names`).

`build_test_dataset(label, variable_entry_map, output_entry, is_validation)` (lignes 61-121) interpole chaque variable assignée (**linéairement**, `np.interp`) sur la grille propre de la grandeur de sortie — jamais l'inverse. Ce choix est délibéré : la vraisemblance (`pm.MvNormal`, §7.5) est construite contre la matrice de covariance mesurée de la grandeur de sortie, qui n'a de sens que sur la grille où elle a réellement

été calculée (une matrice $N \times N$ ne se ré-échantillonne pas sur une autre grille sans être recalculée depuis les tirages bruts, que l'entrée de bibliothèque ne conserve pas) ; chaque autre variable n'est, elle, qu'une valeur ponctuelle nécessaire à l'équation en chaque point de cette même grille. Une `LibraryAdapterError` est levée si la grille propre d'une variable ne couvre pas entièrement la plage `[min, max]` de la grille de sortie (l'interpolation extrapolerait sinon silencieusement), ou si moins de 2 points de grille sont disponibles, ou si la forme de la matrice de covariance ne correspond pas à `(N, N)`.

`per_test_or_shared_covariance(tests, mode)` (lignes 131-158, `COVARIANCE_MODES = ("per_test", "shared_mean")`, lignes 124-128) résout le choix de covariance de vraisemblance : `per_test` (nouveau réglage par défaut — corrige le bug de MAMBA, qui utilisait toujours la moyenne inter-essais même quand la covariance propre à chaque essai était disponible) renvoie la matrice propre de chaque essai ; `shared_mean` (l'ancien comportement systématique de MAMBA, conservé comme option explicite) exige que toutes les matrices aient exactement la même forme et renvoie leur moyenne élément par élément (`np.mean` sur la pile des matrices).

7.5 Construction et échantillonnage du modèle PyMC (`core/calib_pymc_model.py`)

Tous les imports `pymc`/`pytensor` de ce module sont **différés** — à l'intérieur du corps des fonctions, jamais au niveau du module — afin qu'importer `ui.main_window` (et donc démarrer l'application) n'exige jamais que ces bibliothèques soient installées. `pymc_available()` (lignes 52-66) est une sonde mise en cache (`import pymc` et `import pytensor` doivent tous deux réussir) que le reste de l'application interroge pour désactiver le bouton Run de façon proactive plutôt que d'échouer seulement après un clic.

`build_model(parsed, priors, tests, covariances)` (lignes 108-189) construit — sans échantillonner — le `pm.Model` : un `pm.Uniform` par paramètre `uniform` (réutilisé identiquement dans l'évaluation de chaque essai) ; pour chaque paramètre `normal_hier`, l'hyperprior Normal–Chi²-inverse de §7.3 puis `pm.Normal(name, mu=mu, sigma=sigma, shape=n_tests)` (un effet aléatoire par essai) ; les paramètres `fixed` sont de simples constantes Python fusionnées dans l'espace de noms, jamais des variables aléatoires. Pour chaque essai j, l'espace de noms (variables réservées déjà interpolées sur la grille de sortie de cet essai, plus valeurs fixes/pooled/hiérarchiques du moment) est passé à la **même** fonction `core.calib_equation.evaluate()` que le chemin d'aperçu numérique, mais avec le backend PyTensor construit à la volée par `_build_pytensor_backend()` (lignes 77-93, qui mappe chaque fonction reconnue vers son équivalent `pytensor.tensor`, ex. `pt.arccos` pour `acos`) : c'est le bénéfice direct de l'analyseur d'équation à backend unique décrit en §7.2. La matrice de covariance mesurée de chaque essai est une **donnée fixe, pas une variable aléatoire** — son facteur de Cholesky est calculé en numpy ordinaire (`np.linalg.cholesky`), une fois par essai, hors du graphe PyTensor — et la vraisemblance est `pm.MvNormal(f"obs_{j}_{label}", mu=phi_j, chol=chol_j, observed=test.observed)`.

`sample_model(built, draws, tune, chains, cores, target_accept, init, progress_cb, cancel_check)` (lignes 199-242) lance l'échantillonnage NUTS (`pm.sample(..., progressbar=False, return_inferencedata=True)`, la barre de progression textuelle native de PyMC étant désactivée pour ne pas écrire sur la sortie standard hors du contrôle de l'application). Valeurs par défaut (lignes 192-196) : `DEFAULT_DRAWS=1000`, `DEFAULT_TUNE=1000`, `DEFAULT_CHAINS=4`, `DEFAULT_TARGET_ACCEPT=0.9` (le défaut propre de PyMC — MAMBA codait `0.99` en dur et ne l'exposait jamais à l'utilisateur), `DEFAULT_INIT="auto"`.

Avertissement de portée, à signaler explicitement : cette sandbox de développement n'a pas d'accès réseau et n'a ni `pymc`, ni `pytensor`, ni `arviz` installés — `build_model`/`sample_model` ont donc été écrits avec soin contre l'API documentée de PyMC 5/PyTensor, mais n'ont **jamais pu être exécutés contre les bibliothèques réelles** pendant leur développement. Un essai manuel sur la machine de l'utilisateur (un cas synthétique simple à 2 essais/1 paramètre, vérifié à la main) reste nécessaire avant de considérer ce module numériquement validé, au-delà du simple fait qu'il compile et que son couplage à l'interface fonctionne (voir §7.9/§7.10). Tous les autres modules de cette fonctionnalité (`calib_equation.py`, `calib_priors.py`, `calib_library_adapter.py`) sont, eux, entièrement testés avec de vraies données dans cette même sandbox — cette exception est isolée ici précisément pour ne pas se propager au reste.

7.6 Orchestration d'une exécution (`core/calib_run.py`)

`run_calibration(equation_str, priors, test_assignments, covariance_mode, sampler_settings, progress_cb, cancel_check)` (lignes 97-187) est la fonction unique que le bouton Run appelle (via le fil d'exécution partagé, §7.8) et que les tests d'intégration de l'interface substituent pour vérifier le câblage Qt sans avoir besoin d'un vrai PyMC. Ordre des étapes : analyser l'équation (`parse_equation`) → vérifier qu'un prior existe pour chaque paramètre libre détecté → `validate_priors` → point de contrôle d'annulation → construire les jeux de données de test (`_build_datasets`, lignes 64-94, qui sépare les lignes "Validation only" des lignes de calibration et lève une `CalibRunError` si aucune ligne n'est fournie, ou si toutes sont marquées validation-seule) → résoudre le mode de covariance (§7.4) → vérifier `pymc_available()` (sinon `PymcUnavailableError`) → point de contrôle d'annulation → fusionner les réglages d'échantillonneur avec les défauts → `build_model` → `sample_model` → empaqueter un `CalibrationResult` (dataclass, lignes 38-56 : équation source, variables/paramètres utilisés, priors, mode de covariance, noms des paramètres pooled/hiérarchiques/fixes, résumé postérieur, `idata` (l'`InferenceData` arviz complet, conservé pour les diagnostics du §7.7), listes des essais de calibration et de validation, réglages d'échantillonneur effectivement utilisés).

Chaque exception interne (`EquationError`, `PriorError`, `LibraryAdapterError`) est systématiquement relevée comme `CalibRunError` (`ValueError`), de sorte que l'étiquette de statut unique de la page n'a jamais besoin de connaître le type d'exception d'origine ; `PymcUnavailableError` (un `RuntimeError`) reste, elle, distincte, puisque c'est un problème d'environnement, pas de saisie utilisateur.

7.7 Diagnostics postérieurs et validation croisée réelle (`core/calib_diagnostics.py`)

Comme pour PyMC (§7.5), deux niveaux de "testabilité réelle" coexistent délibérément dans ce module : `posterior_predictive_at`/`posterior_correlation_matrix` ne prennent que des tableaux/dictionnaires numpy ordinaires (jamais un objet `arviz.InferenceData`), donc entièrement testés avec de vraies données dans cette sandbox ; `convergence_summary`/`render_energy_plot`/`render_forest_plot` sont de fins encapsulages `arviz` (import différé, même schéma que pymc/pytensor) — génuinement impossibles à tester sans `arviz` installé, donc seulement couverts au niveau du câblage.

`posterior_predictive_at(parsed, param_draws, dataset)` (lignes 132-167) est une **véritable** évaluation prédictive a posteriori : l'équation validée (§7.2) est réévaluée, une fois par tirage postérieur, aux conditions propres du `CalibTestDataset` fourni (sa propre grille, ses propres variables e/v/T/a/b/d) — ceci corrige directement le bug de "validation" de MAMBA, qui ne faisait jamais que superposer l'ajustement de calibration sur le graphique de validation, sans jamais réellement réévaluer le modèle aux conditions

de l'essai de validation. Retourne la grille, la courbe observée, la moyenne postérieure prédictive, et une bande à 95 % (percentiles 2,5/97,5 sur l'ensemble des tirages).

`resolve_predictive_param_draws(free_params, priors, posterior_flat, test_index)` (lignes 75-129) résout, pour chaque paramètre libre, les tirages postérieurs à utiliser : un `fixed` est diffusé comme une constante ; un `uniform` (pooled) est lu directement ; un `normal_hier`, **si** `test_index` est fourni (l'essai en question a réellement fait partie de l'ajustement — carte "Per-test calibration fit", §7.9), utilise sa propre valeur par essai ; **sinon** (un essai de validation réellement tenu à l'écart de l'ajustement — carte "Validation", §7.9), retombe sur l'hyper-moyenne de population du paramètre (`mu_<name>`). C'est une simplification documentée explicitement, aussi bien dans le code que dans le texte d'aide de la carte Validation elle-même : elle prédit avec la tendance centrale de la population et ignore la dispersion inter-essais à laquelle un essai réellement nouveau serait lui aussi soumis — la bande affichée pour un essai de validation est donc plus étroite que l'incertitude prédictive réelle dès qu'un paramètre hiérarchique est impliqué.

`posterior_correlation_matrix(posterior_flat)` (lignes 170-189) calcule une matrice de corrélation de Pearson ordinaire (`np.corrcoef`, sans `arviz`) entre les tirages postérieurs nommés fournis (typiquement chaque paramètre pooled directement, plus l'hyper-moyenne `mu_<name>` de chaque paramètre hiérarchique), noms triés alphabétiquement, exige au moins 2 paramètres. `convergence_summary(idata)` (lignes 196-207) encapsule `arviz.summary` — R-hat/ESS toujours calculés (MAMBA ne les calculait jamais du tout, `forestplot(..., rhat=False)`). `render_energy_plot`/`render_forest_plot` (lignes 210-231) dessinent directement sur un `Axes` matplotlib déjà existant (le diagnostic BFMI/énergie de transition NUTS, et le graphique en forêt avec R-hat).

7.8 Fil d'exécution partagé (`core/qt_worker.py`)

`CalibWorker(QObject)` (signaux `progress`/`finished`/`failed`/`cancelled`) porte hors du thread d'interface l'exécution d'une fonction longue — `run_calibration` ici, mais aussi `core/bayes_visco_calibration.py::ca` pour l'autre page de calibration bayésienne de l'application (chapitre 8, §8.4). Cette classe existait déjà, identique, dans `ui/pages/page_visco_calib.py` (sous le nom `_CalibWorker`) avant ce portage ; elle a été extraite ici en module partagé plutôt que dupliquée une seconde fois, puisqu'elle n'a rien de spécifique à la calibration viscoélastique. `run()` capture `OperationCancelled` (signal `cancelled`) et toute autre `Exception` (signal `failed`, message stringifié) — y compris, de fait, une exception levée à l'intérieur d'un slot connecté au signal `finished` lui-même, puisque `finished.emit(...)` se produit à l'intérieur du même bloc `try`.

7.9 Interface — les onze cartes de la page Bayesian Calibration

La page (`ui/pages/page_bayes_calib.py`) aplatit l'assistant en 4 étapes de MAMBA (Définir le modèle → Paramétrer la calibration → Lancer → Afficher les résultats) en onze cartes toujours visibles sur une page déroulante, selon la convention `Card`/`make_scroll_page` déjà utilisée partout ailleurs dans l'application — pas un assistant pas-à-pas.

1. **Equation** : champ de saisie libre (ex. `A * (1 + B*e^n) * (1 + C*log(v)) * (1 - T^m)`, une forme de type Johnson-Cook), bouton Parse (ou simplement quitter le champ). Le texte d'aide énumère variables réservées et fonctions reconnues directement depuis `RESERVED_VARS`/`KNOWN_FUNCS` (§7.2), jamais recopiées à la main, pour qu'aide et analyseur ne puissent pas diverger.

2. **Free parameters** : une ligne par paramètre libre auto-détecté, reconstruite à chaque analyse réussie. Chaque paramètre démarre en **Normal (hiérarchique)** par défaut — le sous-titre de la carte rappelle le flux de travail recommandé par le manuel de MAMBA : tout lancer en hiérarchique d'abord, inspecter la carte d'interdépendance des paramètres (carte 8) une fois des diagnostics disponibles, puis rétrograder en Uniforme (pooled) les paramètres faiblement identifiés ou trop corrélés entre eux.
3. **Calibration tests** : reconstruite uniquement quand l'ensemble des variables réservées utilisées par l'équation change réellement. Une ligne "Variable" (une par colonne, plus une pour Output) permet de choisir **quelle grandeur physique** une colonne représente réellement, parmi celles enregistrées dans la bibliothèque de calibration (chapitre 1, §3.5) — une lettre réservée comme `e` n'a donc pas un sens figé : elle peut tout aussi bien être assignée à une pression hydrostatique ou une déformation volumétrique qu'à une déformation. Défauts sensés mais non contraignants : `e`→déformation, `v`→vitesse de déformation, `T`→température, Output→contrainte, `a/b/d`→« (any quantity) » (aucun filtre). Changer la grandeur d'une colonne refiltre aussitôt le menu déroulant de chaque ligne pour cette colonne (une entrée dont la grandeur est inconnue — `quantity_key` vide — n'est jamais masquée, faute de provenance connue) ; une sélection déjà faite qui ne correspond plus au nouveau filtre est réinitialisée à « – choose – » plutôt que de rester invisible mais active. Chaque ligne peut être cochée « Validation only » pour être exclue de l'ajustement et réservée à un contrôle croisé réel (carte 10).
4. **Sampler settings** : mode de covariance (Per-test par défaut / Shared mean), Draws/Tune/Chains/Cores (Cores initialisé depuis `project.n_workers`), Target accept (0,9 par défaut), méthode d'initialisation (`auto/adapt_diag/jitter+adapt_diag/advi+adapt_diag`) — avec un message rouge et le bouton Run désactivé si `pymc_available()` est faux.
5. **Run** : Run/Stop/barre de progression/étiquette de statut. L'info-bulle du bouton Stop indique explicitement que PyMC n'offre aucun moyen pris en charge d'interrompre l'échantillonnage une fois réellement démarré — Stop ne fait effet que s'il est cliqué avant que NUTS ne commence (§7.5, §7.10).
6. **Result summary** : moyenne posterior ± écart-type par paramètre pooled, ou hyper-moyenne ± hyper-écart-type par paramètre hiérarchique (l'estimation au niveau population — la valeur propre à chaque essai reste disponible dans le posterior complet mais n'est pas affichée ici), avec un drapeau explicite si R-hat dépasse 1,01.
7. **Per-test calibration fit** : combo des essais de calibration (jamais des lignes "Validation only"), courbe observée vs. moyenne prédictive a posteriori ± bande à 95 % — une véritable réévaluation de l'équation aux conditions propres de l'essai choisi (§7.7), avec, pour un paramètre hiérarchique, sa propre valeur à CET essai précis.
8. **Parameter interdependence** : même présentation en carte de chaleur que `page_visco_calib.py` (chapitre 8, §8.4.5 : `imshow`, palette `RdBu_r`, cellules annotées) — corrélation postérieure entre chaque paire de paramètres calibrés (pooled directement, hiérarchiques via leur hyper-moyenne `mu_<name>`) ; un message simple remplace la carte de chaleur si moins de 2 paramètres sont calibrés.
9. **Posterior diagnostics** : « Posterior density » (histogramme par paramètre, toujours disponible, sans `arviz`) / « Forest plot » / « Energy plot » (ces deux derniers nécessitent `arviz` ; message explicite et propre si absent, plutôt qu'un plantage).
10. **Validation** : n'apparaît que si au moins une ligne "Validation only" a été utilisée lors du dernier run — un contrôle croisé réel par posterior-prédictif (§7.7), pas une simple superposition de l'ajustement de calibration comme le faisait MAMBA. La mise en garde sur le repli à l'hyper-moyenne pour un

paramètre hiérarchique (§7.7) est reprise explicitement dans le texte d'aide de la carte elle-même.

11. **Export** : « Export all figures as PDF… » (une page PDF par graphique actuellement peuplé — nommées `bayes_calib_<carte>.pdf`), « Export posterior summary as .csv/.txt… » (le résumé de `convergence_summary`, §7.7).

7.10 Limitations et avertissements notés dans le code

- **Jamais exécuté contre un vrai PyMC/PyTensor/arviz pendant le développement** (§7.5) : cette fonctionnalité entière (construction du modèle, échantillonnage NUTS, diagnostics arviz) a été écrite avec soin contre l'API documentée, mais un essai manuel de validation numérique sur la machine de l'utilisateur reste requis avant tout usage en conditions réelles.
- **Aucune annulation réelle en cours d'échantillonnage** : PyMC n'expose aucun point d'arrêt pris en charge une fois `pm.sample` démarré ; le bouton Stop ne fait effet que s'il est cliqué avant, et `cancel_check` n'est vérifié qu'une seule fois, juste avant l'appel à `pm.sample` (`core/calib_pymc_model.py::sample_model`, lignes 225-227).
- **`progress_cb` n'est appelé que deux fois** (`(0, 1)` avant, `(1, 1)` après) — pas une progression réelle pas-à-pas, puisque la barre de progression textuelle native de PyMC est désactivée pour ne pas écrire sur la sortie standard hors du contrôle de l'application.
- **Repli à l'hyper-moyenne pour un paramètre hiérarchique en carte Validation** (§7.7, §7.9, carte 10) : la bande prédictive affichée pour un essai de validation ignore la dispersion inter-essais propre à ce paramètre — elle est donc plus étroite que l'incertitude prédictive réelle.
- **Le type de prior d'un paramètre libre est global, pas par essai** : un paramètre est entièrement pooled, entièrement hiérarchique, ou entièrement fixe pour l'ensemble des essais de calibration d'un run donné — aucun mélange partiel n'est possible en une seule exécution.
- **`quantity_key` d'une entrée de bibliothèque n'est jamais validée** contre le contenu réel de l'entrée (chapitre 1, §3.5, limitation n°9) : elle ne sert qu'à filtrer les menus déroulants de la carte "Calibration tests", jamais à vérifier les données elles-mêmes.
- **Liste blanche de l'analyseur d'équation volontairement plus étroite que Python complet** (§7.2) : pas de `%`, `//`, opérateurs de comparaison ou logiques — un choix de sécurité assumé puisque l'entrée est du texte libre saisi par l'utilisateur ; l'arbre validé n'est de toute façon jamais transmis à `eval()`/`exec()`, seulement réévalué par le dispatcheur contrôlé de `core/calib_equation.py`.
- **Question de l'empaquetage PyInstaller explicitement mise de côté** : le choix de PyMC 5 + PyTensor (plutôt qu'une alternative plus légère) a été fait en toute connaissance de cause de son coût pour un exécutable autonome — un compromis assumé par l'utilisateur, non résolu par ce portage.

7.11 Synthèse des renvois fichier/fonction

Sujet	Fichier	Fonction/méthode
Analyse/liste blanche de l'équation	`core/calib_equation.py`	`parse_equation`, `_validate_and_collect`
Évaluation à double backend (numpy/PyTensor)	`core/calib_equation.py`	`evaluate`, `_eval_node`, `NUMPY_BACKEND`
Trois traitements de prior par paramètre	`core/calib_priors.py`	`ParamPrior`, `validate_priors`

Sujet	Fichier	Fonction/méthode
Bibliothèque de covariance → jeu de données d'essai	`core/calib_library_adapter.py`	`build_test_dataset`
Mode de covariance (par essai / moyenne partagée)	`core/calib_library_adapter.py`	`per_test_or_shared_covariance`
Sonde de disponibilité PyMC/PyTensor	`core/calib_pymc_model.py`	`pymc_available`
Construction du modèle (priors, vraisemblance MvNormal)	`core/calib_pymc_model.py`	`build_model`
Échantillonnage NUTS	`core/calib_pymc_model.py`	`sample_model`
Orchestration complète d'un run	`core/calib_run.py`	`run_calibration`
Résolution des tirages de paramètres pour une prédiction	`core/calib_diagnostics.py`	`resolve_predictive_param_draws`
Contrôle posterior-prédictif réel	`core/calib_diagnostics.py`	`posterior_predictive_at`
Matrice de corrélation entre paramètres	`core/calib_diagnostics.py`	`posterior_correlation_matrix`
Résumé de convergence (R-hat/ESS, via arviz)	`core/calib_diagnostics.py`	`convergence_summary`
Fil d'exécution Qt partagé (hors thread UI)	`core/qt_worker.py`	`CalibWorker`
Page de l'onglet Bayesian Calibration	`ui/pages/page_bayes_calib.py`	`BayesCalibPage`

10. Analyse fissure & vitesse d'onde (Crack & Wave Analysis)

Ce chapitre documente l'outil standalone "Crack & Wave Analysis" (`ui/pages/page_crack_wave_analysis.py`), qui calcule ses propres estimations formelles d'incertitude de mesure, comme le module DIC du chapitre 6. Contrairement aux autres chapitres, l'incertitude n'y est **pas** produite par un tirage Monte-Carlo (chapitre 1) : chaque méthode ci-dessous s'appuie sur la covariance asymptotique d'un ajustement par moindres carrés (linéaire ou non linéaire), calculée une seule fois en un point, pas par ré-échantillonnage — voir §7 du chapitre 1 pour la mise en perspective des deux paradigmes.

L'outil travaille sur un jeu de données DIC déjà exporté (comme le chapitre 9 le rappelle pour les autres outils standalone, aucune page ne partage ses résultats DIC en mémoire une fois quittée) et se décompose en quatre étapes, chacune pouvant être utilisée indépendamment des suivantes :

1. **Chargement des données** (`core/crack_wave_io.py`) — un CSV DIC déjà exporté (page Digital Image Correlation, "Export DIC results") plus la séquence d'images/vidéo brute correspondante. Une carte "Data" IDENTIQUE existe sur les deux onglets ("Wave speed analysis" et "Crack analysis") : charger un fichier depuis l'une des deux met à jour l'autre à l'identique (statut, chemins, mode vidéo/séquence), puisqu'il n'existe qu'UN SEUL jeu de données chargé pour toute la page — jamais deux indépendants — la seconde carte n'étant là que pour éviter d'avoir à changer d'onglet pour recharger un fichier.
2. **Vitesse des ondes élastiques** (`core/elastic_wave_speed.py`) — onglet "Wave speed analysis".
3. **Détection/suivi de la pointe de fissure** (saut d'ouverture — position uniquement, pas de K_I/K_{II}, §3) et **facteurs d'intensité de contrainte K_I/K_II par X-FEM-DIC** (corrélation directe enrichie, §4) — onglet "Crack analysis". X-FEM-DIC est la méthode de calcul de K_I/K_{II} exposée dans l'interface ; un ajustement DIC standard en anneau (avec extension T-stress optionnelle) et une intégrale d'interaction (J-integral) restent disponibles dans le code pour un usage programmatique — voir §5.

Les propriétés matériau nécessaires au calcul de K_I/K_II (module d'Young E, coefficient de Poisson ν, état plan de contrainte/déformation) sont saisies dans une carte "Material & Calibration" au sommet de l'onglet "Crack analysis" (seul endroit de la page où elles servent). La taille de pixel (mm/px) — utilisée pour convertir en mm/s ou m/s toutes les vitesses de l'onglet "Wave speed analysis" (fronts d'onde, pointe de fissure), **et** nécessaire au calcul de K_I/K_II sur l'onglet "Crack analysis" — est un champ UNIQUE partagé, mais construit une fois par onglet (même motif que la carte "Data" ci-dessus) : il apparaît à la

fois sur la carte "Material & Calibration" de l'onglet "Wave speed analysis" et sur celle de l'onglet "Crack analysis", les deux instances restant en permanence synchronisées (modifier l'une met immédiatement à jour l'autre) — jamais deux valeurs indépendantes, juste plus d'un endroit pour la voir/l'éditer sans changer d'onglet. Un bouton "Sync from DIC tab", présent sur les deux instances, la resynchronise depuis la page Image Correlation ; elle se **pré-remplit aussi automatiquement** à partir du CSV chargé (carte "Data") quand celui-ci a été exporté avec une taille de pixel renseignée, par la même méthode de récupération que l'import CSV du chapitre 6 (§4.8) — rapport `u_mm/u` et `v_mm/v`, agrégé par médiane sur tout le fichier (`core.crack_wave_io._recover_pixel_size_mm`) — sans jamais écraser une valeur déjà saisie manuellement ou déjà synchronisée : le statut affiché sous le champ dit explicitement ce qui s'est passé (valeur détectée et appliquée, détectée mais non appliquée car une valeur existe déjà, ou rien à détecter si le CSV a été exporté sans taille de pixel).

---

1. Squelette de fissure tracé (`core/crack_skeleton.py`)

Une fissure est représentée comme un arbre (pas une simple polyligne) : `CrackPath`, un graphe de noeuds `(x, y)` reliés par des arêtes, sans racine privilégiée — n'importe quel noeud peut être un point de branchement (degré > 2), une extrémité/pointe candidate (degré 1) ou un point isolé (degré 0). Ce choix reflète le fait qu'une fissure réelle peut se ramifier ; chaque branche a sa propre pointe, suivie indépendamment à l'étape 3.

La carte "Crack Skeleton Tracing" qui porte ce tracé se trouve au sommet de l'onglet "Crack analysis" (juste sous la propre carte "Data" de ce même onglet) : le tracé n'est utilisé que par la détection de pointe et X-FEM-DIC (étape 3, toutes deux sur cet onglet), jamais par le picking de front d'onde de l'onglet "Wave speed analysis" (qui travaille directement sur le champ brut, sans tracé préalable, §2 ci-dessous).

Deux façons de construire un `CrackPath` :

- **Manuelle**, point par point, via le widget interactif `ui/crack_skeleton_picker.py`.
- **Automatique** (`detect_cracks`) : seuillage d'un champ de déformation (le même champ MISES/exx/eyy déjà affiché pour le tracé manuel) en un masque binaire de "bande de localisation", amincissement en un squelette 1-pixel via `skimage.morphology.skeletonize` — préféré à une implémentation Zhang-Suen maison : Zhang-Suen est connu pour éroder les bandes diagonales fines (effet d'escalier), un mode de défaillance qui affecterait une bande de fissure diagonale, cas pourtant courant — puis parcours du squelette en graphe, une composante connexe par fissure détectée, permettant de proposer plusieurs fissures en un seul appel.

La simplification de polyligne (Ramer-Douglas-Peucker) est implémentée directement dans le module (algorithme classique, facilement vérifiable), dans le même esprit de "code littéral et auditable" que `core/gaussian_process.py`.

Le détecteur automatique est explicitement une **suggestion de départ**, pas une réponse finale : son résultat est une liste `CrackPath` ordinaire (éditable), modifiable ensuite exactement comme un tracé manuel — obtenir une topologie parfaite (ne jamais fusionner deux doigts voisins d'une même fissure, ne jamais scinder une fissure en deux composantes à cause d'un pixel de bruit) n'est pas l'objectif ; donner un point de départ

rapide et raisonnable l'est. Ce module ne calcule aucune incertitude — c'est un outil de tracé géométrique, pas de mesure physique.

---

2. Vitesse des ondes élastiques par suivi de front (`core/elastic_wave_speed.py`)

2.1 Principe : temps de vol le long d'un front

Pour la direction mesurée (X ou Y), la méthode reproduit la lecture d'une vitesse de propagation sur un "shot gather" sismique ou un B-scan ultrasonore :

1. Le champ de déformation choisi est moyenné le long de la direction PERPENDICULAIRE à celle mesurée, à chaque position de la direction mesurée (ex. moyenne sur y à chaque x, pour une vitesse en X) — un front d'onde à peu près plan est ainsi renforcé, tandis que le bruit/la variation selon Y s'annule au lieu de brouiller le front.
2. Le temps d'arrivée du front est détecté à chaque position, par franchissement d'un seuil relatif (mi-hauteur par défaut) entre le niveau de base et le pic propres à cette position — convention "first break" standard en essais d'onde/impact, robuste à une amplitude qui décroît avec la distance de propagation (un seuil absolu fixe ne le serait pas).
3. Régression linéaire position $= v \cdot t_{arrive} + x_0$ sur toutes les positions ayant une arrivée détectée — une seule régression sur l'ensemble des colonnes/lignes du champ (pas seulement deux points), avec un R^2 directement vérifiable et une ordonnée à l'origine interprétable (où/quand le front aurait démarré).

Cette hypothèse (front plan, propagation majoritairement selon la direction mesurée) est valide près d'un bord d'où l'onde est lancée, tôt dans sa propagation — la situation type visée par cet outil. Un R^2 faible signale un front courbé, contaminé par un signal de fissure plus tardif, ou un seuil ayant capté du bruit plutôt que le vrai front — pas quelque chose à masquer par un ajustement plus sophistiqué. `position_range` permet de restreindre l'AJUSTEMENT (pas la détection d'arrivée, qui continue de tourner sur toute position) à un seul côté d'une source ponctuelle, pour traiter séparément deux fronts opposés qui, sinon, moyanneraient leurs pentes en une pente quasi nulle et quasi dénuée de sens.

2.2 Détection automatique de branches multiples (crête + RANSAC)

Pour un champ montrant plusieurs réflexions (une onde qui fait des allers-retours, chaque rebond inversant son sens de propagation), une seule séparation manuelle par plage de position ne suffit plus. `detect_wave_branches` automatise la séparation :

1. Contrairement à la méthode à arrivée unique ci-dessus (adaptée à un front qui, une fois arrivé, reste élevé), cette approche cherche une **impulsion transitoire** : à chaque instant, elle trouve chaque bande contiguë (selon la position) qui dépasse le seuil d'arrivée — "où le champ est-il élevé maintenant" — et rapporte un point de crête par bande, à son centroïde pondéré par l'amplitude (`_band_centroids`), pas son unique pixel le plus fort — une bande a une largeur physique réelle, et le bruit de mesure par pixel déplace d'une frame à l'autre quel pixel exact est le plus fort ; moyenner sur toute la largeur

de bande annule ce bruit (environ en $1/\sqrt{N}$ sur la largeur de bande) tout en suivant un déplacement sous-pixel réel du centre.

2. Ces points de crête sont ensuite séparés en branches rectilignes par un RANSAC séquentiel : ajustement répété de la droite (position $= v \cdot t + x_0$) avec le plus grand nombre de points "inliers" (à une tolérance de position près), retrait de ces inliers, puis répétition sur ce qui reste jusqu'à ce que trop peu de points subsistent ou que `max_branches` soit atteint — sans connaissance préalable du nombre de fronts ni de leur sens de propagation.

Cette approche par crête/RANSAC suppose un champ montrant une impulsion réellement transitoire (monte puis redescend) — vrai pour une onde élastique sans déformation résiduelle plastique significative. Si le champ montre au contraire une marche permanente (déformation qui reste élevée après le passage du front, ex. le champ près d'une pointe de fissure en croissance, ou une déformation résiduelle non élastique), il n'y a PAS de maximum local à suivre une fois le front passé — utiliser plutôt la méthode à front unique (§2.1) dans ce cas.

2.3 Cueillette manuelle par polygone ROI (`extract_ridge_in_polygon`)

Les deux méthodes automatiques ci-dessus partagent une limite structurelle : elles choisissent un seuil (fixe, ou normalisé par frame) et lui font confiance pour séparer "l'onde" de "tout le reste" sur l'ensemble du champ à la fois. Un enregistrement réel peut montrer plusieurs fronts d'intensités franchement différentes (l'un saturé, l'autre à peine visible), pour lesquels aucun seuil unique ne convient. `extract_ridge_in_polygon` contourne entièrement le seuillage : l'utilisateur trace un polygone lâche autour d'UNE bande visible directement sur le tracé (comme on pointe à la main une crête de fréquence sur un spectrogramme PDV/VISAR), et chaque frame que ce polygone recouvre contribue un point de crête, à l'extremum local (max ou min, signe quelconque) parmi les échantillons inclus — sélectionné parmi les candidats concurrents par proximité de l'étendue propre du polygone plutôt que par la seule intensité, puis raffiné par un centroïde resserré sur son voisinage immédiat. Rien ici ne lit de seuil : le polygone lui-même isole le front d'intérêt, à quelle que soit son intensité réelle. La méthode n'est pas non plus contrainte à une droite (pas de RANSAC/ajustement de droite) : un front réellement courbé (accélérant/décélérant, ou non élastique) est tracé aussi fidèlement qu'un front rectiligne ; un ajustement OLS indicatif (vitesse/R^2) est tout de même rapporté à titre indicatif, via la même fonction `fit_ridge_points` que l'ajustement automatique.

2.4 Absence d'incertitude formelle

Aucune des méthodes de ce stade ne produit d'incertitude formelle sur la vitesse d'onde elle-même : le R^2 de l'ajustement OLS (indicatif, non pondéré) est le seul indicateur de qualité rapporté, à interpréter qualitativement (un R^2 faible signale un ajustement peu fiable, sans quantifier une barre d'erreur sur la pente) — contrairement au traitement de la pointe de fissure (§3 ci-dessous), qui dispose d'une estimation d'incertitude formelle.

---

3. Détection et suivi de la pointe de fissure (`core/crack_tip_tracking.py`)

3.1 Vue d'ensemble des méthodes disponibles

Cinq méthodes de détection de la position de la pointe le long du squelette tracé existent dans le code, testées et réutilisables ; une seule, "opening_jump" (§3.2), est câblée par défaut dans l'interface actuelle.

- **"threshold"** (`track_tip_threshold`) : parcourt le chemin (rééchantillonné finement) depuis l'origine et retient le point le plus éloigné où le champ choisi dépasse encore une valeur absolue donnée (en tolérant quelques échantillons consécutifs sous le seuil), puis interpole linéairement le franchissement exact (`_interpolate_tip`) plutôt que de rapporter le point de la grille rééchantillonnée brut — sans cette interpolation, la pointe ne peut se placer que sur l'un des pas fixes du rééchantillonnage, ce qui produit un escalier non physique (plusieurs frames d'affilée à la position exacte, puis un saut) même quand le champ sous-jacent avance en réalité de façon régulière. Limite : un seuil absolu unique doit séparer "fissuré" de "pas encore fissuré" à chaque frame, alors que l'intensité globale du champ le long du chemin peut dériver sur toute la séquence (endommagement évolutif, contraste/bruit changeant) — un seuil adapté aux premières frames peut devenir trop haut ou bas plus tard dans la séquence.
- **"singular_fit"** (`track_tip_singular_fit`) : en chaque point candidat le long du chemin, ajuste le champ échantillonné dans un cône/anneau orienté vers l'avant contre la décroissance radiale $1/\sqrt{r}$ du champ LEFM en pointe de fissure au premier ordre, et retient le point au meilleur ajustement (R^2 le plus élevé, parmi ceux à pente positive/croissante vers la pointe) — plus physiquement motivé, mais nécessite une résolution de champ suffisante pour résoudre une vraie tendance en $1/\sqrt{r}$, et est plus sensible au bruit et à la précision de la tangente locale du chemin (utilisée pour orienter le cône).
- **"ridge" (recommandée, non câblée directement mais réutilisée via ROI)** : sans seuil. Traite la concentration de champ en pointe de fissure exactement comme le chapitre 2 traite un front d'onde élastique : un pic mobile, physiquement localisé, balayant le chemin dans le temps, dont la carte (position d'arc x temps) est une "image" 2D dans laquelle une crête peut être tracée (`build_tip_field_profile`). Dans l'interface, ceci se manifeste comme "ridge (ROI)" : l'utilisateur trace un polygone autour de la bande d'avancée visible et `extract_tip_ridge_in_polygon` extrait un point par frame couverte, à l'extremum local le plus fort à l'intérieur du polygone — aucun seuil d'arrivée, donc un signal faible ou dérivant est aussi facile à pointer qu'un signal fort.
- **"arrival (ROI)"** (`extract_tip_arrival_time_in_polygon`) : à l'inverse de "ridge" (qui demande, frame par frame, "où est le pic le plus fort maintenant"), celle-ci demande, position par position le long du chemin, "la première fois que la valeur du champ en ce point atteint un seuil absolu fixe, à quel instant était-ce" — puis relie ces couples (temps d'arrivée, position) dans l'ordre où ils surviennent. Pour un champ comme la déformation équivalente de von Mises, où une zone réellement fissurée saute à un niveau nettement séparé du fond non endommagé et y **reste** ensuite, ceci est quasi monotone par construction. Ce n'est pas le même algorithme que "threshold" ci-dessus : "threshold" compare chaque position, à chaque frame, contre un seuil unique, donc doit continuer de fonctionner malgré une dérive globale d'intensité sur toute la séquence ; "arrival" regarde au contraire l'historique temporel complet de chaque position une seule fois et demande seulement si/quand elle a franchi le seuil, donc un démarrage plus lent ou une rampe d'intensité ailleurs n'affecte en rien une position déjà franchie.
- **"opening_jump" (méthode retenue par défaut, seule câblée dans l'interface actuelle)** — voir §3.2.

Limite commune à "ridge (ROI)"/"arrival (ROI)" et à l'outil de pointage manuel "manual (click)" (toutes trois travaillant exactement sur le chemin tracé) : elles suivent un champ dérivé (déformation, ou tout autre `field_key` choisi), qui peut s'atténuer ou se décaler légèrement de la ligne exacte du chemin tracé avant que la fissure n'ait réellement cessé de s'ouvrir à cet endroit. "opening_jump" (§3.2), qui mesure directement la discontinuité de déplacement plutôt qu'un champ dérivé, est la méthode câblée dans l'interface actuelle ; "ridge (ROI)" et "manual (click)" restent disponibles dans le code pour réutilisation, et "arrival (ROI)" reste disponible dans l'interface en complément quand un franchissement de champ dérivé est effectivement ce qui est recherché.

3.2 Méthode retenue : saut d'ouverture (`crack_opening_jump_profile`, `track_tip_opening_jump`)

Plutôt que de seuiller/pointer un champ dérivé (déformation) le long du chemin, cette méthode mesure directement la DISCONTINUITÉ DE DÉPLACEMENT u/v brute de part et d'autre du chemin : une fissure réellement ouverte montre ici un saut net et non ambigu, même là où un champ dérivé est déjà devenu trop faible ou décalé pour être seuillé/pointé de façon fiable.

En chaque point du chemin tracé, les champs u/v sont échantillonnés en deux points décalés de `offset_px` de part et d'autre du chemin, le long de sa NORMALE locale (perpendiculaire à la tangente — donc "de chaque côté de la face de fissure", pas avant/arrière le long du chemin), et la différence absolue est prise :

$$\text{jump}_u = |u(+\text{offset}) - u(-\text{offset})| \qquad \text{jump}_v = |v(+\text{offset}) - v(-\text{offset})| \qquad \text{jump}_{mag} = \sqrt{\text{jump}_u^2 + \text{jump}_v^2}$$

`jump_mag` combine les deux composantes exactement comme la propre détection Heaviside-jump de X-FEM-DIC combine son propre déplacement de saut ajusté (§4.6) — le seuil par défaut, 0.15 px, est donc partagé et directement comparable entre les deux méthodes, pas une coïncidence. `offset_px` doit être assez grand pour que les deux points d'échantillonnage tombent réellement de part et d'autre de la discontinuité (potentiellement large de plusieurs pixels sur une image réellement corrélée), mais assez petit pour rester une mesure "près de la pointe" plutôt que de capter un mouvement de champ lointain sans rapport.

`opening_extent_s`/`opening_extent_xy` répondent à "jusqu'où le long du chemin `jump_mag` reste-t-il continûment au-dessus de `threshold_px`, en marchant depuis l'origine du chemin (indice 0)" — même algorithme `_threshold_walk`/`_interpolate_tip` que la méthode "threshold" (§3.1), appliqué à `jump_mag` plutôt qu'à un champ exporté brut. Ceci suppose que `path_positions[0]` est déjà réellement ouvert — vrai pour un chemin tracé depuis le point d'initiation réel de la fissure. `max_gap_samples` tolère quelques échantillons consécutifs sous le seuil (bruit de corrélation/un point isolé) sans tronquer la marche prématurément, y compris juste à l'origine elle-même.

Robustesse au passage d'une onde élastique sur le chemin tracé (`smooth_window_samples`, `advance_confirm_frames`) — signalé en pratique sur un granite réel : le passage d'une onde élastique dans le champ de vue DIC produit des perturbations de déplacement très localisées (quelques échantillons de large le long du chemin) et de forte amplitude, qui se déplacent d'une frame à l'autre — exactement le profil qu'un filtrage par seuil unique ne peut pas distinguer d'une vraie ouverture de fissure. Deux mécanismes de rejet, complémentaires, s'attaquent chacun à un axe différent du problème ; aucun des deux ne suppose ou ne borne une vitesse de propagation de fissure, un choix délibéré : la vitesse d'une fissure peut varier

fortement dans un matériau hétérogène comme le granite, si bien qu'une borne de vitesse fixe rejetterait à tort une avancée rapide mais réelle.

(1) Filtrage spatial, dans le profil d'UNE frame — `smooth_window_samples` (1 par défaut = désactivé) fait passer `jump_mag_px` (avant la marche à seuil) par un filtre médian glissant le long du chemin (`_nanmedian_filter_1d`, fenêtre centrée, ignore les échantillons non finis plutôt que de les propager) ; `opening_extent_s`/`opening_extent_xy` sont ensuite calculés sur ce signal lissé (`jump_mag_smoothed_px`), pas sur `jump_mag_px` brut, qui reste néanmoins rapporté tel quel dans le profil retourné (export CSV, tracé "Vérifier visuellement") pour comparaison directe. Une médiane, pas une moyenne, rejette franchement un échantillon isolé aberrant plutôt que de le mélanger dans le résultat — exactement la distinction "pic étroit vs. plateau soutenu" recherchée : une ouverture réelle reste large par construction (chaque point du chemin déjà ouvert le reste aux échantillons suivants), un pulse d'onde ne l'est pas. La largeur de fenêtre doit être choisie un peu plus large que l'extension visible du pulse (dans le tracé "Vérifier visuellement"), mais nettement plus étroite que la plus courte ouverture réelle attendue près de l'initiation — une fenêtre trop large entamerait une vraie ouverture courte.

(2) Confirmation temporelle, à travers les frames — le filtrage ci-dessus n'empêche pas qu'une frame entière, pendant que l'onde traverse le chemin, rapporte un `opening_extent_s` ponctuellement bien au-delà de la tendance réelle (le pulse peut occuper le chemin sur une largeur suffisante pour rester une "ouverture continue" au sens de la marche, même lissée). L'ancien comportement (avant l'ajout de ce paramètre) de la contrainte "avancée monotone seulement" (`enforce_monotonic`, §3.1) acceptait immédiatement toute avancée vers l'avant et relevait aussitôt, de façon permanente, le maximum courant accepté à cette valeur — un bug identifié directement dans le code : la seule protection existante (`backslide_tolerance_px`) ne borne le RECUL toléré derrière ce maximum, jamais une AVANCÉE anormalement grande au-delà. Une seule frame polluée par une onde pouvait ainsi faire "sauter" définitivement le maximum courant loin en avant, ce qui rejetait ensuite systématiquement tous les points suivants, pourtant réels, tant que la tendance réelle ne l'avait pas rattrapé (si jamais elle le rattrapait avant la fin de la plage de frames) — exactement le mécanisme reproduisant la courbe de saut rendue "très très hasardeuse" par le passage de l'onde.

`advance_confirm_frames` (1 par défaut = comportement inchangé, une avancée est acceptée immédiatement comme avant) corrige ceci sans introduire de borne de vitesse : une candidate qui avance de plus de `backslide_tolerance_px` au-delà du maximum courant n'est acceptée que si `advance_confirm_frames` points bruts consécutifs, à partir d'elle inclusivement, restent tous à ce niveau avancé (à `backslide_tolerance_px` près) — l'avancée doit donc "tenir" sur plusieurs frames, pas seulement apparaître sur une seule, avant d'être considérée fiable. Une avancée qui reste dans la bande de tolérance autour du maximum courant continue d'être acceptée immédiatement, comme avant. Avec `advance_confirm_frames=1`, cette fenêtre de confirmation se réduit au point lui-même, qui se confirme trivialement — reproduisant exactement l'ancien comportement d'acceptation immédiate, terme à terme, y compris pour toute frame déjà polluée. En fin de plage de frames, il peut ne pas rester assez de points ultérieurs pour confirmer une avancée réelle et rapide — ces derniers points (au plus `advance_confirm_frames - 1`) sont alors rejetés par prudence plutôt que de risquer de figer un pic non confirmé (voir §6 pour cette limite documentée).

3.3 Incertitude de position (`opening_extent_position_uncertainty_px`)

Activée par la case "Estimer l'incertitude de mesure" de la carte "Crack Tip Detection" — construite entièrement à partir de données que cette méthode calcule déjà, sans hypothèse d'imagerie supplémentaire requise (un plancher de bruit DIC déclaré, une tolérance de mire d'étalonnage, …), même si en fournir davantage améliore le résultat. La méthode, en une phrase : perturber CHACUN des trois paramètres réglables du détecteur, un par un (les deux autres tenus à leur valeur nominale), et mesurer de combien la position détectée se déplace ; combiner ces contributions en quadrature avec un plancher de discrétisation. Quand `smooth_window_samples` (§3.2) est actif, cette estimation reste cohérente avec la détection nominale : `opening_extent_position_uncertainty_px` reçoit ce même paramètre et relance chaque perturbation ci-dessous sur `jump_mag_smoothed_px`, jamais sur `jump_mag_px` brut — sans quoi l'enveloppe d'incertitude reviendrait silencieusement à mesurer la sensibilité d'un signal différent de celui réellement utilisé pour détecter le front.

(1) Seuil, contre le bruit propre du signal de saut — `_local_noise_sigma`, un estimateur de bruit "aveugle" (famille Immerkaer, l'analogue 1D de `core.dic_uncertainty.estimate_noise_sigma` en 2D) : à partir de la dispersion des différences premières consécutives de `jump_mag` dans une fenêtre locale (15 échantillons de chaque côté par défaut) autour du front détecté (un signal sous-jacent lisse contribue peu à une différence première, surtout sur une fenêtre aussi étroite, donc la différence est majoritairement du bruit). Pour un bruit gaussien i.i.d. d'écart-type σ, deux échantillons indépendants diffèrent d'une gaussienne d'écart-type $\sigma\sqrt{2}$, dont la valeur absolue moyenne est $\sigma\sqrt{2}\cdot\sqrt{2/\pi} = \sigma\cdot 2/\sqrt{\pi}$ — inversé pour retrouver σ :

$$\hat{\sigma}_{jump} = \overline{|\Delta\,\mathrm{jump}_{mag}|}\cdot\frac{\sqrt{\pi}}{2}$$

Le seuil nominal est ensuite perturbé à threshold$\pm\hat{\sigma}_{jump}$, la marche relancée pour chacune des deux valeurs perturbées, et la demi-largeur de l'enveloppe des positions candidates (nominale incluse) retenue comme cette contribution :

$$\delta_{seuil} = \frac{\max(\text{candidats}) - \min(\text{candidats})}{2}$$

(2) Tolérance de trou (`max_gap_samples \pm 1`) — même relance de la marche, sur le même `jump_mag` déjà calculé, avec une tolérance de trou augmentée/diminuée d'un échantillon ; même formule de demi-largeur d'enveloppe, δ_{gap}.

(3) Décalage d'échantillonnage (`offset_px \pm \max(1\,\text{px},\ 25\%\cdot\text{offset\_px})`) — la seule des trois perturbations nécessitant un véritable RE-échantillonnage des champs bruts (pas seulement une relance de la marche sur un `jump_mag` déjà calculé), puisqu'elle change où les deux points de part et d'autre du chemin sont eux-mêmes placés ; même formule de demi-largeur, δ_{offset}.

(4) Plancher de discrétisation du chemin, indépendant des trois précédents — le pas de rééchantillonnage du chemin tracé lui-même (lu directement dans les données, pas codé en dur, bien qu'il vale 2 px par défaut sur cette page), divisé par $\sqrt{12}$ — la variance standard d'une erreur d'arrondi uniformément distribuée sur un pas :

$$\delta_{grille} = \frac{\text{pas de rééchantillonnage}}{\sqrt{12}}$$

Combinaison finale, en quadrature (hypothèse d'indépendance des erreurs — simplification, puisque les perturbations de seuil et de décalage agissent en réalité toutes deux à travers le même champ bruité sous-jacent, mais une approximation raisonnable pour une estimation pratique, toujours au moins aussi grande qu'une seule contribution isolée, donc jamais silencieusement optimiste) :

$$\sigma_{position} = \sqrt{\delta_{seuil}^2 + \delta_{gap}^2 + \delta_{offset}^2 + \delta_{grille}^2}$$

Une perturbation qui échoue à détecter un front du tout (la marche n'atteint jamais le seuil) est simplement écartée de la somme de cette frame — ce qui sous-estime alors l'incertitude pour cette contribution-là ; documenté explicitement dans le code plutôt que masqué, puisqu'un cas perturbé réellement indétectable signale en général que la détection nominale elle-même est déjà marginale pour cette frame.

3.4 Vitesse locale et son incertitude

Vitesse locale point à point (`trace_velocity_px_per_s`) : dérivée par différences finies centrées (unilatérale aux deux extrémités) de la position d'arc par rapport au temps —

$$v_i = \frac{s_{i+1} - s_{i-1}}{t_{i+1} - t_{i-1}} \quad \text{(intérieur)} \qquad v_0 = \frac{s_1 - s_0}{t_1 - t_0} \qquad v_{n-1} = \frac{s_{n-1} - s_{n-2}}{t_{n-1} - t_{n-2}}$$

Délibérément non lissée/ajustée (voir §3.5 pour cela) — c'est la vitesse instantanée brute, bruitée, point par point, dont a besoin l'historique de vitesse d'une fissure en propagation réelle, pas une seule moyenne en droite.

Incertitude de cette vitesse locale (`trace_velocity_uncertainty_px_per_s`) : propagation d'erreur ordinaire à travers cette MÊME formule de différences finies, en traitant `position_uncertainty_px[i+1]` et `[i-1]` comme indépendantes (chaque position de front est détectée séparément, frame par frame) :

$$\sigma_{v,i} = \frac{\sqrt{\sigma_{s,i+1}^2 + \sigma_{s,i-1}^2}}{t_{i+1} - t_{i-1}} \quad \text{(intérieur, et la règle analogue aux deux extrémités)}$$

`NaN` partout où `trace_velocity_px_per_s` elle-même serait indéfinie (moins de 2 points, pas de temps nul), et partout où l'une des deux incertitudes de position d'entrée est `NaN` (le `NaN` se propage à travers la racine — une incertitude de position inconnue rend l'incertitude de vitesse résultante inconnue, pas nulle). C'est cette paire de tableaux qui alimente les enveloppes ombrées du tracé "position d'arc vs temps" de l'onglet "Vérifier visuellement" (position à gauche, vitesse locale à droite, en pointillés) et sa colonne CSV `local_speed_uncertainty_px_per_s/_mm_per_s` (voir §3.6).

3.5 Vitesse ajustée (régression, pondérée quand une incertitude est disponible)

`fit_tip_ridge_points` fournit, en plus de la vitesse locale ci-dessus, une vitesse "indicative" par régression linéaire (temps, position d'arc) sur l'ensemble de la trace :

- **Sans incertitude fournie** : régression OLS ordinaire non pondérée (`np.polyfit`), avec R^2.
- **Avec `uncertainty_px` fourni** (une incertitude de position par point, §3.3) : devient une régression par moindres carrés pondérée par l'inverse de la variance ($w_i = 1/\sigma_{s,i}^2$), avec une erreur-type propre sur la pente ajustée, issue de la covariance de régression pondérée — résultat classique pour une régression pondérée quand les poids sont de véritables inverses de variance :

$$\text{Var(pente)} = \frac{S_w}{S_w S_{wtt} - S_{wt}^2} \qquad \text{où } S_w = \sum w_i,\ S_{wt} = \sum w_i t_i,\ S_{wtt} = \sum w_i t_i^2$$

Plus de points, ou une incertitude de position plus serrée point par point, resserrent tous deux cette erreur-type comme une vraie mesure de confiance le devrait. **C'est ici qu'intervient la fréquence d'enregistrement** : des points plus nombreux et plus rapprochés dans le temps (espacement temporel plus serré entre frames acceptées = la fréquence d'image elle-même) resserrent directement cette erreur-type, sans qu'aucune "incertitude de fréquence d'image" séparée ne soit nécessaire pour cet effet-là. Un paramètre optionnel `frame_rate_uncertainty_frac` (le champ UI "Frame rate uncertainty (%)", 0 par défaut) ajoute en plus, en quadrature, une véritable tolérance de base de temps caméra (gigue/dérive du déclenchement) — celui-ci, en revanche, doit être fourni par l'utilisateur, car il n'est pas observable depuis les seules données de la trace elle-même :

$$\sigma_{pente}^{final} = \sqrt{\sigma_{pente}^2 + (\text{pente} \times \text{frame_rate_uncertainty_frac})^2}$$

Retombe sur l'ajustement non pondéré (dernier champs `None`) si `uncertainty_px` a moins de deux entrées finies et positives correspondant à des `times`/`arc_length` valides, ou si les équations normales pondérées sont dégénérées.

Position et vitesse d'incertitude sont rapportées en px (`PolygonTipRidgeResult.position_uncertainty_px`, `speed_stderr_px_per_s`/`_mm_per_s` — converties en mm seulement à l'export/l'affichage, par la taille de pixel de la page) ; chacune vaut `None` quand la case "Estimer l'incertitude de mesure" est décochée — convention "`None` signifie non estimée, jamais incertitude nulle" partagée avec tout le reste de l'outil.

3.6 Export CSV

`export_tip_roi_csv` (`core/crack_tip_tracking.py`) écrit, au format long (une ligne par point de chemin par trace), la position d'arc, la vitesse locale (§3.4), et — quand l'incertitude a été estimée — les colonnes `position_uncertainty_px`/`_mm`, `local_speed_uncertainty_px_per_s`/`_mm_per_s` (par point, §3.4) et `trace_speed_stderr_px_per_s`/`_mm_per_s` (une seule valeur par trace entière, §3.5) ; ces colonnes n'apparaissent dans l'en-tête que si au moins une trace exportée a effectivement une incertitude estimée. Le bouton dédié "Export this plot to CSV…" de l'onglet "Vérifier visuellement" écrit exactement le même contenu, restreint aux traces `opening_jump` de la feuille (leaf) courante — celles que ce tracé affiche réellement.

`export_opening_jump_profile_csv` écrit, pour une seule frame, le profil complet `jump_u_px/jump_v_px/jump_mag_px` le long du chemin (pas seulement ce que montre le tracé — la ligne de seuil et le marqueur de front détecté — mais le signal brut lui-même), avec un indicateur `is_open_at_this_point` par point.

---

4. Facteurs d'intensité de contrainte K_I/K_II : X-FEM-DIC (`core/xfem_dic.py`)

X-FEM-DIC est la méthode de calcul de K_I/K_{II} exposée sur l'onglet "Crack analysis" (voir §5 pour les méthodes additionnelles disponibles dans le code mais non câblées dans l'interface).

4.1 Principe et justification : pourquoi les images brutes

La DIC par subset (`core/dic_processing.py`, chapitre 6) suit chaque point comme un gabarit RIGIDE, purement translaté (corrélation croisée normalisée `cv2.matchTemplate` sur un gabarit de référence fixe). Un gabarit dont le patch d'image chevauche la face de fissure contient des pixels des deux côtés d'une vraie discontinuité de déplacement ; aucune translation rigide ne peut alors correspondre au gabarit correspondant une fois la fissure ouverte, même faiblement — c'est exactement pourquoi l'ajustement en anneau disponible dans le code (§5.1) doit déjà exclure une région `r_min_px` et une bande `flank_exclusion_deg` autour des faces de fissure. Ajuster un modèle plus élaboré (même enrichi) APRÈS COUP sur ce champ déjà exporté ne peut jamais que ré-ajuster les points propres, loin de la fissure — cela ne peut pas récupérer la précision en champ proche que l'enrichissement est censé apporter, car le biais dû à la discontinuité est déjà ancré dans l'entrée avant même qu'un post-traitement quelconque ne la voie.

X-FEM-DIC re-corrèle donc directement la paire d'images brutes référence/déformée elle-même, avec un modèle de déplacement qui contient DÉJÀ la discontinuité de fissure et le champ singulier en champ proche comme termes de premier ordre — une formulation "Integrated DIC" (Hild & Roux ; papiers "eXtended-DIC"/"X-DIC" de Réthoré et al.), résolue directement contre les niveaux de gris des pixels plutôt que contre un nuage de points déjà corrélé.

4.2 Modèle de déplacement et simplifications volontaires

Un code XFEM textbook enrichit des NOEUDS individuels près de la fissure (Heaviside pour les éléments totalement coupés, fonctions de branche de Williams pour les éléments touchant la pointe), chacun portant ses propres degrés de liberté locaux. Deux simplifications délibérées gardent ce module traitable, tout en restant ancrées dans la littérature (proche de la façon dont les formulations X-DIC de Réthoré procèdent réellement — un enrichissement singulier global plutôt que nodal est un choix publié, pas un raccourci ad hoc) :

1. **Le terme singulier de pointe est GLOBAL**, pas nodal : exactement deux inconnues scalaires supplémentaires pour tout le maillage, K_I et K_{II}, multipliant les mêmes fonctions de base Williams/Irwin fermées que l'ajustement en anneau (`_williams_basis` — délibérément les mêmes formules/signe/convention κ, pour que les K_I/K_{II} de cette méthode restent directement comparables à ceux des méthodes additionnelles du §5), fenêtrées de façon lisse pour tendre vers zéro loin de la pointe (`_tip_window`) afin qu'il ne puisse pas fuir vers le champ de fond et le biaiser. K_I/K_{II} sortent

donc directement de la corrélation comme deux de ses paramètres ajustés — pas de post-ajustement en anneau séparé nécessaire.

2. **Le saut de face de fissure est nodal mais NON DÉCALÉ** : $H(x, y)$ est évalué au point de requête lui-même (pas la forme standard décalée $H(x) - H(x_i)$) — plus simple, au prix d'un mélange légèrement moins local juste à la frontière d'un élément coupé avec ses voisins non enrichis, sans conséquence à la densité d'échantillonnage utilisée ici.

Le reste — maillage Q4 structuré, fonctions de forme bilinéaires, Gauss-Newton "forward-additive" contre les niveaux de gris avec un jacobien analytiquement exact (le modèle de déplacement est LINÉAIRE en chacune de ses inconnues ; seule la déformation de l'image elle-même est non linéaire) — est standard.

4.3 Maillage Q4 centré sur la pointe (uniforme ou gradué)

Le maillage de fond (`Q4Mesh`) est une grille d'éléments quadrilatéraux bilinéaires (Q4), stockée par les coordonnées physiques de chaque colonne/ligne de noeuds (`x_edges`/`y_edges`, strictement croissantes) plutôt que par une origine + une taille d'élément uniforme — ce qui permet à `build_graded_tip_centered_mesh` de graduer la taille d'élément de façon continue, fine près de la pointe et grossière loin d'elle, alors que le reste du module (localisation, fonctions de forme, l'ajustement lui-même) reste inchangé pour un maillage uniforme (`build_q4_mesh`, cas particulier où les pas sont constants). Rester axis-aligned rend la localisation d'un point physique (quel élément le contient, et ses coordonnées isoparamétriques locales) directe (une recherche `searchsorted` par axe) — pas de résolution de Newton par point nécessaire, contrairement à un maillage EF général non structuré.

`build_tip_centered_mesh(tip_xy, roi_half_width_px, element_size_px)` construit la version uniforme ; `build_graded_tip_centered_mesh` la version graduée, avec une zone fine (`element_size_px`) sur un rayon `r_enrich_px` autour de la pointe, croissant jusqu'à `element_size_px * coarsen_factor` sur le reste de `roi_half_width_px` — meilleure résolution près de la pointe et moins d'éléments (plus grands) loin de la fissure pour la même région globale, sans coût Gauss-Newton supplémentaire (l'ajustement ne se soucie pas de la taille d'un élément donné, seulement de la localisation `(ξ,η)` par échantillon).

Le bouton "Suggest settings" de l'interface (`recommend_mesh_parameters`) pré-remplit `element_size_px`, `r_enrich_px` et `roi_half_width_px` à partir de la distance de fissure déjà suivie (et, si l'image est déjà chargée, plafonne `roi_half_width_px` pour rester dans l'image) — des heuristiques assumées comme telles dans le code (pas dérivées d'une analyse de premier principe), à affiner ensuite au besoin avec "Preview mesh". Il propose aussi une valeur de départ pour l'espacement d'échantillonnage (`sample_step_px`, "Sample spacing" dans l'interface, voir §4.5 pour ce que ce réglage fait exactement) : `element_size_px / 8`, plafonné entre 1 et 10 px — grossièrement plusieurs échantillons à travers même le plus petit élément (celui près de la pointe sur un maillage gradué), sans descendre arbitrairement sous la longueur de corrélation propre à un vrai mouchetis. Cette proposition n'est qu'un point de départ raisonnable, pas une valeur optimale calculée : elle reste éditable, et le §4.7 détaille pourquoi la diminuer sans discernement en dessous a un coût sur l'incertitude formelle rapportée, pas seulement sur le temps de calcul.

4.4 Classification de l'enrichissement (`classify_mesh_enrichment`)

Un noeud est enrichi Heaviside quand un élément qui le porte est "coupé" par le chemin de fissure déjà tracé, en ne comptant que la partie DÉJÀ OUVERTE (arc de chemin $\leq$ tip_s). Un élément est classé "coupé"

quand les distances signées de ses 4 coins au chemin changent de signe, que le croisement le plus proche est réellement à proximité (moins de 1.5 fois la taille locale de l'élément — une bande LOCALE, pas globale, pour rester correcte sur un maillage gradué où un élément lointain grossier peut être bien plus grand que la taille la plus fine du maillage) et qu'il n'est pas clairement au-delà de la pointe actuelle. La projection se fait toujours sur le chemin COMPLET (jamais tronqué à `s <= tip_s`) puis filtrée après coup par arc length projeté — tronquer d'abord la polyligne biaiserait la projection d'un noeud proche de la pointe vers l'extrémité artificielle de la troncature plutôt que la géométrie réelle de la fissure.

4.5 Résolution : Gauss-Newton amorti, mise à l'échelle de Jacobi

Les résidus de niveaux de gris comparés à chaque itération ne sont pas évalués à chaque pixel ni aux noeuds du maillage, mais sur une grille RÉGULIÈRE de points d'échantillonnage construite indépendamment du maillage Q4 lui-même (`_sample_points(mesh, step_px)`, espacement `sample_step_px`, "Sample spacing" dans l'interface, 2 px par défaut) — cette grille ignore complètement les frontières d'éléments ; le maillage ne sert qu'à interpoler le déplacement en chaque point de cette grille (fonctions de forme) et à classifier l'enrichissement (§4.4). Un espacement trop grossier prive les plus petits éléments en jeu (ceux proches de la pointe sur un maillage gradué, §4.3) d'échantillons suffisants dans leurs bornes : leurs noeuds de coin peuvent alors se retrouver sous-contraints et signalés non fiables (`node_reliable=False`, laissés en blanc sur les isocontours). Un espacement trop fin, à l'inverse, n'apporte plus d'information réelle une fois sous la longueur de corrélation propre du mouchetis réel (typiquement quelques pixels) : les échantillons voisins deviennent de plus en plus redondants plutôt que plus informatifs, pour un coût qui croît en $1/\text{pas}^2$ par itération — et, comme détaillé au §4.7, peut faire paraître l'incertitude formelle sur K_I/K_{II} plus petite qu'elle ne l'est réellement.

Chaque frame est résolue par Gauss-Newton "forward-additive" directement contre les niveaux de gris de l'image déformée (interpolée par spline bicubique `RectBivariateSpline`), avec un vecteur d'inconnues a organisé ainsi : $[a_x\ (n_{noeuds}),\ a_y\ (n_{noeuds}),\ b_x\ (n_{Heav}),\ b_y\ (n_{Heav}),\ K_I,\ K_{II}]$. À chaque itération :

1. Le déplacement courant (u, v) est évalué à chaque point d'échantillonnage (fonctions de forme du maillage de fond + saut de Heaviside pondéré par le signe local + terme singulier fenêtré), donnant la position déformée avertie $(x + u, y + v)$.
2. Le résidu de niveau de gris $I_{ref} - I_{def}(x + u, y + v)$ est calculé, ainsi que le gradient de l'image déformée au point averti (chaîne analytique exacte pour tous les degrés de liberté, sauf le facteur du gradient de l'image lui-même, ré-évalué chaque itération).
3. Un pas Gauss-Newton amorti est résolu par moindres carrés creux (`scipy.sparse.linalg.lsqr`, avec un paramètre d'amortissement `damping`), APRÈS mise à l'échelle de Jacobi des colonnes : les degrés de liberté de fond/Heaviside sont $O(1)$ en PIXELS, mais K_I/K_{II} sont $O(10^6)$ en Pa·√m — sans cette mise à l'échelle, une valeur d'amortissement unique ne peut régulariser équitablement les deux groupes (elle contraindrait à peine les degrés de liberté à l'échelle du pixel, ou écraserait les colonnes K_I/K_{II} à près de zéro, laissant leur estimation au niveau du bruit).
4. Convergence quand le déplacement maximal de tout degré de liberté (hors K_I/K_{II}) entre deux itérations descend sous `convergence_tol_px` (2×10^{-3} px par défaut), ou après `max_iterations` (15 par défaut, réglable de 5 à 500 dans l'interface).

Ce critère de pas ne dit rien sur l'amélioration réelle de l'ajustement, seulement sur sa taille — et

cela produit un comportement observé en pratique sur des données réelles : à grand déplacement de pointe, le signal (gradient de niveau de gris dû au vrai mouvement) domine largement le bruit de corrélation et le bruit numérique de la dérivée de spline, donc les corrections Gauss-Newton décroissent géométriquement et franchissent `convergence_tol_px` en quelques itérations ; à petit déplacement de pointe (fissure à peine ouverte), ce même signal devient comparable à ce bruit, les équations normales de chaque itération sont alors dominées par du bruit plutôt que par un vrai gradient de descente, et comme `damping` est FIXE (pas de schéma Levenberg-Marquardt adaptatif), le pas ne décroît pas nécessairement vers zéro : il peut chahuter indéfiniment autour d'un plancher de bruit au-dessus de `convergence_tol_px`, quel que soit `max_iterations` — ce n'est alors pas une convergence lente qu'un budget d'itérations plus grand résoudrait, mais un pas qui ne peut structurellement pas atteindre CE seuil précis sur ce type de frame.

Deux mécanismes, orthogonaux au critère de pas ci-dessus, répondent à cela :

- **Arrêt sur plateau du résidu** (`stall_patience`, 5 itérations par défaut ; `stall_rtol`, 0.1% par défaut) : le RMS des niveaux de gris de chaque itération est comparé au meilleur RMS observé jusqu'ici sur cette frame ; le compteur "itérations sans amélioration réelle" ne se remet à zéro que si l'amélioration dépasse `stall_rtol` (relatif) — une variation marginale ou négative (typique du chahutement) le laisse courir. Après `stall_patience` itérations consécutives sans amélioration franche, la boucle s'arrête, rapportée `stalled=True` — gardé strictement distinct de `converged=True` (une vraie convergence par taille de pas), pour qu'un appelant qui préfère déjà un candidat convergé à un candidat non convergé (§4.6, recherche automatique de pointe) ne se voie jamais silencieusement remettre un ajustement en plateau comme s'il s'agissait d'une convergence propre.
- **État de meilleur RMS observé, pas simplement le dernier itéré** : quel que soit le critère qui arrête la boucle — convergence propre, plateau, ou simple épuisement de `max_iterations` sans qu'aucun des deux ne se déclenche — l'état des degrés de liberté effectivement rapporté est celui qui a produit le RMS le plus bas sur TOUTE la trajectoire (y compris son tout dernier itéré), jamais nécessairement le dernier itéré de la boucle. Pour une frame qui converge déjà proprement par taille de pas, ceci ne change rien : son état de meilleur RMS coïncide, dans l'immense majorité des cas, avec son état final, puisque le RMS décroît de façon quasi monotone dans ce régime bien conditionné. Pour une frame qui chahute, ceci garantit qu'un arrêt anticipé (par plateau) ou tardif (par épuisement du budget) ne fige jamais le résultat sur un point du chahutement arbitrairement pire qu'un autre déjà rencontré.

Dans le texte de statut de l'onglet "Crack analysis" et dans `export_xfem_k_csv` (colonne `xfem_stalled`, §4.9), une frame en plateau est distinguée d'une frame simplement non convergée : contrairement à cette dernière, augmenter "Max iterations" n'a structurellement aucune raison d'aider une frame en plateau (elle ne convergeait pas lentement, elle a cessé de s'améliorer) — le message affiché le dit explicitement plutôt que de répéter le même conseil ("essayez un plus grand Max iterations") qui s'applique à l'autre cas.

4.6 Détection automatique de la position de la pointe

La méthode réutilise toujours le CHEMIN de fissure déjà tracé à l'étape 1 — "chemin fixe" est le périmètre délibérément choisi pour cette fonctionnalité (pas de re-détection de la géométrie de fissure elle-même). Ce qui PEUT être trouvé indépendamment, frame par frame, est la POSITION de la pointe le long de ce chemin déjà tracé :

- **Trace fixe (`auto_detect_tip=False`, par défaut)** : la position à chaque frame est lue directement

depuis le suivi déjà réalisé à l'étape 3 (`positions_xy/arc_length_s`) — inchangé, sans re-détection.

- **Détection automatique (`auto_detect_tip=True`)**, deux algorithmes au choix (`tip_detection_mode`) :
  - **"residual" (par défaut)** : recherche par grille de positions candidates le long du même chemin, dans un rayon `tip_search_radius_px` autour de la position nominale de CETTE frame (chaque frame est ancrée indépendamment à l'estimation de l'étape 3, PAS à la position auto-trouvée de la frame précédente — pour éviter qu'une seule erreur de recherche ne dérive de façon irréversible toutes les frames suivantes), au pas `tip_search_step_px`, retenant le candidat CONVERGÉ au résidu de corrélation en niveaux de gris le plus faible (repli sur le meilleur candidat non convergé seulement si aucun n'a convergé).
  - **"heaviside_jump"** : ajuste un maillage d'essai délibérément sur-couvrant (démarrant à la position nominale, puis poussé plus loin par pas croissants bornés, tant que la fissure semble encore ouverte jusqu'au bord même de l'essai), puis lit la position raffinée directement sur l'amplitude du saut de Heaviside ajustée par ce fit (`heaviside_disp_x/y`), en localisant où l'ouverture de face tombe sous `heaviside_jump_threshold_px` — généralement bien moins coûteux (1-2 ajustements complets par frame au lieu de $2 \times \text{radius}/\text{step} + 1$), au prix d'une résolution spatiale plus grossière (précise à environ un à deux largeurs d'élément, pas au pixel près comme la recherche par résidu).

Les deux modes imposent la même contrainte physique : une fissure ne peut pas rétrécir — la position ACCEPTÉE à chaque frame ne peut jamais reculer (en arc length, dans le sens de croissance déduit de la tendance globale de l'étape 3) par rapport à la dernière position que cette méthode a réellement acceptée.

4.7 Incertitude formelle (covariance de Gauss-Newton asymptotique)

`estimate_uncertainty=True` (opt-in, coche "Estimer l'incertitude de mesure (K_I/K_II)") calcule `K_I_stderr_pa_sqrt_m/K_II_stderr_pa_sqrt_m` à partir de la covariance asymptotique de Gauss-Newton que tout ajustement non linéaire par moindres carrés admet en son point convergé — exactement la même covariance de moindres carrés (linéaire, au point de linéarisation locale) que le §5.1, appliquée ici à la linéarisation locale de l'ajustement non linéaire plutôt qu'à un modèle exactement linéaire (la façon classique, manuel de rapporter l'incertitude de paramètre d'un ajustement non linéaire par moindres carrés) :

1. **Le jacobien résiduel J est ré-assemblé UNE FOIS DE PLUS à la solution FINALE convergée** — pas réutilisé depuis la dernière itération de la boucle Gauss-Newton, dont le propre J a été évalué un pas EN ARRIÈRE par rapport à l'état final a.
2. La variance résiduelle en niveaux de gris est $\sigma^2_{gris} = SSE/(n_{effectif} - n_{ddl})$, où $n_{effectif}$ est le nombre d'échantillons réellement dans les bornes de l'image déformée à cette itération finale (pas $n_{samples}$ brut) et n_{ddl} le nombre total de degrés de liberté du modèle (fond + Heaviside + K_I/K_{II}).
3. La covariance des paramètres est $\text{cov} = \sigma^2_{gris} \cdot \text{pinv}(J^T J)$ — avec la même mise à l'échelle de Jacobi des colonnes qu'à la résolution du §4.5 : les colonnes de K_I/K_{II} (valeurs de base de Williams pondérées par le gradient d'image) diffèrent des colonnes de déplacement de fond/Heaviside par plusieurs ordres de grandeur, et inverser la matrice normale BRUTE tronquerait silencieusement la variance minuscule-mais-réelle de K_I/K_{II} au lieu de la rapporter honnêtement.

$$\sigma^2_{gris} = \frac{SSE}{n_{eff} - n_{ddl}} \qquad \text{cov} = \sigma^2_{gris} \cdot \text{pinv}\left(\left(\frac{J}{\text{col_scale}}\right)^T \left(\frac{J}{\text{col_scale}}\right)\right) \Big/ (\text{col_scale}_i \cdot \text{col_scale}_j)$$

Cette estimation est opt-in (contrairement à l'ajustement en anneau du §5.1, toujours actif) car ré-assembler J et inverser la matrice normale $n_{ddl} \times n_{ddl}$ résultante coûte un temps réel non négligeable sur un maillage fin (n_{ddl} croît avec le nombre de noeuds du maillage), en plus d'un ajustement déjà, par la propre documentation du module, "bien plus lourd" que la simple résolution linéaire de l'ajustement en anneau.

Une source d'incertitude que cette formule ne capture pas : des résidus corrélés par un `sample_step_px` trop fin (§4.5). $\sigma^2_{gris} = SSE/(n_{eff} - n_{ddl})$ et la matrice normale $J^T J$ traitent implicitement chaque échantillon de la grille comme une observation INDÉPENDANTE — l'hypothèse standard de toute covariance de moindres carrés (déjà relevée comme non universellement vérifiée au point 2 du §6). Un vrai mouchetis n'a cependant qu'une longueur de corrélation finie (typiquement quelques pixels) : en dessous, deux échantillons voisins voient essentiellement la même information de niveau de gris plutôt que deux mesures véritablement indépendantes. Échantillonner plus fin que cette longueur gonfle n_{eff} sans ajouter de contrainte réellement nouvelle sur K_I/K_{II} — σ^2_{gris} et, par suite, K_I_stderr/K_{II}_stderr peuvent alors ressortir plus petits que l'incertitude réelle ne le justifie, pour la même raison qu'une moyenne sur des mesures répétées mais corrélées sous-estime sa propre erreur-type si on la traite comme portant sur des mesures indépendantes. Le code ne corrige pas cet effet (pas de correction de degrés de liberté effectifs à la Bretherton ni de repondération par la longueur de corrélation) : la seule protection est de ne pas descendre `sample_step_px` arbitrairement bas — le §4.3/§4.5 recommandent de ne l'affiner sous la valeur proposée par "Suggest settings" que tant que K_I/K_{II} eux-mêmes continuent réellement de changer, pas par habitude ou pour "plus de précision".

Cas de la détection automatique de pointe (`auto_detect_tip=True`) : la recherche elle-même (par résidu ou par saut de Heaviside) n'est JAMAIS invitée à estimer une incertitude pour chacun de ses (potentiellement nombreux) candidats testés — cela multiplierait le coût déjà élevé de cette recherche par le nombre de candidats évalués, pour une incertitude immédiatement jetée sur $N - 1$ ajustements. Une fois la position gagnante connue, cette position est réajustée UNE seconde fois avec `estimate_uncertainty=True`, et ce second ajustement (déterministe — mêmes K_I/K_{II} que le premier, seuls ses champs d'incertitude diffèrent) devient la valeur retenue pour cette frame.

`K_I_stderr_pa_sqrt_m`/`K_II_stderr_pa_sqrt_m` restent `None` si les entrées diagonales pertinentes de la matrice normale mise à l'échelle ressortent non finies ou négatives (ajustement dégénéré/à rang déficient), plutôt que de rapporter un nombre trompeur.

4.8 Convention "K_I critique" et enveloppe d'incertitude sur le tracé

$K_I \leq 0$ n'a pas de sens physique (une pointe de fissure fermée/comprimée n'a pas d'intensité de contrainte en mode I) — contrairement à K_{II}, dont le SIGNE est physiquement significatif (le sens du cisaillement), donc jamais filtré de cette façon. La valeur "critique" rapportée pour K_I — et les marqueurs de bornes de domaine sur les tracés — reflète donc uniquement la portion positive d'une trace, sur chaque intervalle continu (une trace réelle, bruitée, repasse fréquemment sous zéro plus d'une fois, pas seulement à son tout début ou sa toute fin) :

$$\overline{K_I}^{crit} = \text{moyenne}\big(\{K_I(s) : K_I(s) > 0\}\big)$$

implémentée par `positive_value_mean_and_domain_bounds` — les échantillons non finis sont écartés en premier, jamais traités comme une borne de domaine ; les échantillons sont triés par distance avant de chercher les intervalles, pour que "contigu" signifie contigu EN DISTANCE.

Sur le tracé K_I/K_{II}(distance) de l'onglet "Crack analysis", un menu déroulant "Uncertainty envelope" (Off/±1/±2/±3σ) contrôle une enveloppe ombrée (`fill_between`) autour de la courbe X-FEM-DIC quand elle dispose d'une incertitude formelle (case "Estimer l'incertitude de mesure" cochée pour ce calcul, §4.7) — avec la courbe elle-même toujours dessinée par-dessus sa propre bande (via l'ordre de calque `zorder`). La ligne de moyenne K_I/K_{II} critique (dans le texte de statut et sur le tracé) n'est affichée que si la case "K_I"/"K_II" correspondante est cochée.

Le tracé affiche, par trace, une courbe K_I (trait plein, marqueur rond) et une courbe K_{II} (tirets, marqueur carré), avec la même couleur par trace et l'étiquette de légende `"{trace} -- K_I"/"{trace} -- K_II"`.

4.9 Export CSV (`export_xfem_k_csv`)

Écrit, au format long (une ligne par échantillon de frame par trace), les colonnes suivantes : identifiants de trace/frame, distance d'arc (px et mm), K_I/K_{II} (X-FEM-DIC) avec leurs erreurs-types (§4.7, vides si non estimées), `xfem_converged` et `xfem_stalled` (§4.5 – jamais tous deux à 1 sur la même ligne), un indicateur `is_default_critical`, et la moyenne critique (§4.8) de K_I positif, répétée identiquement sur chaque ligne de la trace correspondante, pour un filtrage/pivotement aisé dans un tableur.

---

5. Méthodes additionnelles disponibles dans le code (`core/crack_k1c.py`, `core/crack_interaction_integral.py`)

Deux méthodes supplémentaires de calcul de K_I/K_{II} sont implémentées et testées (`test_crack_k1c.py`, `test_crack_interaction_integral.py`), pour un usage programmatique, mais ne sont pas câblées dans l'onglet "Crack analysis" où X-FEM-DIC (§4) est la seule méthode exposée. Toutes deux utilisent en entrée le champ de déplacement u/v déjà exporté par la DIC standard (chapitre 6) plutôt qu'une nouvelle corrélation sur les images brutes.

5.1 Ajustement DIC standard en anneau (`fit_k_single_frame`, `compute_k_time_series`)

Cette méthode échantillonne le champ de déplacement dans un anneau autour d'une position de pointe donnée, le tourne dans le repère local (tangente, normale) de la fissure, et l'ajuste contre le champ de déplacement en pointe de fissure LEFM, mode I + II, au premier ordre (Williams/Irwin) :

$$u' = \frac{K_I}{2\mu}\sqrt{\frac{r}{2\pi}}\cos\frac{t}{2}\Big(\kappa - 1 + 2\sin^2\frac{t}{2}\Big) + \frac{K_{II}}{2\mu}\sqrt{\frac{r}{2\pi}}\sin\frac{t}{2}\Big(\kappa + 1 + 2\cos^2\frac{t}{2}\Big) + T_x$$

$$v' = \frac{K_I}{2\mu}\sqrt{\frac{r}{2\pi}}\sin\frac{t}{2}\Big(\kappa+1-2\cos^2\frac{t}{2}\Big) - \frac{K_{II}}{2\mu}\sqrt{\frac{r}{2\pi}}\cos\frac{t}{2}\Big(\kappa-1-2\sin^2\frac{t}{2}\Big) + T_y$$

où (r,θ) sont les coordonnées polaires centrées sur la pointe ($\theta=0$ droit devant la pointe, $\theta=\pm\pi$ le long des deux faces ouvertes de la fissure), κ la constante de Kolosov ($3-4\nu$ en déformation plane, $(3-\nu)/(1+\nu)$ en contrainte plane) et $\mu = E/(2(1+\nu))$ le module de cisaillement. T_x, T_y sont un décalage de corps rigide local INCONNU, inclus comme deux paramètres d'ajustement libres supplémentaires plutôt que supposés nuls — un champ de déplacement DIC est mesuré par rapport au repère (arbitraire) de référence, pas par rapport à la pointe de fissure elle-même ; sans eux, l'ajustement absorberait silencieusement ce décalage dans un K_I/K_{II} biaisé.

Ce modèle est LINÉAIRE dans ses 4 inconnues (K_I, K_{II}, T_x, T_y), donc un simple ajustement par moindres carrés ordinaires (OLS) suffit à chaque frame, avec un R^2 par frame directement vérifiable. L'échantillonnage couvre un anneau complet autour de la pointe (l'ouverture de la fissure au travers de ses deux faces, θ proche de $\pm\pi$, est là où l'ajustement est le plus sensible à K_I), exclu seulement au niveau des faces elles-mêmes (`flank_exclusion_deg`), là où la corrélation DIC standard échoue couramment sur la discontinuité réelle de l'image. `r_min_px`/`r_max_px` bornent l'anneau radialement, excluant respectivement le bruit/la décorrélation DIC trop près de la pointe et l'écart au champ singulier au premier ordre trop loin d'elle.

`recommend_flank_exclusion_deg` (§5.3) fournit une valeur de départ pour `flank_exclusion_deg` adaptée à la trace analysée plutôt qu'une constante fixe pour toutes les fissures.

Cette méthode est "statique d'abord" : un ajustement dynamique (type Freund, noyau dépendant de la vitesse de fissuration) est une extension possible, non implémentée.

Incertitude formelle (covariance OLS, mise à l'échelle de Jacobi) : le modèle étant linéaire, la théorie OLS classique s'applique directement et exactement (pas de linéarisation/approximation nécessaire, contrairement à X-FEM-DIC, §4.7) : avec une variance résiduelle $\sigma^2 = SS_{res}/(n_{eq} - n_{param})$ et une matrice de plan X (2 lignes par point valide — une pour u, une pour v — 4 colonnes, une par inconnue), la covariance des paramètres est $\sigma^2 \cdot (X^TX)^{-1}$, et l'erreur-type de chaque paramètre est la racine carrée de son propre terme diagonal. Ceci est TOUJOURS calculé (pas de case à cocher, contrairement à X-FEM-DIC, §4.7) car le coût est quasi nul par rapport à la résolution `lstsq` déjà effectuée — inverser une matrice 4×4, pas refaire un ajustement.

Les colonnes propres à K_I/K_{II} (valeurs de la base de Williams en champ proche) valent environ 10^{-13} (en mètres par Pa·√m), tandis que les colonnes de T_x/T_y valent exactement 1.0 — un écart d'environ 10^{13} qui fait que les valeurs propres de X^TX s'étalent sur ~26 ordres de grandeur. Le seuil `rcond` par défaut de `pinv` est relatif à la PLUS GRANDE valeur singulière, donc il traiterait silencieusement les directions K_I/K_{II} (minuscules mais réelles) comme numériquement nulles et les éliminerait — ce qui ne fait pas exploser leur variance (comme un ajustement mal conditionné/quasi singulier le ferait normalement) mais la tronquerait au contraire à presque rien. Mettre chaque colonne à l'échelle d'une norme L^2 unitaire avant `pinv` (le même artifice de préconditionnement de Jacobi que la résolution Gauss-Newton de X-FEM-DIC, §4.5/§4.7) égalise l'ordre de grandeur de chaque colonne avant que `pinv` ne voie la matrice, de sorte que son seuil `rcond` ne discrimine plus entre elles ; la vraie covariance est alors récupérée en "dé-échelant" (division par le carré de l'échelle de chaque colonne) :

$$\text{col_scale}_j = \sqrt{\sum_i X_{ij}^2} \qquad \text{cov} = \frac{\sigma^2 \cdot \text{pinv}\big((X/\text{col_scale})^T (X/\text{col_scale})\big)}{\text{col_scale}_i \cdot \text{col_scale}_j}$$

`K_I_stderr_pa_sqrt_m`/`K_II_stderr_pa_sqrt_m` valent `None` dès que moins de 3 points valides sont disponibles (en dessous du plancher `n_valid < 6` déjà appliqué en amont) ou que la matrice normale mise à l'échelle reste singulière (variance non finie ou négative).

La convention "K_I critique = moyenne des valeurs positives" (§4.8) et `positive_value_mean_and_domain_bounds` sont partagées avec X-FEM-DIC.

5.2 Extension optionnelle : terme de T-stress (série de Williams étendue)

Le modèle du §5.1 ne retient que le terme $n = 1$ (singulier, en $1/\sqrt{r}$) de la série de Williams complète. Le terme $n = 2$ suivant est un déplacement induit par une contrainte UNIFORME parallèle aux faces de la fissure — le "T-stress" T, classiquement défini par $\sigma_{xx} = T$ (repère local, faces de fissure selon x) en plus du champ singulier. Il s'ajoute comme un CINQUIÈME paramètre d'ajustement linéaire, A_2, sur les mêmes colonnes u'/v' que le §5.1 :

$$u'_T = A_2 \cdot \frac{\kappa + 1}{2\mu}\, x' \qquad v'_T = A_2 \cdot \frac{\kappa - 3}{2\mu}\, y' \qquad T = 4A_2$$

où (x', y') sont les coordonnées cartésiennes locales (tangente, normale) déjà utilisées pour tourner le champ. Le facteur $\kappa - 3$ est négatif pour tout coefficient de Poisson physique, ce qui reproduit automatiquement, une fois $T = 4A_2$ substitué, le résultat classique $v = -(\nu T/E)\, y$ d'une contrainte uniforme sans qu'aucun signe supplémentaire n'ait besoin d'être ajouté à la main.

Le modèle reste LINÉAIRE (5 inconnues au lieu de 4), donc toujours un simple ajustement OLS ; son erreur-type, $T_{stderr} = 4\sqrt{\text{Var}(A_2)}$, sort de la MÊME matrice de covariance 5×5 que $K_I/K_{II}/T_x/T_y$ (§5.1), avec la même mise à l'échelle de Jacobi des colonnes appliquée à la fois à la résolution `lstsq` elle-même et à `pinv` : les colonnes propres à $K_I/K_{II}/T$ (valeurs de base en champ proche, $\sim 10^{-13}$–10^{-14}) restent des ordres de grandeur plus petites que les colonnes T_x/T_y (=1.0 exactement), et un seuil `rcond` non compensé par cette mise à l'échelle tronquerait silencieusement la direction de plus petite magnitude (le T-stress, la plus petite des trois).

Ce contrôle n'est pas un simple raffinement de précision : c'est un DEUXIÈME contrôle de cohérence sur l'ajustement en anneau lui-même (au-delà du R^2, §5.1) — un T-stress ajusté anormalement grand, ou dont l'erreur-type n'est pas petite devant sa valeur, indique typiquement que l'anneau échantillonné (`r_max_px` trop grand, ou un champ de fond réellement non uniforme sur le domaine) capte un gradient de champ lointain que le modèle au premier ordre seul absorberait silencieusement dans un K_I/K_{II} biaisé — une piste de diagnostic directement actionnable (réduire `r_max_px`) plutôt qu'une seule valeur numérique de plus à ignorer.

5.3 Intégrale d'interaction / J-integral (`core/crack_interaction_integral.py`)

Approche mathématiquement distincte de l'ajustement en anneau : une intégrale de domaine énergétique plutôt qu'un ajustement ponctuel de déplacement — une technique classique et bien établie de la mécanique

de la rupture (Yau, Wang & Corten, 1980 ; Moran & Shih, 1987), adaptée à la DIC par Réthoré, Gravouil, Morestin & Combescure (2005). Le champ RÉEL (le même déplacement u/v déjà exporté par la DIC standard, entrée identique au §5.1) est combiné, sous forme d'intégrale sur un domaine annulaire pondéré par une fonction q, contre un champ AUXILIAIRE connu — le champ de Williams/Irwin en mode I ou en mode II pur, à $K = 1$:

$$M^{(1,2)} = \iint \left[\sigma_{ij}^{(1)} u_{i,1}^{(2)} + \sigma_{ij}^{(2)} u_{i,1}^{(1)} - W^{(1,2)}\delta_{1j}\right] q_{,j}\, dA \qquad W^{(1,2)} = \sigma_{ij}^{(1)}\varepsilon_{ij}^{(2)}$$

où (1) désigne le champ réel et (2) le champ auxiliaire (ou l'inverse — l'intégrale est symétrique). La relation standard reliant cette intégrale aux facteurs d'intensité de contrainte est $M = (2/E^*)(K_I^{(1)}K_I^{(2)} + K_{II}^{(1)}K_{II}^{(2)})$, avec $E^* = E$ en contrainte plane et $E^* = E/(1-\nu^2)$ en déformation plane — en choisissant le champ auxiliaire comme un mode I pur à $K_I^{(2)} = 1$ (respectivement mode II pur), l'intégrale isole directement le $K_I^{(1)}$ (resp. $K_{II}^{(1)}$) du champ réel.

Le domaine d'intégration réutilise le même anneau que le §5.1 : mêmes `r_min_px`/`r_max_px`/`flank_exclusion_deg`, pour que les deux méthodes restent directement comparables sur un domaine de données identique. La fonction de pondération q retenue est délibérément simple — conique, $q = 1$ en r_{min}, $q = 0$ en r_{max}, linéaire entre les deux — plutôt que la forme "plateau" (constante sur l'essentiel du domaine, décroissance rapide seulement près du bord) plus habituelle dans la littérature, pour la comparabilité architecturale avec l'anneau du §5.1 plutôt que pour une précision numérique optimale.

Différence avec le §5.1 sur `r_min_px` : cette méthode exige `r_min_px` STRICTEMENT positif, alors que l'ajustement en anneau (et X-FEM-DIC) tolèrent `r_min_px=0` ("désactivé") sans problème. La raison est physique : le champ auxiliaire à $K = 1$ utilisé pour isoler K_I/K_{II} a une CONTRAINTE singulière en $r = 0$ (le point même de la singularité que K_I/K_{II} caractérisent), donc un domaine qui atteindrait $r = 0$ intégrerait à travers cette singularité plutôt que d'évaluer une fonction de base finie (si grande soit-elle) en ce point — `fit_k_interaction_integral_single_frame` lève explicitement une erreur si `r_min_px<=0`.

L'intégrale elle-même est évaluée par quadrature du point milieu sur une grille (r, θ) régulière (12 points radiaux × 24 points angulaires par défaut), pas par une règle de Gauss d'ordre supérieur — cohérent avec le reste de l'application ("code littéral et auditable" plutôt qu'optimisé).

Le gradient du champ réel (nécessaire au calcul de $W^{(1,2)}$ et $\sigma_{ij}^{(1)}$) est obtenu par différences finies (`np.gradient`) sur la grille u/v déjà exportée, interpolé aux points de quadrature puis projeté dans le repère local (tangente, normale) — pas de dérivation analytique, puisque le champ réel n'a par définition pas de forme fermée connue. Comme dans le reste de l'application, `u/v` et les coordonnées de grille `xs/ys` sont TOUTES DEUX en pixels : le gradient discret est donc déjà sans dimension et numériquement identique à la déformation physique, sans conversion d'unité supplémentaire.

Auto-normalisation : chaque mode est calculé comme $K_{mode} = M(\text{réel}, \text{aux}_{mode})/M(\text{aux}_{mode}, \text{aux}_{mode})$, plutôt que par la formule fermée standard $K = (E^*/2) \cdot M$ — c'est-à-dire en divisant par la PROPRE auto-interaction du même champ auxiliaire à $K = 1$, calculée sur le MÊME domaine (potentiellement tronqué). E^* s'annule algébriquement dans ce rapport. La formule fermée standard $K = (E^*/2) \cdot M$ suppose implicitement un domaine angulaire COMPLET (θ de $-\pi$ à π) autour de la pointe ; exclure une bande près des faces de fissure (`flank_exclusion_deg`, nécessaire par ailleurs) rompt cette hypothèse de

façon NON UNIFORME entre les deux modes, le mode II (antisymétrique) perdant proportionnellement plus de signal près de la bande exclue que le mode I (symétrique) — la normalisation par auto-interaction élimine cette dépendance au domaine tronqué quel que soit l'angle d'exclusion, sur un mode I pur, un mode II pur, un mode mixte, une tangente de fissure quelconque et en contrainte comme en déformation plane.

Diagnostic de confiance : indépendance au sous-domaine (inner/outer half) — contrairement à l'ajustement en anneau (dont le R^2, §5.1, mesure directement l'adéquation du modèle aux données), cette méthode n'est pas un ajustement et n'a donc pas de résidu à rapporter. Le diagnostic de confiance retenu à la place est l'INDÉPENDANCE AU DOMAINE : K_I/K_{II} sont recalculés séparément sur la moitié interne (r_{min} à r_{mid}) et la moitié externe (r_{mid} à r_{max}) du même anneau (`K_I_inner_half_pa_sqrt_m`/`K_I_outer_half_pa_sqrt_m` et les champs analogues pour K_{II}) — pour un champ LEFM au premier ordre réellement bien décrit par les hypothèses du modèle, les deux moitiés doivent s'accorder étroitement entre elles ET avec la valeur pleine-anneau ; un désaccord marqué signale que le champ réel s'écarte du modèle idéal sur une partie du domaine (viscoplasticité en champ proche, ou gradient de champ lointain en champ plus éloigné) — un rôle analogue au R^2 de l'ajustement en anneau ou au statut "converged" de X-FEM-DIC (§4.6), propre à cette méthode faute d'alternative.

Réglage de `flank_exclusion_deg` : bien que la normalisation ci-dessus annule le biais systématique dû à la troncature du domaine, un angle d'exclusion plus petit reste préférable en pratique : il conserve davantage du signal proche des faces de fissure (là où la sensibilité à K_I/K_{II} est la plus forte) tout en restant suffisamment loin d'une discontinuité de corrélation DIC réelle sur les faces elles-mêmes. `recommend_flank_exclusion_deg` (`core/crack_k1c.py`) fournit une valeur de départ adaptée à la trace analysée plutôt qu'une constante fixe pour toutes les fissures : elle compare, à la position d'arc de la pointe suivie, la tangente locale BRUTE (échantillon du chemin le plus proche) à la tangente LISSÉE (fenêtre de lissage $2 \times r_{max,px}$, la même que celle déjà utilisée en interne par `compute_k_time_series`) — un désaccord important entre les deux est le même signal, déjà documenté pour `path_tangents` (§3), qu'un chemin tracé sur grille de pixels zigzague localement plus qu'une fissure réelle ; ce désaccord (en degrés) est ajouté à une marge de base de 15°, bornée entre 8° et 40° :

$$\text{flank_exclusion_deg} = \text{clip}\big(15^\circ + \text{désaccord tangente}(^\circ),\ 8^\circ,\ 40^\circ\big)$$

Cette méthode ne calcule pas d'incertitude formelle par covariance (ce n'est pas un ajustement de moindres carrés — il n'y a ni Jacobien ni résidu au sens du §4.7/§5.1) ; le diagnostic d'indépendance au sous-domaine ci-dessus en tient lieu comme signal de confiance.

---

6. Limitations et remarques notées dans le code

1. **Deux paradigmes d'incertitude coexistent dans l'application** sans être directement comparables terme à terme : le tirage Monte-Carlo du chapitre 1 (rejoue tout le pipeline physique N fois) contre la covariance asymptotique de moindres carrés de ce chapitre (calculée une seule fois, au point ajusté, par linéarisation locale) — voir chapitre 1, §7 pour la mise en perspective.
2. **L'incertitude de l'ajustement en anneau (§5.1) et de X-FEM-DIC (§4.7) est celle du MODÈLE ajusté** (Williams/Irwin ou FE+Heaviside+singulier), pas une incertitude de mesure DIC de bout

en bout : elle ne couvre pas, par exemple, une éventuelle erreur systématique du champ u/v en amont (chapitre 6) qui affecterait TOUS les points de l'anneau/de l'image de façon corrélée plutôt qu'indépendante — l'hypothèse de résidus indépendants sous-jacente à toute covariance de moindres carrés n'est pas vérifiée explicitement.

3. **Pas d'incertitude formelle sur la vitesse d'onde élastique** (chapitre 2) — seul un R^2 indicatif est rapporté, contrairement au traitement plus détaillé de la pointe de fissure (§3).
4. **La détection automatique de la position de pointe pour X-FEM-DIC (§4.6) est une recherche sur grille/gloutonne, pas une optimisation globale** : elle peut se stabiliser sur un minimum local erroné, en particulier avec des images bruitées ou un pas de recherche grossier — chaque frame est ancrée indépendamment à l'estimation de l'étape 3 plutôt que de chaîner sur l'auto-détection de la frame précédente (§4.6), pour limiter la dérive d'une erreur sans l'éliminer entièrement pour une seule frame isolée.
5. **L'incertitude de position par saut d'ouverture (§3.3) sous-estime silencieusement sa propre contribution dès qu'une perturbation échoue à détecter un front du tout** — documenté explicitement dans le code plutôt que masqué (voir §3.3, fin).
6. **X-FEM-DIC et l'ajustement en anneau (§5.1) sont "statiques d'abord"** : aucun des deux ne modélise de dépendance à la vitesse de propagation de la fissure (un noyau dynamique type Freund) — extension possible non implémentée.
7. **Le saut de face de fissure de X-FEM-DIC est non décalé** (§4.2, simplification 2) — mélange légèrement moins local à la frontière d'un élément coupé, jugé sans conséquence à la densité d'échantillonnage utilisée.
8. **La classification de l'enrichissement (§4.4) utilise une bande de tolérance locale (1.5× la taille d'élément)**, pas une intersection géométrique exacte — un choix de robustesse pratique, pas une garantie topologique absolue sur un maillage très irrégulier.
9. **Le terme de T-stress optionnel de l'ajustement en anneau (§5.2) n'est pas une amélioration systématique de K_I/K_{II} eux-mêmes** — c'est un contrôle de cohérence du modèle (un T-stress anormalement grand signale un anneau échantillonné trop large ou un champ de fond non uniforme) ; l'activer ou non change en général très peu K_I/K_{II} sur un champ déjà bien décrit par le modèle au premier ordre seul.
10. **La formule fermée standard reliant l'intégrale d'interaction à K_I/K_{II} ($K = (E^*/2)M$) suppose un domaine angulaire complet autour de la pointe** — une hypothèse rompue par `flank_exclusion_deg`, de façon non uniforme entre les modes (le mode II, antisymétrique, est affecté davantage que le mode I) ; la normalisation par auto-interaction (§5.3) élimine cette dépendance au domaine tronqué quel que soit l'angle d'exclusion, plutôt que d'utiliser la formule fermée standard.
11. **L'intégrale d'interaction n'a pas de covariance/erreur-type formelle** (ce n'est pas un ajustement de moindres carrés) : son diagnostic de confiance propre est l'accord entre les K_I/K_{II} calculés sur la moitié interne et la moitié externe du même anneau (§5.3).
12. **La taille de pixel auto-détectée depuis le CSV chargé (§0, `_recover_pixel_size_mm`) n'est récupérable que si l'export d'origine avait lui-même une taille de pixel renseignée** (colonnes `u_mm`/`v_mm` non vides) — un export réalisé en pixels purs sur la page Image Correlation ne laisse rien à détecter, et le champ reste à 0 (px/s uniquement) jusqu'à une saisie manuelle ou un "Sync from DIC tab" ; le statut affiché sous le champ le dit explicitement plutôt que de laisser croire à un étalonnage qui n'existe pas.

13. **`r_min_px=0` ("désactivé"), un réglage normal partagé avec l'ajustement en anneau (§5.1) et X-FEM-DIC, est une configuration INVALIDE pour l'intégrale d'interaction (§5.3)** : cocher "Also compute via interaction integral" sans relever `r_min_px` fait échouer ce contrôle croisé sur chaque frame. Ce cas de configuration est distingué, dans le texte de statut par-frame, du cas où les données elles-mêmes sont insuffisantes pour un `r_min_px` par ailleurs valide.
14. **`advance_confirm_frames` (§3.2) peut rejeter, par prudence, jusqu'à `advance_confirm_frames - 1` points authentiques en toute fin de plage de frames** : une avancée réelle et rapide qui survient trop près de la dernière frame analysée peut ne pas disposer d'assez de points ultérieurs pour se confirmer, et est alors écartée comme le serait un pic d'onde non corroboré — un compromis délibéré (mieux vaut perdre quelques points de fin de trace que risquer de figer un artefact), pas un défaut de détection à cet endroit précis. Choix de conception tout aussi délibéré : ni ce paramètre ni `smooth_window_samples` (§3.2) ne bornent ou ne supposent de vitesse de propagation de fissure — une borne de vitesse a été explicitement écartée, la vitesse d'une fissure pouvant varier fortement dans un matériau hétérogène comme le granite.
15. **`stall_patience`/`stall_rtol` (§4.5) sont des heuristiques réglées sur un budget d'itérations pratique (5 itérations, 0.1% relatif), pas dérivées d'une analyse formelle du bruit de corrélation propre à chaque frame** — une frame dont le signal est juste à la limite de ce seuil peut s'arrêter un peu tôt ou un peu tard selon des variations de bruit d'une frame à l'autre ; ce n'est pas exposé comme réglage dans l'interface (contrairement à `max_iterations`), au même titre que `damping`/`convergence_tol_px` eux-mêmes déjà non exposés. Un `stalled=True` signale que le RMS a cessé de s'améliorer significativement, pas que l'ajustement rapporté est nécessairement mauvais – l'état effectivement rapporté reste toujours le meilleur RMS observé sur toute la trajectoire de cette frame (voir §4.5), donc souvent déjà proche de ce qu'une convergence complète aurait donné ; il reste néanmoins prudent de traiter un K_I/K_{II} en plateau avec la même réserve qu'un ajustement non convergé.
16. **Le ratio `SAMPLE_STEP_RATIO=8.0` et le plancher `1.0` px utilisés par "Suggest settings" pour proposer `sample_step_px` (§4.3/§4.5) sont eux aussi une heuristique choisie, pas une valeur dérivée d'une mesure de la longueur de corrélation du mouchetis réellement utilisé** — deux mouchetis de granulométrie différente n'ont pas la même longueur de corrélation, que le code ne mesure jamais explicitement (ni au moment de l'export DIC, chapitre 6, ni ici) ; la proposition reste donc un point de départ raisonnable sur un maillage donné, pas un optimum garanti. Corollaire direct du §4.7 : rien n'empêche techniquement de descendre `sample_step_px` bien en dessous de cette proposition (jusqu'au plancher `0.5` px du réglage lui-même) — le code ne le refuse ni ne l'avertit à l'exécution – seul le texte de ce chapitre documente pourquoi ce n'est généralement pas souhaitable.

Annexe — Synthèse des points de vigilance relevés dans le code

Cette annexe rassemble, chapitre par chapitre, l'ensemble des points signalés dans le corps du document : incohérences apparentes entre deux chemins de calcul, hypothèses implicites non documentées, unités ambiguës, ou fonctionnalités d'interface qui ne transmettent pas toutes les informations attendues par le calcul sous-jacent. Chaque point est formulé tel qu'observé dans le code, sans correction ni réinterprétation ; il est destiné à servir de liste de vérification unique, en complément des sections « Limitations » détaillées à la fin de chaque chapitre.

Chapitre 1 — Incertitudes Monte-Carlo et covariance

- Les paramètres d'entrée sont tirés indépendamment les uns des autres : aucune corrélation d'entrée n'est modélisée (par exemple entre diamètre et masse volumique d'une même barre usinée).
- Seules deux lois de tirage sont supportées (Normale, Uniforme) — pas de log-normale, triangulaire, etc.
- Le paramètre `dispersion` d'un `MCValue` a un sens différent selon la loi choisie (écart-type pour la loi Normale, demi-étendue pour la loi Uniforme).
- Deux méthodes d'agrégation distinctes coexistent dans l'application : `ui/pages/page_process.py::_aggregate_qua` tronque les tirages à la longueur commune et utilise un écart-type **biaisé** (`ddof=0`), tandis que `core/covariance.py` interpole chaque tirage sur une grille commune et utilise `numpy.cov` (**non biaisé**, `ddof=1`) — les deux écarts-types peuvent donc légèrement différer pour une même quantité.
- Dans le module de covariance, un tirage dont la courbe ne couvre pas toute la plage de déformation demandée est silencieusement exclu (pas d'extrapolation) ; le nombre de tirages effectivement utilisés peut être significativement inférieur au nombre de tirages réussis.
- Aucune matrice de corrélation normalisée n'est calculée nativement par l'application : seule la covariance brute est produite.
- La bande d'incertitude d'un capteur personnalisé (`core/custom_sensor.py`) est une combinaison additive déterministe, pas un tirage Monte-Carlo en soi ; elle n'est traitée comme un écart-type gaussien qu'au moment où elle réintègre un calcul Monte-Carlo aval.
- La correction de sortie thermique variable dans le temps d'un capteur personnalisé (`core/custom_sensor.py::Custom` §4.4) n'applique délibérément aucun modèle de retard/dynamique thermique entre le profil de température fourni et la jauge : la jauge est supposée suivre ce profil instantanément, quelle que soit la vitesse réelle de diffusion thermique à travers son collage — un choix de conception assumé

(pour rester valide aussi bien en dynamique rapide qu'en quasi-statique lent), pas une approximation non documentée. Elle n'est par ailleurs pas restreinte aux capteurs de rôle jauge de déformation : la case peut être cochée pour n'importe quel capteur, sans avertissement si son signal n'est pas une déformation.
- Un tirage Monte-Carlo qui échoue (exception dans le pipeline physique) est simplement retiré de la moyenne/écart-type final, sans nouvelle tentative.
- La grandeur physique (`quantity_key`) associée à une entrée de la bibliothèque de calibration (chapitre 1, §3.5) est une étiquette purement indicative, jamais validée contre le contenu réel de l'entrée — seul le filtrage des menus déroulants de la page Bayesian Calibration (chapitre 9, §7) s'appuie dessus.

Chapitre 2 — Quasi-Statique (QS) et géométries d'éprouvette

- Sur l'onglet QS, `comp_check`/`tens_check` transmis à `process_qs()` dépendent uniquement du mode cinématique choisi (Compression/Tension/Notched/Shear), jamais de la géométrie réellement configurée pour l'éprouvette (`project.geometry`) — contrairement aux onglets SHPB et Direct Impact, qui dérivent bien ces indicateurs de la géométrie. En conséquence, les géométries Carrée/Rectangulaire et Plate semblent inaccessibles en pratique depuis l'onglet QS (la section circulaire $\pi(D/2)^2$ est utilisée dans tous les cas).
- Toujours sur l'onglet QS, `shear_check` n'est jamais transmis : seul le cisaillement simple (facteur 1) est atteignable, les géométries Hat-shaped et Double cisaillement restant inaccessibles depuis cet onglet.
- `notch_angle` est déclaré avec l'unité « deg » dans `core/models.py`, mais utilisé directement dans `math.cos()` (qui attend des radians) sans conversion visible dans le code, y compris côté tirage Monte-Carlo.
- Plusieurs champs géométriques déclarés dans `Specimen` (`total_length`, `H3`, `R3`, `head_width`, etc.) ne sont jamais lus par `core/qs_processing.py`, bien qu'ils soient bien utilisés par les chaînes SHPB et Direct Impact.
- Aucune des quatre cinématiques QS ne recalcule la section résistante en fonction de la déformation courante (pas de correction de type striction) ; seule la conversion nominal→vrai est appliquée.
- L'hypothèse d'adiabaticité du modèle de température (Taylor-Quinney) n'est pas justifiée explicitement dans le code.

Chapitre 3 — SHPB (deux barres)

- Avertissement explicite du module `process_shpb.py` : le moteur n'a pas été validé indépendamment sur données réelles pour la matrice complète d'options, seulement testé sur signaux synthétiques dans la combinaison la plus simple.
- La correction de poinçonnement (`punch_check`) applique toujours la formule de contrainte/déformation vraies **de compression**, y compris pour un essai en traction — alors que le chemin sans cette correction bifurque correctement entre compression et traction.
- La correction de poinçonnement introduit, pour le cisaillement, une division par 2 de la déformation absente du chemin sans correction.
- La correction de friction applique en pratique un facteur m^2 (et non m) à la contrainte vraie finale,

du fait de l'ordre des opérations, alors que la contrainte nominale n'est réduite que d'un facteur m.
- La correction d'inertie pour une éprouvette entaillée utilise un angle fixe de 45°, indépendant de l'angle d'entaille réellement configuré.
- Le bras viscoélastique du calcul (`disp_visco`) suppose implicitement que les deux barres (entrée/sortie) partagent la même rhéologie, sans que cette hypothèse soit signalée comme délibérée dans le code.
- Le module complexe fréquentiel du bras viscoélastique est indexé directement échantillon de temps par échantillon de temps plutôt que via une transformée de Fourier/Fourier inverse complète — traitement non standard, à valider.
- Le mode de test global (`type_check`) est déterminé par le réglage « Test mode » de la page Test Setup et non par onglet ; un garde-fou (`_require_notched_geometry`) prévient du risque de traiter un essai avec la géométrie d'un autre mode.
- La correction de toe recoupe désormais chaque tirage Monte-Carlo individuellement (plus seulement la courbe agrégée), mais le compteur `toe_n_removed` qui sert à recaler un capteur de confinement/laser resté en signal brut est une **médiane** sur l'ensemble des tirages, pas un décompte exact tirage par tirage — approximation raisonnable tant que la variation inter-tirages du nombre de points retirés reste faible.

Chapitre 4 — Impact Direct (DI)

- Avertissement explicite du module `process_di.py` : seule la combinaison d'options la plus simple a été testée avec des signaux réels/synthétiques ; le croisement complet chaleur × plateau × poinçonnement × friction × inertie × barre viscoélastique × 4 cinématiques n'a pas été revalidé indépendamment.
- Lorsque `plat_check=True` et `heat_check=False`, le module et la célérité utilisés redeviennent ceux de la barre alors que la masse volumique reste celle du plateau : triplet (E, ρ, C_0) potentiellement incohérent.
- Pour une éprouvette entaillée (`type_check==3`), le champ `displacement_mm` semble, par un suivi strict des unités du code, exprimé en mètres malgré son nom et malgré un commentaire adjacent affirmant l'inverse.
- Le recalcul viscoélastique de la branche cisaillement utilise systématiquement la géométrie « hat-shaped », même pour un cisaillement double ou simple — sans conséquence pratique tant que cette branche reste inaccessible depuis l'interface (voir point suivant).
- La contrainte vraie recalculée après correction de poinçonnement utilise toujours la formule « compression », y compris en traction.
- La correction de friction retranche la contribution de friction une seconde fois dans la seule branche entaillée, équivalent à appliquer m^2 au lieu de m.
- L'option `disp_visco` (bras viscoélastique) n'est actuellement jamais activée depuis l'onglet Direct Impact de l'interface, bien que la fonctionnalité existe et soit opérationnelle côté code.
- `SG_unc_d` et `f_inc_velocity`, bien que définis comme paramètres Monte-Carlo par tirage côté `process_di()`, reçoivent systématiquement une valeur constante depuis l'interface DI : leur incertitude propre n'est donc pas échantillonnée.

Chapitre 5 — Confinement (quasi-œdométrique)

- Ni le modèle linéaire ni le modèle non-linéaire de cellule n'imposent de borne (« clamp ») à la plage de calibration : au-delà de $[0, \varepsilon_{ref}]$, la pondération entre régimes devient une extrapolation linéaire non signalée comme telle dans le code.
- Le polynôme de calibration non-linéaire est forcé à passer par l'origine (terme constant éliminé), ce qui améliore la cohérence physique mais dégrade la qualité de l'ajustement par rapport à un polynôme libre.
- L'itération à point fixe de `radial_strain()` effectue systématiquement 50 itérations sans aucun test de convergence.
- Les fonctions `radial_strain()`, `volumetric_strain()`, `equivalent_strain()` utilisent une convention de signe **inversée** (compression négative) par rapport au reste de l'application (compression positive) ; l'appelant doit explicitement négocier ce changement de signe. Cette inversion ne s'applique pas aux contraintes (σ_r, p, q).
- La déformation équivalente combine une déformation axiale vraie (logarithmique) avec une déformation radiale technique — combinaison reprise telle quelle du code d'origine, sans justification supplémentaire documentée.
- Dans le modèle non-linéaire, aucune incertitude Monte-Carlo équivalente aux 2 % appliqués au module de cellule du modèle linéaire n'est modélisée sur les courbes de calibration elles-mêmes.
- Toutes les fonctions de `core/confinement.py` tronquent silencieusement leurs tableaux d'entrée à la longueur minimale commune, sans avertissement.
- Sur SHPB et Impact Direct, le calcul réel des grandeurs de confinement dans la boucle Monte-Carlo n'est activé que si le capteur de cellule a en plus sa case « Include this cut in the Monte-Carlo loop » cochée ; sinon l'application retombe silencieusement (message explicite mais facultatif à lire) sur l'enveloppe synthétique de repli.
- Sur l'onglet Quasi-Statique, le signal de cellule de confinement n'est jamais varié par tirage Monte-Carlo (signal unique, fixe, partagé par tous les tirages) même quand la contrainte/déformation axiale l'est ; l'incertitude qui en résulte pour les grandeurs de confinement y est donc réelle mais partielle.
- Les grandeurs dérivées de la vélocimétrie laser (`axial_stress_laser`, `pressure_laser`, `deviator_laser`) sont toujours calculées par l'enveloppe synthétique de repli, jamais dans la boucle Monte-Carlo, indépendamment de la disponibilité de données réelles pour les autres grandeurs de confinement du même résultat.
- Quand le capteur de confinement/laser n'est pas recoupé par tirage (signal brut), `_refresh_confinement()` corrige désormais le décalage en tête qu'introduirait une correction de toe appliquée après coup, mais uniquement ce décalage : une différence de cadence d'échantillonnage entre ce capteur et la courbe contrainte-déformation reste, elle, ni détectée ni corrigée (troncature position-à-position à la longueur minimale, sans base de temps commune).

Chapitre 6 — Corrélation d'images (DIC)

- Le suivi est rigide en translation par subset (pas de subset déformable) : approprié aux déformations modérées à dominante uniaxiale, pas à un champ fortement hétérogène.
- Le calcul de déformation locale (`_grid_strain`) est un gradient au premier ordre en petites déformations, pas un calcul DIC grande déformation rigoureux.

- La conversion déformation nominale → vraie ($\varepsilon = \ln(1+e)$) n'est exacte qu'en 1D uniaxial et n'est jamais appliquée à la composante de cisaillement.
- Le calcul de force dynamique sur la ROI suppose une section constante (pas de striction).
- Aucune synchronisation automatique caméra/jauges n'est effectuée : le recalage temporel est manuel.
- Le modèle d'incertitude DIC est **prédictif** (construit à partir du bruit image et de la qualité de corrélation), pas calibré empiriquement contre un essai « zéro » ou une déformation synthétique connue ; il ne couvre pas la distorsion optique, le mouvement hors-plan, ni le biais d'interpolation sous-pixel du raffinement parabolique.
- Les trois termes d'incertitude DIC sont combinés en quadrature en supposant leur indépendance statistique, et la propagation vers déformation/vitesse/force suppose des erreurs indépendantes entre composantes qui partagent en réalité le même bruit sous-jacent de déplacement.
- Le nombre de points effectif utilisé pour corriger le moyennage spatial du bruit sur la ROI repose sur une approximation grossière de la longueur de corrélation spatiale.

Chapitre 7 — Dispersion élastique

- Avertissement explicite en tête de module : le portage n'a pas pu être re-testé contre un cas de référence connu.
- Aucun garde-fou n'existe sur la convergence de la recherche de racine (`sympy.nsolve`) en régime élastique haute fréquence, contrairement au chemin viscoélastique qui dispose d'un repli `try/except`.
- Aucun fenêtrage temporel (apodisation) n'est appliqué avant la FFT du pulse, et la recherche de racine elle-même ne dispose d'aucun garde-fou de divergence propre.
- Un bornage optionnel de la bande passante propre de la correction (`_bandwidth_rolloff_weight`, activé par défaut, seuil réglable — 0,6 par défaut) limite désormais l'amplification du bruit haute fréquence en plafonnant le gain de la correction au-delà d'une fréquence normalisée réglable ; il reste distinct du filtre passe-bas optionnel appliqué au signal brut (`Bar.lowpass_enabled`), et sa valeur par défaut est une heuristique pragmatique, pas un optimum démontré.
- L'origine physique exacte des facteurs de correction d'amplitude M_1/M_2 n'est pas documentée dans le code ; leur forme est cohérente avec un facteur de type Bancroft/Davies, sans confirmation explicite.
- Le positionnement temporel des fenêtres réfléchie/transmise utilise une célérité constante C_0 même en régime élastique corrigé — seule la correction spectrale elle-même est dispersive.
- Le paramètre `n=n` de `np.fft.irfft` est un point de correction critique et non optionnel pour les fenêtres de longueur impaire (cas courant) : son omission produirait un signal reconstruit numériquement faux sur toute sa longueur, pas seulement décalé.

Chapitre 8 — Dispersion viscoélastique, calibration bayésienne, amortissement

- Seul le modèle Standard Linear Solid (SLS) est actuellement branché sur la correction de dispersion effective de la page « Dispersion Corrections » ; les modèles Kelvin-Voigt, Generalized Maxwell (2 branches) et Burgers ne sont disponibles que pour la calibration.
- L'expansion Monte-Carlo de barre viscoélastique (`core/models.py::Bar.expand_viscoelastic`) reste câblée en dur sur le modèle SLS, indépendamment des quatre modèles rhéologiques disponibles pour la calibration.

- La formule du modèle SLS correspond, après vérification algébrique documentée dans le code, à la topologie « ressort en série avec cellule de Kelvin-Voigt ».
- Le modèle Burgers a un module statique exactement nul, ce qui impose un traitement spécial de la fréquence nulle (évaluation à la première fréquence non nulle, partie réelle seule conservée) pour éviter un crash de conversion complexe→réel.
- La recherche de racine de Pochhammer-Chree viscoélastique dispose d'une formulation de repli lorsque la première échoue à converger ; dans ce cas, l'atténuation n'est pas recalculée mais reportée du bin de fréquence précédent, sans justification physique détaillée dans le code.
- `core/visco_calibration.py` porte un avertissement explicite de non-validation indépendante contre des données expérimentales réelles ou la sortie de l'application d'origine.
- La matrice de corrélation a posteriori entre paramètres calibrés indique une non-identifiabilité de la mesure (deux paramètres dont seule une combinaison est contrainte par les données), pas un couplage physique réel — et aucun budget de recherche supplémentaire ne peut la résoudre.
- Le pulse prédit final affiché n'est pas recalculé à la moyenne posterior des paramètres (risque de convergence très lente du solveur dans cette région), mais réutilisé depuis le meilleur point réellement évalué durant la recherche bayésienne.
- Dans `core/decay.py`, l'unité annotée pour le facteur d'amortissement complet (« rad/m ») ne correspond pas dimensionnellement à la formule utilisée ($\zeta \cdot 2\pi f_n$, qui donne des rad/s pour f_n en Hz) — incohérence relevée sans conversion supplémentaire visible dans le code environnant.
- Toujours dans `core/decay.py`, la recherche des points de mi-puissance balaie l'ensemble du spectre depuis l'indice zéro (pas seulement un voisinage du pic), sans garde-fou contre un spectre bruité ou multi-pics.
- Le même bornage optionnel de bande passante (chapitre 7) s'applique à la correction viscoélastique, mais avec une vitesse de référence propre à chaque branche (incidente/réfléchie/transmise) puisqu'un milieu viscoélastique n'a pas de célérité unique constante — ce choix n'est pas exposé ni réglable séparément du cas élastique dans l'interface.

Chapitre 9 — Outils standalone

- Le calculateur CPG (`core/cpg_calc.py`) est un portage littéral non revalidé indépendamment ; sa propagation d'incertitude en température utilise directement l'écart-type relatif de la température (σ_T/T), ce qui ne correspond pas à une propagation rigoureuse à travers le terme $[1+SG_T(T-T_{ref})]$ réellement présent dans la formule — l'incertitude sur le coefficient de température de jauge SG_T lui-même n'est par ailleurs pas prise en compte.
- La correction de sensibilité transverse optionnelle du calculateur CPG (chapitre 9, §1.7) est dérivée pour une jauge axiale unique sous contrainte uniaxiale, et appliquée de la même façon quel que soit le type de pont sélectionné (Quarter/Half/Full) — une approximation de premier ordre, la note technique de référence (Vishay TN-509) ne couvrant explicitement que le cas d'une jauge seule ou d'une rosette à deux jauges, pas le cas d'un pont câblé avec compensation électrique de Poisson.
- La correction de sortie thermique optionnelle du calculateur CPG (chapitre 9, §1.8) porte sur la déformation (un décalage additif), pas sur le coefficient CPG lui-même (un facteur multiplicatif) : elle n'est donc pas repliée dans la valeur synchronisée vers `SG_coeff` ni appliquée automatiquement ailleurs dans l'application — le résultat (déformation apparente et tension équivalente) est affiché à titre informatif, à appliquer manuellement.

- La correspondance trigger <-> impulsion d'exposition, dans l'outil de synchronisation caméra/vidéo, s'effectue en espace d'index d'échantillon plutôt qu'en temps réel, ce qui suppose implicitement un échantillonnage régulier du fichier d'horloge, non vérifié explicitement par le code.
- Toujours dans cet outil, la détection de pics peut sur-détecter sur un signal bruité lorsqu'aucune période attendue n'est fournie ; le nombre de pics détectés différent du nombre de frames chargées ne bloque jamais le calcul, seulement un avertissement affiché à l'utilisateur.
- Dans l'analyse par lot (`core/batch_analysis.py`), la correction isotherme traite le dernier point de chaque courbe de façon asymétrique par rapport aux autres (réutilisation de l'avant-dernière valeur brute), comportement non expliqué dans les commentaires du code source.
- Toujours dans l'analyse par lot, le rééchantillonnage sur grille commune (méthode « target_x ») utilise une recherche du dernier point sous la cible (pas d'interpolation linéaire), avec un cas particulier de repli à l'indice 1 (et non 0) lorsqu'aucun point de X n'est sous la cible — non documenté dans le code source.
- Le seuil de bascule entre méthode de rééchantillonnage monotone (« target_x ») et non monotone (« index_fraction »), fixé à 15 % de rétrogradation relative, est un choix empirique du code, non dérivé d'un critère physique.
- La calibration bayésienne de lois de comportement (portage de MAMBA, chapitre 9 §7) n'a jamais été exécutée contre un vrai PyMC/PyTensor/arviz pendant son développement (aucun accès réseau ni bibliothèque installée dans la sandbox de développement) — un essai manuel de validation numérique sur la machine de l'utilisateur reste requis avant tout usage en conditions réelles (chapitre 9, §7.5, §7.10).
- Dans cette même fonctionnalité, PyMC n'offre aucune annulation réelle en cours d'échantillonnage (le bouton Stop ne fait effet qu'avant le démarrage de `pm.sample`) et la barre de progression n'est mise à jour qu'avant/après l'échantillonnage, jamais pendant (chapitre 9, §7.5, §7.10).
- Toujours dans cette fonctionnalité, la carte "Validation" retombe sur l'hyper-moyenne de population pour un paramètre hiérarchique à un essai de validation réellement tenu à l'écart de l'ajustement, ignorant la dispersion inter-essais propre à ce paramètre — la bande affichée y est donc plus étroite que l'incertitude prédictive réelle (chapitre 9, §7.7, §7.9, §7.10).
- Le type de prior (fixe / pooled / hiérarchique) d'un paramètre libre de calibration bayésienne s'applique globalement à l'ensemble des essais d'un run donné, sans mélange partiel possible en une seule exécution (chapitre 9, §7.10).

Chapitre 10 — Analyse fissure & vitesse d'onde (Crack & Wave Analysis)

- X-FEM-DIC (`core/xfem_dic.py`) est la seule méthode de calcul de K_I/K_{II} exposée dans l'interface de l'onglet "Crack analysis" ; un ajustement DIC standard en anneau avec extension T-stress optionnelle (`core/crack_k1c.py`) et une intégrale d'interaction (`core/crack_interaction_integral.py`) restent disponibles dans le code, testées, pour un usage programmatique — voir chapitre 10, §5.
- Deux paradigmes d'incertitude coexistent sans être directement comparables terme à terme : le tirage Monte-Carlo (chapitre 1) et la covariance asymptotique de moindres carrés (X-FEM-DIC et l'ajustement en anneau), calculée une seule fois au point ajusté par linéarisation locale plutôt que par ré-échantillonnage ; l'intégrale d'interaction n'entre dans aucun des deux, puisqu'elle n'est pas un ajustement de moindres carrés — sa confiance s'évalue par un diagnostic d'indépendance au sous-domaine (inner/outer half).

- L'incertitude formelle de X-FEM-DIC et de l'ajustement en anneau est celle du modèle ajusté, pas une incertitude de mesure DIC de bout en bout ; l'hypothèse de résidus indépendants sous-jacente à toute covariance de moindres carrés n'est pas vérifiée explicitement (une erreur systématique corrélée en amont du champ u/v n'est pas couverte).
- Aucune incertitude formelle n'est calculée sur la vitesse d'onde élastique (`core/elastic_wave_speed.py`) — seul un R^2 indicatif est rapporté, contrairement au traitement de la pointe de fissure.
- La détection automatique de la position de pointe pour X-FEM-DIC (`auto_detect_tip=True`) est une recherche sur grille/gloutonne, pas une optimisation globale ; elle peut se stabiliser sur un minimum local erroné sur une frame isolée, malgré l'ancrage indépendant de chaque frame à l'estimation de suivi de pointe.
- L'incertitude de position par saut d'ouverture (`opening_extent_position_uncertainty_px`) sous-estime silencieusement sa propre contribution dès qu'une perturbation (seuil, tolérance de trou, décalage d'échantillonnage) échoue à détecter un front du tout — documenté explicitement dans le code plutôt que masqué.
- X-FEM-DIC et l'ajustement en anneau sont volontairement « statiques d'abord » : aucun des deux ne modélise de dépendance à la vitesse de propagation de la fissure (noyau dynamique type Freund).
- Le saut de face de fissure de X-FEM-DIC est non décalé (forme `H(x,y)` évaluée au point de requête, pas la forme standard décalée), et la classification de l'enrichissement des noeuds utilise une bande de tolérance locale (1.5x la taille d'élément) plutôt qu'une intersection géométrique exacte.
- Le terme de T-stress optionnel de l'ajustement en anneau (`core/crack_k1c.py`) n'est pas une amélioration systématique de K_I/K_{II} eux-mêmes : c'est un contrôle de cohérence du modèle (un T-stress anormalement grand signale un rayon d'anneau externe trop large ou un champ de fond non uniforme), pas une correction automatique.
- La formule fermée standard reliant l'intégrale d'interaction à K_I/K_{II} suppose un domaine angulaire complet autour de la pointe, hypothèse rompue de façon non uniforme entre les modes par l'angle d'exclusion des faces de fissure (`flank_exclusion_deg`) — l'intégrale d'interaction normalise donc chaque mode par l'auto-interaction du champ auxiliaire sur le même domaine (potentiellement tronqué) plutôt que d'utiliser la formule fermée standard, ce qui élimine cette dépendance au domaine tronqué.

Fin du document.